PhotoGlow

Production-Ready Social Platform Leveraging Next-Generation JavaScript Technology Stacks

Author: Pure-Peace (Albert Wu)

https://github.com/pure-peace/photoglow

GitHub Edition

November 4, 2026

Abstract

PhotoGlow presents an innovative social media platform that advances the state-of-the-art in cloud-based photo sharing applications through its novel architectural approach.

The platform introduces a highly modular, type-safe system architecture based on monorepo design patterns, which significantly improves development efficiency and application reliability.

By leveraging Bun as a next-generation JavaScript runtime environment and embracing modern software engineering principles, PhotoGlow achieves exceptional performance while maintaining code maintainability.

The implementation demonstrates how contemporary architectural patterns, including hybrid rendering and containerized deployment, can be effectively applied to create sophisticated social platforms, providing valuable insights for small to medium enterprises in building scalable web applications.

This work contributes both practical engineering solutions and architectural guidance to the broader development community, supported by comprehensive documentation and development workflows.

Contents

Abstract **1**

1. Introduction **5**
1.1 Project Background and Motivation . . . 5
1.2 Project Objectives . . . 6
1.3 Main Work Accomplished . . . 7
1.3.1 Technical Foundation and Architecture . . . 7
1.3.2 Core Functionality Implementation . . . 8
1.3.3 Cloud Infrastructure Integration . . . 9
1.3.4 Deployment and Operations . . . 9
1.3.5 Quality Assurance and User Engagement . . . 9
1.4 Structure . . . 10

2. Literature Review **11**
2.1 The Co-evolution of Web Technologies and Social Media . . . 11
Early Web Technologies and Social Forms . . . 11
2.1.2 Breakthroughs in the Era of Dynamic Interaction . . . 11
2.1.3 Mobile Internet-Driven Transformation . . . 11
2.1.4 The Era of Algorithm-Driven Personalization . . . 12
2.1.5 The Shared Future of Technology and Social Interaction . . . 12
2.2 JavaScript Full-Stack Evolution and Front-End Engineering Progress . . . 12
2.2.1 JavaScript: From Browser Scripts to Server Runtime . . . 14
2.2.2 TypeScript and Modern Front-End Engineering . . . 14
2.2.3 Build Tool Evolution and Modularization Challenges . . . 14
2.2.4 Package Management Ecosystem: Prosperity and Challenges . . . 15
2.2.5 The Rise of Monorepo Architecture . . . 15
2.2.6 Bun: A Unified Full-Stack Development Solution . . . 15
2.3 Evolution of Modern Web Development Paradigms . . . 15
2.3.1 Front-End Framework Innovation . . . 16
2.3.2 The Revival of Server-Side Rendering . . . 16
2.3.3 Hybrid Rendering: Evolution of Best Practices . . . 16
2.3.4 Evolution of Styling Solutions . . . 17
2.3.5 Architecture Choice Trade-offs . . . 17
2.4 Technology Selection Theory for Social Media Platforms . . . 18
2.4.1 Balance Between Performance and User Experience . . . 18
2.4.2 Scalability Design Considerations . . . 18
2.4.3 Technical Support for Real-Time Interaction . . . 18
2.4.4 Technical Guarantees for Security and Privacy . . . 18
2.5 Summary . . . 18

3. System Design and Implementation **20**
3.1 System Architecture and Implementation . . . 20
3.1.1 Overall Architecture Design . . . 20
3.1.2 Technology Stack Selection . . . 22
3.1.3 Deployment Architecture . . . 23
3.1.4 Communication Mechanism . . . 23
3.1.5 System Architecture Diagram . . . 24
3.1.6 Data Flow Design . . . 25
3.1.7 Security Architecture Design . . . 25
3.1.8 Implementation Challenges and Solutions . . . 29
3.2 Frontend Design and Implementation . . . 29
3.2.1 Design Goals and Principles . . . 29
3.2.2 Technology Stack Selection . . . 30
3.2.3 Architecture Design . . . 31
3.2.4 Rendering Strategy . . . 33
3.2.5 UI/UX Design . . . 33
3.2.6 Frontend Implementation . . . 98

3.2.7 Performance Optimization and Monitoring . . . 108
3.2.8 Future Optimization Plans . . . 108
3.3 Backend Design . . . 108
3.3.1 Technology Stack Selection . . . 108
3.3.2 Architecture Design . . . 109
Project Structure . . . 109
3.3.3 API Endpoint Design . . . 111
3.3.3 Core Functionality Implementation . . . 112
3.3.4 Security Design . . . 121
3.3.5 Performance Optimization . . . 121
3.3.6 Scalability Design . . . 122
3.4 Database Design . . . 122
3.4.1 Data Model Design . . . 122
3.4.2 Relationship Design . . . 127
3.4.3 Indexing Strategy . . . 127
3.4.4 Data Integrity Assurance . . . 128
3.4.5 Scalability Design . . . 128
3.4.6 Security Design . . . 129
3.5 Gateway Layer Design . . . 129
3.5.1 Architecture Design . . . 129
3.5.2 Nginx Basic Configuration Design . . . 130
3.5.3 Route Forwarding and Service Integration . . . 130
3.5.4 Security Protection Mechanism . . . 131
3.5.5 Performance Optimization Design . . . 132
3.5.6 Scalability Design . . . 132
3.5.7 Operations Support . . . 133
3.6 Admin Backend Design . . . 133
3.6.1 Architecture Design . . . 133
3.6.2 Functional Module Design . . . 134
3.6.3 Data Model Design . . . 134
3.6.4 UI/UX Design . . . 141
3.6.5 Deployment Architecture . . . 150
3.6.6 Development Tool Support . . . 150
3.6.7 Future Plans . . . 151
3.7 SDK Design . . . 151
3.7.1 Design Goals . . . 152
3.7.2 Technical Architecture . . . 152
3.7.3 Multi-language Support Implementation . . . 153
3.7.4 Generation Process Design . . . 153
3.7.5 Documentation Generation . . . 155
3.7.6 Security Design . . . 157
3.7.7 Extensibility Design . . . 157
3.7.8 Future Plans . . . 158
3.8 Containerized Deployment Design . . . 158
3.8.1 Containerized Architecture Design . . . 158
3.8.2 Service Container Configuration Design . . . 159
3.8.3 Build Process Optimization . . . 160
3.8.4 Deployment Environment Configuration . . . 161
3.8.5 Deployment Process Automation . . . 161
3.8.6 Extensibility Design . . . 162
3.8 Recommendation System Design . . . 162
3.8.1 Available Technical Resources . . . 164
3.8.2 System Architecture Design . . . 164
3.8.3 Data Collection Design . . . 164
3.8.4 Recommendation Algorithm Design . . . 165
3.8.5 Cold Start Strategy . . . 165
3.9 Current Implementation Status and Future Plans . . . 165

4. System Testing and Evaluation **168**
4.1 Deployment Environment . . . 168
4.2 Performance Testing Methodology . . . 168
4.3 Performance Test Results and Analysis . . . 168

5. Project Management **173**
5.1 Project Planning . . . 173
5.1.1 Time Management . . . 173
5.1.2 Development Tools . . . 173
5.1.3 Quality Management . . . 174
5.2 Risk Management . . . 174
5.2.1 Risk Identification . . . 174
5.2.2 Risk Response Strategies . . . 174
5.3 Project Background Considerations . . . 174
5.3.1 Legal and Ethical Considerations . . . 174
5.3.2 Health and Safety Considerations . . . 175
5.3.3 Equality, Diversity, and Inclusion . . . 175
5.3.4 Sustainable Development . . . 175
6.1 Project Summary . . . 176
6.2 Innovation Points . . . 176
6.3 Future Work . . . 177
6.4 Project Reflection . . . 177

References **179**

1. Introduction

1.1 Project Background and Motivation

In the era of mobile internet, social media platforms that combine visual content and social interaction have become an integral part of people's daily lives. Traditional photo storage services have evolved into comprehensive social platforms where users not only share images but also create, discover, and engage with diverse forms of content. With the widespread adoption of smartphones and high-speed mobile networks, these platforms have seen exponential growth in both user base and content volume. This evolution has fundamentally changed how people share experiences and consume information, while simultaneously creating multifaceted technical challenges.

Modern social content platforms face complex technical demands across multiple dimensions. At the architectural level, platforms must handle image storage, content distribution, and user interactions seamlessly. Performance and scalability requirements become increasingly critical as user bases grow - systems must scale smoothly while maintaining consistent response times. Additionally, operational challenges in deployment, monitoring, and reliability require sophisticated engineering solutions.

The technical landscape of web development is currently undergoing a significant transformation. The emergence of Bun, a JavaScript runtime built with the modern systems programming language Zig, represents a significant leap forward in JavaScript execution performance. Bun achieves up to three times faster execution speeds compared to Node.js [1], thanks to its innovative design and low-level optimizations. Beyond runtime performance, it also revolutionizes the developer experience with near-instant package installation and an integrated toolchain that includes a bundler, test runner, and package manager. The widespread adoption of TypeScript has brought robust type safety to large-scale application development, while modern ORM tools have simplified database operations. These technological advancements have created new possibilities for building next-generation social content platforms.

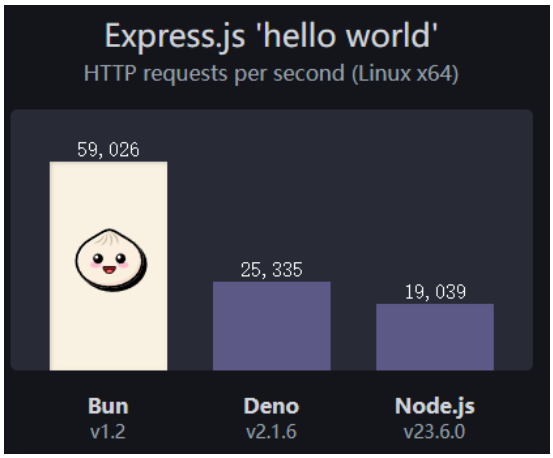


Figure 1: Bun is faster (Origin from: https://bun.sh/)

However, the industry lacks comprehensive case studies demonstrating how to effectively integrate these emerging technologies into a complete engineering system. A successful platform requires not just application-level implementation, but end-to-end solutions incorporating deployment strategies, cloud services integration, and advanced features like content recommendation systems. This gap in practical implementation examples affects development teams' technology decisions and potentially slows industry-wide innovation.

1.2 Project Objectives

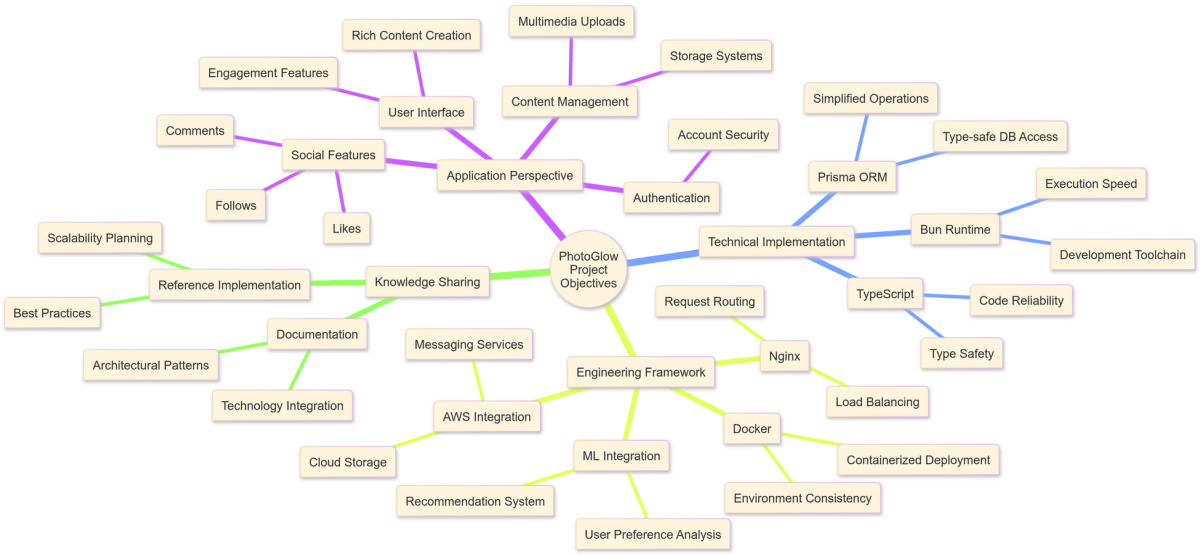


Figure 2: Objectives

The PhotoGlow project aims to develop a modern social content platform while exploring and validating the integration of cutting-edge web technologies. This overarching goal represents both a practical implementation challenge and an opportunity to advance the understanding of modern web development practices. The project's objectives can be detailed across several key dimensions:

From an application perspective, PhotoGlow seeks to establish a comprehensive social platform that enables users to share and discover visual content effectively. A primary objective is to create an intuitive user interface that supports rich content creation and engagement features. This includes implementing robust user authentication to ensure account security, developing efficient content management systems for handling multimedia uploads and storage, and building dynamic social interaction features such as comments, likes, and follows. The platform should deliver a seamless user experience across different devices and network conditions, ensuring that content loads quickly and interactions feel responsive.

From a technical implementation standpoint, the project aims to leverage the latest advancements in web development technologies to create a high-performance, maintainable system. A key objective is to utilize the Bun JavaScript runtime, taking advantage of its superior execution speed and integrated development toolchain to achieve optimal performance in both development and production environments. The project also aims to maintain complete type safety through TypeScript, ensuring code reliability and maintainability across the entire application stack. The use of Prisma ORM serves another critical objective: establishing a type-safe database access layer that reduces the risk of runtime errors while simplifying data operations.

From an engineering perspective, the project aims to establish a comprehensive deployment and operations framework that demonstrates modern web application architecture. This includes implementing containerized deployment using Docker to ensure consistency across different environments, configuring Nginx for efficient request routing and load balancing, and integrating AWS cloud services for scalable storage and messaging capabilities. Furthermore, the project seeks to lay the groundwork for advanced features such as a machine learning-based recommendation system that can effectively analyze user preferences and suggest relevant content.

From a knowledge-sharing perspective, PhotoGlow aims to serve as a practical reference implementation for the web development community. By documenting the integration of modern technologies and architectural patterns, the project seeks to provide valuable insights into building large-scale web applications. This includes demonstrating how to effectively combine different technologies while maintaining system performance and code quality, and how to plan for future scalability and feature expansion.

By achieving these objectives, PhotoGlow not only seeks to create a functional social content platform but also aims to contribute to the broader understanding of modern web application development. The

project's outcomes will serve as a valuable reference for development teams facing similar technical challenges, particularly in the context of building content-centric social platforms with high performance and reliability requirements.

This comprehensive set of objectives reflects both the immediate practical goals of building a functioning platform and the broader ambition of advancing web development practices. Through careful attention to both technical excellence and practical usability, PhotoGlow aims to demonstrate how modern web technologies can be effectively integrated to create sophisticated social platforms.

1.3 Main Work Accomplished

The development of PhotoGlow involved a series of systematic efforts across multiple technical domains, from architectural design to real-world deployment and user engagement. This section outlines the key efforts undertaken in developing this modern social content platform.

1.3.1 Technical Foundation and Architecture

The first phase focused on establishing the technical foundation through careful technology selection and architecture design. After thorough evaluation, we adopted a monorepo architecture to manage the project's multiple packages efficiently. The selection of Bun as the JavaScript runtime environment was a crucial decision, driven by its superior performance characteristics and integrated development toolchain. We implemented comprehensive type safety throughout the entire application stack using TypeScript and chose Prisma as our ORM solution to ensure reliable database operations. These foundational choices formed the technical backbone of the platform.

1.3.2 Core Functionality Implementation

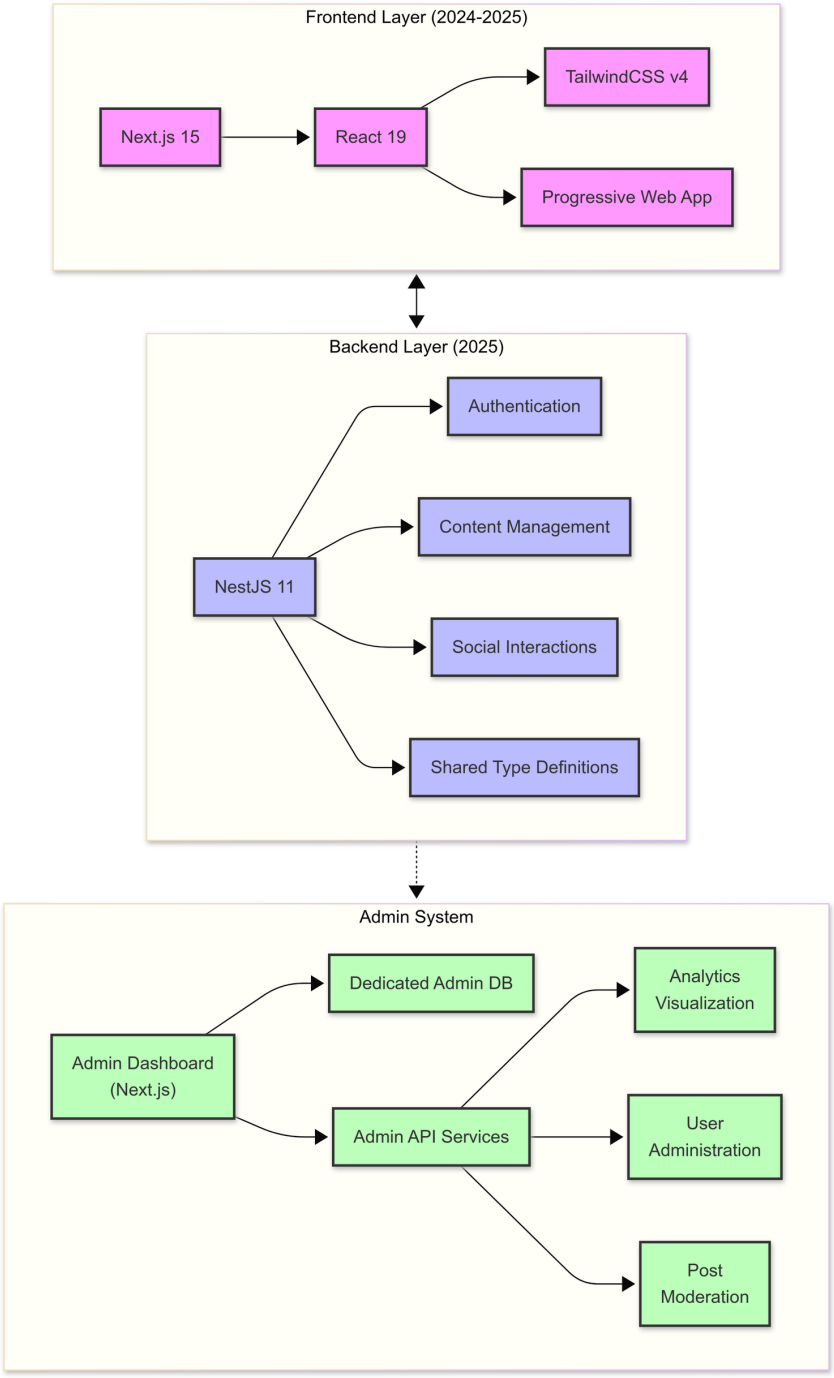


Figure 3: Overview

The second phase centered on implementing core functionality. On both frontend and backend, we undertook the ambitious challenge of working with the most recent releases in the JavaScript ecosystem.

The frontend utilizes Next.js 15 and React 19, both released in late 2024, along with TailwindCSS v4, which was released in January 2025. These technology choices represented a significant technical challenge, as we were among the early adopters working with these frameworks immediately after their release. The user interface design, including interaction flows, UI/UX experience, and responsive layouts, was personally crafted to ensure an intuitive and engaging user experience.

A notable feature of the frontend implementation is its Progressive Web App (PWA) support, enabling

users to install and use the platform as a native application on both desktop and mobile devices, providing a seamless experience comparable to traditional installed applications. This hands-on approach to design allowed us to create a unique platform identity while leveraging the powerful features introduced in these new framework versions, such as enhanced server components and improved rendering patterns.

On the backend, we built a robust API service using NestJS 11 (released in January 2025), implementing essential features such as user authentication, content management, and social interactions. We paid particular attention to ensuring type safety between frontend and backend through shared type definitions and automated API client generation.

Despite the challenges of working with such recently released technologies, including encountering and resolving previously undocumented issues, this cutting-edge stack enabled us to implement sophisticated features and achieve optimal performance levels that would not have been possible with older versions.

Complementing the main application, we developed a separate full-stack administrative dashboard using Next.js. This independent system operates with its own dedicated admin database and API services, maintaining strict isolation from the main user-facing services. The admin dashboard provides comprehensive platform management capabilities, including user administration, post moderation, and detailed analytics visualization. This separation of concerns ensures robust security and operational efficiency in platform management while preventing any potential interference with the main application's performance.

1.3.3 Cloud Infrastructure Integration

The third phase involved cloud service integration, focusing on establishing reliable infrastructure support. We integrated AWS services strategically throughout the platform, using S3 for secure image storage and SES for reliable email delivery. This integration required careful consideration of security configurations and performance optimizations to ensure efficient resource utilization while maintaining cost-effectiveness.

1.3.4 Deployment and Operations

The fourth phase addressed deployment and operational concerns, culminating in a full production deployment. We began by implementing containerized deployment using Docker, ensuring consistent behavior across different environments. The configuration of Nginx as a reverse proxy enhanced the platform's request handling capabilities and improved overall system reliability.

To move the platform into production, we executed a comprehensive deployment strategy. This involved procuring and configuring an AWS EC2 instance to host the application, carefully considering factors such as computational resources, storage requirements, and network configuration. We registered a domain name for the platform and implemented a robust security infrastructure through Cloudflare's services.

This integration provided multiple critical security and performance benefits:

- DNS management for reliable domain resolution
- SSL certificate provisioning for encrypted data transmission
- Protection against common cyber threats such as DDoS attacks and malicious traffic

1.3.5 Quality Assurance and User Engagement

Quality assurance was primarily conducted through comprehensive manual testing, with a focus on user experience, performance, and security. While we implemented basic automated tests for critical components, the majority of testing efforts involved thorough manual verification across different scenarios and edge cases. This hands-on approach allowed us to identify and address subtle issues that might have been missed by automated testing alone.

The successful production deployment led to the platform's actual adoption by an initial user base consisting of friends and fellow students. This real-world usage provided invaluable insights into how the platform performs under genuine conditions. Users particularly appreciated the platform's responsive interface and smooth content sharing capabilities, offering positive feedback about the overall user experience. Their engagement with the platform validated many of our technical decisions, from the choice of modern frontend technologies to our infrastructure setup.

Throughout these phases, we maintained a strong focus on code quality and system performance. While the groundwork for advanced features such as the recommendation system and comprehensive DevOps

pipeline has been laid, their full implementation is planned for future iterations. The modular architecture we adopted not only simplified current development but also provided clear pathways for future enhancements.

This systematic approach to platform development has resulted in a robust foundation that demonstrates the practical application of modern web technologies in building social content platforms. The progression from technical implementation to actual user adoption represents a comprehensive validation of the project's approach. The work accomplished thus far not only provides valuable insights into the challenges and solutions involved in implementing such systems but also establishes a clear direction for future improvements based on real user feedback and operational experience.

1.4 Structure

This thesis is organized into six chapters, with the following structure:

Chapter 1, Introduction, begins by presenting the project background and research motivation, establishing the necessity and significance of this research endeavor. It then clearly defines the project objectives and provides an overview of the main work accomplished, including technical foundation and architecture, core functionality implementation, cloud infrastructure integration, deployment and operations, as well as quality assurance and user engagement.

Chapter 2, Literature Review, systematically examines the theoretical foundations and technological developments relevant to this project. The chapter covers the co-evolution of web technologies and social media, the evolution of JavaScript full-stack development and front-end engineering progress, the evolution of modern web development paradigms, and technology selection theory for social media platforms. This comprehensive review establishes the theoretical groundwork for the project's design and implementation.

Chapter 3, System Design and Implementation, serves as the core chapter of this thesis, providing detailed explanations of the system's overall architecture design and specific implementation details. The chapter begins with an overview of the system's general architecture design, followed by in-depth descriptions of the system's design and implementation process from multiple perspectives: frontend, backend, database, gateway layer, admin backend, SDK development, containerized deployment, and recommendation system. Each section encompasses detailed technical stack selection, architecture design, and specific implementation solutions.

Chapter 4, System Testing and Evaluation, presents a detailed description of the deployment environment, introduces the performance testing methodology, and provides an in-depth analysis of the test results. This chapter validates the rationality of the system design and the effectiveness of its implementation.

Chapter 5, Project Management, summarizes the management experience gained during the project development process from three main aspects: project planning, risk management, and project background considerations. The project planning section covers time management, development tools, and quality management. Risk management includes risk identification and response strategies. Project background considerations encompass legal and ethical considerations, health and safety, equality, diversity and inclusion, and sustainable development.

Chapter 6, Conclusion and Future Work, provides a comprehensive summary of the entire project, highlighting its innovation points and outlining directions for future improvements. This chapter also includes reflections and insights gained throughout the development process.

Each chapter closely revolves around the project's central theme, ensuring both comprehensiveness and systematic coverage while emphasizing the depth of key chapters. The thesis maintains a balance between theory and practice, combining in-depth theoretical discussions with detailed implementation specifics and practical experience summaries.

2. Literature Review

2.1 The Co-evolution of Web Technologies and Social Media

Web technologies and social media platforms have developed in a deeply symbiotic relationship. Technological innovations have driven the evolution of social media forms, while users' demands for social interaction have, in turn, catalyzed web technology innovations. This bidirectional interaction has shaped the landscape of the modern internet.

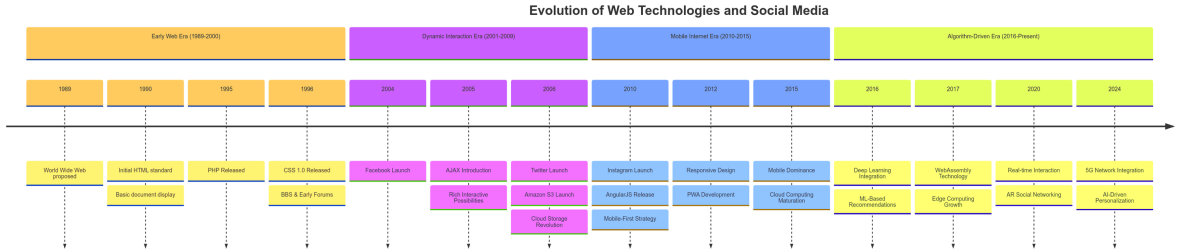


Figure 4: Evolution of Web Technologies

Early Web Technologies and Social Forms

The inception of web technologies traces back to 1989 when Tim Berners-Lee, a British scientist at CERN, proposed an information management system that laid the foundation for the World Wide Web [2]. The initial HTML standard, developed in 1990, facilitated the display of simple documents, enabling basic hyperlinking and text formatting [3].

During this early phase, online social interactions were predominantly conducted through Bulletin Board Systems (BBS). Despite their rudimentary functionalities, BBS platforms exhibited distinct community characteristics. Users engaged in text-based communications, fostering virtual social networks that mirrored real-world communities [4].

The release of CSS 1.0 in December 1996 introduced fundamental style control capabilities to web pages, allowing for the separation of content and presentation [5]. This advancement, coupled with the emergence of dynamic web technologies like PHP—first released by Rasmus Lerdorf in June 1995—paved the way for the development of the first generation of web forums [6].

These early forums utilized server-side rendering techniques, wherein the server generated the complete HTML content for each user interaction, necessitating full page reloads.

2.1.2 Breakthroughs in the Era of Dynamic Interaction

In 2005, Jesse James Garrett first introduced the concept of AJAX [7], a technological innovation that fundamentally changed Web interaction patterns. AJAX enabled web pages to asynchronously update partial content, providing richer interactive possibilities for social media platforms. The success of Facebook (2004) and Twitter (2006) fully demonstrated the potential of AJAX technology in social networks.

During the same period, the concept of Single Page Applications (SPA) [8] began to take shape. In 2010, Google's release of the AngularJS framework [9] marked the beginning of front-end engineering. This paradigm shift enabled social media platforms to provide user experiences comparable to native applications.

2.1.3 Mobile Internet-Driven Transformation

The proliferation of smartphones and deployment of 4G networks triggered a new revolution in social media [10]. Instagram's success (2010) validated the value of a mobile-first strategy. Research data shows that by 2015, mobile had become the primary means of social media usage. This trend drove the development of technologies such as responsive design and Progressive Web Applications (PWA) [11].

The maturation of cloud computing technology also provided crucial support for this transformation. Cloud storage services like Amazon S3 (2006) [12] significantly reduced the cost of media content storage

and distribution.

2.1.4 The Era of Algorithm-Driven Personalization

Currently, social media platforms have evolved into complex algorithmic systems. The application of machine learning technology enables platforms to accurately understand user interests and content characteristics. Research shows that deep learning-based recommendation algorithms significantly improved user engagement rates compared to traditional timeline displays [13]. The success of short video platforms like TikTok further confirms the advantages of algorithm-driven content distribution models.

Web technologies continue to evolve to support these new requirements. WebAssembly technology [14] has made complex browser-side computations possible, while the development of Edge Computing has optimized content distribution efficiency. Research indicates that enterprises adopting edge computing have seen significant improvements in real-time data processing efficiency and substantial reductions in response times [15].

2.1.5 The Shared Future of Technology and Social Interaction

With the popularization of 5G networks and advancement of artificial intelligence technology, social media platforms are evolving toward more intelligent and personalized directions. The emergence of new forms such as real-time interaction and augmented reality (AR) social networking requires continuous innovation in Web technology in terms of performance, security, and scalability.

In this context, building next-generation social media platforms requires consideration of multiple technical dimensions. This project has paid special attention to these technology trends in its technical selection and architectural design, adopting the latest Web frameworks and toolchains to ensure performance while providing quality user experience.

In the co-evolution of Web technology and social media, front-end technology plays an increasingly crucial role. As front-end projects grow in scale and complexity, traditional front-end development methods can no longer meet the demands of modern Web applications. This has prompted a series of revolutionary changes in the front-end technology field, from breakthroughs in JavaScript runtime environments to innovations in package management tools and project architecture evolution, each step pushing forward the update of Web development paradigms.

2.2 JavaScript Full-Stack Evolution and Front-End Engineering Progress

The development history of front-end technology demonstrates the profound transformation of Web development paradigms. From a single browser scripting language to a complete full-stack development solution, the evolution of the JavaScript ecosystem reflects the technical demands brought by the increasing complexity of Web applications. This section will discuss in detail the key breakthroughs and important milestones in this evolutionary process.

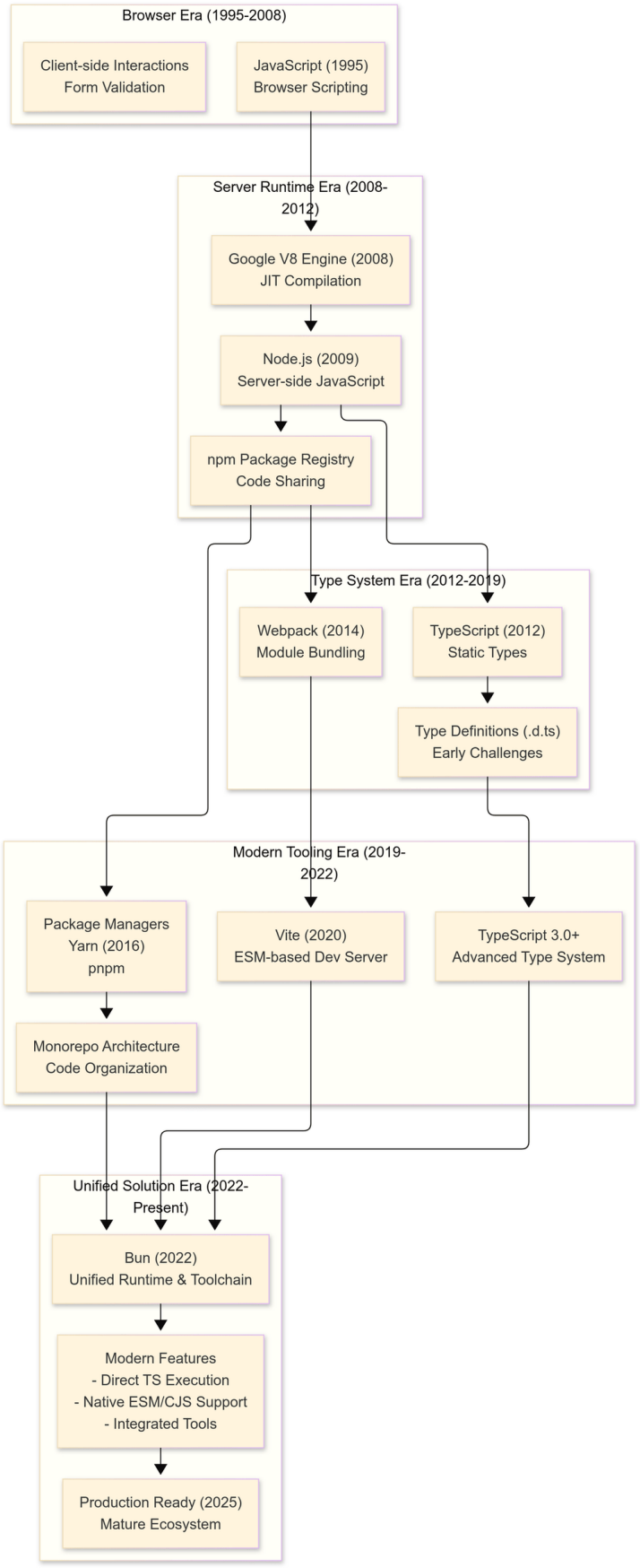


Figure 5: Evolution of JavaScript Technologies

2.2.1 JavaScript: From Browser Scripts to Server Runtime

JavaScript was born in 1995, originally developed by Brendan Eich at Netscape [16], with the initial purpose of providing simple client-side interaction capabilities for web pages. For a long time, JavaScript was confined to the browser environment, mainly used for handling basic user interactions and form validation. This limitation continued until 2008, when Google released the V8 engine [17], marking a turning point in JavaScript's development. The V8 engine adopted Just-In-Time compilation technology, directly compiling JavaScript code into machine code for execution, which demonstrated JavaScript's potential as a high-performance language for the first time.

In 2009, Ryan Dahl created Node.js [18] based on the V8 engine, marking an important milestone in JavaScript's history. Node.js first broke through the browser's limitations, enabling JavaScript to run on the server side. Node.js's non-blocking I/O model and event-driven architecture were particularly suited for handling high-concurrency network applications, features that quickly gained recognition in the Web server domain.

2.2.2 TypeScript and Modern Front-End Engineering

As Web applications grew in scale, JavaScript's limitations as a dynamically typed language became increasingly apparent. In large projects, type-related errors accounted for a significant proportion of production environment issues. In 2012, Microsoft released TypeScript [19], attempting to address these problems by introducing a static type system. However, TypeScript's development path was not smooth sailing.

In the first few years after its release, TypeScript not only failed to effectively solve problems but also brought new challenges to development teams. First, TypeScript required an additional build step to compile code into JavaScript, which increased project complexity. Second, early type definition files (.d.ts) varied in quality, and many popular JavaScript libraries lacked type definitions, requiring developers to write their own or use poor-quality community-contributed versions. Additionally, the learning curve of the type system itself was quite steep, especially when handling complex type inference.

TypeScript's real turning point appeared around 2019. Multiple factors contributed to this change: First, TypeScript's type system became increasingly mature, particularly with advanced features like conditional types introduced after version 3.0, making type definitions for complex scenarios more elegant. Second, mainstream IDE support for TypeScript improved significantly, with tools like VS Code providing powerful type inference and auto-completion features. Most crucially, the community ecosystem finally reached a critical point—the vast majority of popular npm packages provided high-quality type definitions, and the number of contributors to the DefinitelyTyped repository [20] grew rapidly.

By 2020, TypeScript's position in front-end development underwent a fundamental change. Survey data shows that TypeScript adoption rates in enterprise-level Web projects increased significantly. This transformation was reflected not only in adoption rates but more in the change in developer attitudes—TypeScript evolved from an "optional add-on" to an "essential tool" [21].

2.2.3 Build Tool Evolution and Modularization Challenges

With the popularization of TypeScript and increasing complexity of front-end projects, the build toolchain became another key challenge. JavaScript's split module system (CommonJS and ES Modules) brought serious compatibility issues. For example, a project using ESM would often encounter module resolution errors when trying to import CommonJS-based libraries. This situation required developers to configure complex build tools to handle module conversion.

Webpack (2014) [22] became the most popular build tool during this period, solving module compatibility issues through its powerful module bundling system. However, as project scale grew, Webpack's build process became increasingly slow. In large projects, extended build times seriously impacted development efficiency.

In 2020, the release of Vite [23] marked a major breakthrough in build tools. Vite adopted an ESM-based development server, eliminating the need for bundling in the development environment, significantly improving the development experience. Compared to Webpack, Vite could substantially reduce cold start times in the development environment.

2.2.4 Package Management Ecosystem: Prosperity and Challenges

With the popularization of Node.js, npm (Node Package Manager) [24] gradually developed into the world's largest software package registry. The flourishing of the open-source community, especially the rise of the GitHub platform, greatly promoted code sharing and reuse.

However, as Web project scales expanded, dependency management became increasingly complex. A typical modern Web application might contain hundreds of dependency packages, bringing huge challenges to package installation speed and version management. To solve these problems, new-generation package management tools emerged:

Yarn (2016) [25] significantly improved dependency installation speed through parallel installation and offline caching mechanisms. Subsequently, pnpm [26] innovatively adopted hard links to share dependencies, not only improving installation speed but also greatly reducing disk space usage. Research shows that in large projects, pnpm can significantly improve dependency installation efficiency.

2.2.5 The Rise of Monorepo Architecture

As application scales grew, the traditional approach of maintaining multiple related projects in separate repositories exposed numerous problems, such as complex version management, difficult code sharing, and low build efficiency. Monorepo architecture emerged in response, unifying multiple related projects under a single code repository [27]. Practices by tech giants like Google and Facebook have proven that Monorepo can significantly improve development efficiency and code quality.

Early Monorepo tools (such as Lerna, Nx) solved basic multi-package management needs but still had shortcomings in performance and usability. Developers needed additional configuration and tools to handle dependency management, build optimization, and other issues.

2.2.6 Bun: A Unified Full-Stack Development Solution

In 2022, the release of Bun [28] marked a major breakthrough in JavaScript development toolchains. As a newly designed JavaScript runtime, Bun not only significantly outperforms Node.js in performance but, more importantly, provides a unified solution to address many pain points in front-end development.

Bun's innovation is reflected in multiple aspects: First, it can directly execute TypeScript code without a separate compilation step, greatly simplifying the development process. Second, Bun natively supports both ESM and CommonJS module systems, fundamentally solving module compatibility issues. Third, its built-in build tool is several times faster than traditional build tools [1]. Additionally, by integrating test runners, bundling tools, and other functions, Bun nearly eliminates the dependency on external build tools, significantly simplifying project configuration.

Notably, until early 2025, Bun's production environment support became mature and stable. This timing provided an opportunity for our project to practice new technology. By adopting Bun in the PhotoGlow project, we not only validated its feasibility in production environments but also fully utilized its development efficiency improvements. Practice has proven that after adopting Bun, the project's overall development efficiency improved significantly.

These performance improvements and development experience enhancements make Bun an important force in driving Web development paradigm innovation. It not only solves many historical issues in front-end development but also simplifies the development process through a unified toolchain, representing the development direction of modern Web development tools.

2.3 Evolution of Modern Web Development Paradigms

As Web application complexity continues to increase, traditional JavaScript direct DOM manipulation development methods can no longer meet the needs of modern Web applications. Developers need more structured and maintainable solutions to build complex user interfaces. This has driven the birth and development of modern front-end frameworks while also spawning new rendering strategies and styling solutions.

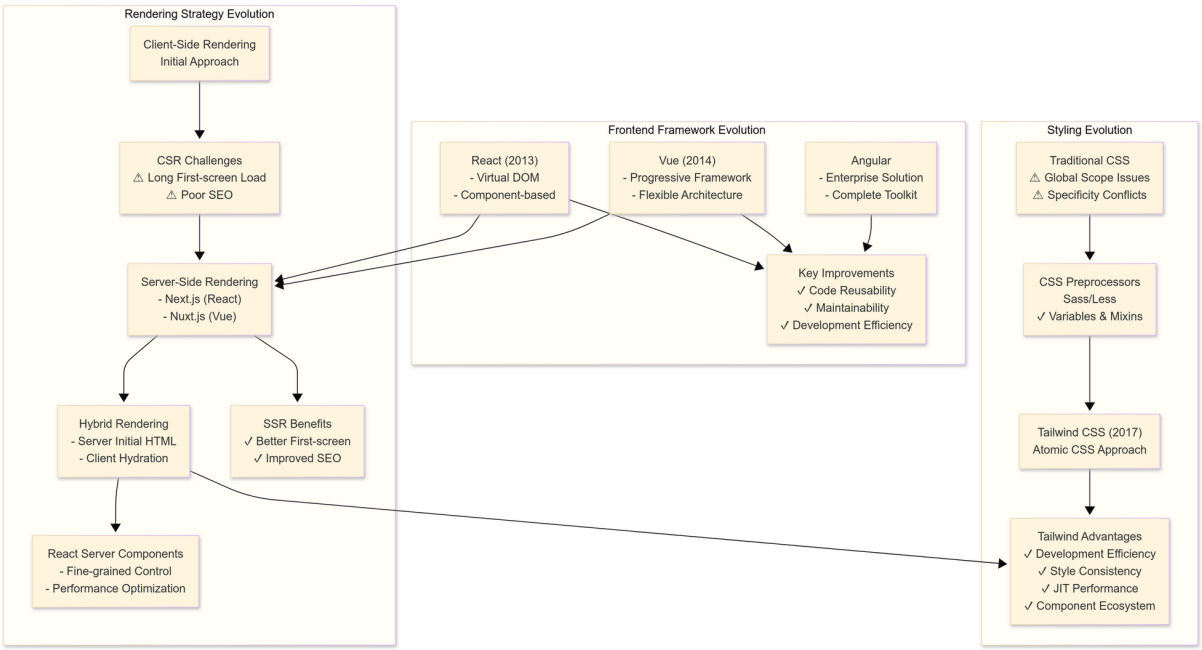


Figure 6: Dev Evolution

2.3.1 Front-End Framework Innovation

Between 2013 and 2015, three mainstream front-end frameworks emerged: Angular **angular?**, React [29], and Vue [30]. These frameworks fundamentally changed the way Web development is done. React, open-sourced by Facebook in 2013, introduced revolutionary concepts of virtual DOM and component-based development. Vue was released in 2014, providing a more progressive development experience. Angular, as Google's complete solution, provided a comprehensive toolkit for enterprise application development.

The emergence of these frameworks solved several core problems in traditional Web development. First, they improved code reusability and maintainability through componentization. Second, the introduction of virtual DOM significantly improved development efficiency.

2.3.2 The Revival of Server-Side Rendering

However, pure client-side rendering frameworks also brought new challenges. Long first-screen loading times and Search Engine Optimization (SEO) difficulties became major pain points. To solve these problems, Server-Side Rendering (SSR) technology was revitalized. Full-stack frameworks like Next.js (React ecosystem) [31] and Nuxt.js (Vue ecosystem) [32] emerged, providing complete server-side rendering solutions.

The innovation of these frameworks lies in integrating data fetching and rendering logic into the same project. Developers can write data fetching logic directly in page components, and the framework executes this logic on the server side to generate complete HTML. This approach not only optimizes first-screen loading performance but also significantly improves SEO effectiveness. Applications adopting server-side rendering have achieved notable improvements in first-screen loading time and search engine indexing efficiency [33].

2.3.3 Hybrid Rendering: Evolution of Best Practices

Practice has shown that pure server-side rendering is not a perfect solution either. It can lead to increased server load [33] and user interaction responsiveness is not as good as client-side rendering. Therefore, Hybrid Rendering [34] gradually became industry consensus. This approach generates initial HTML on the server side to optimize first-screen loading and SEO, followed by "hydration" on the client side to become a fully interactive application.

React Server Components [35] introduced in Next.js 13 pushed this concept to new heights. It allows developers to precisely control the rendering location of each component, achieving more fine-grained

performance optimization. Applications adopting hybrid rendering strategies have achieved significant results in both user experience and performance.

2.3.4 Evolution of Styling Solutions

In terms of styling development, Web development has gone through an evolution from original CSS to preprocessors (like Sass [36], Less [37]), and then to atomic CSS frameworks. Early CSS faced problems such as global scope pollution and selector specificity conflicts. The emergence of Sass and Less brought features like variables and mixins, improving the maintainability of style code, but did not fundamentally solve these problems.

In 2017, the release of Tailwind CSS [38] marked a major shift in styling development paradigms. It adopted the concept of atomic CSS, breaking styles down into minimal functional classes. This approach was initially questioned, being considered not fundamentally different from inline styles. However, Tailwind CSS quickly proved its advantages:

1. Development Efficiency: Through predefined functional classes, developers can directly combine styles in HTML, achieving a WYSIWYG development experience. Research shows that using Tailwind CSS can significantly improve team styling development speed.
2. Consistency: The predefined design system ensures style consistency, particularly suitable for small teams without professional designers. Through the default theme system, dark mode and theme switching can be easily implemented.
3. Performance: Thanks to its unique JIT (Just-In-Time) engine, Tailwind CSS can generate highly optimized production environment style files.
4. Component Ecosystem: The community has developed numerous open-source component libraries based on Tailwind CSS, such as Headless UI and shadcn/ui [39], providing high-quality reusable interface components.

In the PhotoGlow project, we fully utilized these advantages of Tailwind CSS. Through its powerful theme system, we easily implemented dark mode and custom theme functionality. More importantly, leveraging its WYSIWYG development mode, we could quickly iterate and optimize user interfaces without complex design toolchains. Practice has shown that this development approach significantly shortened the feedback loop between interface design and implementation, improving development efficiency and final product quality.

These advances in modern Web development paradigms provide powerful tool support for building high-quality user interfaces. Hybrid rendering strategies solve the balance between performance and user experience, while Tailwind CSS simplifies the styling development process. The combination of these technologies enables small teams to efficiently develop professional-grade Web applications. In the PhotoGlow project, our adoption of these best practices fully validated their effectiveness in practical projects.

2.3.5 Architecture Choice Trade-offs

Although modern full-stack frameworks like Next.js provide the ability to perform database operations directly in page components, this "full-stack unified" approach is not always the best choice in all scenarios. In actual enterprise application development, separating front-end rendering from backend services still has its unique advantages.

When selecting the architecture solution for the PhotoGlow project, after thorough technical evaluation, we ultimately decided to use Next.js as the front-end framework while maintaining an independent Nest.js [40] backend API service. This separated architecture design was primarily based on the following considerations:

First, Separation of Concerns [41] is an important principle in software engineering. Independent backend API services make business logic encapsulation clearer and easier to maintain and extend. For example, when we need to provide API support for mobile applications, independent backend services can be reused by different clients without rewriting business logic.

Second, in terms of security, separated architecture allows us to implement more fine-grained access control and security policies. Backend APIs can be independently deployed in internal networks and accessed through gateways for access control, providing an additional security protection layer. Research shows

that layered security architecture can significantly reduce the risk of security incidents when handling sensitive data.

Finally, this architecture also has advantages in team collaboration and development processes. The front-end team can focus on user interface and interaction experience optimization, while the backend team can independently perform service optimization and expansion. This division not only improves development efficiency but also facilitates independent testing and deployment.

2.4 Technology Selection Theory for Social Media Platforms

2.4.1 Balance Between Performance and User Experience

In the technology selection for social media platforms, balancing performance and user experience is the primary consideration. Research shows that increased page loading time significantly affects user experience and bounce rates. For image-sharing social platforms, this impact is even more significant. Therefore, the choice of rendering strategy requires particular caution.

Traditional Client-Side Rendering (CSR), while providing smooth interaction experience, has longer first-screen loading times [33]. In comparison, Server-Side Rendering (SSR) can significantly improve first-screen loading speed [33] but may affect interaction responsiveness. The hybrid rendering solution provided by Next.js effectively solves this problem—static content is quickly rendered through SSR, while dynamic interactive parts are handled on the client side. Practice has shown that this approach effectively balances first-screen loading speed and interaction response time.

2.4.2 Scalability Design Considerations

User growth in social platforms often shows exponential characteristics, requiring technical architecture to have good scalability. Microservice architecture shows clear advantages in this aspect but also brings increased system complexity [42].

At the storage layer, hybrid storage strategies are increasingly favored. Relational databases are used for storing structured data like user profiles, while in-memory databases like Redis are used for handling real-time interaction data. This approach achieved significant success in Twitter's technical transformation, effectively improving information flow query performance [43].

2.4.3 Technical Support for Real-Time Interaction

The real-time interactive features of social platforms place special demands on technical architecture. While WebSocket is the standard solution for implementing real-time communication, it faces connection maintenance challenges in large-scale deployment [44]. Server-Sent Events (SSE) [45] provides a lighter alternative in some scenarios, particularly suitable for one-way real-time notification pushing. Proper use of SSE can significantly reduce server resource consumption.

2.4.4 Technical Guarantees for Security and Privacy

Security design for social platforms needs to be implemented at multiple levels. Beyond basic HTTPS [46] encryption and JWT authentication [47], content encryption for transmission and storage is becoming increasingly important. Research shows that while end-to-end encryption brings certain system overhead, it can significantly increase user trust. To balance performance and security, the industry generally adopts selective encryption strategies—applying end-to-end encryption only to private messages and sensitive data.

In terms of content security, real-time content moderation combined with artificial intelligence has become the mainstream solution. This includes not only image recognition and text analysis but also consideration of protection against new threats like deep fakes. An effective content moderation system can quickly handle violating content while maintaining a low false positive rate.

2.5 Summary

Through deep consideration and trade-offs in these dimensions, we have chosen technical solutions for the PhotoGlow project that can both meet current needs and possess future expansion potential. These choices will be detailed and justified in subsequent chapters.

Through reviewing the co-evolution of Web technology and social media, we can clearly see how technological innovation has driven the transformation of social platform forms.

From early static pages to modern rich interactive applications, from single JavaScript scripts to complete full-stack development solutions, Web technology has continuously broken through its own limitations while providing increasingly powerful technical support for social media platforms.

Particularly noteworthy are the recent innovations in development toolchains, rendering strategies, and architectural design, which have provided new possibilities for building high-performance, scalable social platforms.

These technological advances not only optimize development efficiency but more importantly can provide better user experiences.

Based on in-depth analysis of these technology development trends, we will next discuss in detail the system design solution for the PhotoGlow project, demonstrating how to practically apply these advantages of modern Web technology in actual development.

3. System Design and Implementation

System design is the crucial phase that transforms modern web technology theories into practical solutions. The PhotoGlow project adopts a layered architecture based on microservices principles, building a high-performance, scalable cloud-based photo sharing and social media platform through thoughtful technology selection and module organization.

3.1 System Architecture and Implementation

PhotoGlow adopts a modern system architecture based on microservices principles, organizing and managing multiple related but independent service components through a Monorepo (single code repository) approach.

The project uses Bun as a unified package management and monorepo management tool, which not only provides execution speeds 3 times faster [1] than traditional Node.js/npm but also achieves near-instant dependency installation and updates.

This architectural design ensures both excellent code organization and management capabilities while guaranteeing system scalability and maintainability. This section will detail the overall system architecture design.

3.1.1 Overall Architecture Design

PhotoGlow's system architecture employs a layered design, using Bun's workspace functionality to manage the following core components:

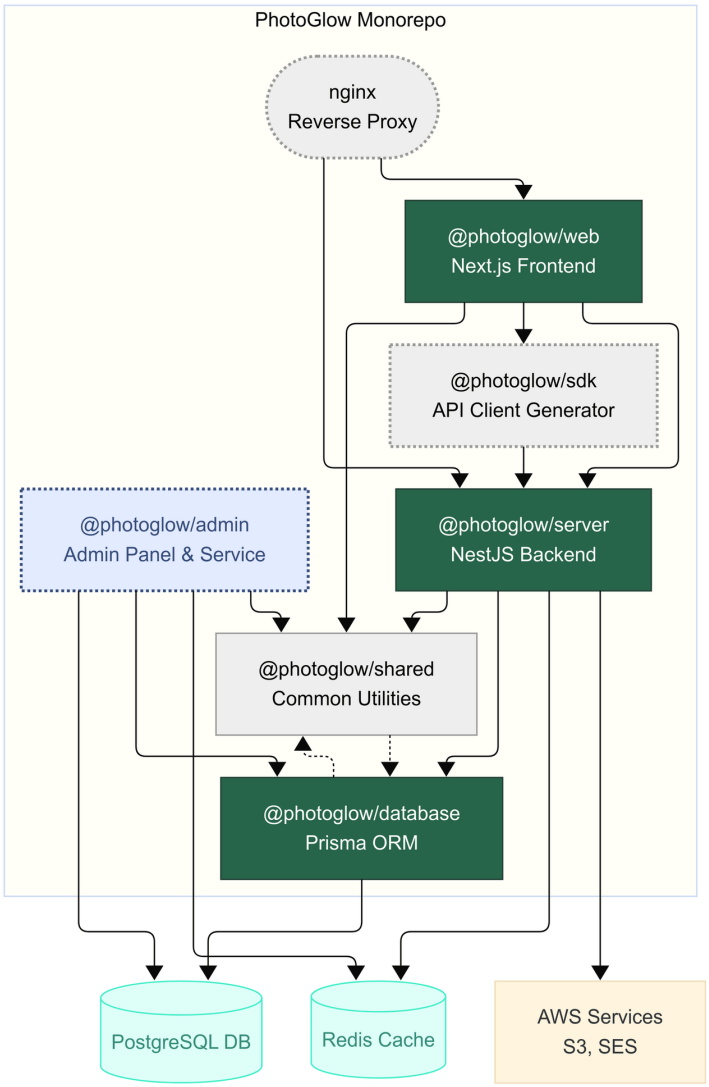


Figure 7: Monorepo

1. **Frontend Application Layer (`@photoglow/web`)**
   - User interface built with Next.js 15 and React 19
   - Hybrid rendering strategy combining server-side rendering (SSR) and client-side rendering advantages
   - Beautifully designed UI with excellent UX and page performance
   - Modern responsive design using TailwindCSS v4
   - Progressive Web App (PWA) features for native-like experience
2. **Backend Service Layer (`@photoglow/server`)**
   - API service built with Nest.js 11
   - Core business logic and data processing with complete type safety
   - RESTful API and WebSocket real-time communication support
   - Third-party cloud service integration (AWS S3 storage, email services, etc.)
   - Redis for caching and session management
   - Comprehensive Open API and Swagger documentation
   - Complete data validation
3. **Data Persistence Layer (`@photoglow/database`)**
   - Database access layer based on Prisma ORM
   - PostgreSQL 16+ as primary data storage

- Type-safe database operations
- Database modeling and relationship definition using Prisma Schema
- Database version management and automated migration support
- Database seeding and reset functionality

4. **Gateway Layer (`@photoglow/nginx`)**
    - Nginx as reverse proxy and load balancer
    - SSL termination and request routing capability
    - Request rate limiting and security protection
    - Static resource distribution optimization
5. **Admin Dashboard (`@photoglow/admin`)**
    - Independent management system for platform operations and monitoring
    - Same technology stack as main application
    - Independent database and API services
    - Strict access control and audit logging
    - Role-based access control
6. **Shared Module (`@photoglow/shared`)**
    - Cross-service shared utilities and type definitions
    - Environment configuration management
    - Common validation rules
    - Constant definitions
7. **SDK Module (`@photoglow/sdk`)**
    - Auto-generated API client library
    - Multi-language support
    - Type-safe API access
    - Complete API documentation and Postman Schema generation
8. **Containerized Deployment Architecture (Docker)**
    - Service orchestration using Docker Compose
    - Independent containerized environment for each core service
    - Unified environment variable management
    - Automated build and deployment process
    - Independent container network space
    - Horizontal scaling support (migration to k8s)
    - Development and production environment consistency

Each sub-container communicates through Docker networking, forming a complete microservices architecture system.

Each component runs in its own container, ensuring environment isolation and portability.

The system implements flexible configuration management through environment variables and configuration files, making transitions between different environments (development, testing, production) simple and controllable.

3.1.2 Technology Stack Selection

The system adopts the latest released modern web technologies in its technology selection:

1. **Runtime Environment**
    - Bun as primary JavaScript runtime, providing 3x execution speed compared to Node.js
    - Built-in bundler and test runner
    - Near-instant package installation capability
2. **Frontend and Admin Dashboard Frameworks**
    - Next.js 15 (released October 2024) for server-side rendering and routing management
    - React 19 (released December 2024) providing enhanced component system

- TailwindCSS v4 (released January 2025) implementing atomic CSS design
- TypeScript
- ESlint

3. **Backend Framework**
   - Nest.js 11 providing enterprise-level backend architecture
   - Prisma ORM implementing type-safe database access
   - Socket.IO supporting real-time communication functionality
   - TypeScript
   - ESlint
4. **Data Storage**
   - PostgreSQL 16+ as main database
   - Redis for caching and real-time data
   - AWS S3 for media file storage

3.1.3 Deployment Architecture

The system adopts a containerized deployment strategy, using Docker Compose to orchestrate multiple services:

1. **Container Services**
   - PostgreSQL container: Database service
   - Redis container: Cache service
   - Server container: Backend API service
   - Web container: Frontend application service
   - Nginx container: Reverse proxy service
2. **Network Configuration**
   - Independent Docker network creation (photoglow-network)
   - Inter-service communication through internal network
   - Only Nginx ports 80 and 443 exposed to external network
3. **Security Measures**
   - SSL/TLS encryption implementation
   - Security response header configuration
   - API request rate limiting (5 requests/second)

3.1.4 Communication Mechanism

The system implements multiple communication mechanisms to support data exchange needs in different scenarios:

1. **HTTP/HTTPS Communication**
   - RESTful API for regular data interaction
   - Versioned API design (/api/v1/)
   - JSON format for request responses
2. **WebSocket Communication**
   - Real-time chat functionality
   - Message push system
   - Real-time notifications
3. **Internal Service Communication**
   - Direct communication between services via Docker network
   - Service addresses configured through environment variables
   - Type-safe API calls

This layered architectural design not only ensures system maintainability and scalability but also provides excellent performance through appropriate technology selection.

While adopting the latest released technology stack brings certain technical risks, it also brings significant performance improvements and better development experience.

In subsequent sections, we will discuss the specific implementation of each component in detail.

3.1.5 System Architecture Diagram

To visually demonstrate PhotoGlow's system architecture, the following diagram describes the core components and their interaction relationships:

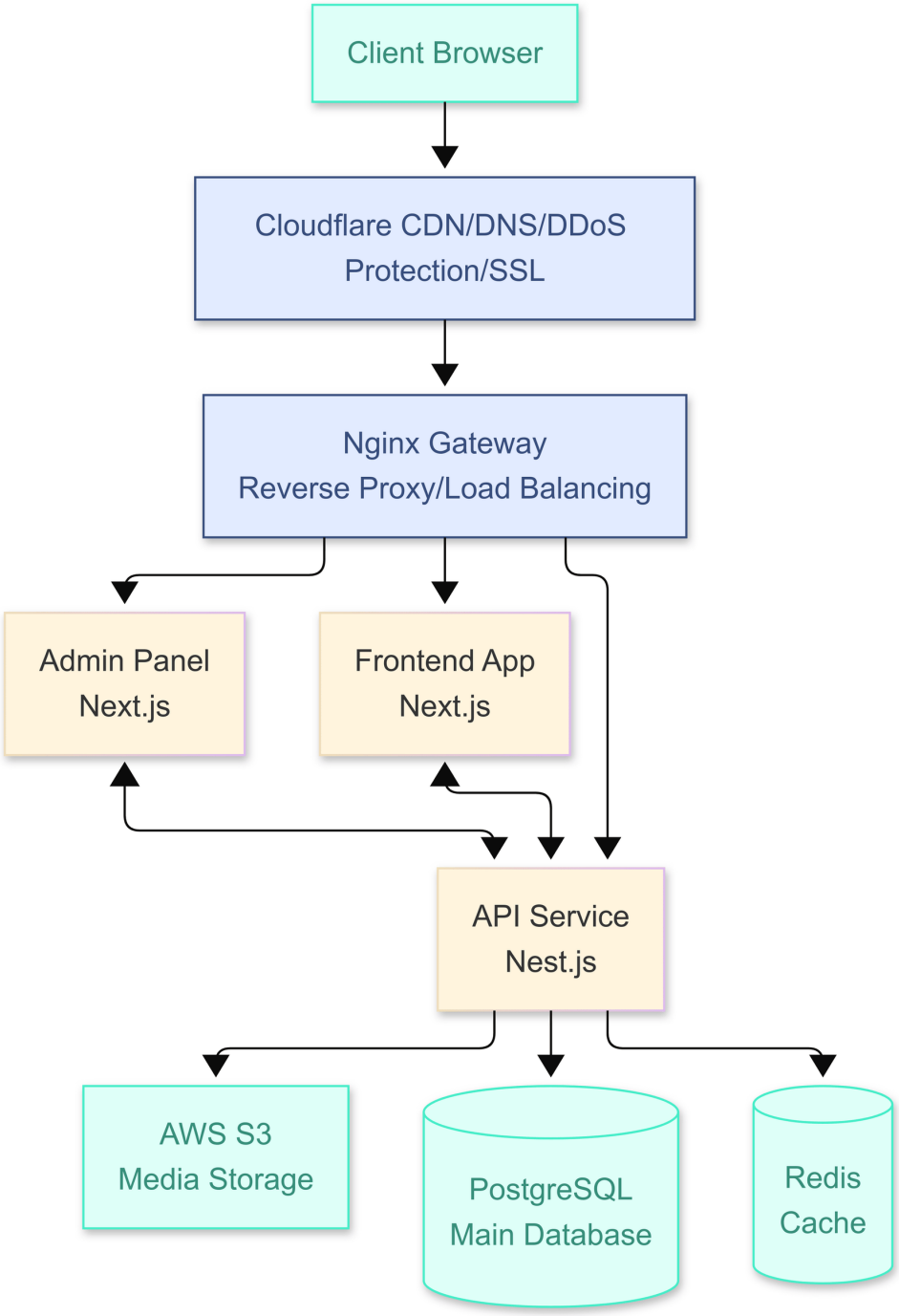


Figure 8: Architecture Diagram

This architecture diagram shows the system's multi-layer structure, clearly presenting the interaction relationships between components from user access entry point to data storage layer.

3.1.6 Data Flow Design

PhotoGlow's data flow design follows clear paths, ensuring efficient communication between system components:

1. **User Content Publishing Flow**
   - User uploads image → Frontend application preprocessing and compression → API service validation → Media file upload to AWS S3 → Metadata storage to PostgreSQL → Related users notification
2. **Content Retrieval Flow**
   - User requests content → Nginx routes request → Next.js server-side renders initial content → API service retrieves dynamic data → Media files load from CDN → Client hydrates into complete interactive application
3. **Real-time Notification Flow**
   - System event trigger → API service processing → WebSocket push to related users → Frontend real-time update

The system implements multi-level caching strategies, including browser cache, CDN cache, application-level cache, and database query cache, significantly improving performance and response speed.

3.1.7 Security Architecture Design

PhotoGlow's security architecture adopts a defense-in-depth strategy, combining implemented core security features with planned security enhancement solutions. This section will detail current security implementations and future security architecture planning.

3.1.7.1 Implemented Core Security Features

3.1.7.1.1 Authentication and Authorization Mechanism This system implements a dual-token JWT (JSON Web Token) authentication strategy, achieving an optimal balance between security robustness and user experience. The architecture employs a sophisticated token management approach that enhances both security measures and operational efficiency.

The authentication mechanism consists of two primary components: the access token and the refresh token, each serving distinct purposes in the security framework:

Access Token Implementation The access token operates exclusively in the client browser's memory space, with a deliberately short validity period of one hour. This ephemeral nature provides enhanced security, as the token is automatically invalidated upon page refresh. Rather than utilizing cookies, the system transmits the access token through the Authorization header using the Bearer scheme, which ensures broader platform compatibility, particularly for mobile applications where cookie handling can present technical challenges.

Refresh Token Architecture The refresh token employs a more persistent approach, leveraging HTTP-only cookies set during the initial authentication response. This implementation provides robust protection against Cross-Site Scripting (XSS) attacks, as the browser's security policies prevent JavaScript-based access to HTTP-only cookies. Furthermore, the refresh token is persistently stored in the database, enabling comprehensive session tracking and allowing users to manually terminate active sessions when necessary.

Authentication Flow Process The system implements a sophisticated authentication flow:

1. Upon successful authentication, the server responds with an access token in the JSON payload while simultaneously setting the refresh token as an HTTP-only cookie.
2. The client-side application interprets the presence of an access token as a valid authentication state, triggering appropriate UI updates through conditional rendering.

3. During page reloads or subsequent site visits, the system employs a server-side rendering (SSR) optimization strategy. When the initial page load occurs with an expired or absent access token, the server automatically processes the refresh token cookie to generate a new access token.
4. This server-side token refresh mechanism is seamlessly integrated into the HTML response, eliminating the potential for authentication-related UI flicker that commonly occurs in purely client-side implementations.

3.1.7.1.2 Authentication Process Design The system's authentication process encompasses the following key scenarios:

Initial Login Flow

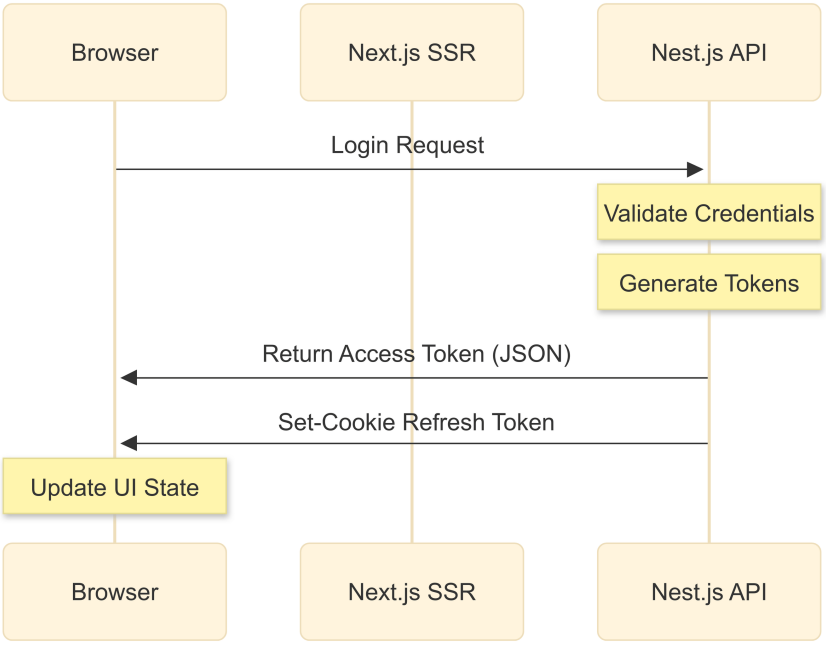


Figure 9: Initial Login Flow

Page Refresh/Revisit Flow

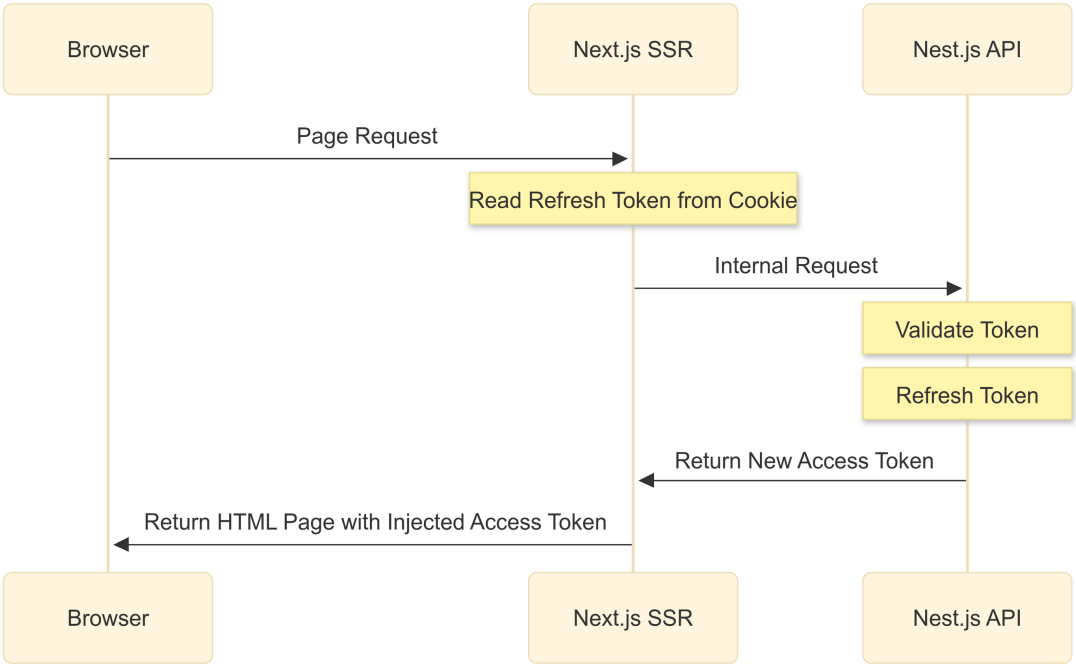


Figure 10: Page Refresh/Revisit Flow

Security Architecture and Implementation The system implements a comprehensive security framework that spans multiple layers, from token management to infrastructure protection. This section details the security mechanisms and their implementation rationale.

Token Security Implementation The authentication system employs a sophisticated dual-token architecture that prioritizes both security and user experience. This implementation is characterized by several key security features:

Token Lifecycle Management
The system implements a differentiated token validity strategy based on exposure risk and usage patterns. The access token, being frequently transmitted and stored in client memory, maintains a restricted one-hour validity period to minimize potential security breaches. Conversely, the refresh token, protected by HTTP-Only cookie constraints, extends to a 30-day validity period, offering enhanced user convenience while maintaining robust security measures.

Session Management Infrastructure
The system maintains comprehensive session control through persistent storage of refresh tokens and associated session metadata in the database. This architecture supports advanced session management capabilities, including:

- Programmatic interfaces for session monitoring and control
- Web-based interface for active session visualization
- Remote session termination functionality
- Device-specific session management

Infrastructure Security Layer The system's security architecture extends beyond token management to encompass comprehensive infrastructure protection:

SSL/TLS Implementation
The system leverages Cloudflare's SSL/TLS encryption services for site-wide HTTPS implementation. This approach offers several advantages over traditional self-managed SSL certificates:

- Automated certificate lifecycle management, eliminating manual renewal processes
- Enhanced reliability through automated certificate rotation
- Advanced security features including TLS 1.3 support and HSTS policy implementation

- Reduced operational complexity while maintaining high security standards

API Access Control Architecture
The system implements a multi-layered API security framework:

1. **Network Isolation**
    - Strategic port exposure limited to HTTP (80) and HTTPS (443)
    - Internal service isolation through Docker network segmentation
    - Administrative services restricted to internal network access
2. **Administrative Interface Security**
    - Complete isolation of admin API endpoints within Docker internal network
    - Server-side rendering architecture for enhanced security validation
    - Comprehensive middleware-based permission verification

File Upload Security Implementation
The system employs a dual-layer file upload security mechanism:

1. **Gateway-Level Control**
    - Configurable Nginx upload size limitations through client_max_body_size directives
    - Graceful handling of oversized file uploads with proper error responses
2. **Application-Level Validation**
    - Granular file size verification through middleware implementation
    - Business logic-driven upload constraints
    - Resource consumption protection mechanisms

Data Security Planning

a) **Password Security**

Our implemented solution leverages the built-in Argon2i algorithm in the Bun runtime for password encryption. This choice offers significant technical advantages, as Argon2i has been specifically engineered for password hashing with exceptional security characteristics. The algorithm demonstrates robust protection against GPU-based cracking attempts and brute force attacks through its memory-hardness and computational complexity mechanisms.

The security guarantees provided by this implementation are comprehensive. Even in the event of a database breach, attackers would be unable to reverse-engineer users' original passwords. Additionally, unauthorized database access by internal personnel cannot reveal password plaintext. All password verification processes are conducted exclusively server-side, eliminating the need to transmit password hashes over the network.

b) **Input Validation and Data Access Control**

The system has successfully implemented comprehensive input validation and data access control mechanisms. The technical implementation utilizes Nest.js's Data Transfer Objects (DTOs) for request data validation and transformation, while integrating zod for rigorous type validation and data sanitization. Fine-grained data access control is achieved through Prisma's middleware capabilities.

These implementations have demonstrated effective results in practice. All API requests undergo strict validation before entering business logic processing. The system ensures users can only access data within their permission scope, effectively preventing unauthorized data access and injection attacks. Type safety is maintained consistently throughout the entire request handling pipeline.

Security Monitoring Planning

a) **Logging System**

We recommend implementing a centralized log collection system, with ELK Stack being the primary technology consideration. The monitoring focus should be placed on detecting abnormal access patterns and authentication failures.

b) **Alert Mechanism**

The proposed solution includes establishing a multi-level alert system with automated response triggers based on threat severity levels. The alert notification system should incorporate multiple communication channels, including email and SMS notifications.

3.1.8 Implementation Challenges and Solutions

1. **Authentication System Challenges**

   The system addresses authentication challenges through careful design decisions in both access and refresh token implementations, focusing on security, usability, and performance. The implementation includes features such as rate limiting, secure token storage, and comprehensive session management.

2. **Request Security Challenges**

   Request security is managed through a comprehensive rate-limiting strategy implemented using Redis for distributed rate limiting. The system enforces a general limit of 5 requests per second per IP address, with more stringent limitations on critical APIs. Additionally, the system includes automatic IP blocking for detected anomalous behavior.

3. **Input Validation Challenges**

   Input validation is handled through multiple security layers including:

   - Mandatory server-side validation for all user inputs
   - Runtime type checking implementation using zod
   - Special character escaping and HTML tag filtering
   - Strict file upload type and size restrictions

This implementation ensures a robust security architecture while maintaining system performance and user experience.

3.2 Frontend Design and Implementation

The frontend application `@photoglow/web` leverages cutting-edge web technologies to deliver a modern, high-performance user experience. This section provides a detailed examination of the frontend's design philosophy, technology selection, and key implementations.

3.2.1 Design Goals and Principles

The frontend design centers on user experience while emphasizing development efficiency and code quality. The primary design objectives include:

1. **Performance Priority**

The performance strategy focuses on delivering optimal user experience through strict metrics:

- First contentful paint controlled within 1.5 seconds
- Interaction response time capped at 100ms
- Smooth animation delivery at 60fps
- Optimized resource loading strategies

2. **Maintainability**

Code maintainability is ensured through robust development practices:

- Rigorous TypeScript type checking implementation
- Modular component architecture
- Unified code style standards
- Comprehensive documentation and code comments

3. **User Experience**

The user experience design emphasizes:

- Responsive layouts adapting across multiple devices
- Smooth state transitions

- Intuitive interaction feedback
- Seamless offline support capabilities

4. **Security**

Security measures encompass:

- Frontend sensitive information protection
- Cross-site scripting (XSS) prevention
- Cross-site request forgery (CSRF) protection
- Comprehensive input validation

3.2.2 Technology Stack Selection

The technology stack selection was based on careful consideration of various factors:

1. **Runtime Environment - Bun**

Bun was selected for its significant advantages:

- Offers over 3x execution speed compared to Node.js
- Integrated build toolchain simplifying development workflow
- Native TypeScript support
- Built-in test runner and package manager

2. **Core Framework**

The core framework selection prioritizes modern capabilities:

- **Next.js 15**
  - App Router providing intuitive routing organization
  - React Server Components enabling granular rendering optimization
  - Built-in performance optimization toolchain
  - Comprehensive SSR support
- **React 19**
  - Enhanced concurrent rendering
  - New Hooks API implementation
  - Improved server component support
  - Optimized state management capabilities

3. **State Management - Zustand**

Zustand was chosen for its balanced features:

- Lightweight architecture
- Clean API design
- Superior TypeScript integration
- Performance advantages over Context API

4. **UI Framework and Styling**

The UI layer implements modern solutions:

- **TailwindCSS v4**
  - Atomic CSS methodology
  - JIT engine support
  - Powerful responsive design capabilities
  - Minimal production bundle size
- **shadcn/ui**
  - High-quality components based on Radix UI
  - Accessibility compliance
  - Theme customization capabilities
  - Modular design with on-demand importing

5. **Tool Chain**

The development toolchain includes:

- **React Hook Form**: High-performance form handling
- **Zod**: Runtime type validation
- **Framer Motion**: Professional-grade animation effects
- **Socket.IO**: Real-time communication support

This comprehensive technology stack enables the development of a robust, performant, and maintainable frontend application.

3.2.3 Architecture Design

Directory Structure The project's codebase is organized using the following directory structure, which has been carefully designed to promote maintainability, scalability, and ease of navigation:

```
web/
  src/                    # Source code directory
    api/                 # API integration layer
      endpoints/        # API endpoint definitions
        auth.ts       # Authentication endpoints
        chat.ts       # Chat related endpoints
        comments.ts # Comment management
        interactions.ts # User interactions
        media.ts      # Media handling
        mentions.ts # User mentions
        moderation.ts # Content moderation
        notifications.ts # User notifications
        posts.ts      # Post management
        search.ts     # Search functionality
        settings.ts # User settings
        social.ts     # Social interactions
        topics.ts     # Topic management
        user.ts       # User management
      index.ts         # API client configuration

    app/                # Next.js app directory
      chat/           # Chat feature
      explore/        # Content exploration
        [topicName]/ # Topic-specific pages
      following/      # Following feed
      notifications/ # Notifications center
      profile/        # User profiles
        [username]/  # User-specific pages
          photo/   # Photo viewing pages
      search/         # Search interface
      settings/       # User settings
      upload/         # Content upload
      context/        # React contexts
      error/          # Error handling

    components/         # React components
      auth/           # Authentication components
      chat/           # Chat components
      common/         # Shared components
      explore/        # Exploration components
      following/      # Following components
      linkify/        # Text linking components
      nav/            # Navigation components
      notifications/ # Notification components
      post/           # Post-related components
      profile/        # Profile components
      search/         # Search components
```

```
        settings/      # Settings components
        text-editor/   # Rich text editor
        theme/         # Theme management
        ui/            # UI components library
        upload/        # Upload components

    hooks/             # Custom React hooks
        auth.ts        # Authentication hooks
        hydrate.ts     # Hydration utilities
        use-onscreen.ts # Visibility detection
        use-socket.ts   # WebSocket hooks

    lib/               # Utility functions
        auth.ts        # Auth utilities
        date.ts        # Date formatting
        regex.ts       # Regular expressions
        utils.ts       # General utilities

    stores/            # State management
        auth.ts        # Auth state
        notifications.ts # Notifications state
        post-dialog.ts  # Post dialog state
        upload.ts       # Upload state
        sidebar-store.ts # Sidebar state

    styles/            # Global styles
        globals.css     # Global CSS

    middleware.ts      # Next.js middleware

public/              # Static files and assets
.next/               # Next.js build output
node_modules/        # Project dependencies
.vscode/            # VS Code configuration
next.config.ts      # Next.js configuration
tsconfig.json       # TypeScript configuration
postcss.config.mjs  # PostCSS configuration
prettier.config.mjs # Prettier configuration
eslint.config.mjs   # ESLint configuration
components.json     # UI components configuration
package.json        # Project dependencies and scripts
```

Key directories and their purposes:

- `src/api/`: API integration layer
  - `endpoints/`: Organized API endpoints for different features
  - Type-safe API client configuration and utilities
- `src/app/`: Next.js App Router pages and layouts
  - Feature-specific pages (chat, explore, profile, etc.)
  - Dynamic routes for topics and user profiles
  - Global layout and error handling
- `src/components/`: React component library
  - Feature-specific components organized by domain
  - Common UI components and shared utilities
  - Comprehensive UI component library in `ui/`
  - Navigation and layout components
  - Rich text editor implementation

- `src/hooks/`: Custom React hooks
  - Authentication and authorization hooks
  - Real-time communication via WebSocket
  - Utility hooks for common patterns
- `src/lib/`: Utility functions and helpers
  - Authentication utilities
  - Date formatting and manipulation
  - Regular expressions and validation
  - General purpose utilities
- `src/stores/`: State management
  - Authentication state
  - UI state (dialogs, sidebar)
  - Feature-specific states (notifications, uploads)
- `src/styles/`: Global styling
  - Global CSS and theme definitions

State Management Architecture Implements a layered state management strategy:

1. **Local State**: Component-level useState
2. **Shared State**: Zustand store
3. **Persistent State**: LocalStorage + Zustand persist

3.2.4 Rendering Strategy

The frontend application employs a hybrid rendering strategy to balance initial loading speed and user interaction experience:

1. **Server-Side Rendering (SSR)**
   - Initial page requests are handled by the server
   - Utilizes Server Components for initial data fetching
   - Optimizes first-screen loading performance and SEO
   - Ensures seamless synchronization of authentication states
2. **Client-Side Rendering (CSR)**
   - Subsequent navigation handled on the client side
   - Dynamic content updates
   - Real-time interaction features
   - State management and animation effects
3. **Rendering Optimization**
   - Selective component pre-rendering
   - Dynamic imports and code splitting
   - Image and resource optimization
   - Caching strategies

3.2.5 UI/UX Design

PhotoGlow's user interface design emphasizes intuitiveness, usability, and code reusability, with all components supporting light/dark mode switching. Below are the main components and their functional descriptions:

3.2.5.1 Theme Design In this project, theme design is a core component of the user interface visual experience. We have implemented a theme system based on Tailwind CSS and Shadcn/ui, creating an aesthetically pleasing and functional user interface through carefully designed color schemes, typography rules, and component styles.

3.2.5.1.1 Color System The project's color system is based on the OKLCH [48] color space, a perceptually uniform color model that provides more accurate color expression and better accessibility. The color system includes the following key elements:

Base Colors

We have defined a series of base color variables to build the visual hierarchy of the interface:

- **Background**: Provides the foundation layer for content, pure white (`oklch(1 0 0)`) in light mode and dark (`oklch(0.141 0.005 285.823)`) in dark mode.
- **Foreground**: Primarily used for text and icons, ensuring appropriate contrast with the background.
- **Primary**: Used for emphasizing important interface elements such as buttons and links.
- **Secondary**: Used for secondary elements and auxiliary information.
- **Muted**: Used for interface elements that don't need emphasis, such as backgrounds and borders.
- **Accent**: Used for special emphasis and interaction feedback.
- **Destructive**: Used for error messages and dangerous operation warnings.

Functional Colors

In addition to base colors, we have defined color variables for specific functions:

- **Border**: Boundary lines for interface elements.
- **Input**: Borders and backgrounds for form elements.
- **Ring**: Highlight effects for focus states and selected items.
- **Chart Colors (chart-1 to chart-5)**: A coordinated color scheme for data visualization.

Dark Mode Adaptation

The project implements complete dark mode support, switching themes via the `.dark` class selector. In dark mode, all color variables are redefined to ensure readability and visual comfort on dark backgrounds:

```
.dark {
  --background: oklch(0.141 0.005 285.823);
  --foreground: oklch(0.985 0 0);
  /* Other dark mode variables are in styles/global.css */
}
```

We use a custom variant `@custom-variant dark (&:is(.dark *))` to simplify dark mode style writing, making the development process more efficient.

3.2.5.1.2 Component Design System The theme design includes not only colors but also a series of unified component style definitions:

Cards and Popups

Defines consistent background colors, foreground colors, and border styles for content containers (such as cards and popups), ensuring visual consistency across different contexts.

Sidebar Customization

Defines dedicated theme variables for the application's sidebar area, including background color, foreground color, and accent color, to visually distinguish it from the main content area:

```
--sidebar: oklch(0.985 0 0);
--sidebar-foreground: oklch(0.141 0.005 285.823);
--sidebar-primary: oklch(0.21 0.006 285.885);
/* Other sidebar-related variables are in styles/global.css */
```

Border Radius and Spacing

Defines unified border radius variables to provide consistent appearance for interface elements:

```
--radius: 0.625rem;
--radius-sm: calc(var(--radius) - 4px);
--radius-md: calc(var(--radius) - 2px);
--radius-lg: var(--radius);
--radius-xl: calc(var(--radius) + 4px);
```

This hierarchical border radius design allows elements of different sizes and importance to express their hierarchical relationships through subtle visual differences.

Theme Variable Mapping

Through the `@theme inline` directive, we have established a complete CSS variable mapping system to maintain consistency of theme colors and styles throughout the application:

```
@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  /* Other variable mappings are in styles/global.css */
}
```

This mapping mechanism provides a unified access method for different components, simplifying the development process and ensuring visual consistency.

3.2.5.1.3 Special Interface Element Handling

Button Styles

To enhance user experience, we apply special style rules to button elements:

```
button:not(:disabled),
[role='button']:not(:disabled) {
  cursor: pointer;
}
```

This ensures that all non-disabled button elements display the correct pointer style, enhancing interface interaction cues.

Text Editor

Defines a dedicated style layer for the text editor component (TipTap), handling special cases such as placeholder text:

```
@layer text-editor {
  .tiptap p.is-editor-empty:first-child::before {
    @apply text-muted-foreground pointer-events-none float-left h-0 content-[attr(data-placeholder)]
  }
}
```

3.2.5.1.4 Summary The project's theme design adopts a modular and variable-based approach, establishing a complete visual design system. Through defining a series of color variables, component styles, and responsive adaptation rules, we have achieved visual consistency and smooth user experience. Additionally, comprehensive support for dark mode enhances the application's adaptability across different environments and user preferences.

This theme design approach not only improves the aesthetics of the user interface but also simplifies the development process, making style maintenance and updates more efficient and controllable. Through the tools and methods provided by Tailwind CSS, we have implemented a user interface design system that is both beautiful and practical.

3.2.5.2 User Interface Overview

Definitions

- Pages: Interfaces managed by the global router, rendered by changing the path in the browser's address bar.
- Dialogs: Non-route transitions, rendered as floating overlays on pages, typically with a mask layer. By default, they do not change the browser's address bar path.

Page components are located in the `web/app` directory and are all server components. The main interfaces are as follows:

3 Basic Interfaces:

- `error`: Rendered when an unknown error occurs
- `404`: Rendered when navigating to undefined routes

- `layout`: Website global framework

11 Main Content Interfaces:

- `/`: Website homepage
- `chat`: User chat page
- `explore`: Topic browsing page
- `explore/[topic name]`: Topic post viewing page
- `following`: Posts from users that the current user follows
- `notifications`: User notifications
- `profile/[username]`: User profile page
- `profile/[username]/photo/[post id]`: Post details
- `search`: Search topics, users, and content
- `settings`: User account settings
- `upload`: Mobile post publishing functionality presented as a page

Dialog components are typically located in `web/components` and are generally client-side components. The main interfaces are as follows:

5 Major Interfaces Presented as Dialogs:

- `auth/auth-dialog`: Login, registration, and password reset functionality
- `post/post-dialog`: Post details displayed when clicking a post card
- `post/post-edit-dialog`: Modify published posts
- `profile/profile-dialog`: Edit username, avatar, and personal introduction
- `upload/upload-dialog`: PC post publishing functionality presented as a dialog

3.2.5.3 Global Framework The `layout` website framework is a global component that forms the website, typically divided into three main parts:

- Top bar
- Sidebar
- Page content area

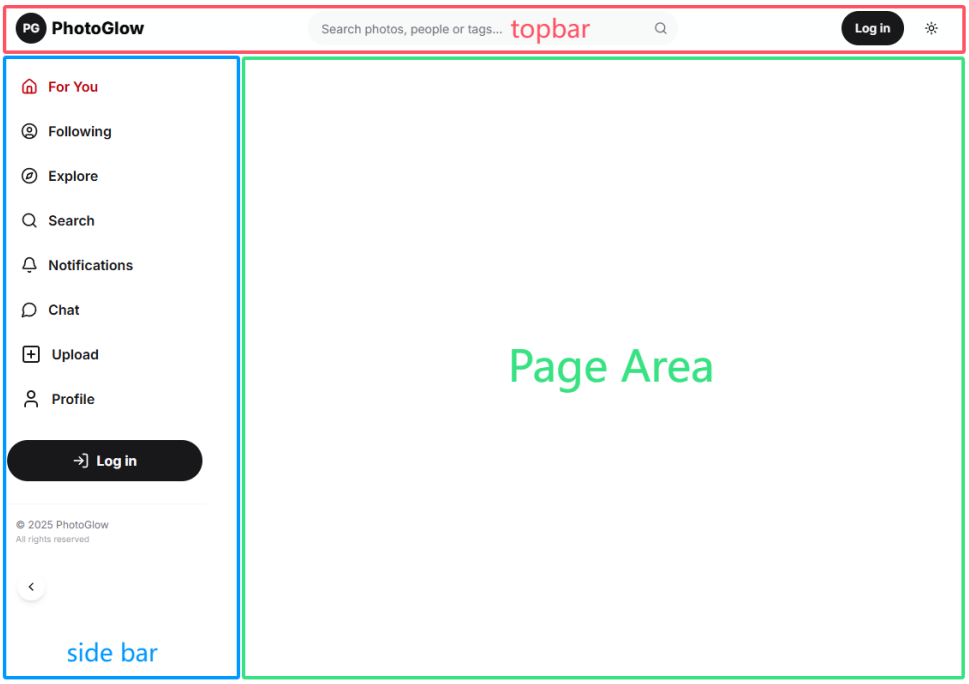


Figure 11: Website Framework View in PC Style

1. **Top Bar**

Features the website logo, search box, login button, and light/dark mode toggle button. Has different styles for logged-in/logged-out states.

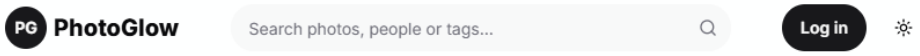


Figure 12: Top Bar in Logged-out State

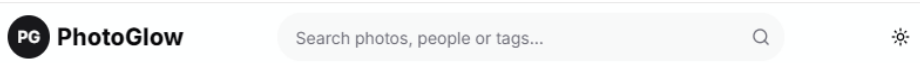


Figure 13: Top Bar in Logged-in State

2. **Side Navigation Bar**

The side navigation bar consists of the following components:

- Navigation menu
- Footer copyright component
- Sidebar collapse button

Navigation menu items highlight the current route in red text. Additionally, the side navigation bar has different styles for logged-in/logged-out states.

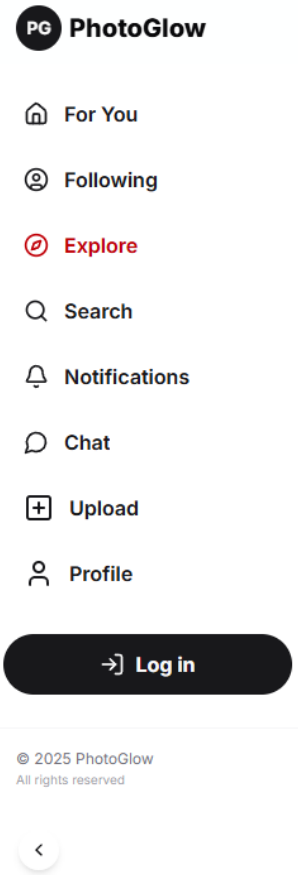


Figure 14: Side Navigation Bar in Logged-out State

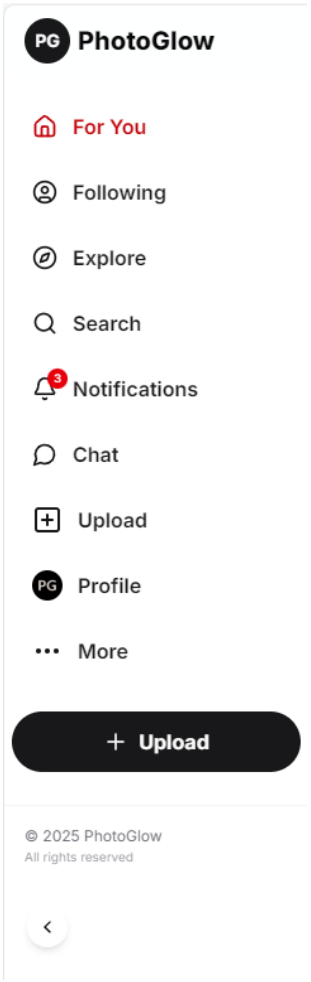


Figure 15: Side Navigation Bar in Logged-in State

Styles and behaviors in logged-in state:

- If there are unread notifications, a floating red bubble displays the number of unread notifications next to the Notifications menu item
- Displays the More navigation item, which shows a dropdown menu when clicked
- Displays a large Upload button instead of the Log in button
- The Profile menu item icon shows the user's avatar instead of the default icon; if no avatar is set, a character avatar is shown

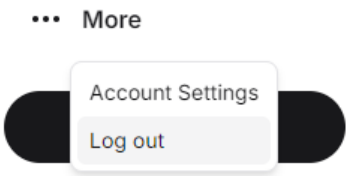


Figure 16: Sidebar - Dropdown Menu

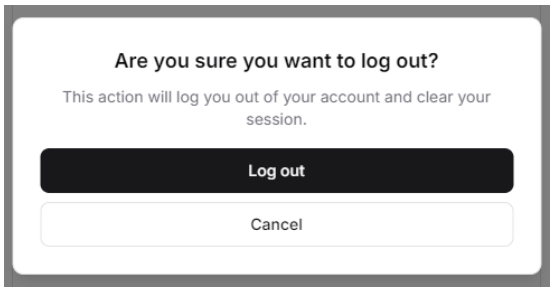


Figure 17: Logout

3. **Content Area** Managed by the global router component, displaying content from the current route's page or the 404 page.

The website framework supports dark/light mode switching and responsive display, with style changes based on device screen size:

a) **Desktop (Greater than or equal to 1024px)**

- Side navigation fully displayed by default
- Other styles at default



Figure 18: Default PC Style

b) **Tablet (768px-1023px)**

- Collapsed sidebar
- Sidebar shows only icons without text

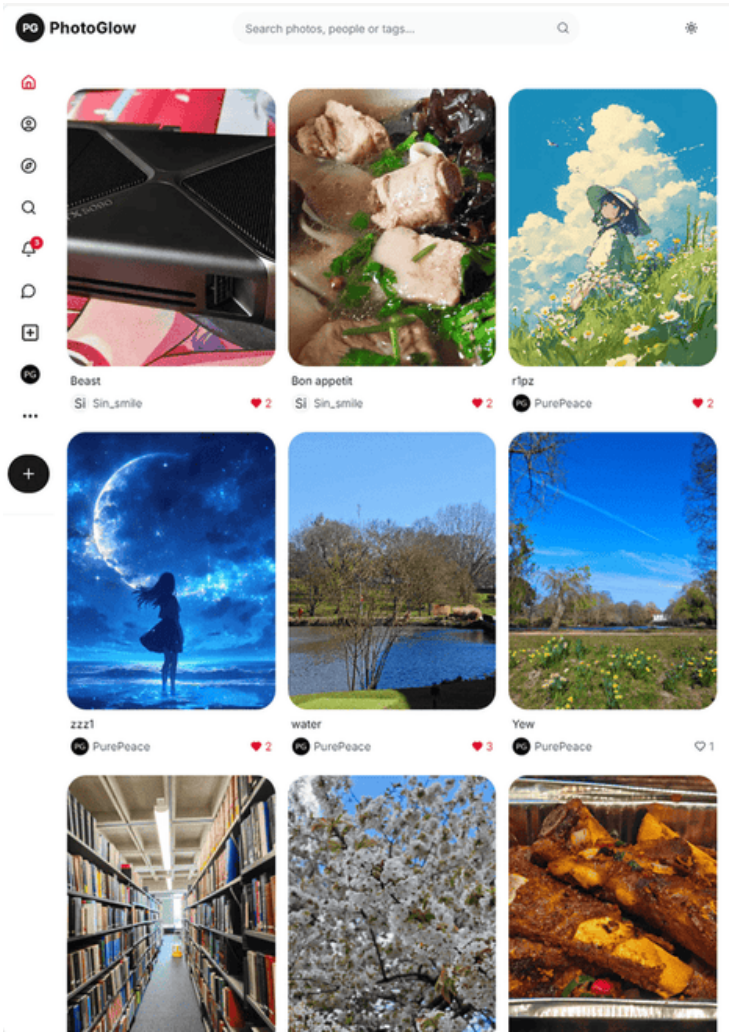


Figure 19: Tablet Style in Logged-in State

c) **Mobile (Less than or equal to 767px)**

- Hidden sidebar
- Hidden top search box and color mode toggle button
- Drawer button and search button displayed at top
- Mobile navigation bar displayed at bottom

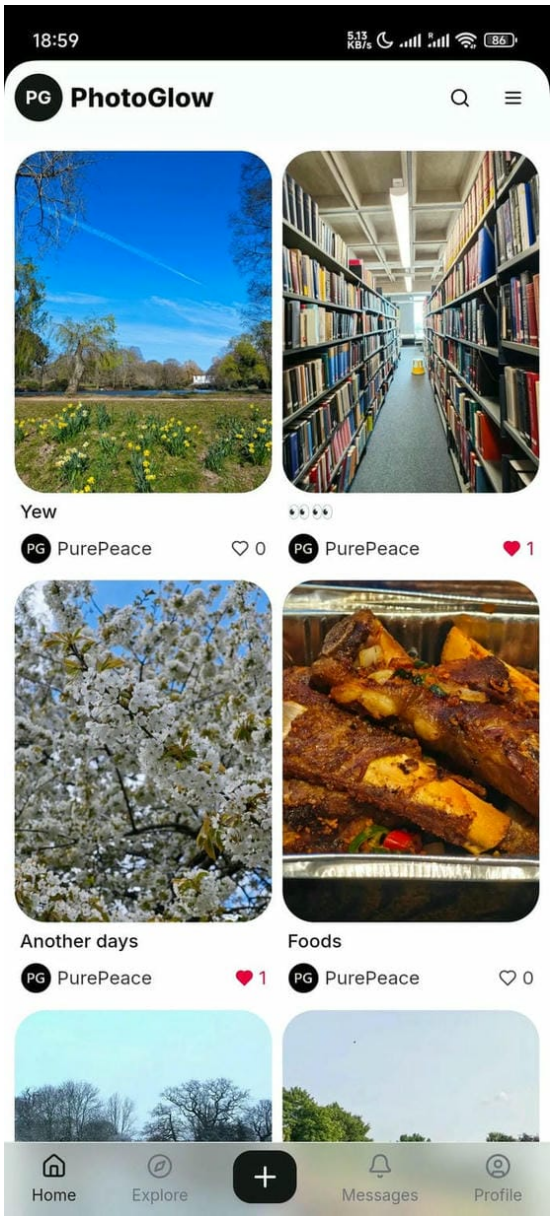


Figure 20: Mobile Navigation Bar Layout

4. **Bottom Navigation Bar**

The mobile bottom navigation bar consists of 5 fixed menu items:

- Home
- Explore
- Upload button
- Notifications
- Profile

When clicking the drawer button on mobile, the drawer menu toggle displays, consisting of:

- User profile button
- Navigation menu
- Theme toggle button
- Logout button

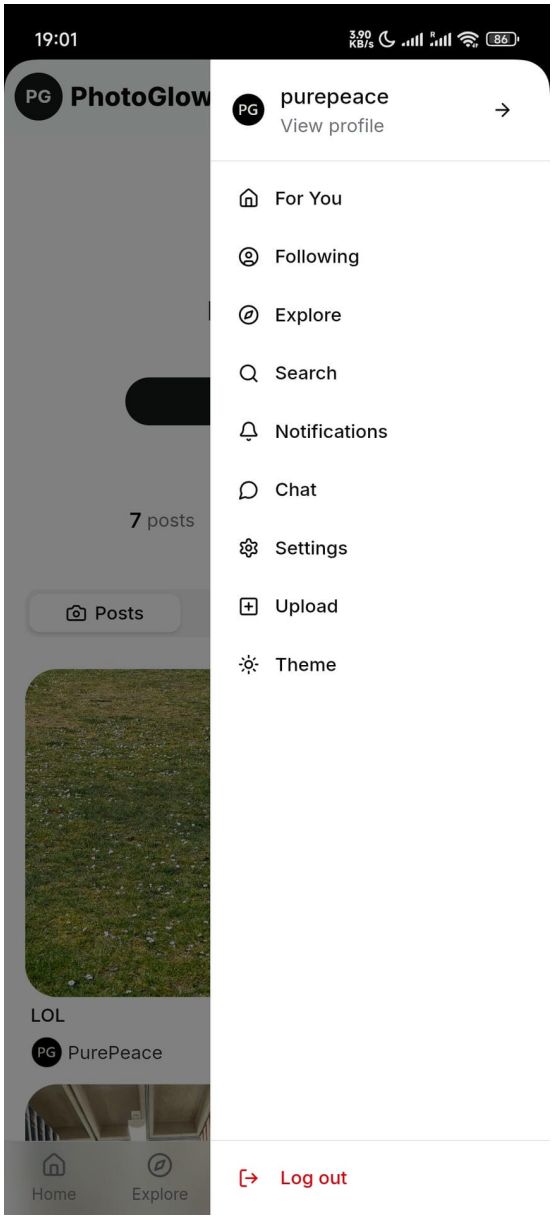


Figure 21: Toggle Drawer Menu

3.2.5.3 Main Page The main page is displayed when users access the website's root domain `/`. It features an infinite-scrolling content card waterfall. As users scroll down the page, new posts continue to load until no more posts are available.

The content card waterfall `post-grid` component uses a grid layout and supports responsive design, automatically adjusting size and display quantity across different device sizes. This waterfall component's design and logic are reused by other pages such as `profile` and `following` to display user posts.

- PC: 6 cards per row
- Tablet: 3 cards per row
- Mobile: 2 cards per row



Figure 22: Main Page - Light Mode



Figure 23: Main Page - Dark Mode

Data Loading Behavior:

- Initial screen data loading occurs on the Next.js server side, providing users with a complete DOM for optimal experience.
- Subsequently, client-side rendering takes over, loading additional post cards as needed.

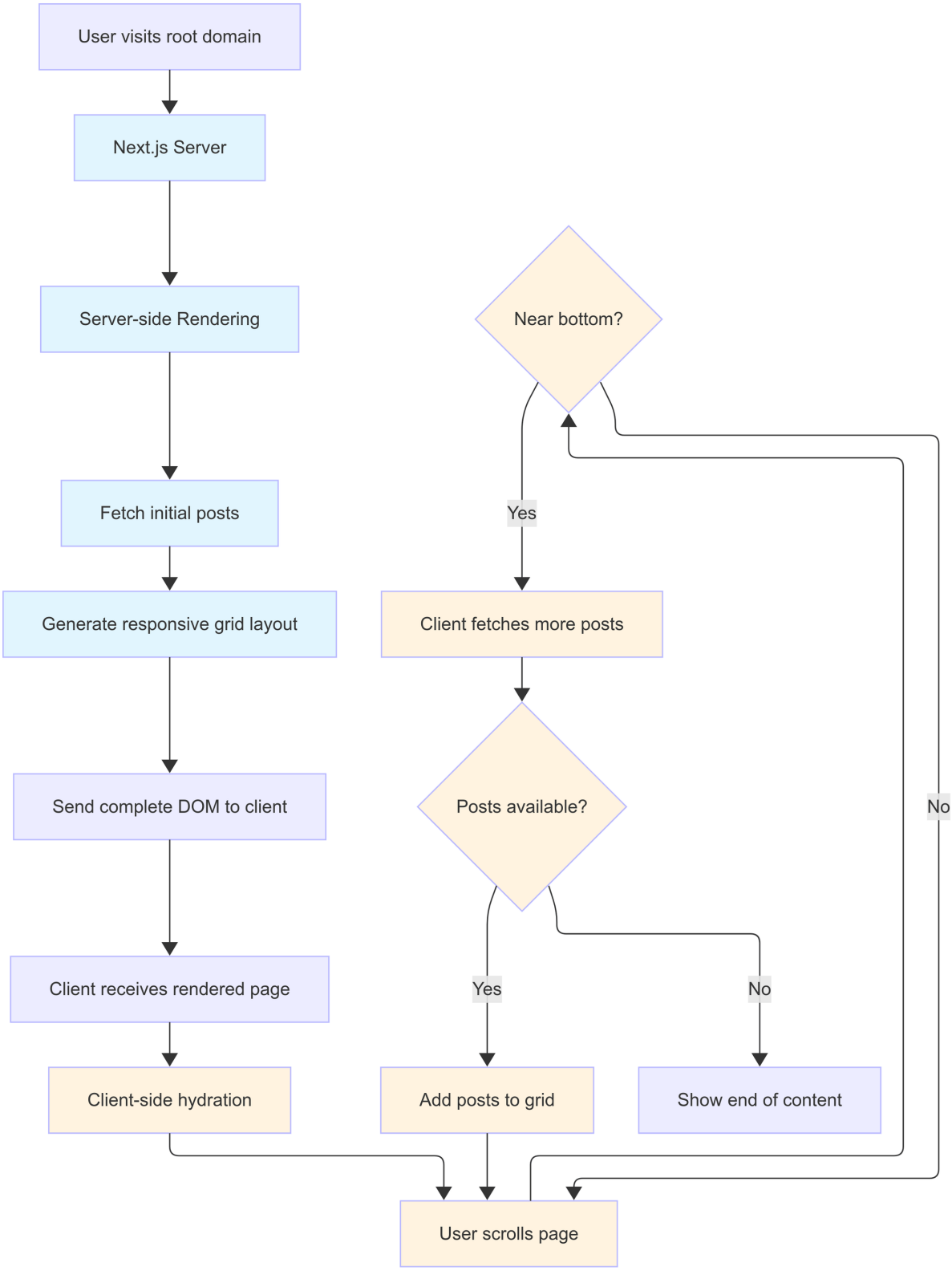


Figure 24: Data Loading Flow Diagram

Content Card Component

The content card `post-card` component is used to display post previews within the content waterfall component. This component is reused by other pages or components rather than duplicating styles.



Figure 25: Post Card

Component breakdown:

- Image section
- Title section
- Author information section
- Interaction buttons/display

Design behaviors:

- Automatically scales based on screen width.
- Card preview images don't load complete post images from AWS S3, but rather use Next.js-optimized and cached .webp covers that load according to device size, providing better performance and bandwidth savings.
- When the user's cursor hovers over an image, it displays a glow effect (photo glow!).
- Clicking the image or post title displays post details.
- When the cursor hovers over the author, it displays a blue link style; clicking navigates to that user's profile page.
- Interaction buttons show the current post's like count and the current user's like status; users can like by clicking the like button.
- If not logged in, clicking the like button prompts for login.

Post detail display behavior:

- On PC and tablet, clicking a card displays post details in dialog form without actual route navigation, supporting keyboard ESC shortcut for closing, providing a good browsing experience.
- On mobile, to accommodate mobile user operations, post details are displayed as route-navigated pages, allowing users to return to the previous level using phone back operation.
- Post comments and more detailed data are loaded asynchronously on the client side when clicking the card, reducing the loading of data users might not browse, improving performance and saving network bandwidth.

3.2.5.4 User Authentication Interface The user authentication interface is presented as a modal dialog `auth/auth-dialog`, containing four main components:

- Email and password login
- Third-party account OAuth login/registration
- Email and password account registration
- Email password reset

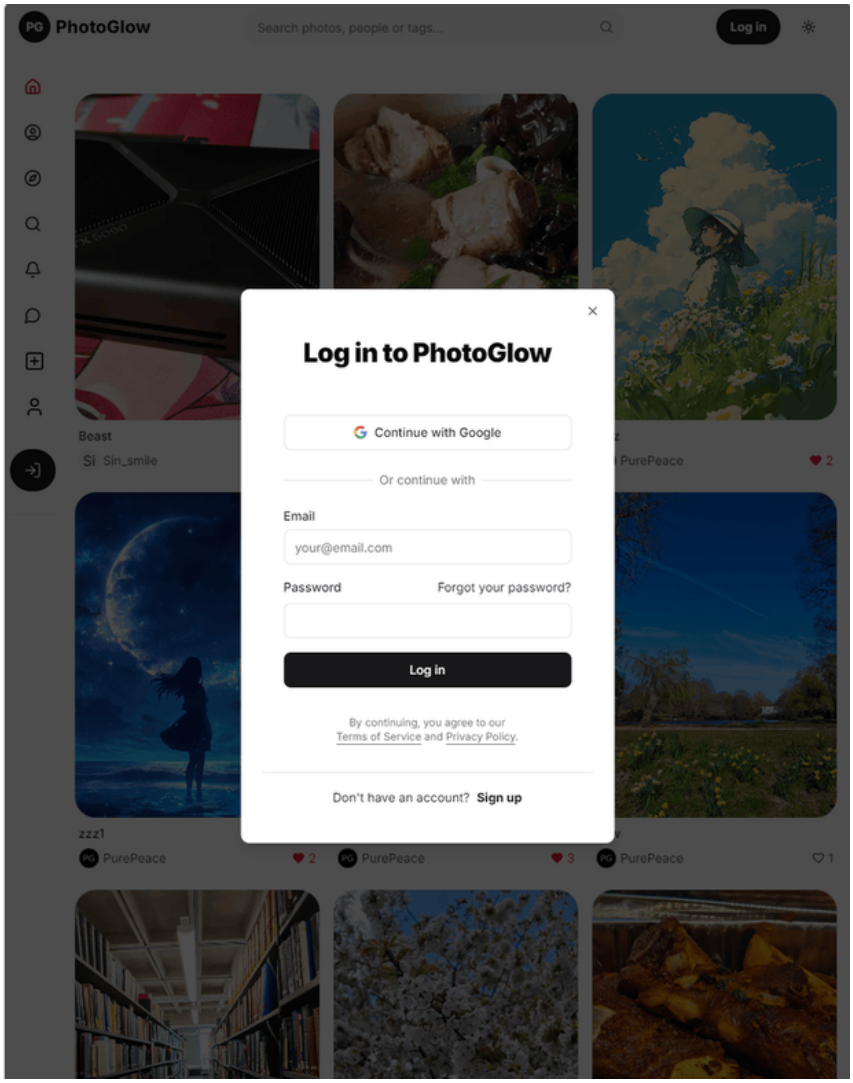


Figure 26: Auth Dialog in Tablet

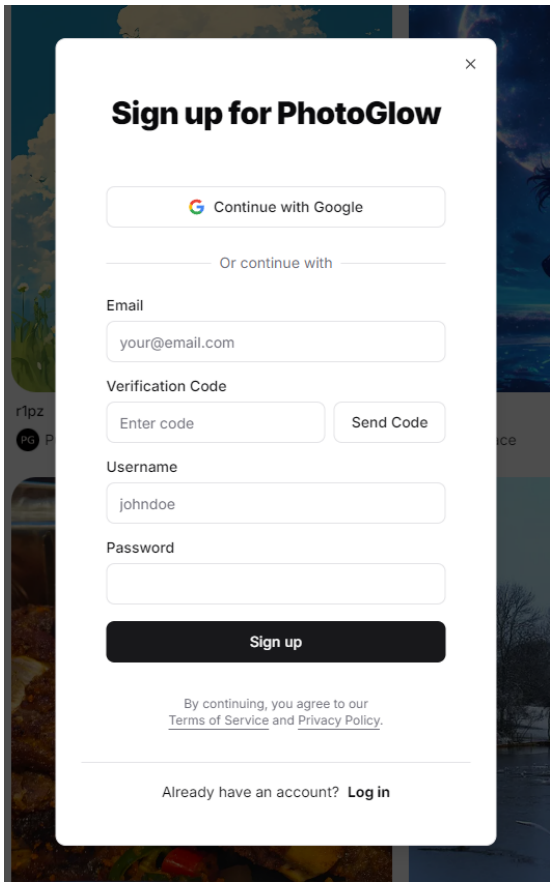


Figure 27: Sign Up Form

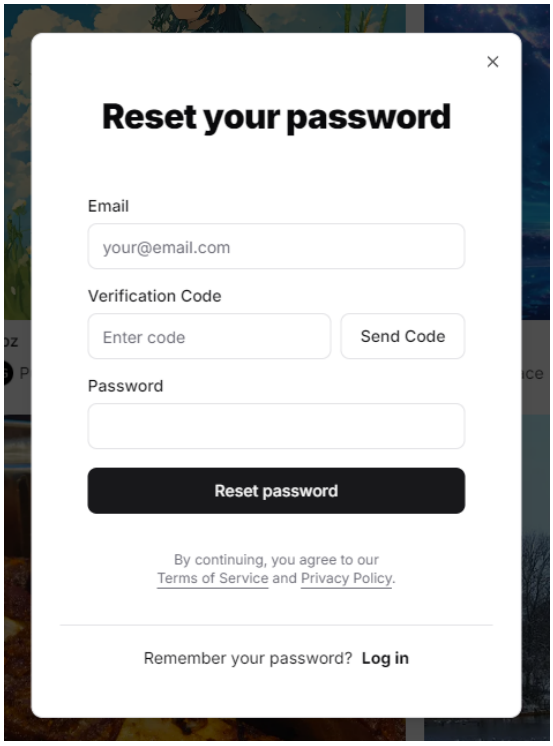


Figure 28: Reset Password Form

Design Behaviors:

- Clicking switch links within the dialog (e.g., register, forgot password) switches dialog content.
- Modal dialog design: After opening the dialog, interaction with other external elements is blocked, and clicking the external mask layer won't close the dialog; must click the close button, preventing accidental closure.
- Responsive design with style differences between mobile and non-mobile versions; mobile version is flatter to save space.
- Third-party OAuth registration/login support.
- Auth dialog has memory functionality: during a single visit, when users reopen the dialog after closing, it remembers their input content, preventing concerns about accidental closure or registration indecision.
- Seamless switching: In login, registration, and password reset forms, form components are reused. For example: if a user enters an email address in the login form but realizes they haven't registered, clicking the register button seamlessly switches the dialog form while retaining the entered email address, avoiding repetitive input - a great experience optimization.
- Form validation: When users click login, register, change password, or other operation buttons, frontend form validation logic runs first, displaying specific problem prompts if not passed.
- Browser native components: Uses native components, thus having browser default behaviors, including form validation prompts and password input field show/hide password text buttons.

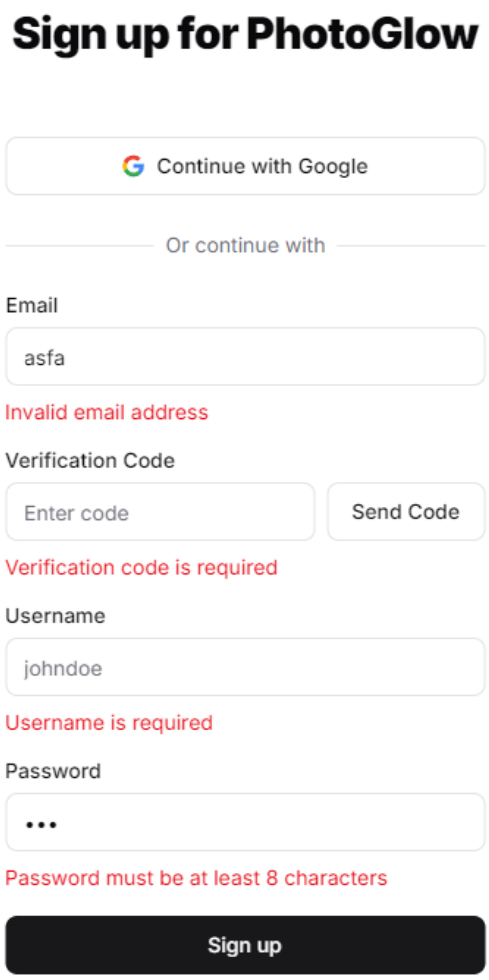


Figure 29: Signup Form Validation

Registration Flow Design:

Users enter their email address to receive a verification code, then can set their username and password for their account, submitting together with the verification code for registration (verification code has

10-minute validity).

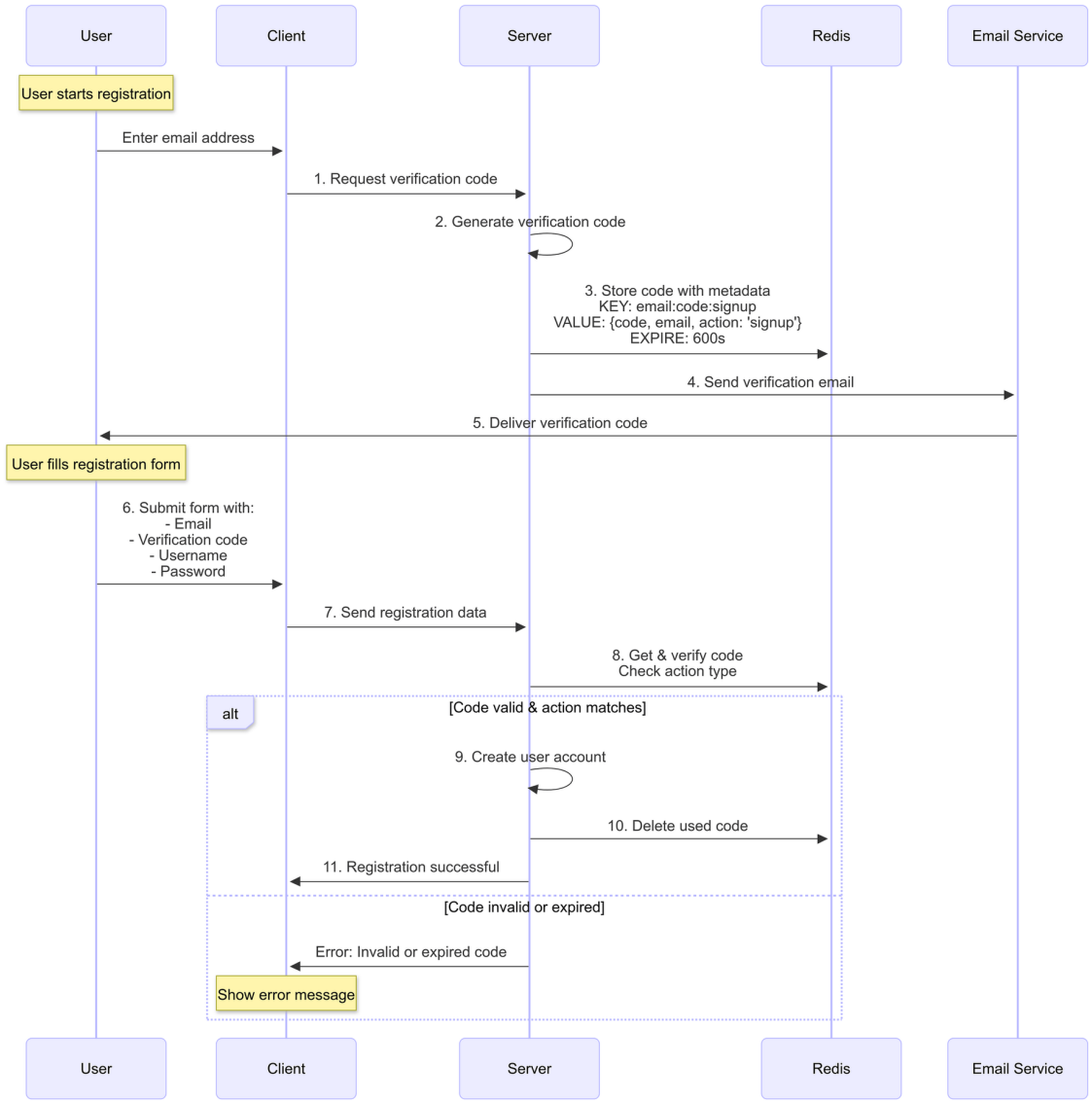


Figure 30: Sign Up Sequence

Password Reset Flow:

Password reset requires entering an email address to receive a verification code, then using the verification code to set a new password.

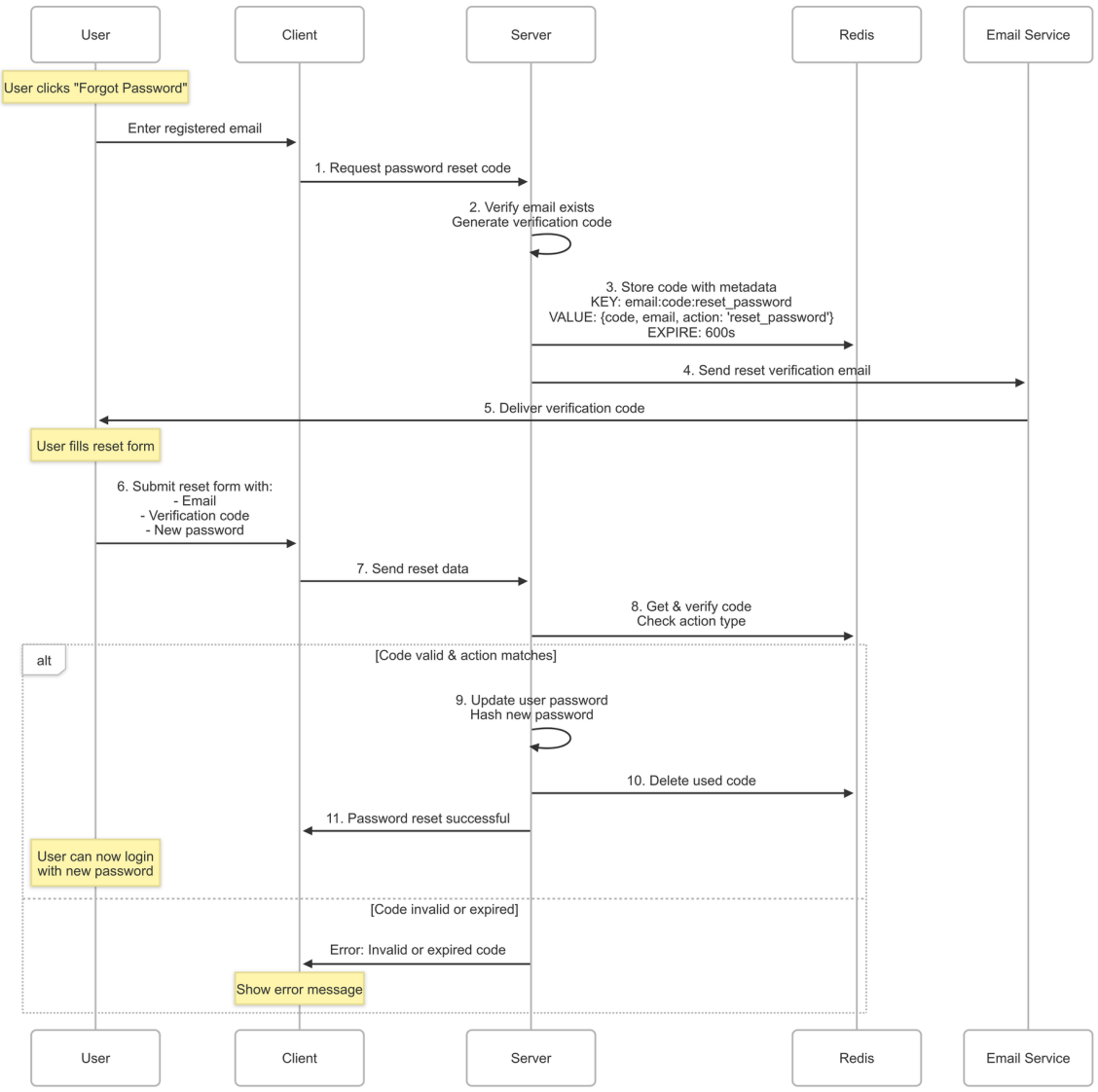


Figure 31: Reset Password Sequence

OAuth Design:

Flow

1. User clicks third-party login button on page, sending request to server API
2. Server redirects to third-party authentication link based on OAuth provider
3. Third-party validates success/failure and calls back to server API
4. Server API completes login/registration/failure logic processing on server
5. Server API writes user's login/registration results back to frontend, achieving seamless login/registration.

Implementing seamless login: For users who have previously completed third-party login authorization and are logged into the third-party account in their browser, clicking the third-party login button instantly completes login, achieving a seamless login user experience.

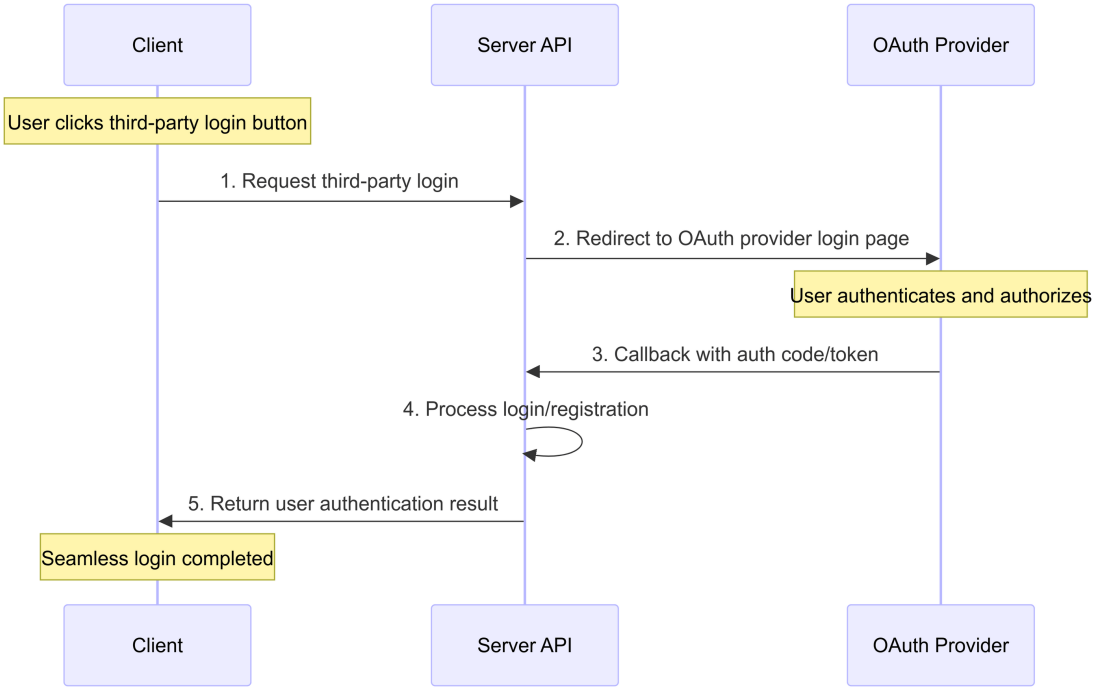


Figure 32: OAuth Flow

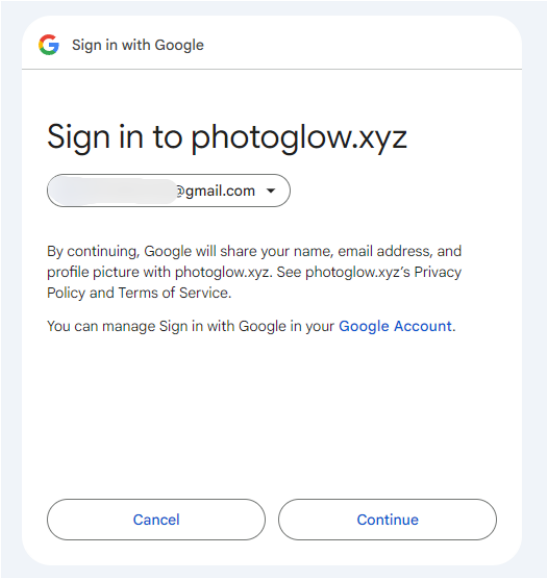


Figure 33: First-time Third-party Login Authentication (Google)

3.2.5.5 Post Detail Interface The post detail interface is a crucial component for displaying post content and comments. This component has two presentation modes - page and dialog - which have different interaction designs but similar basic composition:

- Photo area
- Content area
- Interaction buttons
- Comment area

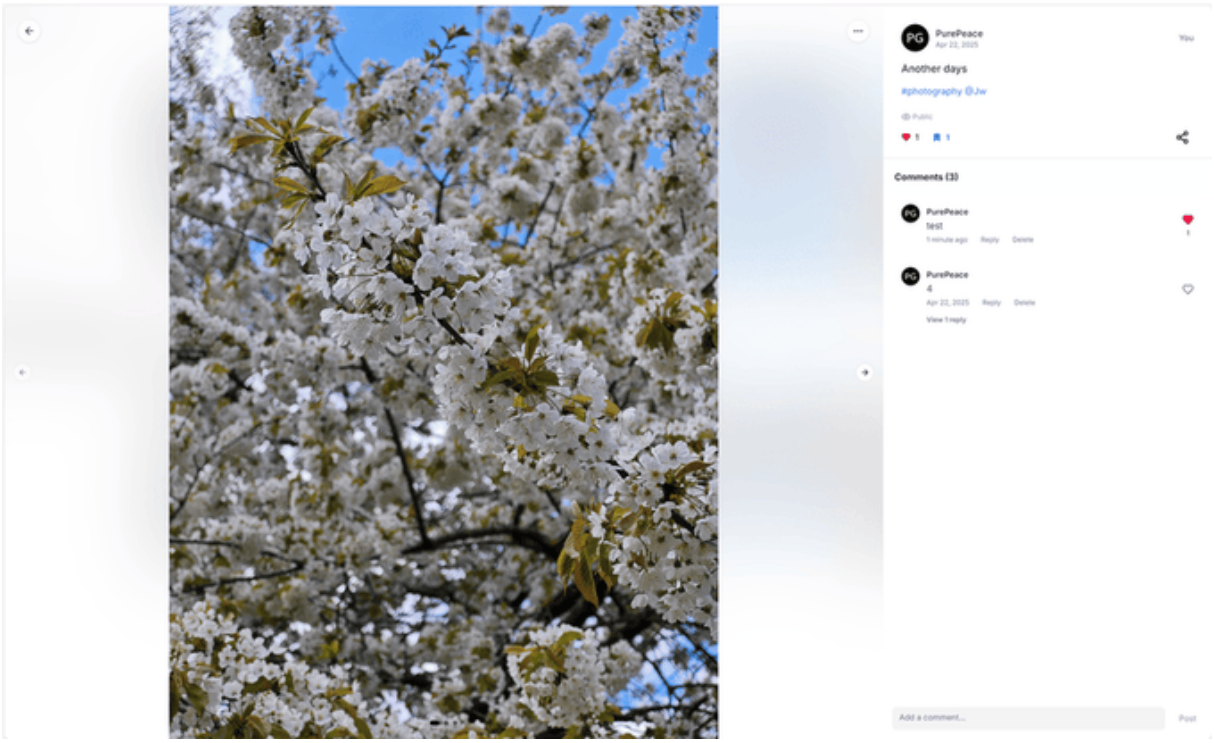


Figure 34: Overview - Post Detail - Dialog Mode

Photo Area:

The photo area can display multiple photos in slideshow format according to the poster's set image order.



Figure 35: Post Detail - Photo Area

Photo area design:

- Photo slideshow: Can switch images by dragging cursor left/right, can click image for original high-resolution display.
- Top left: Close dialog/return to previous page button
- Top right: Dropdown menu button for reporting, sharing. If current user is author, shows share, edit, delete post options.
- Middle sides: If multiple images exist, previous/next image switch buttons appear on middle sides of image.
- Image indicator: Shows when post has multiple images, centered at bottom of image, displays current image position, also allows clicking indicator to switch images.



Figure 36: Post Detail - Photo Area - High-resolution Display Popup

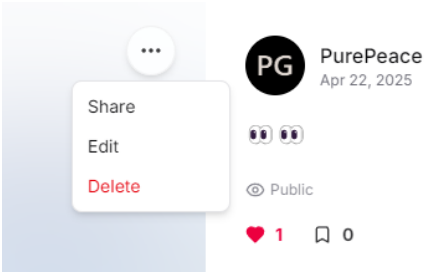


Figure 37: Post Detail - Dropdown Menu - Author View

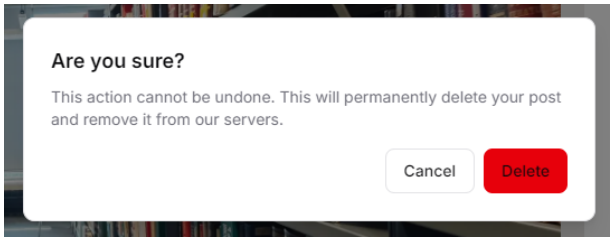


Figure 38: Post Detail - Delete Post Confirmation Dialog

High-resolution Display Popup Design:

- Displays popup and loads high-resolution original image when image is clicked.

- Top left: Zoom in/out/reset button group.
- Top right: Close high-resolution display popup button.
- Can zoom in/out using mouse wheel, mobile gestures.
- Supports screen readers.
- Supports keyboard ESC to close popup.

Post Edit Design:

- Largely reuses content publishing component.
- Can reorder images, delete existing images, upload new images.
- Can modify title, content, including `@` and `#` mentions of users or topics.
- Can modify visibility.
- Detailed design will be covered in `3.2.5.6 Content Publishing Interface`.

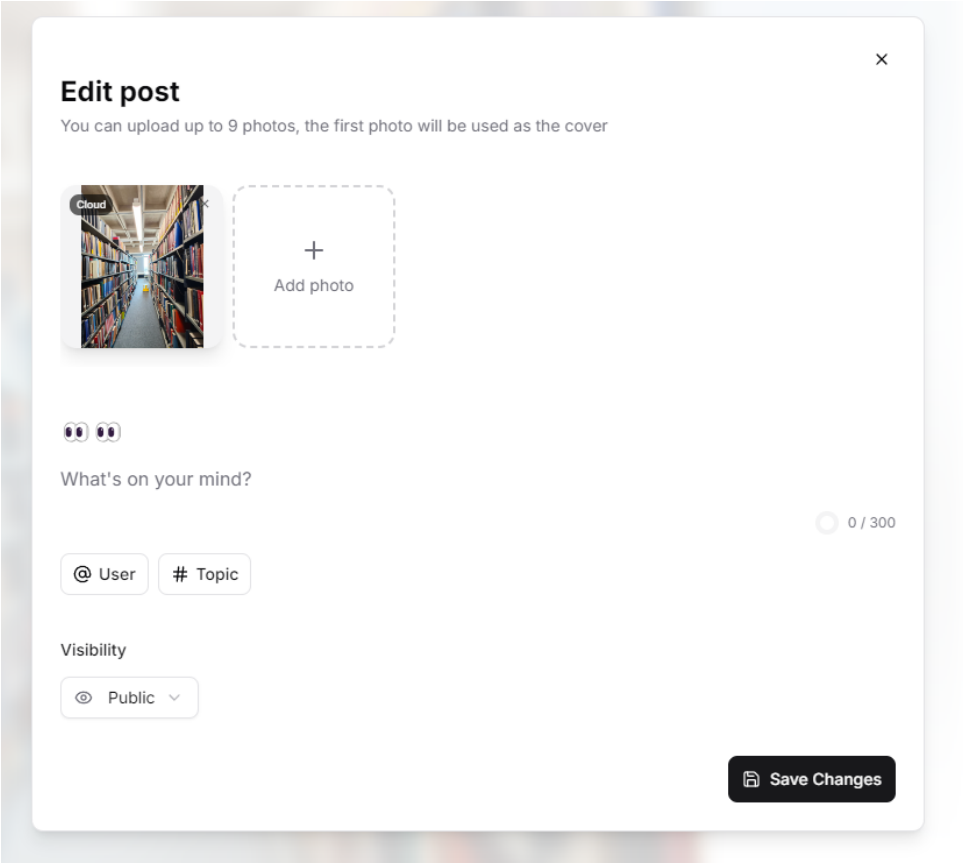


Figure 39: Post Detail - Post Edit

Content Area:

Content area displays post author, post title, post body, and post visibility in top-to-bottom order.



Figure 40: Post Detail - Content Area

Content area design:

- Author display: Shows user's avatar and Display Name, falls back to alternative style if user hasn't set one.
- Timestamp optimization: Shows "time ago" for timestamps within 7 days of local time, shows full date/time beyond that. Can be localized.
- Follow button component shown on author's right side.
- Post body rich text rendering: Renders `@` (user mentions) and `#` (topic mentions) in body as corresponding link styles, users can click these links to jump to corresponding user/topic.
- Post visibility includes: Public, Private, Followers only.

Follow Button Component Design:

- Shows gray "You" label when viewing own post.
- Shows blue "Follows You" when post author follows current user.
- Shows gray followed button when current user follows post author but author hasn't followed back.
- Shows rose-colored "Mutual" when current user and post author follow each other.
- Shows red "Block" when user is blocked by current user.
- This follow button component is reused by other pages/components like `profile` page.



Figure 41: Post Detail - Content Area - Author Follows Current User



Figure 42: Post Detail - Content Area - Current User and Author Follow Each Other



Figure 43: Post Detail - Content Area - Author Blocked by Current User

Interaction Buttons:

Interaction buttons are located at bottom of content area, consisting of three buttons: like, bookmark, share.

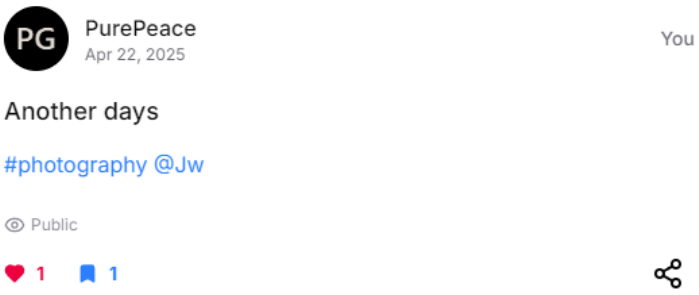


Figure 44: Post Detail - Content Area - Interaction Buttons

Interaction buttons design:

- Shows current post's like and bookmark counts.
- Like and bookmark buttons use rose and blue outline styles respectively.
- When current user has liked or bookmarked, color fills entire icon.
- Share button currently copies post link, can be optimized for complex functionality later (like social media platform sharing).

Comment Area:

Comment area is located below content area, consisting of comment display area and comment publishing area.

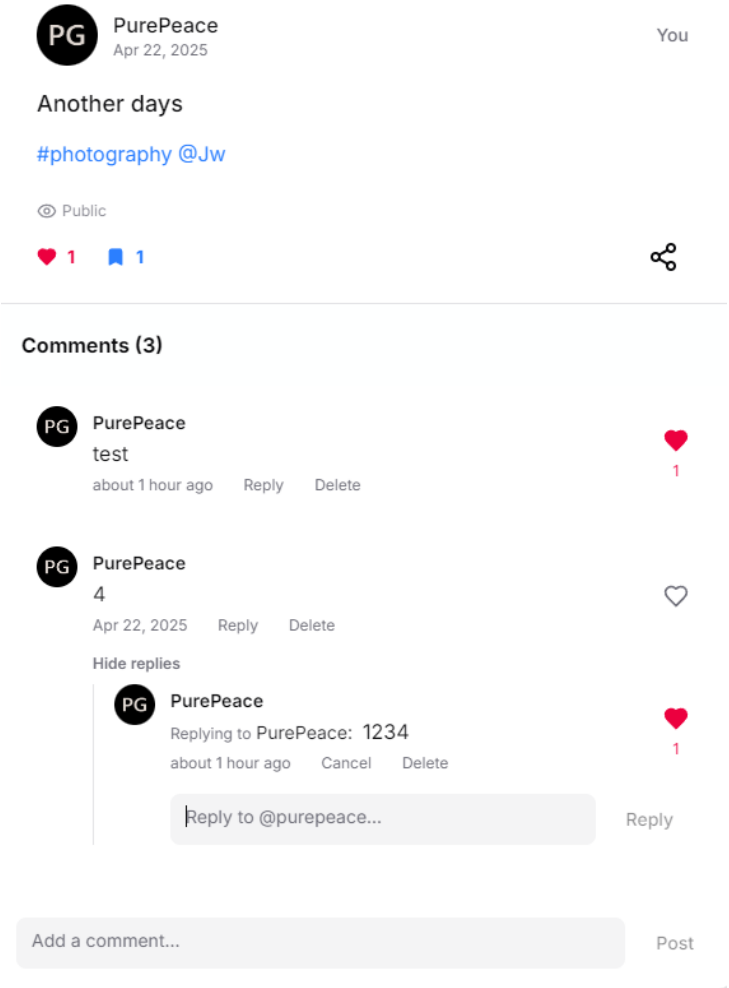


Figure 45: Post Detail - Comment Area

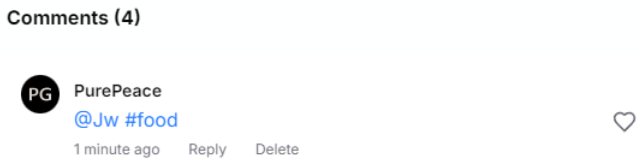


Figure 46: Post Detail - Comment Content Supports Rich Text Rendering

Comment display area design:

- Top shows comment counter displaying total comments on post.
- Timestamp optimization.
- Comment list supports infinite scroll loading until no more comments, showing "No more comments to load".
- Comment content rich text rendering: Renders `@` (user mentions) and `#` (topic mentions) in content as corresponding link styles, users can click these links to jump to corresponding user/topic.
- Supports nested comments, hierarchy: `comments on post -> replies to comments`.
- When comment has replies, shows `View {count} reply` label button, click expands five replies to that comment, can repeat until all loaded, can click Hide to collapse.
- If comment is by current user, shows Delete button, click displays delete confirmation box, confirm completes deletion.
- Clicking Reply button under any comment expands rich text editor box same as comment publishing to post reply to comment.

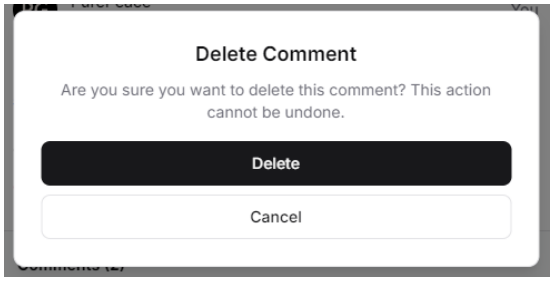


Figure 47: Post Detail - Delete Comment

Comment publishing area design:

- Rich text editor, typing `@` or `#` automatically expands mention suggestion overlay for suggesting users or topics.
- Supports keyboard shortcuts for undo/redo.
- Shows login button when not logged in.

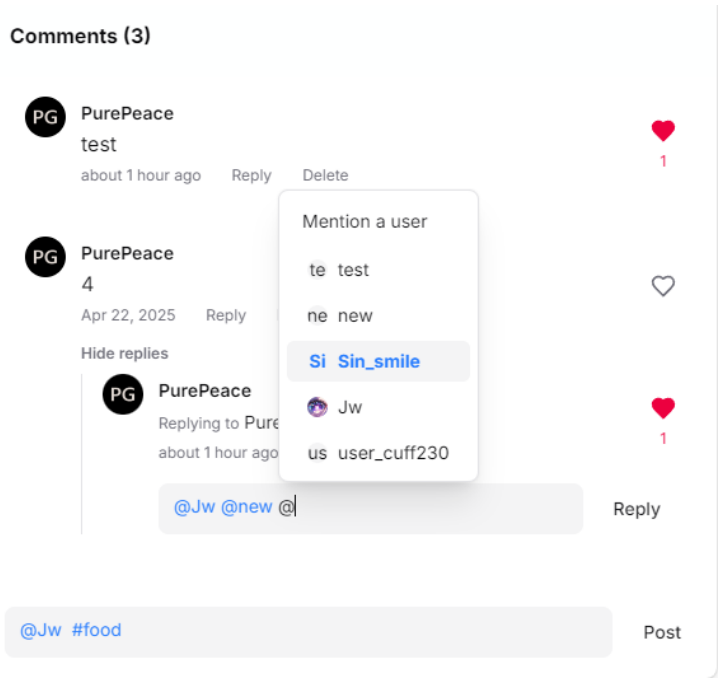


Figure 48: Post Detail - Publish Comment

Dialog Mode:

On non-mobile devices (width greater than 767px), directly clicking any post card displayed in any page will present post content in dialog form.

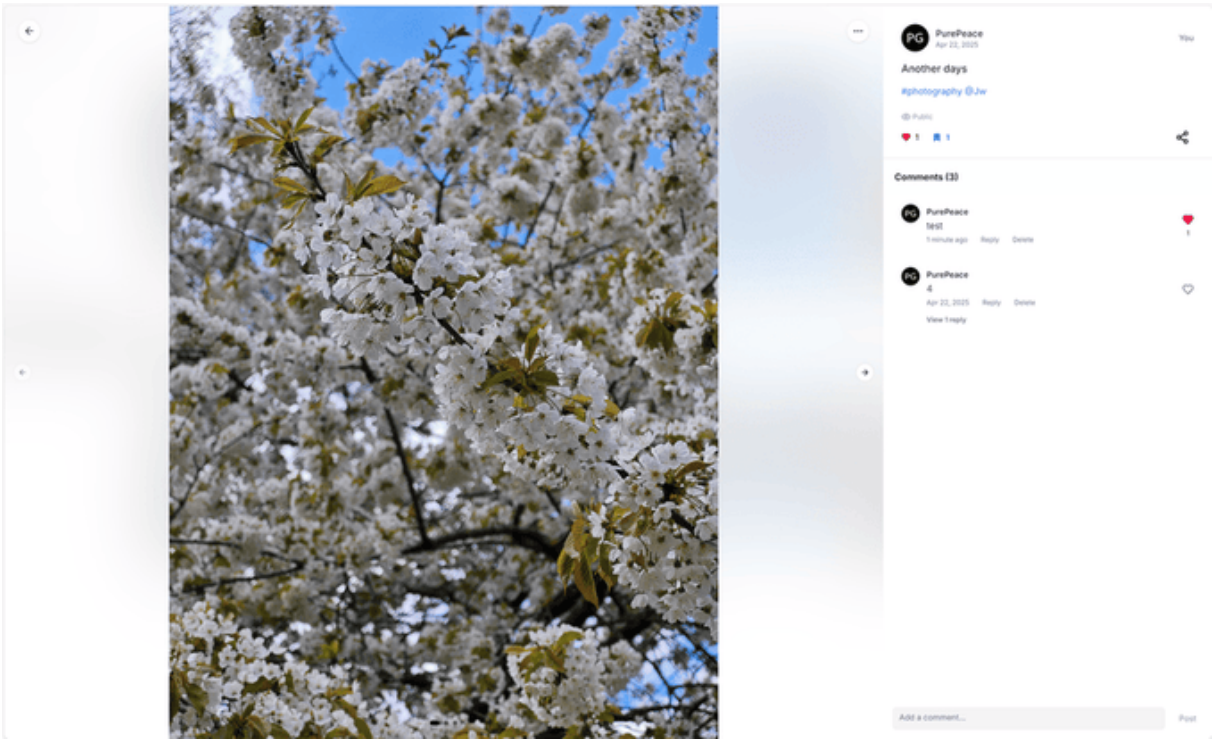


Figure 49: Post Detail - Dialog Mode

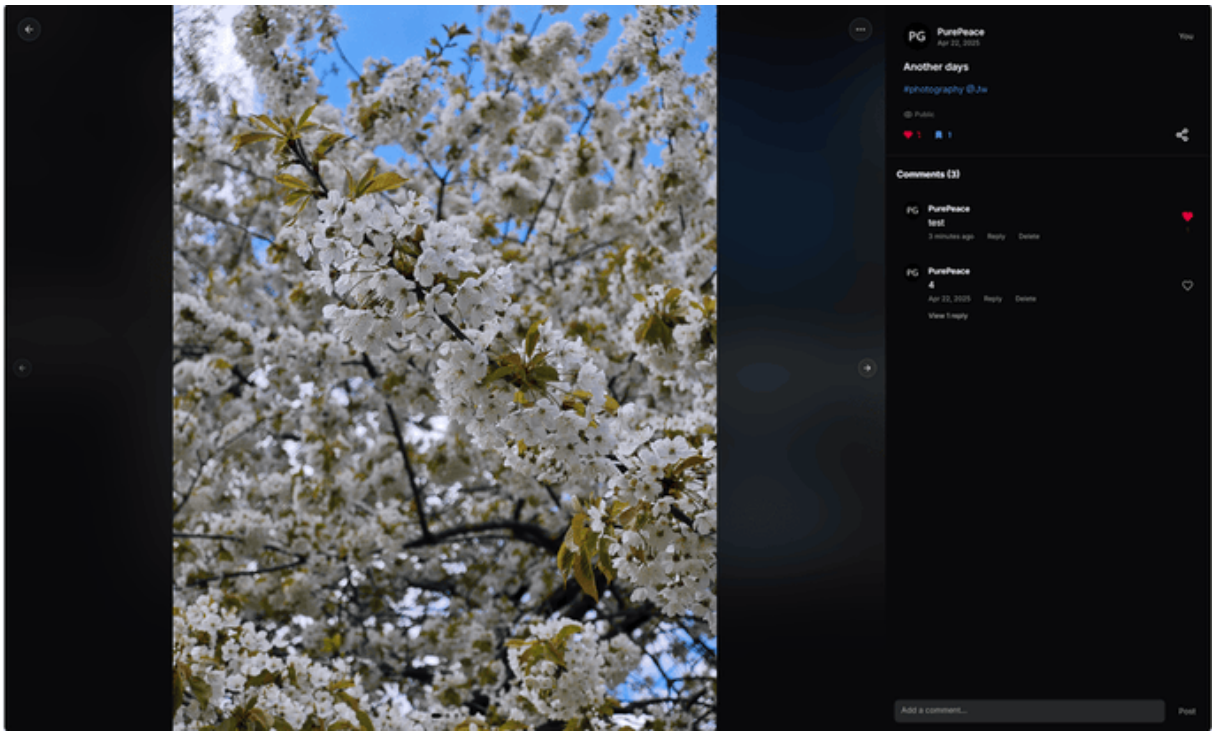


Figure 50: Post Detail - Dialog Mode (Dark)

Dialog mode design notes:

- Component is global component, entire website uses one dialog instance, preventing multiple creations causing memory leaks.
- Can call post detail dialog by referencing function anywhere in frontend code, and can pass initial data when opening dialog for pre-rendering effect, avoiding white screen/jitter from data loading, providing excellent user experience.
  - Example: Post cards in content waterfall have preloaded partial post data, opening dialog passing initial data enables reuse, very efficient.
- Asynchronously loads complete data, even with initial data passed, will update latest content and comment data when dialog opens, keeping content current.
- Dialog mode enables more immersive photo and comment viewing, hiding sidebar and top bar for fullscreen post display.
- Dialog supports accessibility with screen reader support.
- Can use keyboard keys like ESC to close dialog, convenient for PC users' continuous browsing.
- Added slight transparency and frost filter to photo area background for better design feel.
- Pseudo-route optimization design:
  - Dialog display still changes browser address bar path to show complete post URL, convenient for user copying for sharing.
  - Also inserts URL into browser history record so browser forward/back buttons work normally in dialog mode.

Page Mode:

- On mobile devices (width less than 768px), directly clicking post card presents post content as route-navigated page.
- Opening post by copying URL, opening in new tab etc. presents in page mode even on non-mobile devices.

Page route format:

- Default: `domain.com/@{username}/photo/{cuid}`
- Complete: `domain.com/profile/{username}/photo/{cuid}`
- `@` shorthand support implemented by custom Next.js middleware

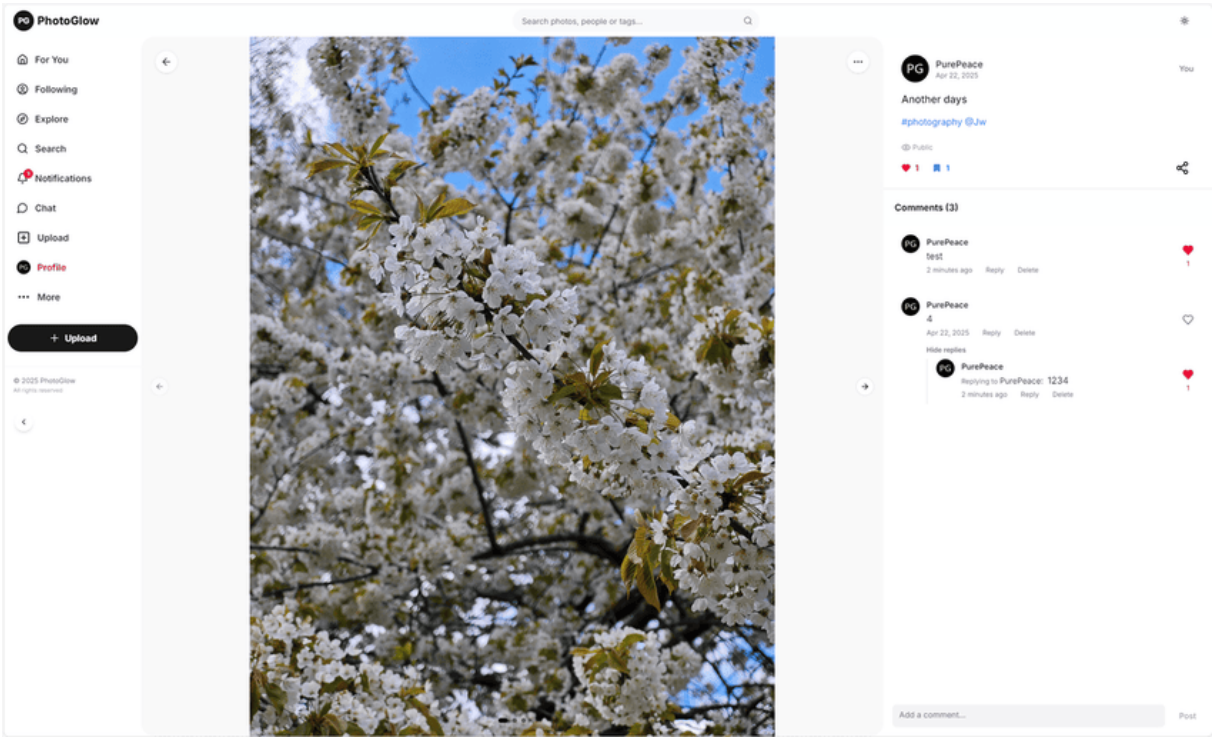


Figure 51: Post Detail - Page Mode

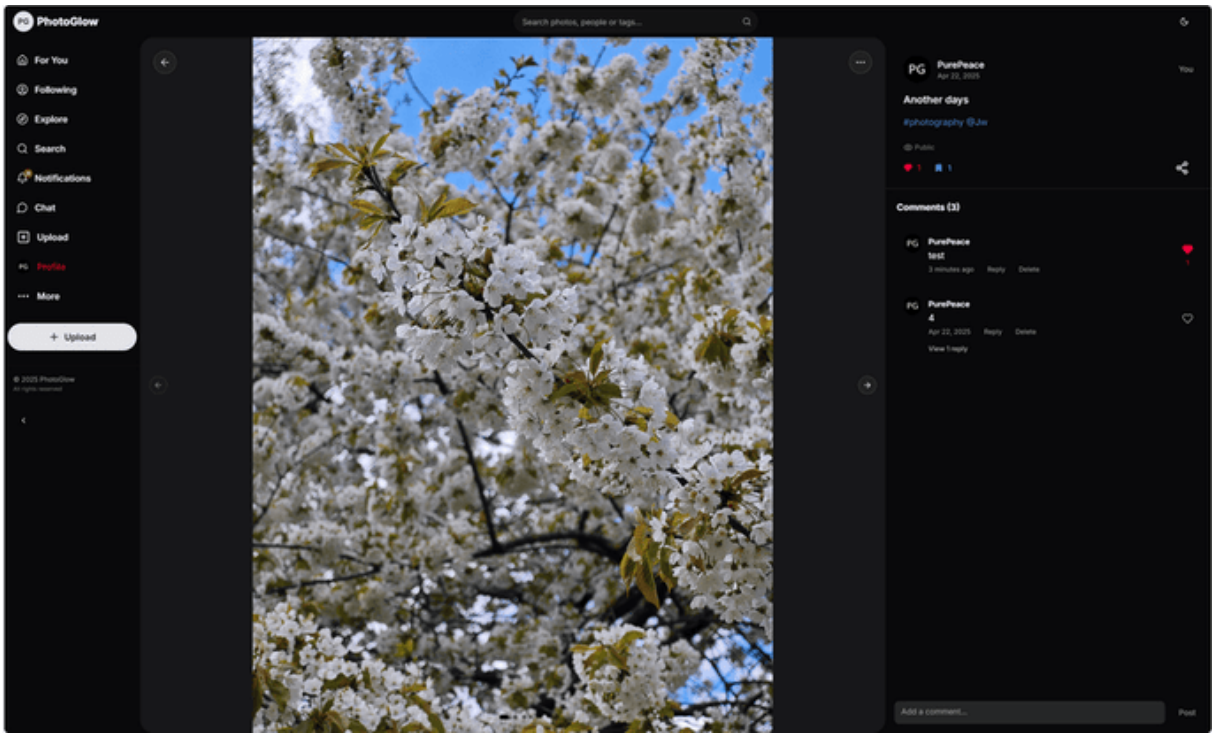


Figure 52: Post Detail - Page Mode (Dark)

Page mode design notes:

- Although dialog can provide good experience, page mode is more suitable for mobile device operation compatibility as pages better suit mobile-specific key and gesture operations.
- When opening posts via complete URL (e.g. sharing scenarios), can fully utilize Next.js server component features for server-side rendering, providing better first-screen experience.

- Considering users might want to browse multiple post contents using mouse middle button or opening new tabs, adding page mode design is very necessary.

Responsive Design:

Post detail page employs responsive design for optimal experience across different device sizes.

- Default: Presents left-right layout, photo area on left, content and comment area on right.
- Narrow screen (less than 1280px): Presents vertical layout, top to bottom sequence of photo area, content area, comment area.
- Mobile (less than 768px): Same as narrow screen, difference is due to mobile bottom navigation bar, comment publishing area positioned above navigation bar.

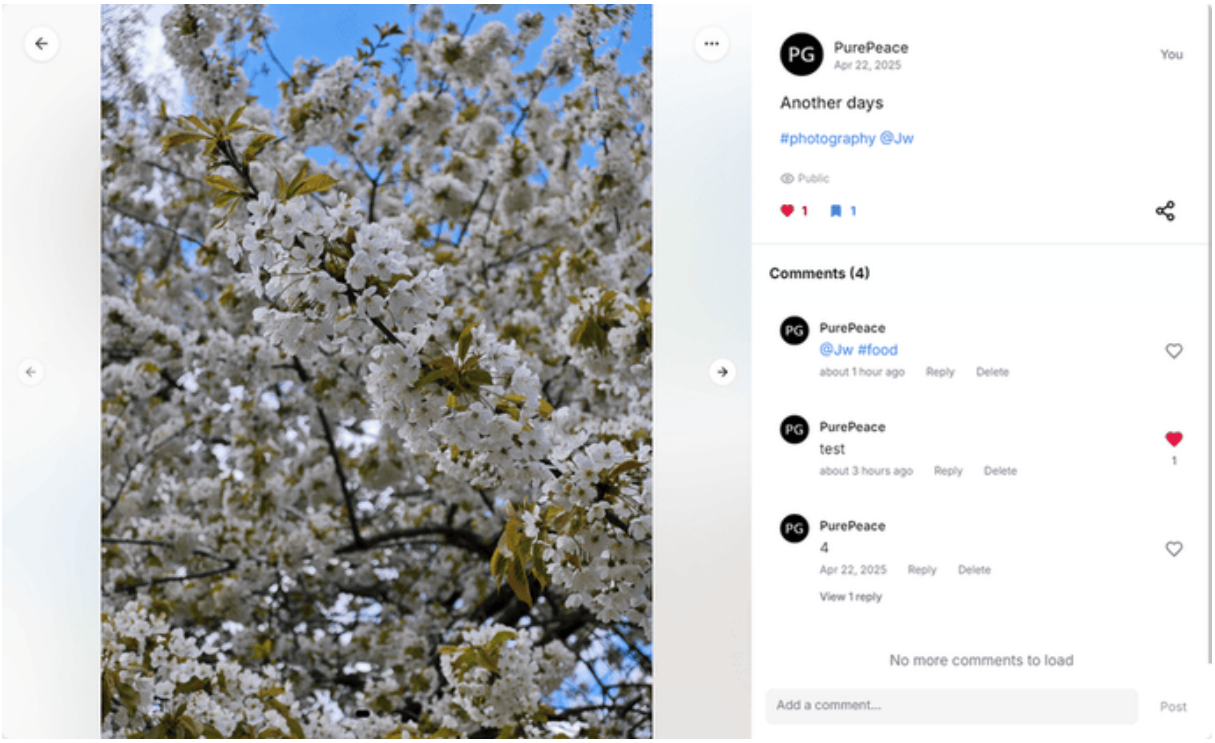


Figure 53: Post Detail - Wide Screen

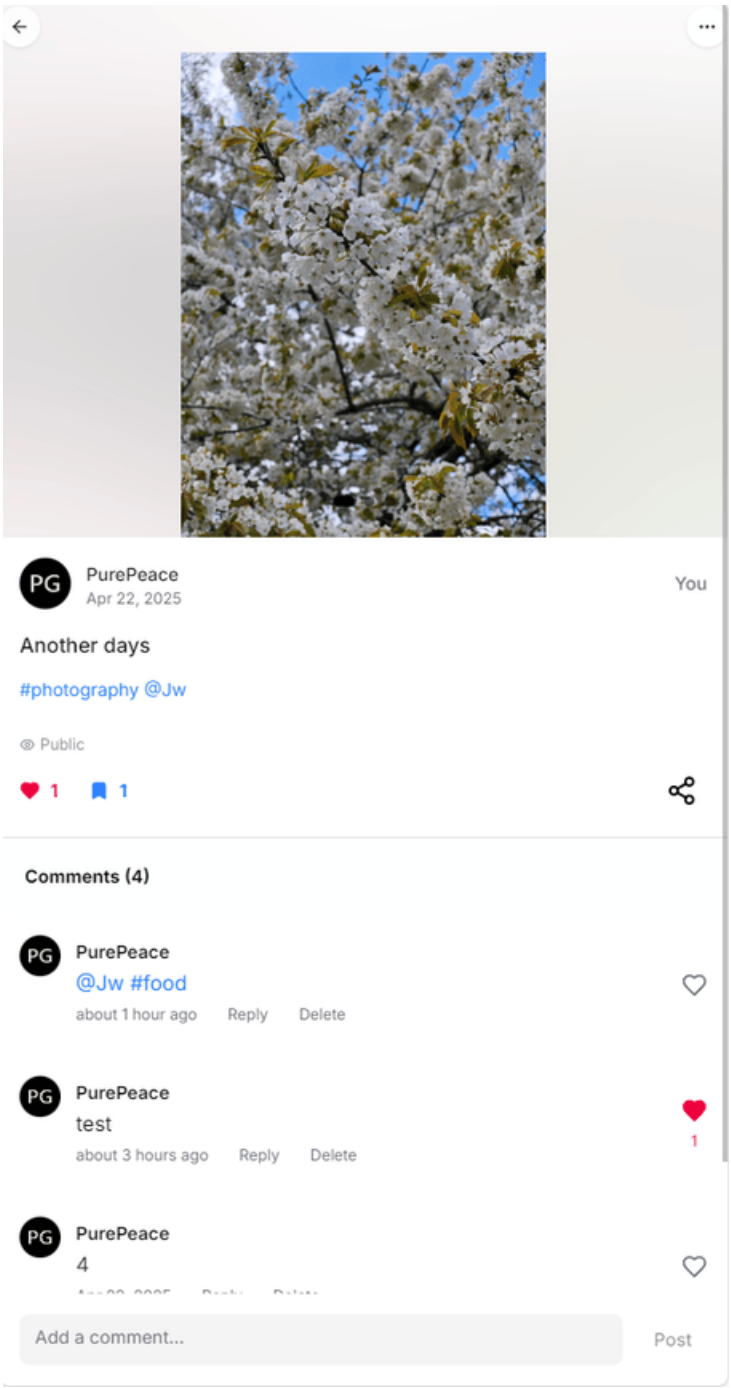


Figure 54: Post Detail - Narrow Screen

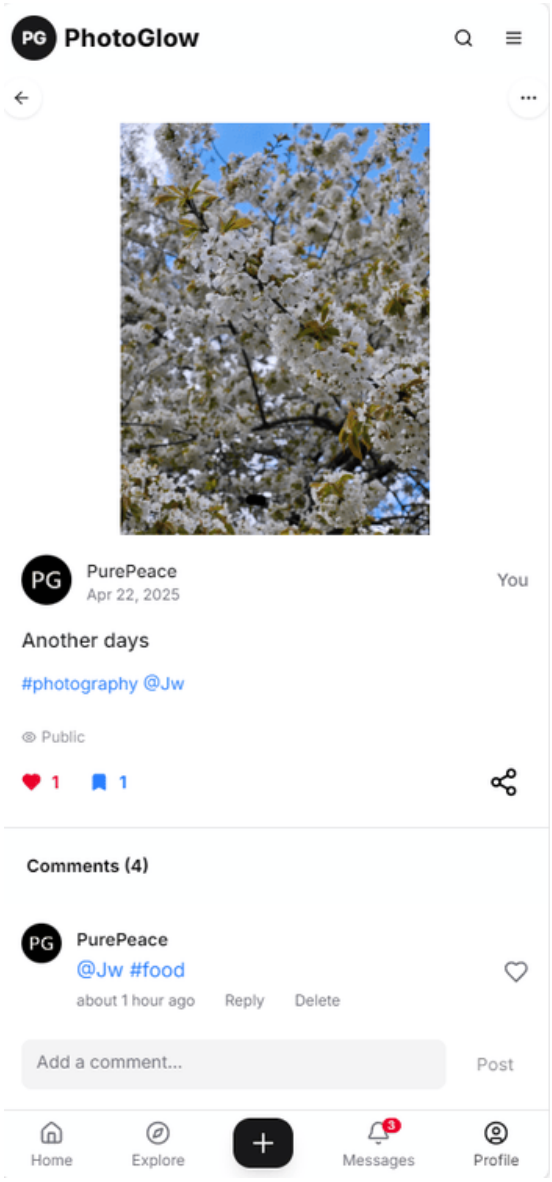


Figure 55: Post Detail - Mobile Device

Responsive design notes:

- Adjusting from horizontal to vertical layout involves scroll container changes:
  - `Only comment list scrolls vertically in left-right layout -> Entire page parent container scrolls vertically in vertical layout`
- To maintain normal comment input box display, uses technology to send it to different element levels based on conditions rather than creating multiple comment input box instances, achieving ultimate optimization.

3.2.5.6 Content Publishing Interface This section provides a detailed examination of the system's content publishing interface design.

As one of the system's core functionalities, this module is dedicated to providing users with an intuitive, efficient, and feature-rich creation experience. Through carefully crafted interface design and interaction optimization, we have implemented a series of features including rich text editing, image editing, sorting, and real-time preview, effectively enhancing users' creative efficiency and usage experience.

The component offers two presentation modes - page and dialog - which feature different interaction designs but share similar basic composition:

- Image editing area
- Content editing area
- Action button area

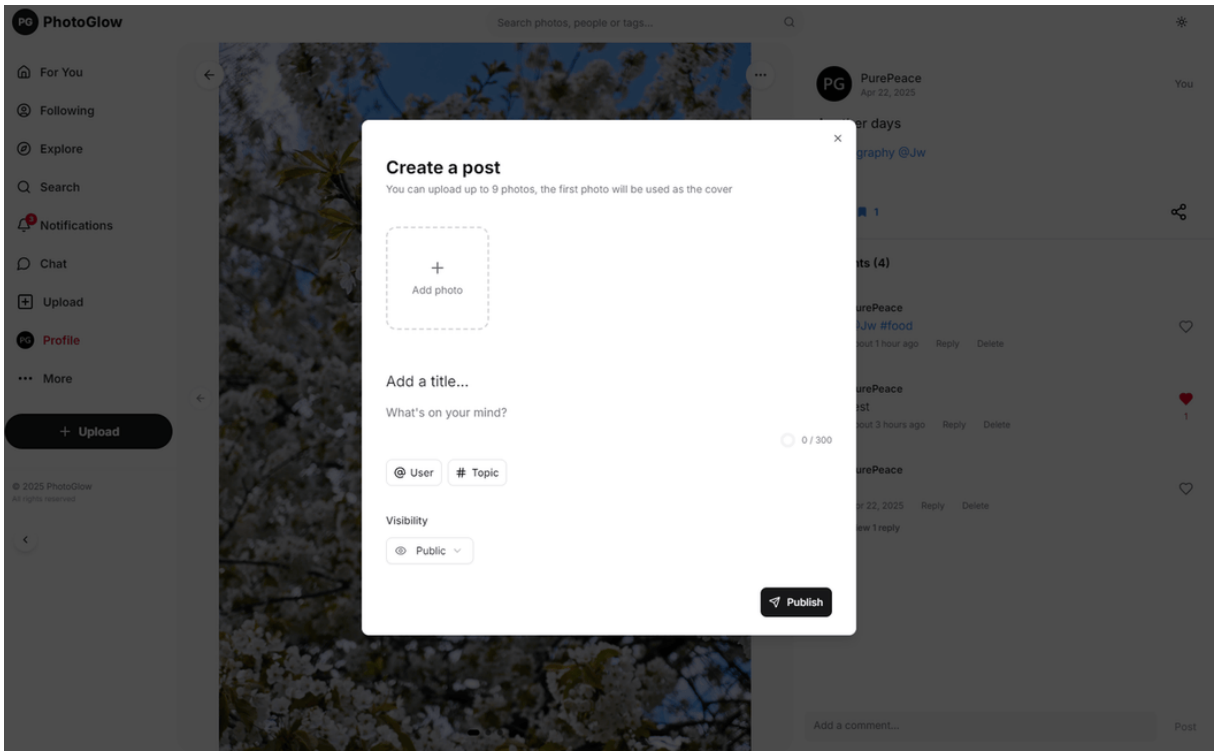


Figure 56: Content Publishing - Dialog

Image Editing Area:

Image editing serves as the core functionality of the content publishing editor. Currently, each post supports publishing up to 9 images, with this area supporting batch image addition, image editing, drag-and-drop sorting, and real-time preview capabilities.

Adding images:

- Clicking Add photo opens the system's file dialog for image selection, supporting mobile devices and multiple image selection at once.
- Supports dragging images from the system to this area for addition.
- After addition, images display as thumbnails in the list.
- Additional images cannot be added once the maximum number is reached.

Editing images:

- On PC, hovering the cursor over an image reveals editing buttons; clicking opens the image editing dialog.
- On mobile devices, directly tapping the image initiates image editing.
- Clicking the X button in the top-right corner of an image removes it.
- Users can preview in real-time, set zoom size and ratio for each image, and define image display range. The component comes preset with a collection of common photo ratios.

Drag-and-drop sorting:

- Images can be directly dragged for sorting, with the first image automatically set as the cover, aligning with user intuition.

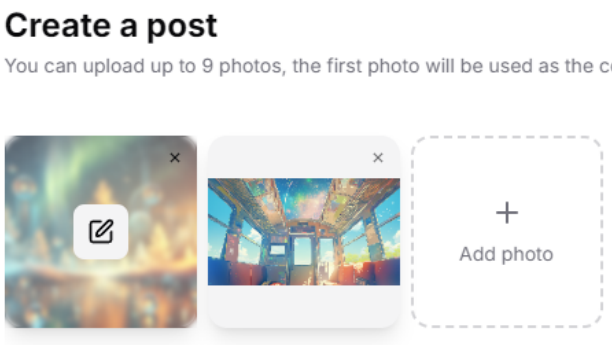


Figure 57: Content Publishing - Image Editing Buttons

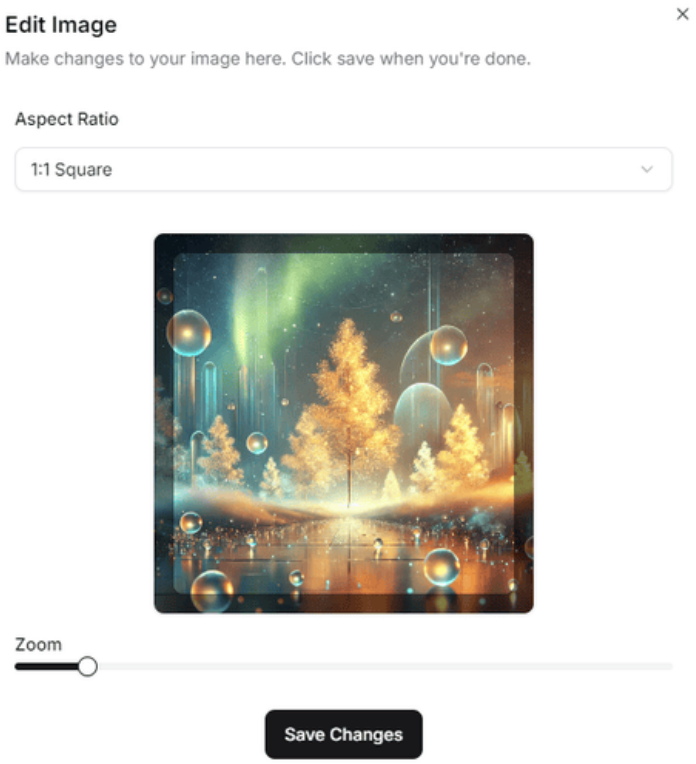


Figure 58: Content Publishing - Image Editing Dialog

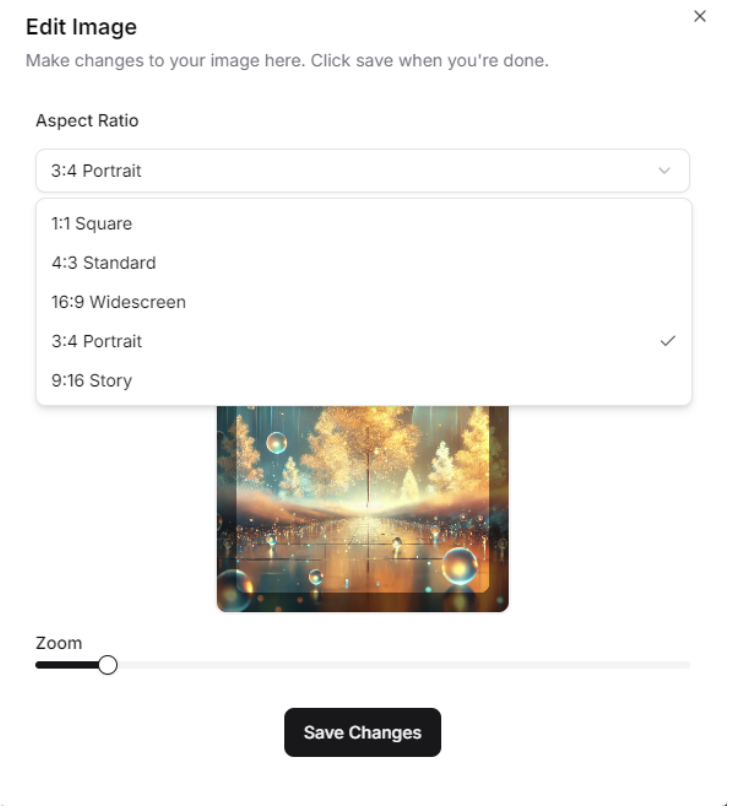


Figure 59: Content Publishing - Image Editing Dialog - Ratio Selection

Content Editing Area:

The content editing area comprises title, body text, writing buttons, and length limit indicators.

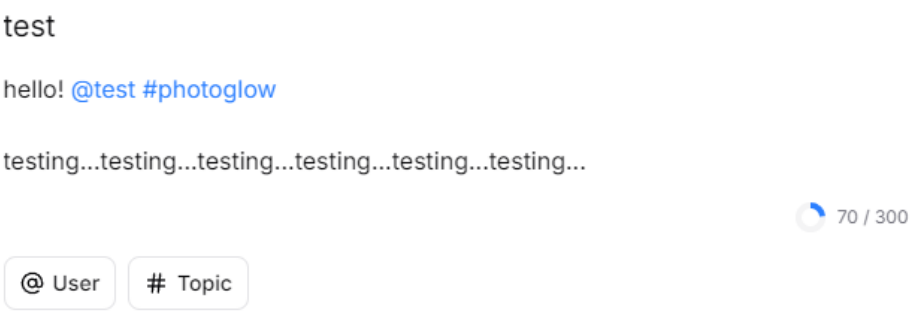


Figure 60: Content Publishing - Content Editing

Content editing area design:

- Only the title field is mandatory.
- Body text is limited to 0-300 characters (`@` and `#` count as single characters, though maximum mention counts are limited by the server API).
- A circular progress bar style length limit indicator appears below the body text.
- The body text uses a rich text editor, automatically expanding a mention suggestion overlay when typing `@` or `#` to suggest users or topics.
- Features content memory functionality and supports keyboard undo/redo shortcuts.
- Automatically creates corresponding topics when non-existent topics are input.

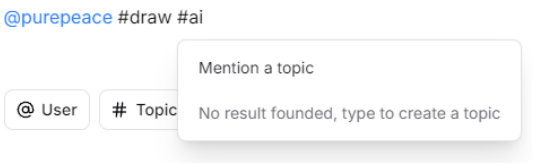


Figure 61: Content Publishing - Creating Topics

Action Button Area:

Currently supports setting post visibility range, with options including:

- Public
- Private: Visible only to self
- Followers Only: Visible only to followers



Figure 62: Content Publishing - Action Buttons

Responsive Design:

On mobile devices, clicking the publish button navigates to the publishing page via routing rather than opening a dialog.

Additionally, this component features mobile-optimized styling to ensure better user experience.

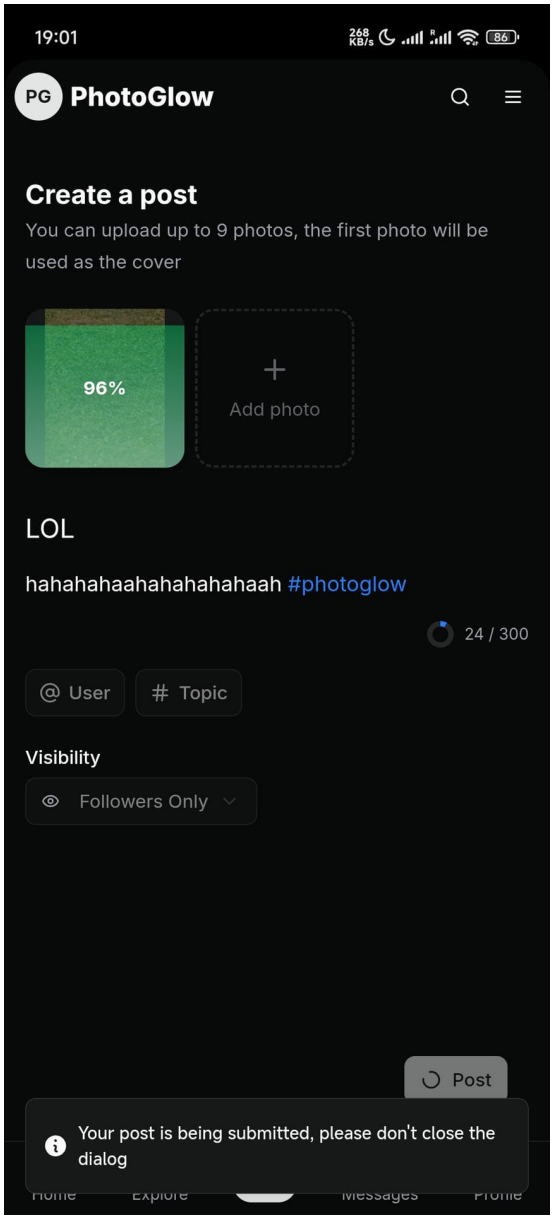


Figure 63: Content Publishing - Mobile Page

Additional Features:

1. Close Confirmation

When users attempt to close the editor with unsaved content, a confirmation dialog appears, effectively preventing accidental content loss through unintended closure.

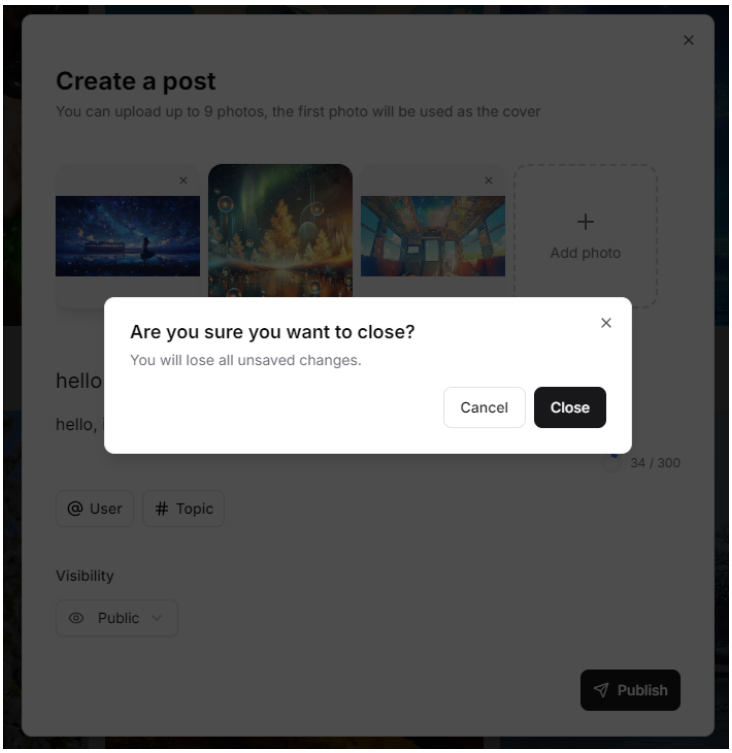


Figure 64: Content Publishing - Close Confirmation

2. Form Validation

The form validation system provides input feedback, including:

- Required field checking
- Format validation
- Character count
- Error messages

These features are presented in a non-intrusive manner, helping users identify and correct issues promptly.

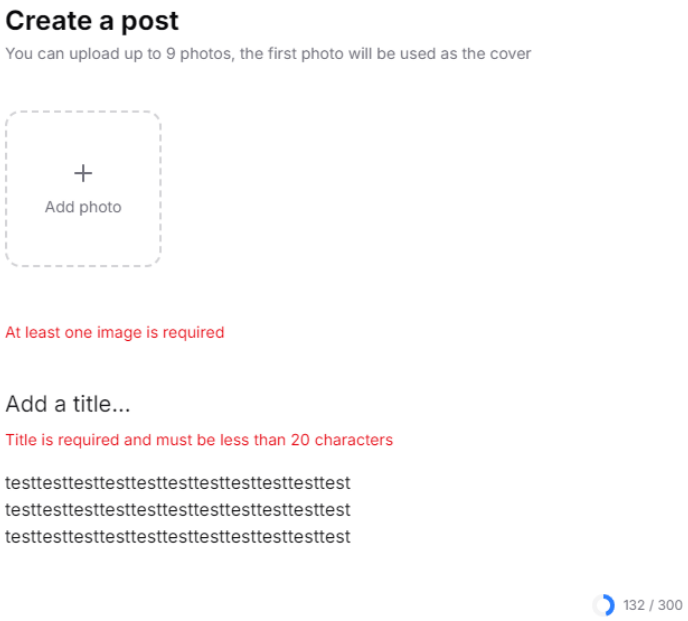


Figure 65: Content Publishing - Form Validation

3. Concurrent Image Upload

The system supports efficient multi-image upload functionality:

- Supports batch uploading
- Displays real-time upload progress
- Provides upload status feedback

These features significantly enhance the user experience.

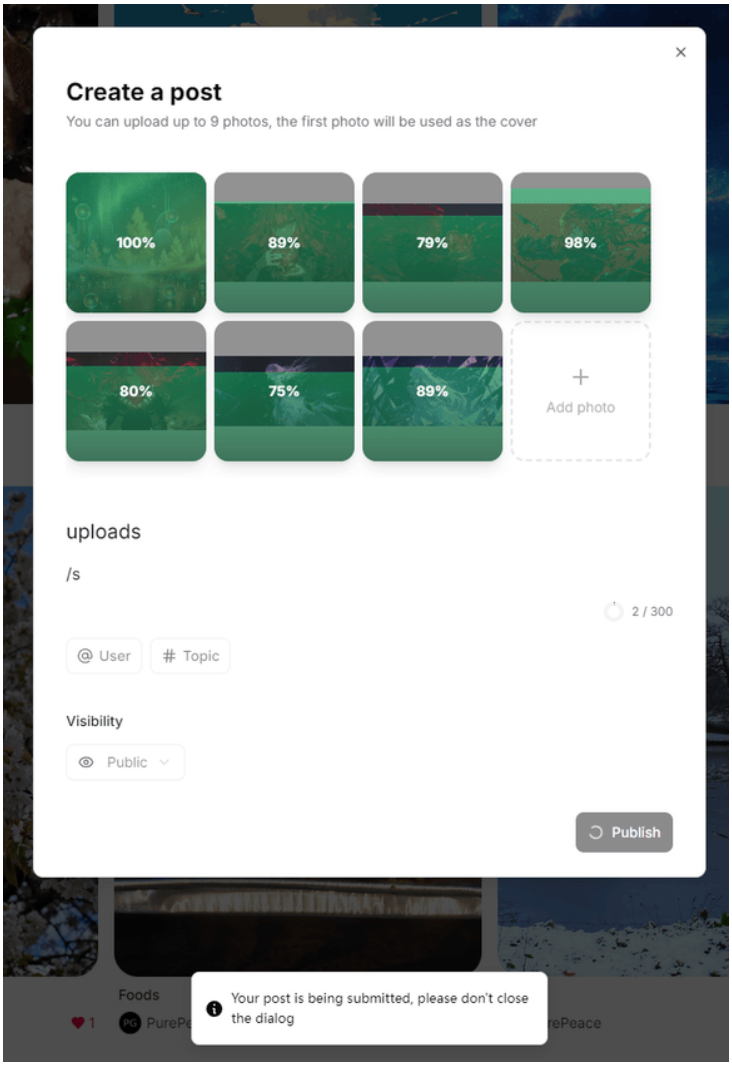


Figure 66: Content Publishing - Concurrent Upload

4. Publishing Failure Recovery Mechanism

The system features an intelligent upload recovery mechanism:

- Automatically saves uploaded content
- Provides upload failure recovery options
- Displays cloud-existing photos (with Cloud label in top-left corner of image thumbnails)

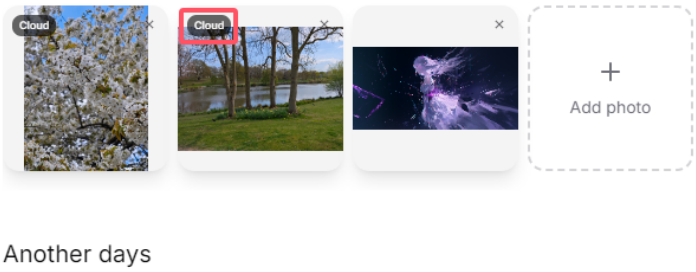


Figure 67: Content Publishing - Recovery Mechanism

3.2.5.7 User Profile Page The user profile page stands as one of the core pages in the system, serving multiple essential functions including profile modification, user information display, and presentation of the user's posts, likes, bookmarks, and follower/following lists.

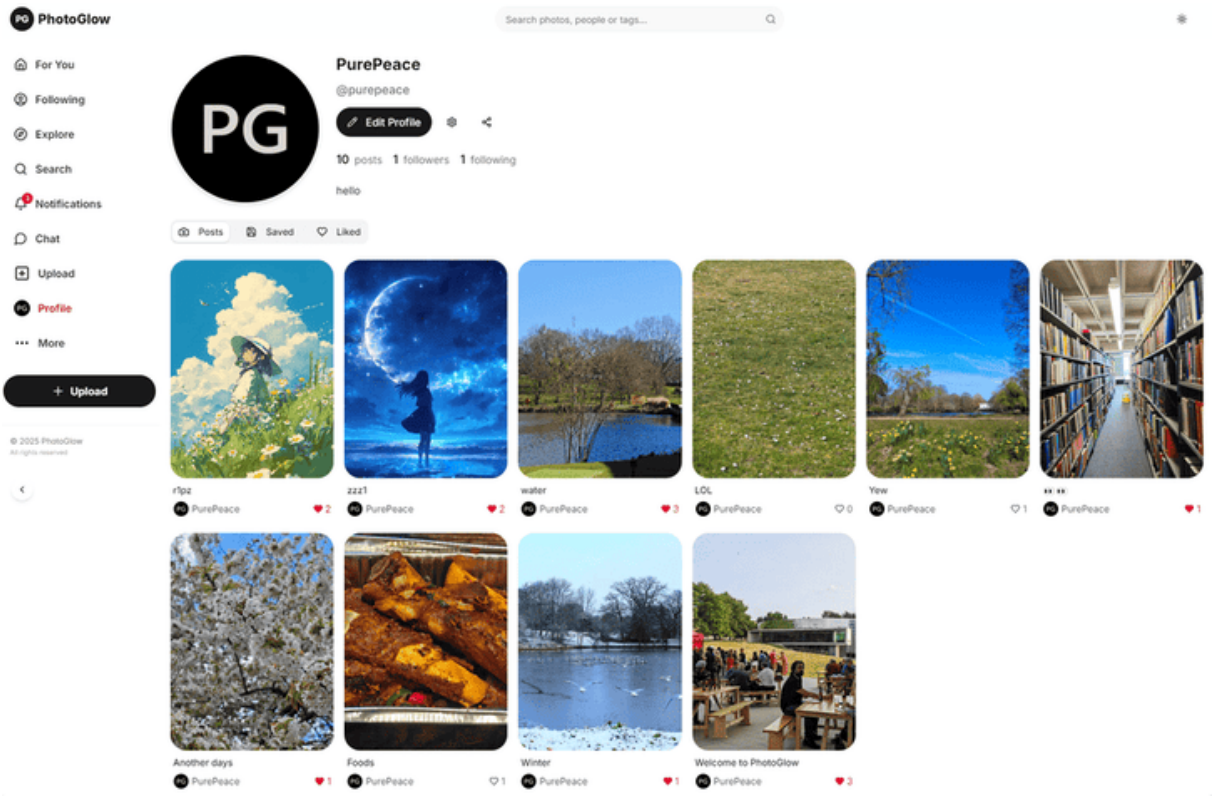


Figure 68: User Profile - Current User Perspective - PC View

The main structure consists of the following components:

- User information area
- Action button area
- Post Tabs (divided into Posts, Saved, and Liked)

User Information Area:

The information area presents user details in a carefully organized layout:

- Left side: User avatar, with a fallback avatar if none is set
- Right side first level username: The user's Display Name, supporting Unicode character set and displayed in primary color
- Right side second level username: The user's unique Username, displayed with a leading @ symbol and shown in secondary color

Action Button Area:

The action area adapts its display based on the viewer's relationship to the profile:

- Own profile view: Displays Edit Profile button, Account Settings button, and Share button
- Other user's view: Shows Follow button, Send Message button, Share button, and More options button

Button functionality:

- Edit Profile: Opens the profile modification dialog when clicked
- Account Settings: Navigates to the `settings` route upon interaction
- Send Message: Redirects to the `chat` route, displaying a private message window with the selected user
- Follow button: Implements identical functionality to the component described in "Section 3.2.5.5 Post Detail Interface"
- More options: Reveals a dropdown menu containing Report and Block options



Figure 69: User Information Area - Current User Perspective - PC View

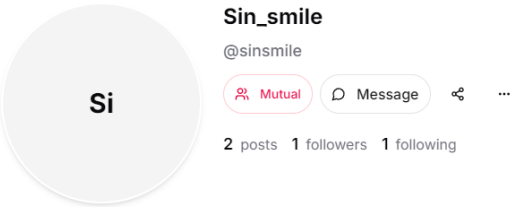


Figure 70: User Information Area - Other User Perspective - PC View

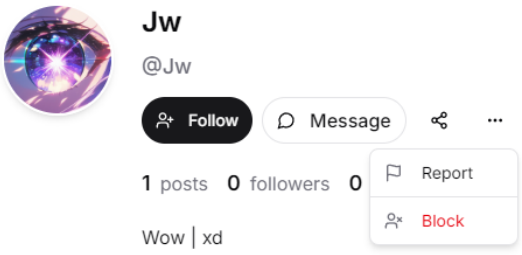


Figure 71: User Information Area - More Options Dropdown Menu

Profile Modification Dialog:

The `profile-dialog` component enables users to modify their profile information through an intuitive interface comprising:

- User avatar
- Display Name
- Username

- Bio (self-introduction)

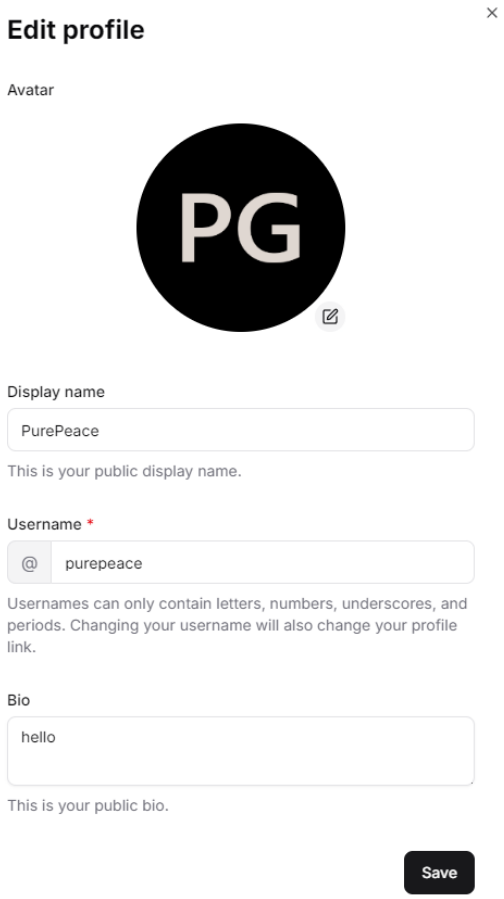


Figure 72: User Profile - Profile Modification Dialog

Avatar Modification:

The avatar modification process offers multiple user-friendly options:

- Clicking the avatar in the profile modification dialog opens the system's file selection interface
- Users can drag and drop an image from their system onto the component
- Upon image selection, the component transforms into an avatar editor, allowing users to:
  - Select the visible area of the avatar
  - Adjust zoom level
  - Remove and reselect an image if desired
- Changes can be saved at any time using the Save button, with immediate updates reflected on the profile page

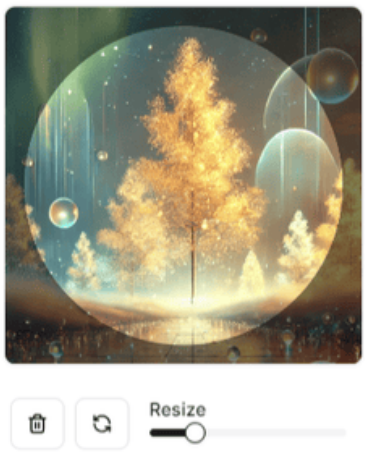


Figure 73: User Profile - Avatar Editor

Form Validation:

The system implements comprehensive validation:

- Display Name and Username are validated using regular expressions
- Bio text is limited to 100 characters

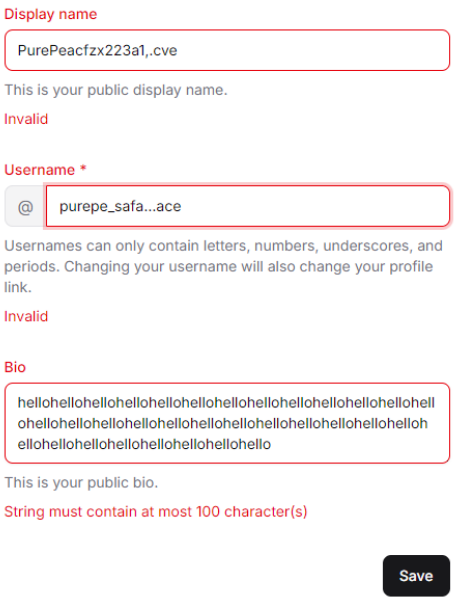


Figure 74: User Profile - Form Validation

Followers List Dialog:

Beyond displaying Follower and Following counts, the user information area enables interactive exploration of connections:

- Clicking the counts opens the followers list dialog
- The dialog facilitates relationship management and reciprocal following

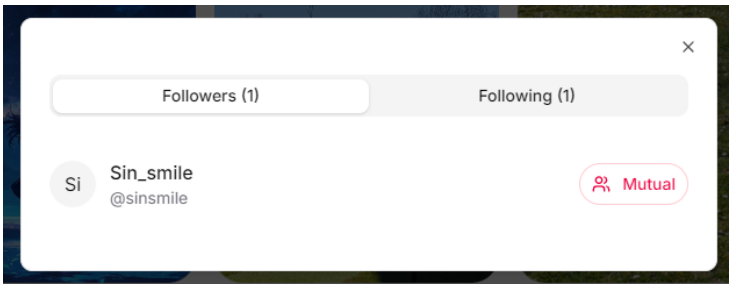


Figure 75: User Profile - Followers List

Followers list features:

- Displays both Followers and Following for easy management and reciprocation
- Clicking on displayed users navigates to their respective profiles
- Implements infinite scroll loading for seamless browsing
- Includes follow button functionality (as described previously)

Post Tabs:

The post tabs section organizes user content into three categories:

- Posts
- Saved
- Liked

Users can switch between views with a simple tab click.

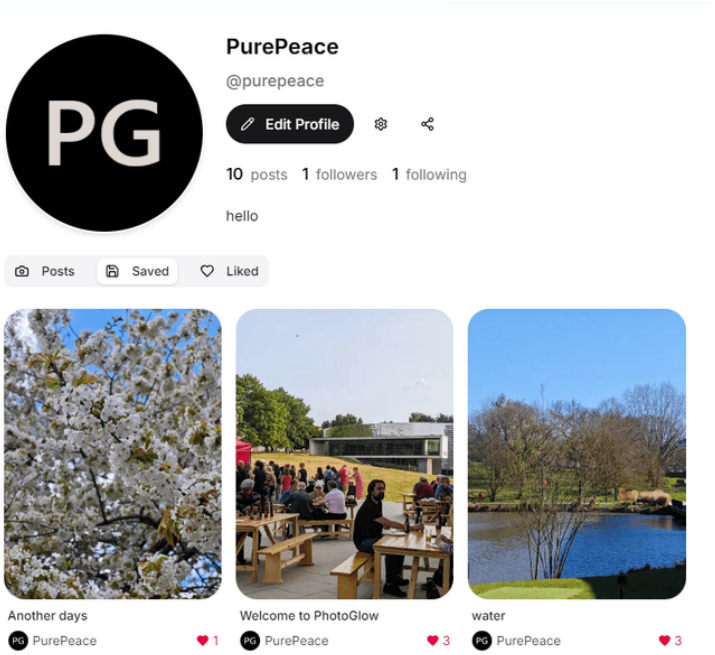


Figure 76: User Profile - Post Tabs

Post Tabs implementation:

The system leverages the reusable `photo-grid` component to display content streams, incorporating all essential features:

- Infinite scroll loading
- Grid layout
- Responsive design

- Post card presentation
- Empty state handling

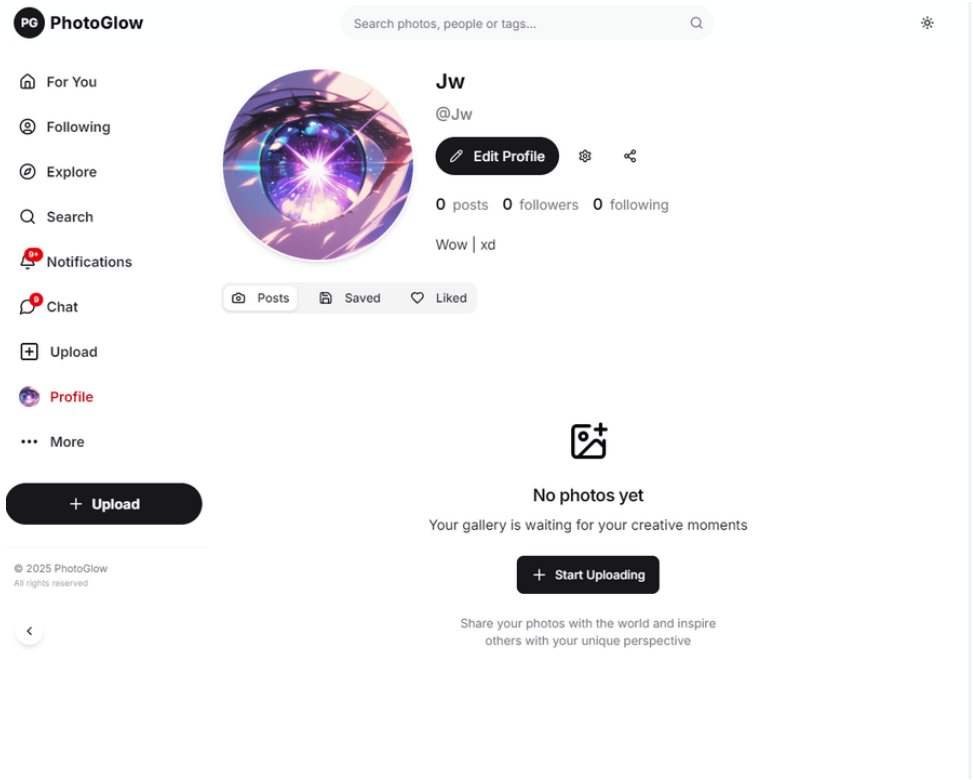


Figure 77: Empty State

Responsive Design:

The user profile adapts its layout across different screen sizes to maintain optimal user experience:

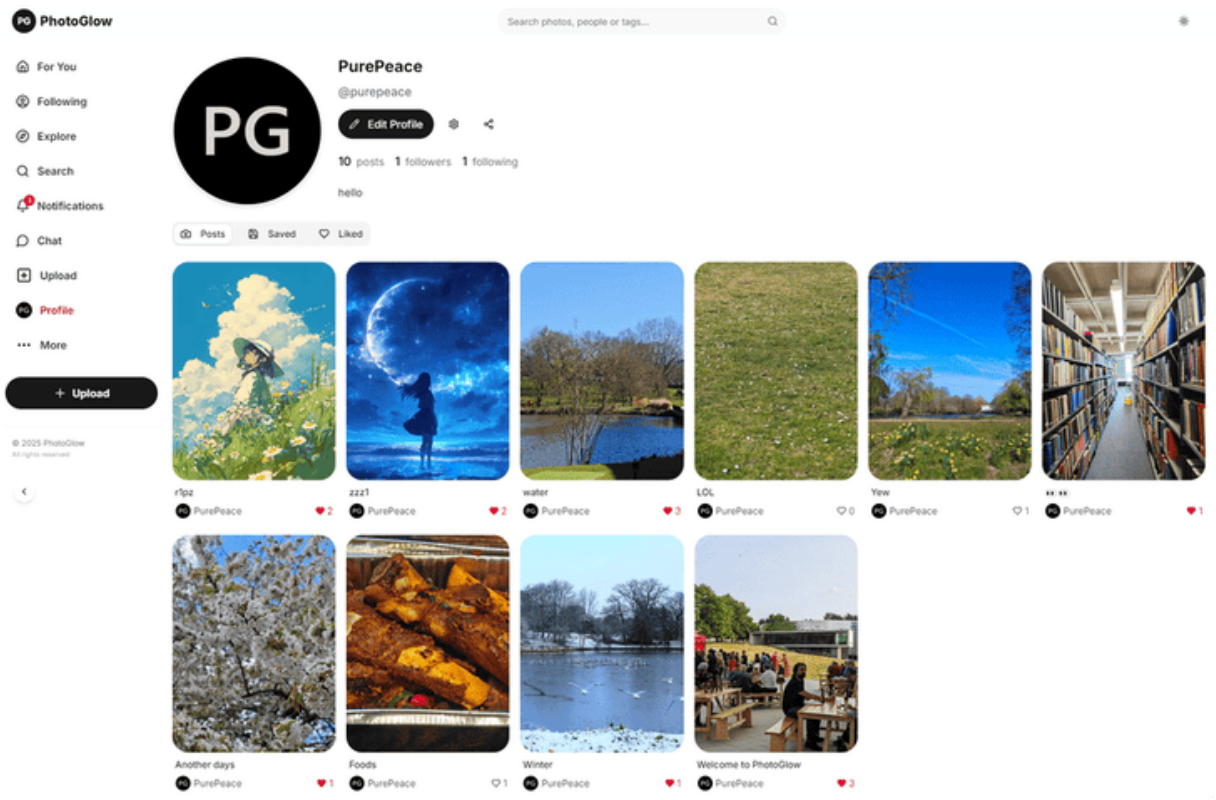


Figure 78: User Profile - Current User Perspective - PC View

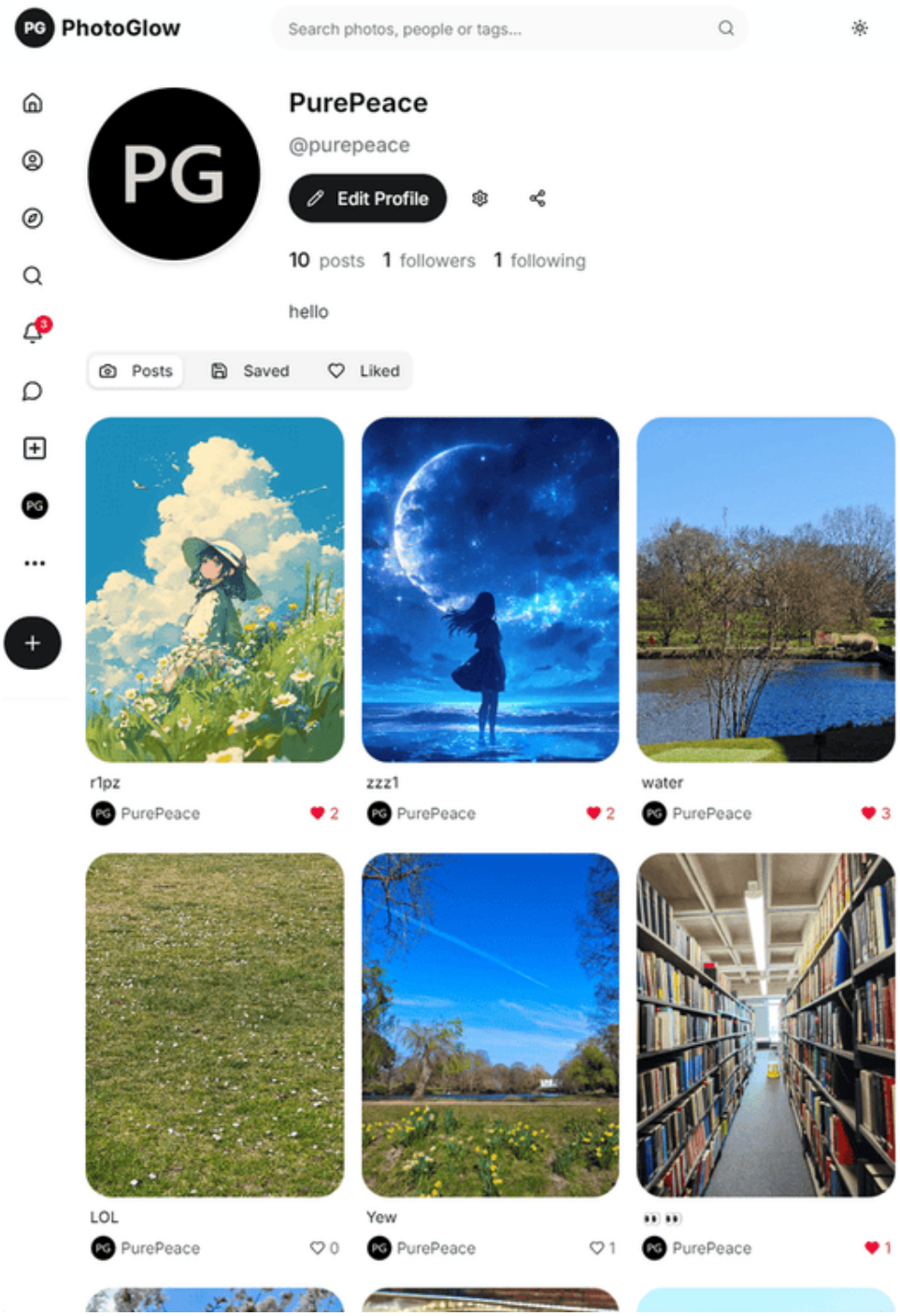


Figure 79: User Profile - Current User Perspective - Tablet View

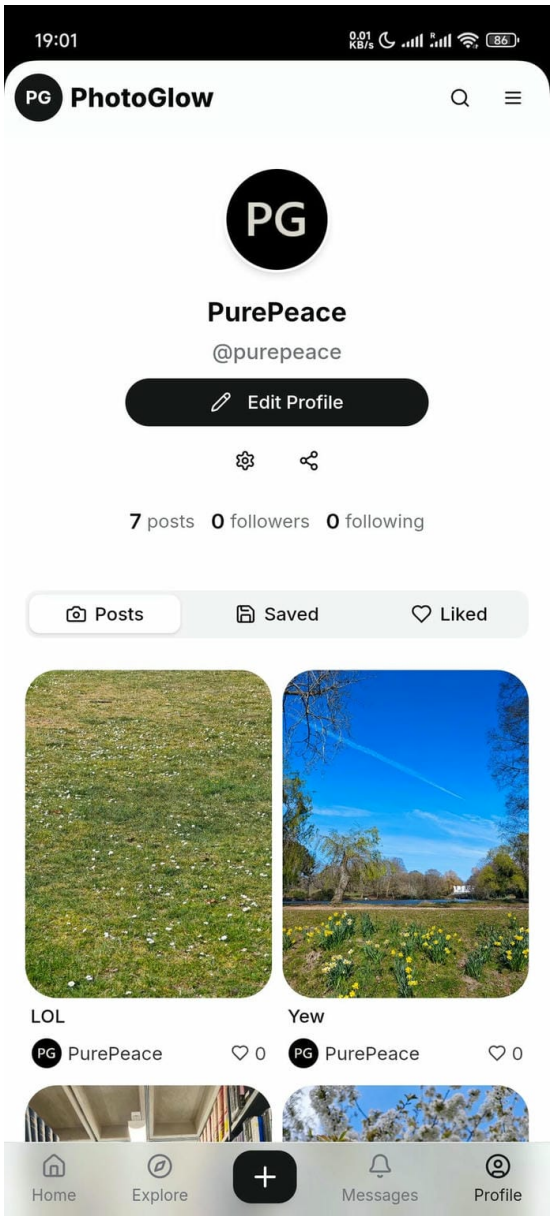


Figure 80: User Profile - Current User Perspective - Mobile View

3.2.5.8 Following Feed Page The `following` page presents a timeline view of posts from users the current user follows.

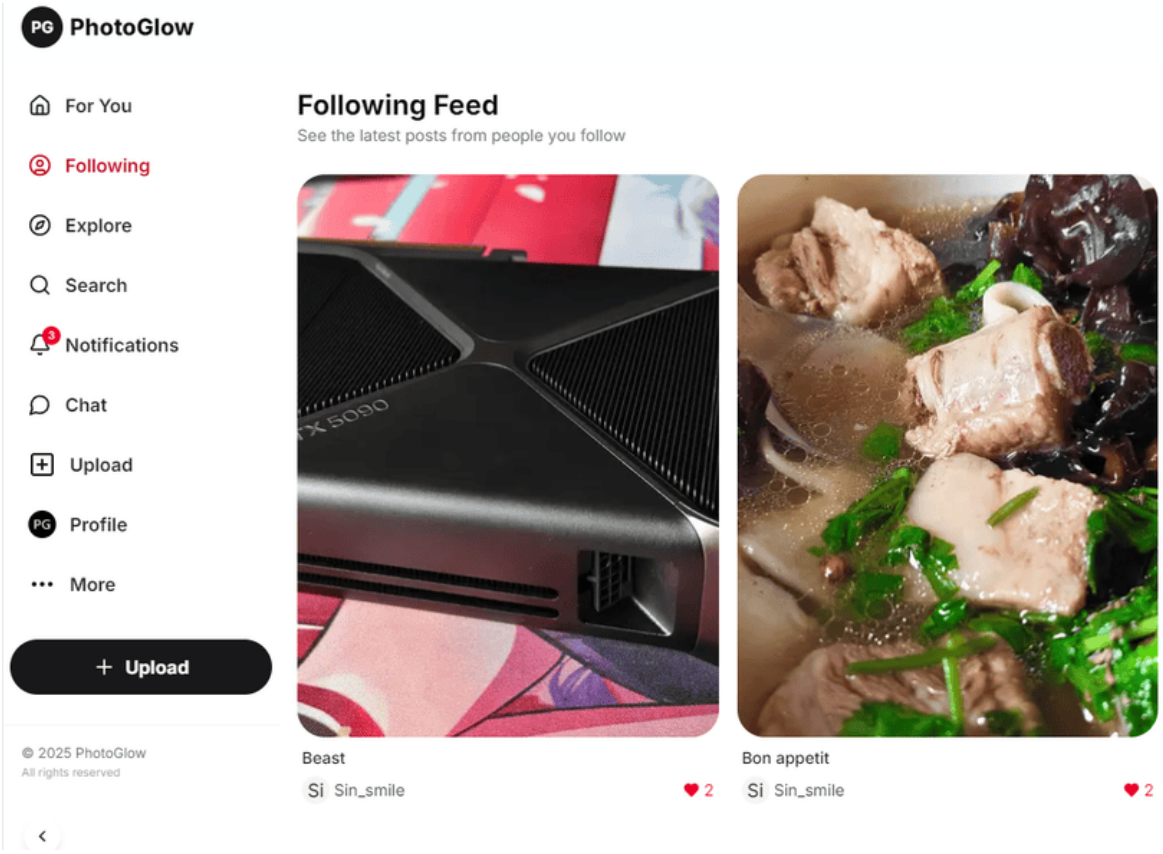


Figure 81: Following Feed

Design implementation:

The page shares many characteristics with the Home page:

- Similar presentation style
- Server-side rendering implementation
- Login requirement message for non-authenticated users
- Reuse of the `photo-grid` component
- Mobile responsiveness and theme mode adaptation
- Design patterns follow Section 3.2.5.3 (Home Page), differing only in server API data source

3.2.5.9 Explore Page The `explore` page showcases popular Topic categories, helping users discover currently trending discussions and posts.

Explore Topics are organized into two groups:

- Popular Topics: Trending topics arranged by popularity
- All Topics: Less trending but browseable topics

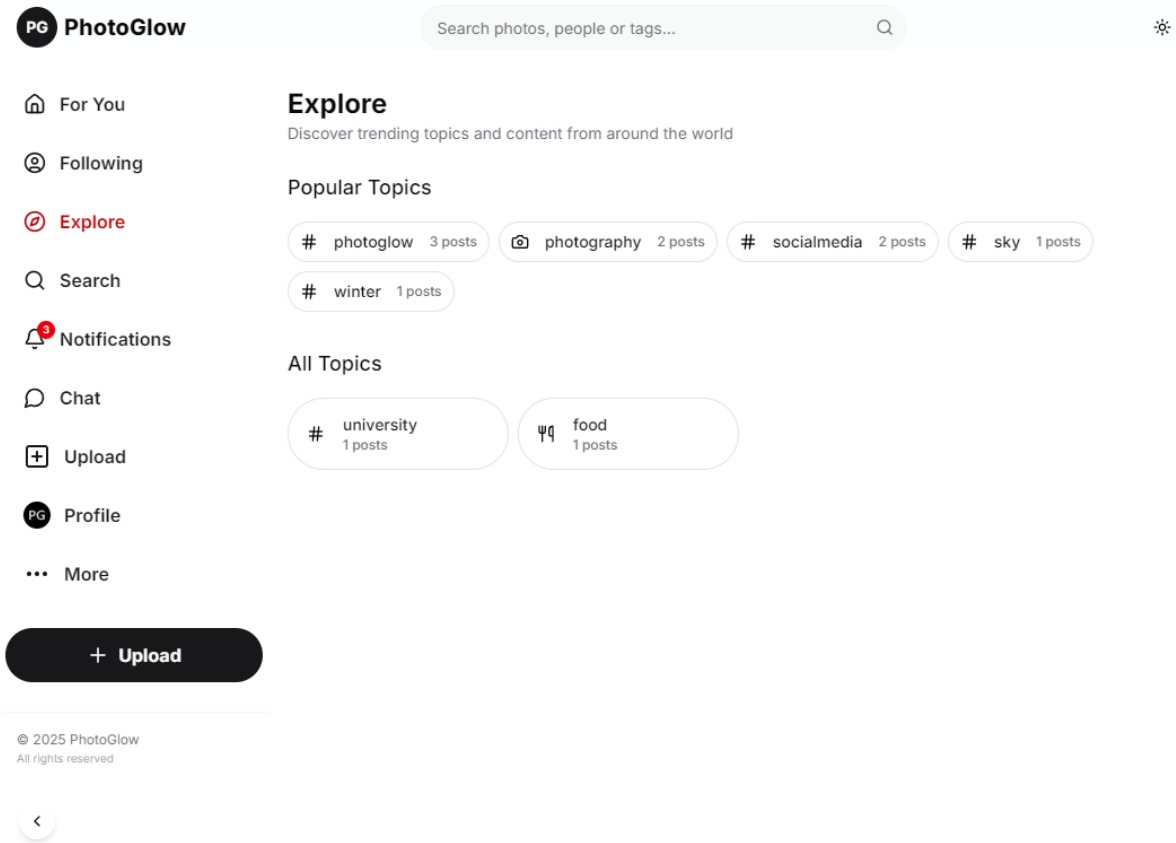


Figure 82: Explore PC

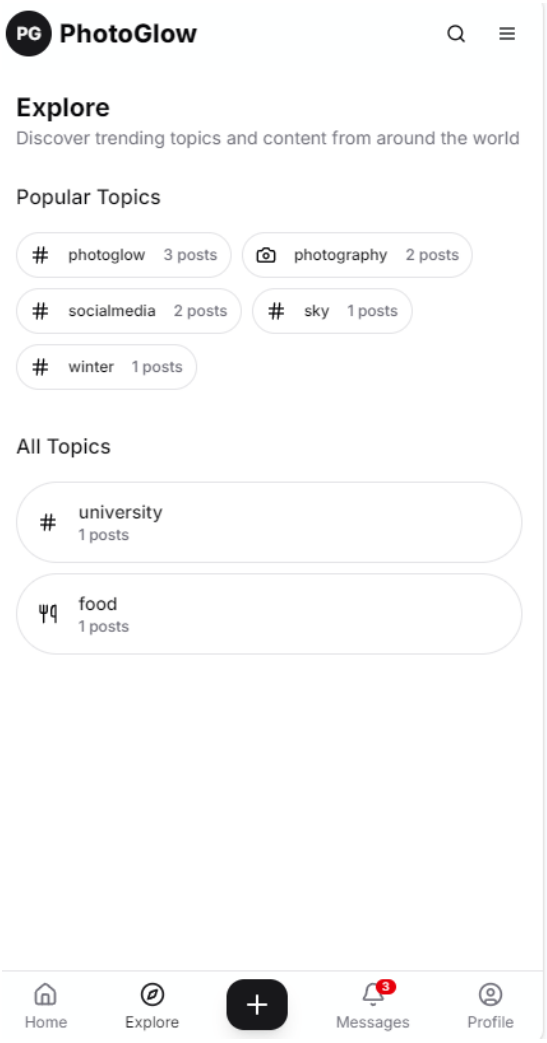


Figure 83: Explore Mobile

Clicking any Topic reveals a post waterfall display that:

- Reuses `home page` logic and the `photo grid` component
- Maintains all core features
- Supports URL sharing in the format `/explore/{topics}`

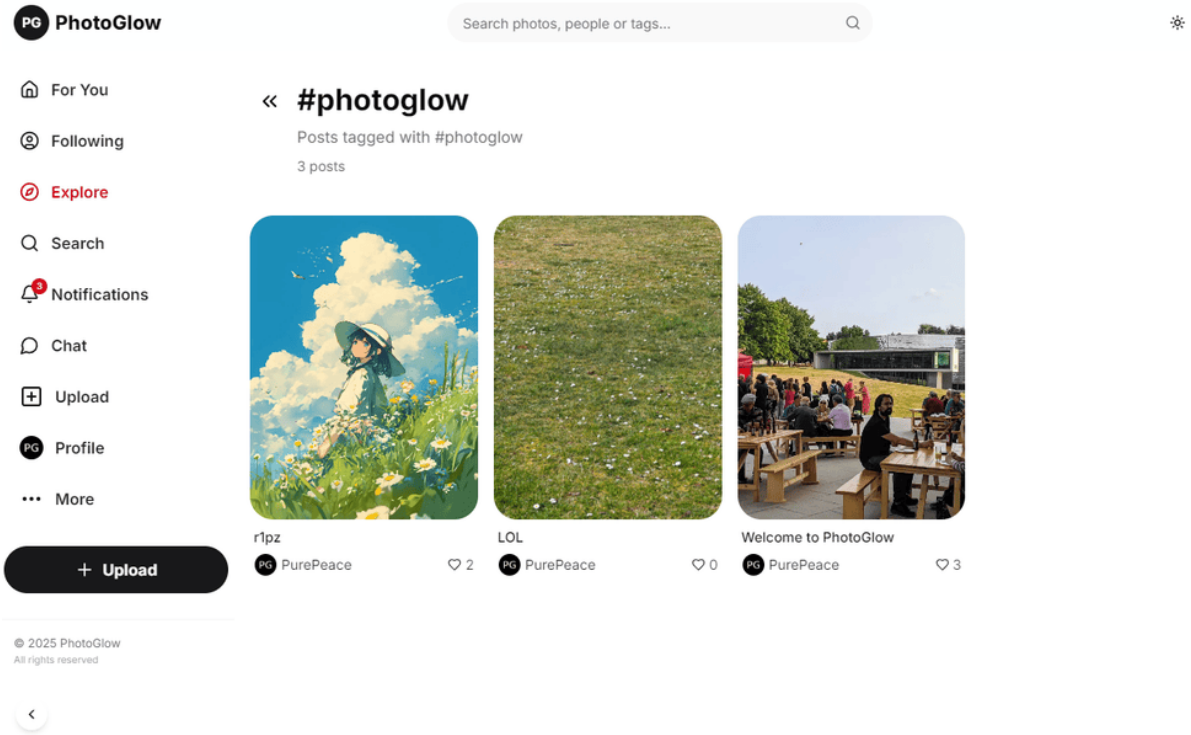


Figure 84: Explore Topic

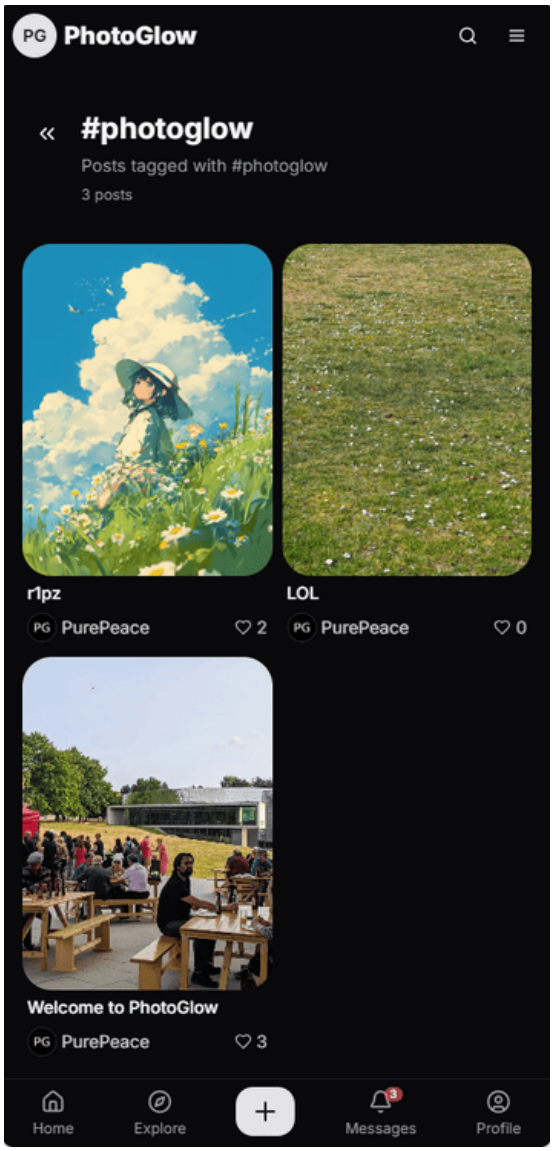


Figure 85: Explore Topic Mobile

3.2.5.10 Search Page The Search page serves as the website's content discovery interface, enabling searches across Topics, People, and Posts.

This page supports the Search bar in the website framework, processing searches initiated from any location.

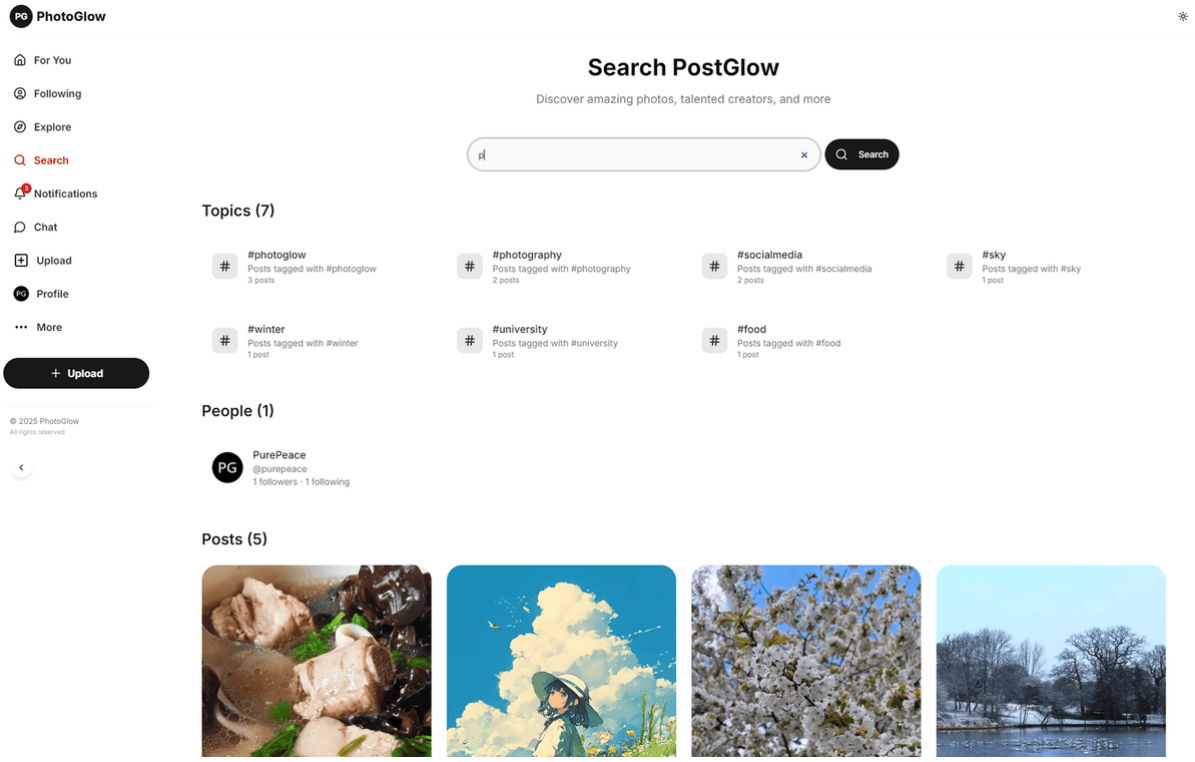


Figure 86: Search

Search implementation features:

- Real-time search: Automatically triggers search upon keyword entry, with debouncing and minimum request interval to prevent excessive requests
- Multiple trigger methods: Enter key and search button clicks
- Comprehensive scope: Covers Topics, People, and Posts
- Topics: Maintains consistency with Explore page logic, linking to `explore/{topics}`
- People: Shows active users with avatars, Display Names, Usernames, and follower counts
- Posts: Implements the `photo-grid` component with consistent behavior
- Full-text search: Prepared for ElasticSearch integration (service pending)
- Responsive design implementation

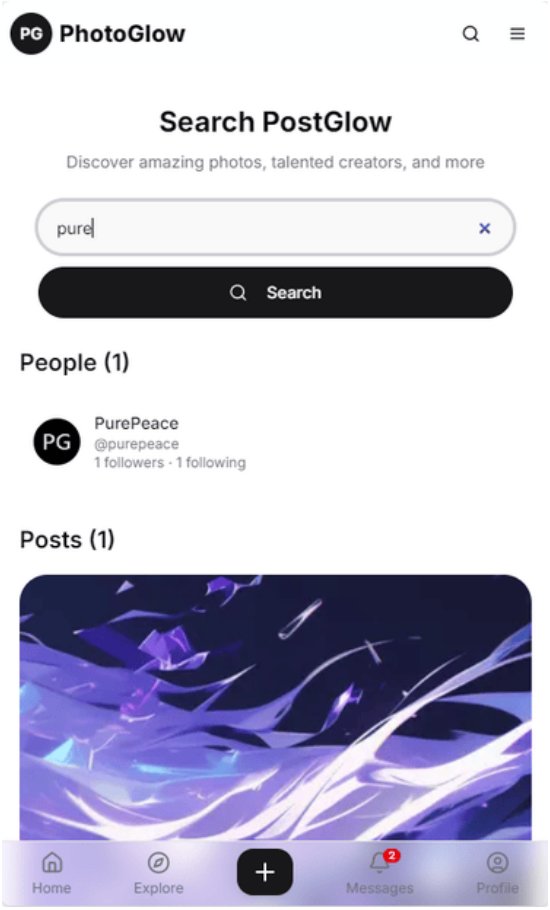


Figure 87: Search Mobile Responsive Design

3.2.5.11 Notifications Page This page displays user notifications and allows users to mark notifications as read or delete them.

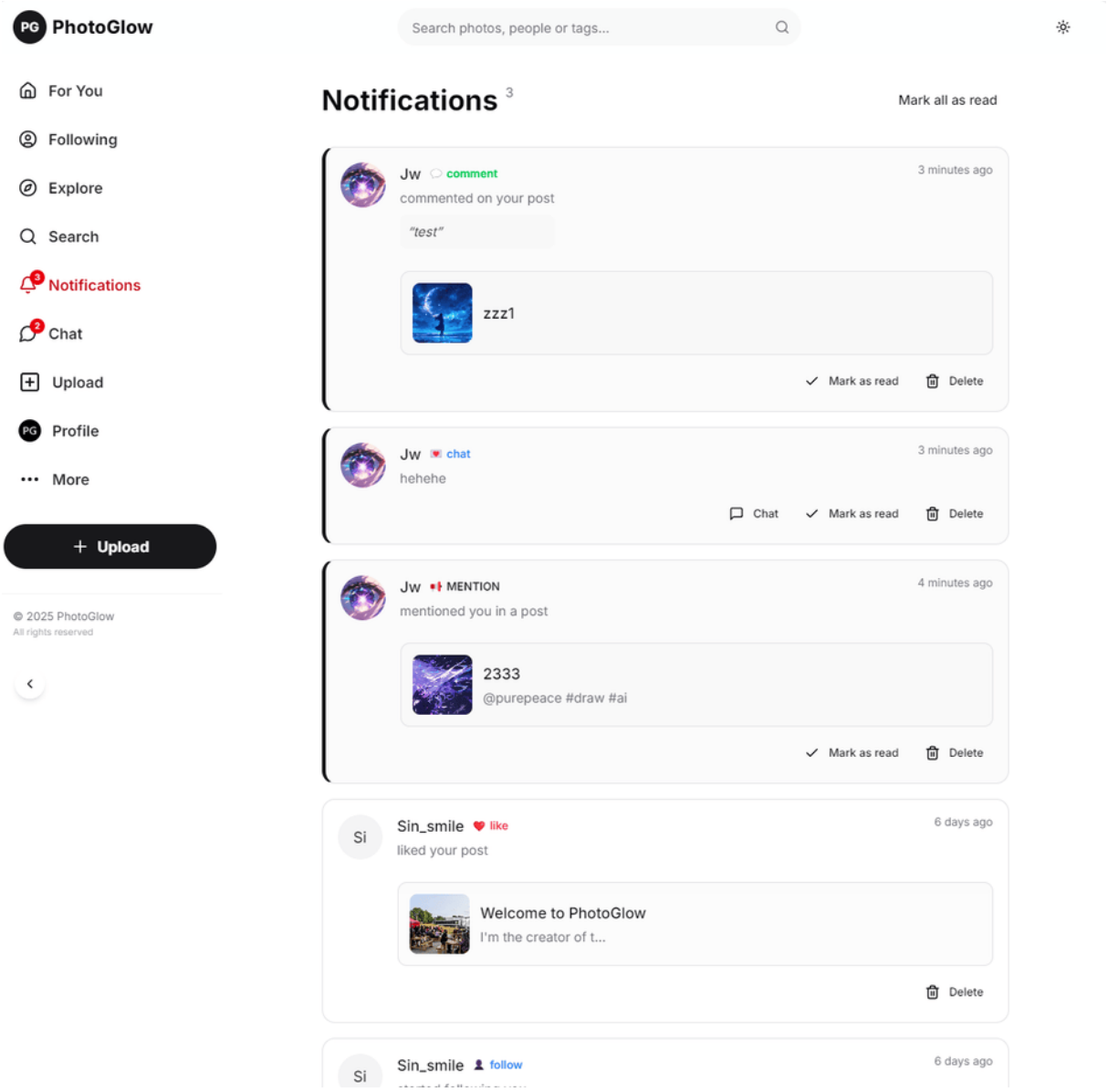


Figure 88: Notifications PC

Notification categories:

- like: Other users liking your posts or replies
- follow: New followers
- comment: Comments on your posts
- reply: Replies to your comments
- mention: Mentions in posts/comments
- chat: Private message notifications

Notifications Page Design:

The notification system implements several user-friendly features:

- Real-time unread notification count displayed as a red bubble in the navigation bar
- Global "Mark all as read" functionality
- Associated post data display including title and summary when available
- Comment content display for user comments
- Chat button in the lower right of chat notifications for quick responses
- "Load More" button for pagination until all notifications are loaded

Responsive Design:

The notifications page maintains optimal user experience across different device sizes:

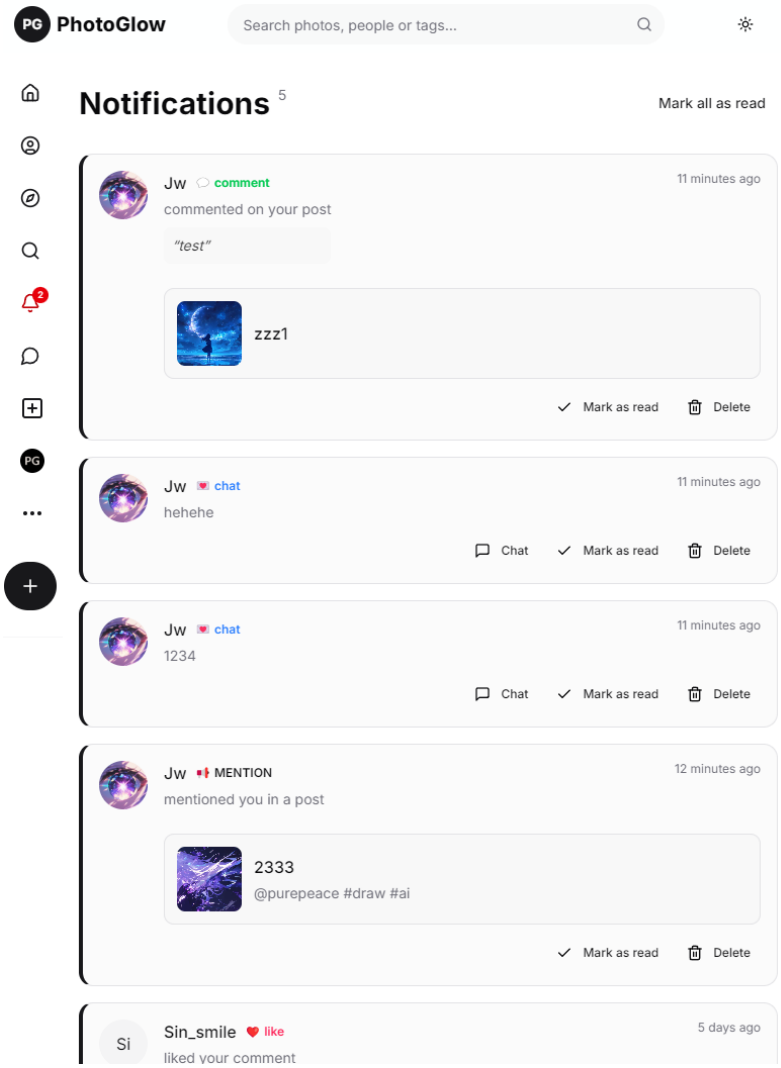


Figure 89: Notifications Tablet

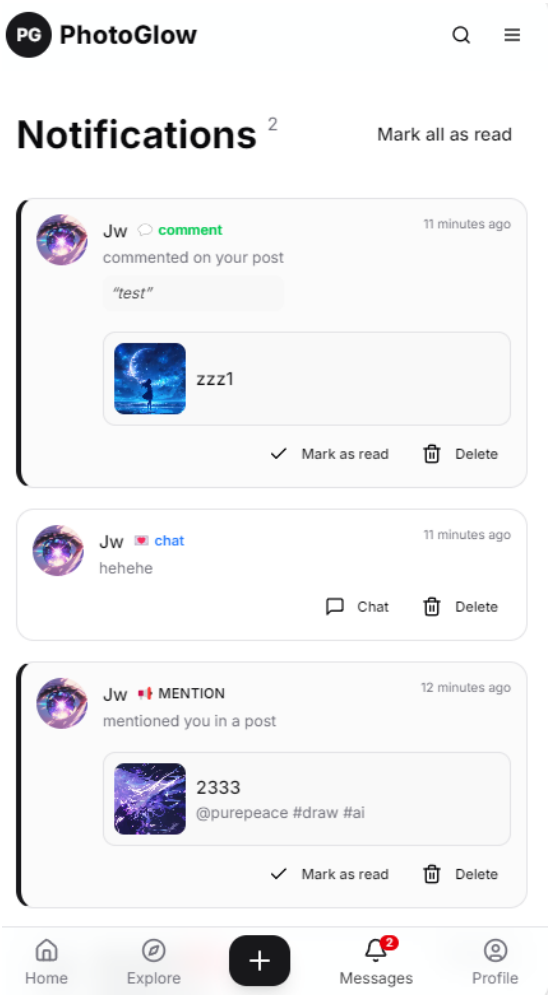


Figure 90: Notifications Mobile

3.2.5.12 Chat Page The Chat feature enables users to send private messages to other users, similar to traditional messaging applications.

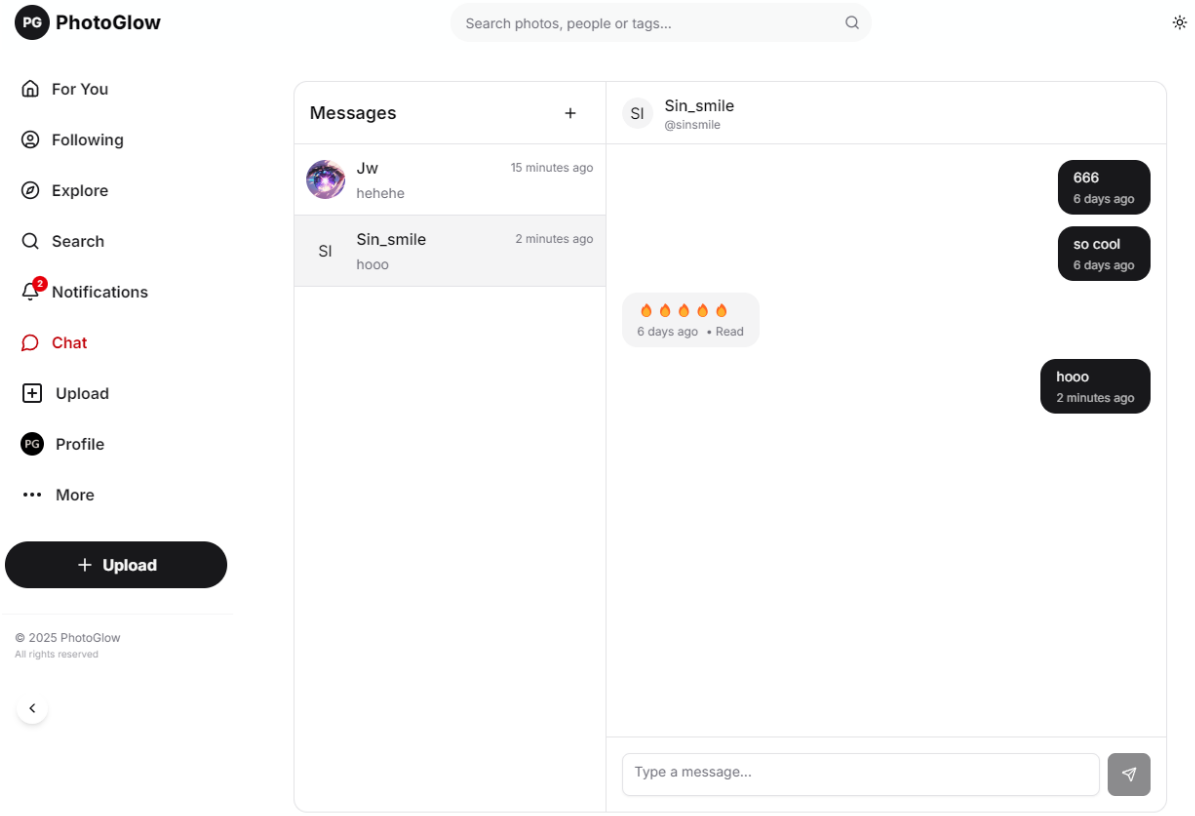


Figure 91: Chat PC

Users can initiate new conversations through multiple entry points:

- Clicking the Send Messages button on another user's profile page
- Responding to notifications by clicking the Chat button in the Notifications section
- Using the + button on the page to open the New Message dialog and entering a specific username

New Message Dialog:

The dialog provides a user search functionality, allowing users to select a recipient and begin a new conversation.

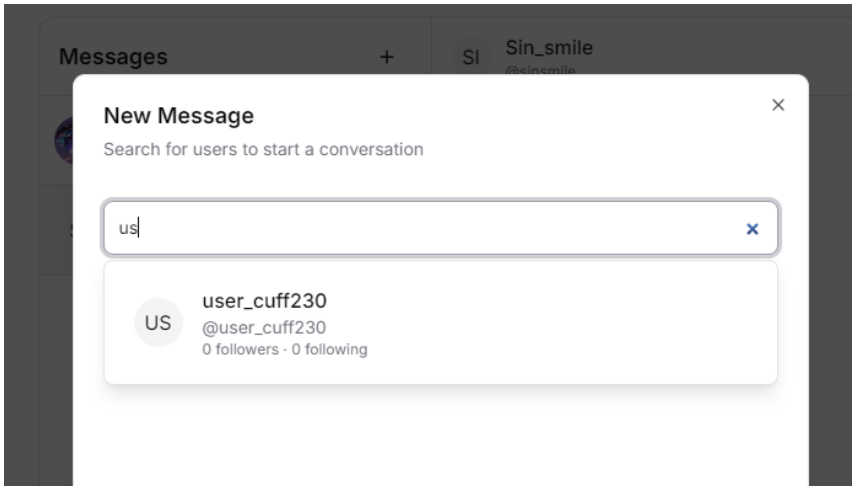


Figure 92: Chat-new-messages

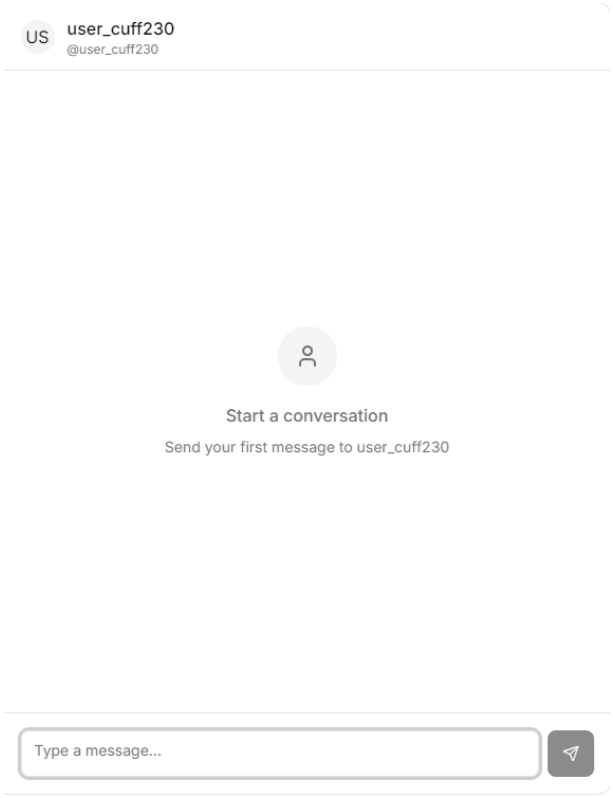


Figure 93: Chat-new-conversation

Chat Implementation:

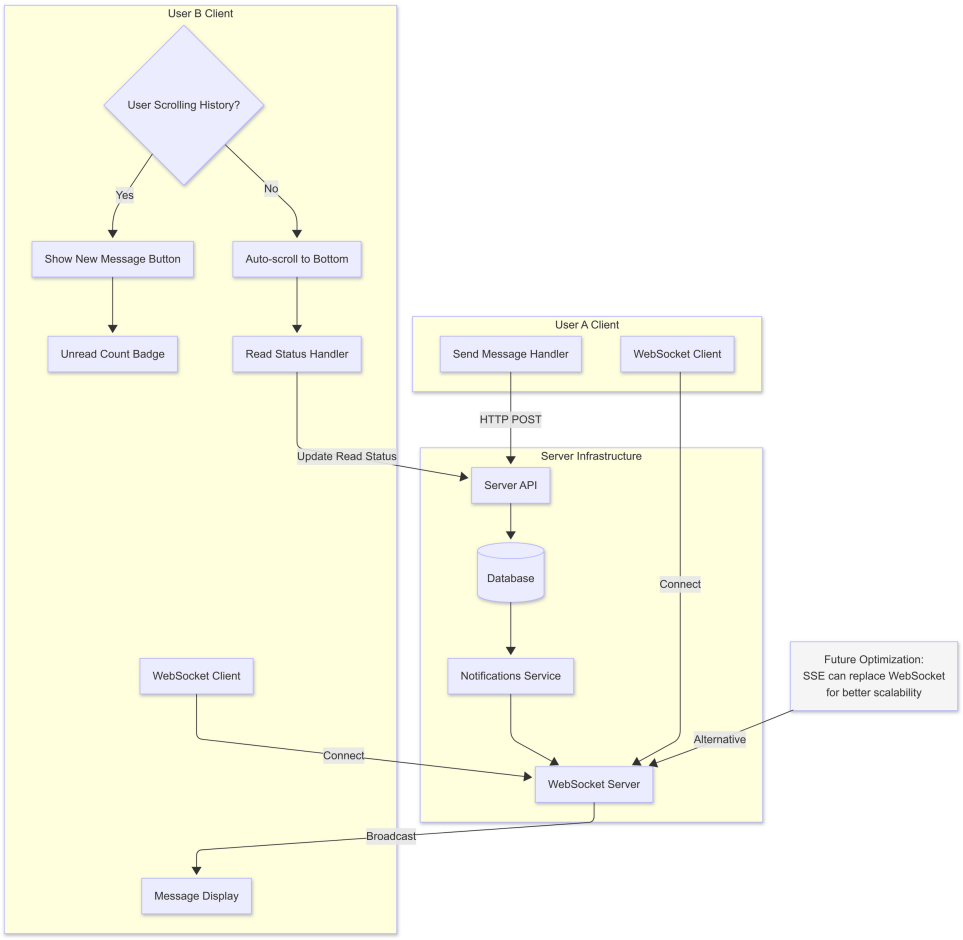


Figure 94: Chat-sequence

The chat system implements several sophisticated features:

- Real-time Push: Attempts to establish real-time communication through WebSocket connections (using socket.io)
- Message Delivery: Due to WebSocket reliability concerns, message sending still utilizes HTTP API. Messages are first persisted in the database and create Notifications, followed by WebSocket broadcast attempts
- Read Receipt: Displays "Read" under message bubbles to indicate message status
- Scroll Optimization: New messages automatically scroll to bottom by default. However, if a user is reviewing message history, the message list maintains its position and displays a button prompting users to view new messages, preventing interruption of message review
- Real-time Reception: Messages are received in real-time when both users have established WebSocket connections with the server API
- Notification Bubbles: Displays unread message count
- TODO: WebSocket push functionality can be replaced with SSE or similar solutions for performance optimization at scale

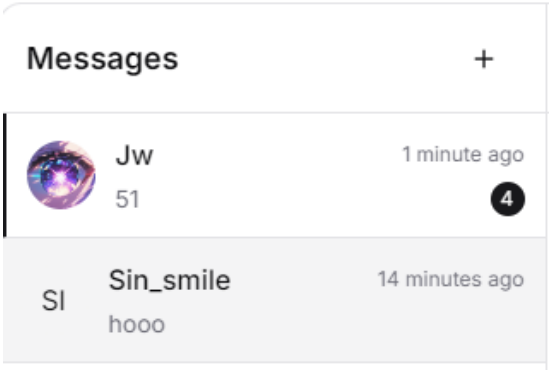


Figure 95: Chat-Notification Bubble Design

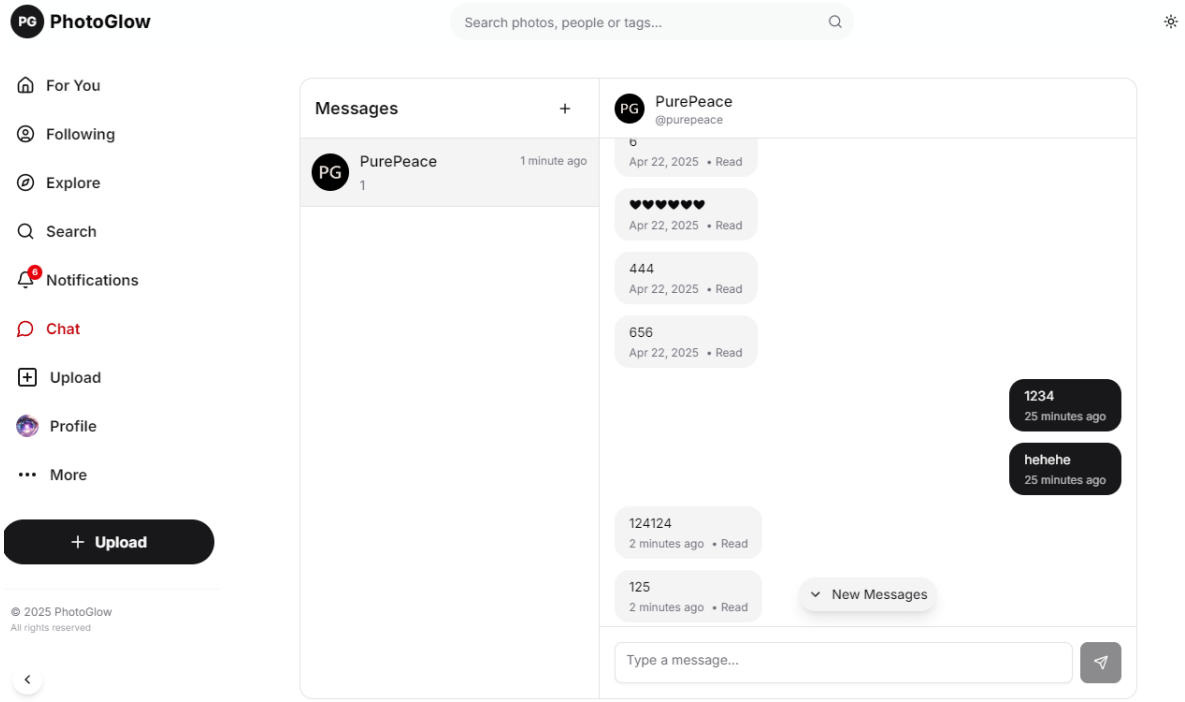


Figure 96: Chat-Message Scroll Optimization

Responsive Design:

On narrow screens and mobile devices (width less than 1024px), the interface adapts to a layout more reminiscent of traditional chat applications.

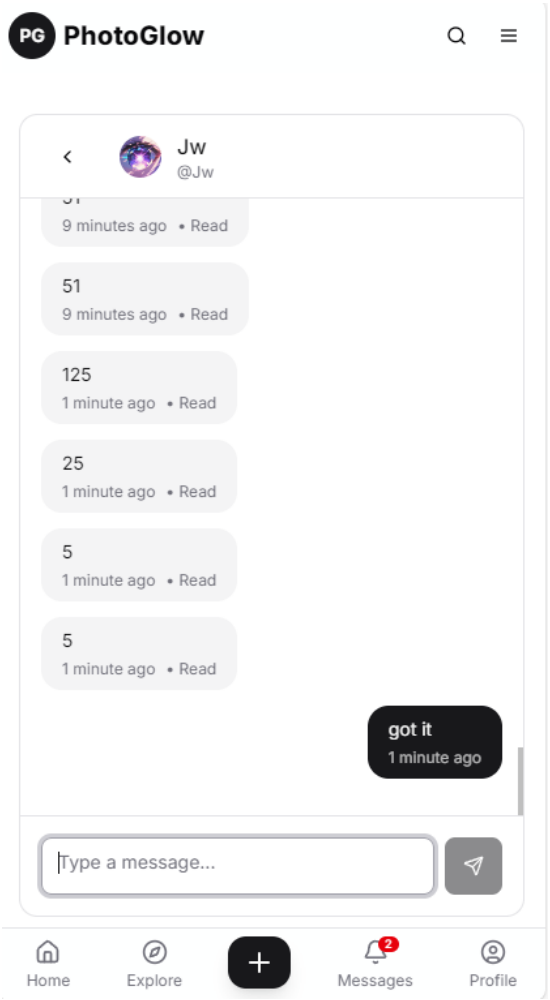


Figure 97: Chat-Message List-mobile

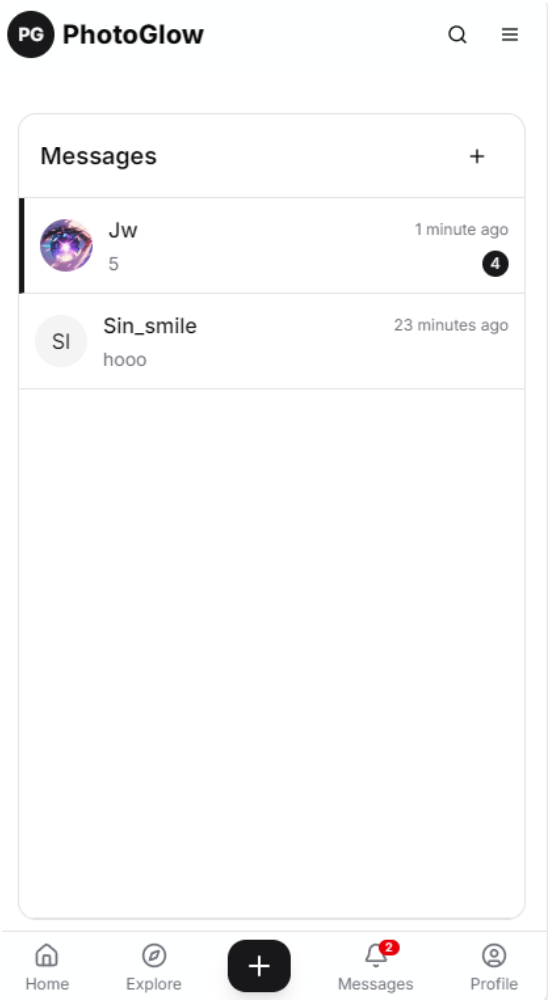


Figure 98: Chat-Chat Page-mobile

3.2.5.13 Account Settings Page The Account Settings page provides comprehensive account management capabilities, including security settings, third-party account linking, blocked user management, and account deletion options.

The page features a navigation bar and a large settings card, organized into the following sections:

- User Info
- Third Party Accounts
- Change Password
- Reset Password
- Change Email
- Blocked Users
- Delete Account

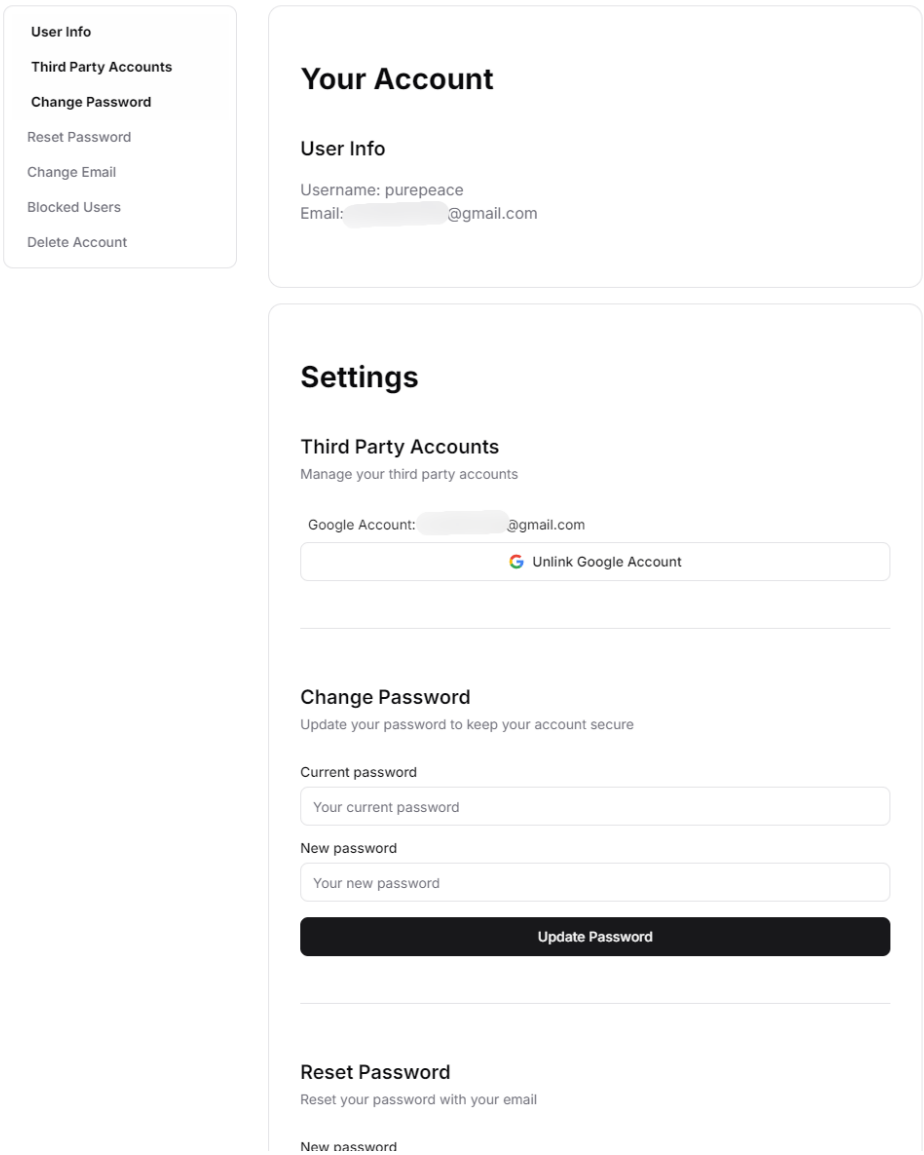


Figure 99: Settings

Implementation details:

- Each section is implemented as an independent component, ensuring isolation between different functionalities
- Every section maintains its own form validation logic
- Navigation items scroll to their corresponding sections when clicked, with the navigation bar highlighting the currently visible sections in bold

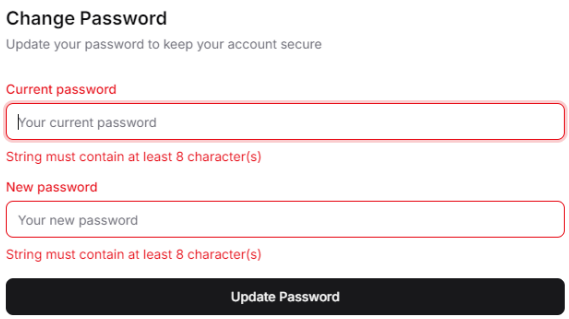


Figure 100: Settings-change-password Form Validation

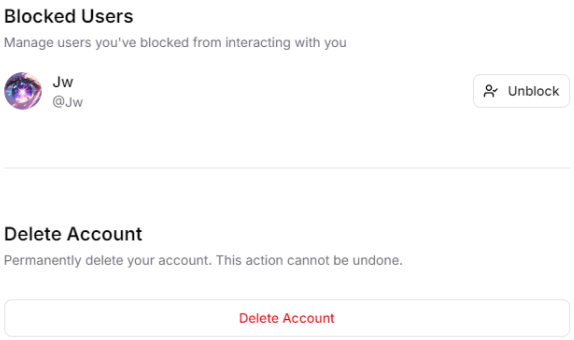


Figure 101: Settings Blocked User Management and Account Deletion Sections

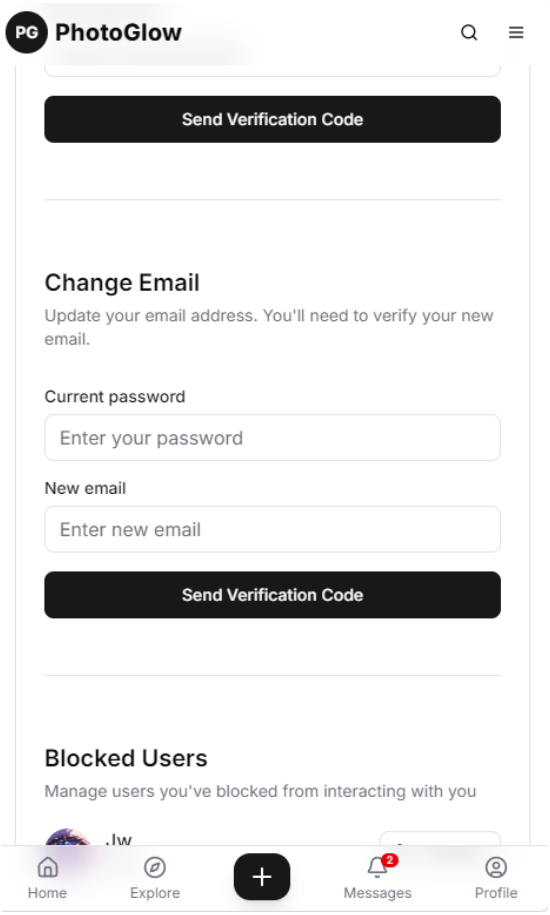


Figure 102: Settings Responsive Design

3.2.6 Frontend Implementation

Due to space constraints, only key code sections are presented.

3.2.6.1 Root Component The root component encapsulates basic server-side rendering logic and is located at `/web/src/app/layout.tsx`:

```
export default async function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode
}>) {
  const authData = await ssrAuth()

  return (
    <html
      lang="en"
      suppressHydrationWarning
      className={`${inter.className} antialiased`}
    >
      <body>
        <NextTopLoader />
        <SSRAuthProvider ssrAuth={authData}>
          <AuthProvider />
          <NotificationProvider />
          <ThemeProvider
            attribute="class"
```

```
            defaultTheme="system"
            enableSystem
            disableTransitionOnChange
          >
            <MainLayout>{children}</MainLayout>
            <AuthDialog />
            <LogoutDialog />
            <PostDialog />
            <UploadDialog />
            <Toaster position="bottom-center" offset={50} duration={3000} />
          </ThemeProvider>
        </SSRAuthProvider>
      </body>
    </html>
  )
}
```

The `RootLayout` function executes on every server-side render, serving as a critical entry point for the application's authentication flow.

3.2.6.1.1 Authentication Verification The most crucial component of the authentication process is the `const authData = await ssrAuth()` function call. This operation performs several essential tasks: it extracts the refresh token from client-side cookies, sends this token to the server API for validation, obtains a new access token, and retrieves the user's information.

The authentication logic is implemented in `/web/src/lib/auth.ts` - ssrAuth:

```
export const ssrAuth = cache(async (): Promise<UserWithAccessToken | null> => {
  const headersList = await headers()
  let accessToken = headersList.get(TokenType.ACCESS_TOKEN)

  // if accessToken is not exist, try to refresh it
  if (!accessToken) {
    const cookie = await cookies()
    const refreshToken = cookie.get(TokenType.REFRESH_TOKEN)?.value

    // if refreshToken is not exist, return null
    if (!refreshToken) return null

    const newAccessToken = await api.refreshAccessToken(refreshToken)
    if (!newAccessToken) return null

    accessToken = newAccessToken as string
  }

  // fetch user info
  const user = await api.getUserInfo(accessToken)
  if (!user) return null

  return {
    accessToken,
    user,
  }
})
```

The `ssrAuth` function is called by multiple server components throughout the application. To optimize performance and maintain consistency, the function implements result caching. This caching mechanism ensures that the authentication process executes only once during a complete server-side rendering cycle, regardless of how many components request authentication data.

After execution, the resulting `authData` is passed to the `SSRAuthProvider`, which serves as a central

authentication context for the application:

```
const SSRAuthContext = createContext<UserWithAccessToken | null>(null)

export const useSSRAuth = () => {
  const context = useContext(SSRAuthContext)
  if (context === undefined) {
    throw new Error('useSSRAuth must be used within a SSRAuthProvider')
  }
  return context
}

// ssr only
export function SSRAuthProvider({
  children,
  ssrAuth,
}: {
  children: React.ReactNode
  ssrAuth: UserWithAccessToken | null
}) {
  return (
    <SSRAuthContext.Provider value={ssrAuth}>
      {children}
    </SSRAuthContext.Provider>
  )
}
```

This Context component plays a crucial role in synchronizing server-side state with client-side components. Through the `useSSRAuth` hook, client-side components can access the authentication state that was established during the server-side rendering phase, ensuring consistency across the application.

After this synchronization, the root component initializes the `AuthProvider`. This component serves a vital function in the authentication flow: once client-side hydration is complete, it sets up the initial state for the Zustand auth store during client-side rendering. This step is essential because once completed, any React component throughout the application can dynamically access the client-side authentication state using the `useAuthStore` hook, creating a seamless authentication experience across the entire application.

```
/**
 * sync auth data from server to client, the data is stored in the browser's memory
 */
export const AuthProvider = () => {
  const ssrAuth = useSSRAuth()
  const { setAccessToken, setUser, clearAuth, setIsReady } = useAuthStore()

  useEffect(() => {
    if (ssrAuth) {
      setAccessToken(ssrAuth.accessToken)
      setUser(ssrAuth.user)
    } else {
      clearAuth()
    }

    setIsReady(true)
  }, [ssrAuth])

  return null
}
```

3.2.6.1.2 Additional Components in the Root Component The root component incorporates several essential components that provide core functionality across the application:

- `NextTopLoader`: Provides a global progress indicator at the top of the page, offering visual feedback during page transitions and loading states.
- `NotificationProvider`: Manages and automatically refreshes the latest notifications throughout the application, ensuring users stay informed of new activity.
- `ThemeProvider`: Enables theme mode switching functionality, allowing users to toggle between different visual themes.
- `MainLayout`: Implements the fundamental page framework that structures the entire application.
- `AuthDialog`: Serves as a global dialog component for authentication-related interactions.
- `LogoutDialog`: Functions as a global dialog component managing logout confirmations.
- `PostDialog`: Acts as a global dialog component for post-related interactions.
- `UploadDialog`: Operates as a global dialog component handling file upload interactions.
- `Toaster`: Provides global toast notifications for system feedback and alerts.

3.2.6.2 Page Framework Implementation The page framework `MainLayout` is implemented in `/web/src/components/main-layout.tsx` and consists of several key structural elements:

- Main Container: `<div className="relative flex min-h-screen flex-col supports-[dvh]:min-h-[100dvh]"` - Creates a responsive container that adapts to the viewport height while maintaining a minimum height requirement.
- `SiteHeader`: Implements the top navigation bar of the application.
- `SiteSidebar`: Provides the side navigation bar for desktop and tablet views.
- `MobileNav`: Delivers a specialized navigation interface optimized for mobile devices.
- `main`: Defines the primary content area where page-specific content is rendered.

```
export function MainLayout({ children }: { children: React.ReactNode }) {
  const { isCollapsed } = useSidebarStore()

  return (
    <div className="relative flex min-h-screen flex-col supports-[dvh]:min-h-[100dvh]">
      <SiteHeader />
      <div
        className={cn(
          'flex flex-1 transition-[padding] duration-150 md:pl-16 lg:pl-60',
          isCollapsed && 'md:pl-16 lg:pl-16'
        )}
      >
        <SiteSidebar />
        <main className="flex-1 pb-14 md:pb-0">{children}</main>
      </div>
      <MobileNav />
    </div>
  )
}
```

3.2.6.3 Middleware The middleware implementation, located at `/web/src/middleware.ts`, serves a specific purpose during server-side rendering by handling path rewrites. Its primary function is to transform user-friendly URLs into their internal routing counterparts.

Specifically, the middleware rewrites URLs from the format `/@{username}` to `/profile/{username}`. This transformation happens seamlessly behind the scenes, allowing users to interact with more intuitive, social media-style URLs while maintaining the application's internal routing structure.

From the user's perspective, they only encounter paths beginning with `/@username`. This design choice represents a significant user experience enhancement, following the pattern popularized by platforms like TikTok. The approach makes profile URLs more memorable, shareable, and aligned with modern social media conventions, while keeping the technical implementation clean and organized.

```
export async function middleware(request: NextRequest) {
  if (request.nextUrl.pathname.startsWith('/@')) {
    const url = request.nextUrl.clone()
```

```
    url.pathname = '/profile' + url.pathname.replace(/@/g, '')
    return NextResponse.rewrite(url)
  }
}
```

It's important to note that this middleware runs in Next.js Edge Runtime rather than Bun or Node.js, and its API is not fully compatible.

3.2.6.4 Server API Encapsulation The `/web/src/api/endpoints` directory contains server APIs used by both server-side rendering and client-side rendering.

Example encapsulation in `/web/src/api/endpoints/posts.ts`:

```
async function submitPost(
  accessToken: string,
  postData: CreatePostData
): Promise<string> {
  const res = await fetch(buildUrl('posts/create'), {
    method: 'POST',
    body: JSON.stringify(postData),
    credentials: 'include',
    cache: 'no-store',
    headers: {
      ...authorizationHeader(accessToken),
      'Content-Type': 'application/json',
    },
  })

  if (!res.ok) {
    throw await respError(res)
  }

  const { id } = await res.json()
  return id
}
```

It's worth noting that the server API address is not fixed between server-side rendering and client-side rendering.

`buildUrl` in `/web/src/lib/utils.ts` is a utility library used for constructing API encapsulation. Here is the code:

```
// API configuration
export const API_URL =
  typeof window === 'undefined'
    ? process.env.INTERNAL_API_URL
    : process.env.NEXT_PUBLIC_API_URL

// Helper function to build API URLs
export const buildUrl = (path: string) => `${API_URL}/api/v1/${path}`
```

As we can see, the `API_URL` variable is dynamically set based on the execution environment. When in server-side rendering, it uses the internal network address; otherwise, it uses the public address.

Moreover, only environment variables prefixed with `NEXT_PUBLIC` are sent to the client browser, so there's no need to worry about leaking other environment variables.

3.2.6.5 Post Submission The post submission process is complex code divided into multiple steps.

Flow diagrams:

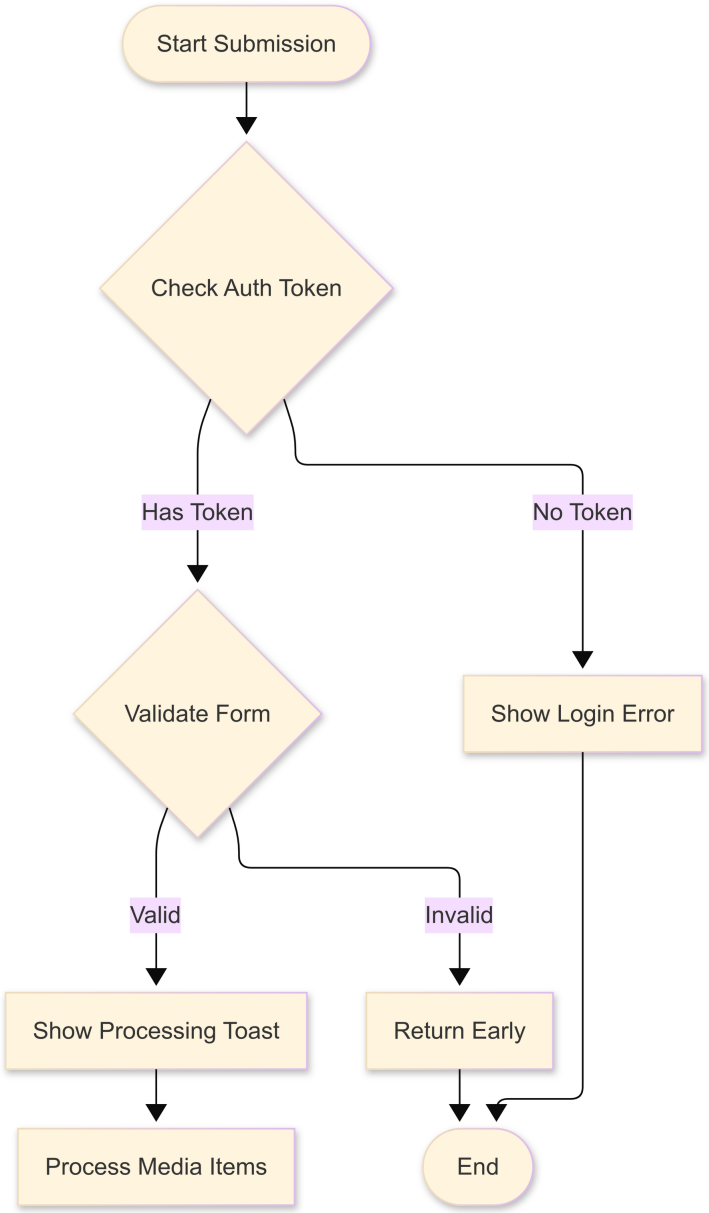


Figure 103: Post Submission 1

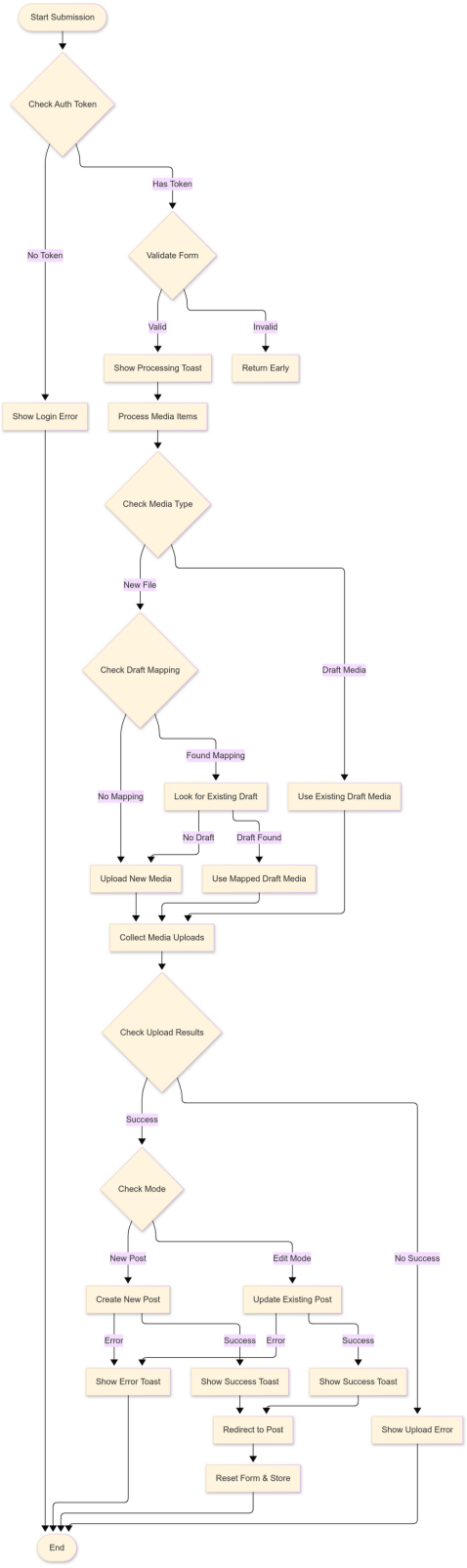


Figure 104: Post Submission 2

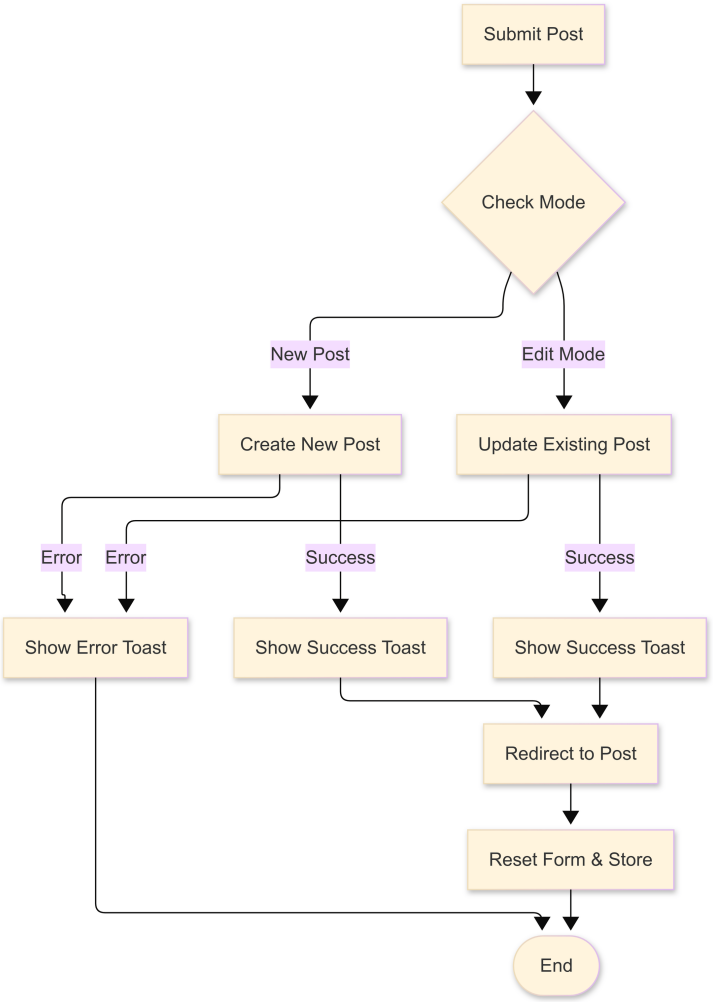


Figure 105: Post Submission 3

The core code is located in the component `web/src/components/upload/upload-main.tsx`

3.2.6.6 Post Detail Dialog This dialog has been specially handled to enable changing the browser's URL and history record when opening the dialog, while supporting browser forward/backward navigation, without triggering Next.js-managed route transitions.

The core code is located in `web/src/components/post/post-dialog.tsx`

3.2.6.7 Writing Hybrid Rendering Pages Taking the main page `/web/src/app/page.tsx` as an example:

1. This component is a server-side async component that executes only on the server.
2. After authentication is completed, it requests the initial data.
3. The initial data is passed to client-side components for rendering. Next.js will complete server-side rendering according to the client component's DOM structure on the server before sending it to the client.

```
export default async function Home() {
  const auth = await ssrAuth()
  const initialPosts = await api.getPostList({
    accessToken: auth?.accessToken,
    limit: 18,
    offset: 0,
  })
```

```
  return <HomePage initialPosts={initialPosts} ssrAuth={auth} />
}
```

3.2.6.8 Real-time Chat Information via WebSocket The core code is located in `/web/src/hooks/use-socket.ts`, which is a reusable hook:

```
export function useSocket(accessToken: string | null) {
  ...
}
```

The client-side component `/web/src/components/chat/chat-main.tsx` using this hook:

```
export function ChatMain({
  authData,
  initialData,
  selectedUserId,
}: ChatMainProps) {
  ...
}
```

3.2.6.9 Rendering Server API Responses as Rich Text The component `web/src/components/linkify/linkify.` implements a reusable rich text rendering component `Linkify`.

Using it anywhere is very simple:

```
<Linkify>{commentContent}</Linkify>
```

3.2.6.10 Implementation of Reusable Custom Rich Text Editor Component The core code is quite extensive and located in the `/web/src/components/text-editor` directory, consisting of 4 components:

- `/web/src/components/text-editor/extension-mention.tsx`: Provides suggestions when typing `@` or `#`.
- `/web/src/components/text-editor/mention-button.tsx`: Button for typing characters in the rich text input field.
- `/web/src/components/text-editor/plugin-suggestion.tsx`: Parent component of `extension-mention`.
- `/web/src/components/text-editor/text-editor-main.tsx`: Editor definition.

Usage example (from `/web/src/components/post/post-comment.tsx`):

```
const editor = useEditor({
  ...editorConfig,
  immediatelyRender: false,
  onUpdate: ({ editor }) => {
    // Move cursor to end when initializing with reply content
    if (comment && editor.getText() === `@${comment.author.username} `) {
      editor.commands.focus('end')
    }
    // Update content state when editor content changes
    setEditorContent(editor.getText())
  },
})

<TextEditorMain
  onKeyDown={(e) => {
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault()
      handleSubmit()
    }
  }}
  editor={editor!}
```

```
  className="min-h-[40px]"
  placeholder={
    comment ? `Reply to ${getReplyToName()}...` : 'Add a comment...'
  }
/>
```

3.2.6.11 Implementation of Reusable Post Card Post Grid and Post Card are used in many components.

```
export function PostCard({ post }: PostCardProps) {
  ...
}
```

3.2.6.12 Reusable, Responsive Basic UI Components These components typically don't have complex logic and can have their styles overridden. They are generally located in the `/web/src/components/ui` directory.

Other components can be implemented by referring to the current component.

3.2.6.13 Reference Implementation for Responsive Design Taking the `/web/src/components/post/post-grid.` component as an example:

```
<div
 className={cn(
   'relative
   grid // Grid layout
   grid-cols-2 // 2 columns
   gap-x-2 // Horizontal gap 2
   gap-y-3 // Vertical gap 3
   sm:grid-cols-2 // Media query, small breakpoint: 2 columns
   md:grid-cols-3 // Media query, medium breakpoint: 3 columns
   md:gap-x-4 // Media query, medium breakpoint: horizontal gap 4
   md:gap-y-6 // Media query, medium breakpoint: vertical gap 6
   lg:grid-cols-4 // Media query, large breakpoint: 4 columns
   xl:grid-cols-5 // Media query, extra-large breakpoint: 5 columns
   2xl:grid-cols-6  // Media query, 2x extra-large breakpoint: 6 columns
   ',
   className
 )}
>
 {posts.map((post, index) => (
   <PostCard key={`${post.id}-${index}`} post={post} />
 ))}

 {/* Loading indicator */}
 {loading && (
   <div className="absolute bottom-0 left-1/2 z-10 h-8 -translate-x-1/2">
     <div className="border-muted-foreground size-6 animate-spin rounded-full border-2 border-t-trans
   </div>
 )}
</div>
```

Responsiveness is primarily achieved through CSS media queries, which Tailwind CSS has encapsulated into custom directives like `sm:xxx, xl:xxx` for convenient use.

3.2.6.4 Summary Above are the most core and necessary frontend code explanations, which handle critical server-side rendering steps. For the remaining client-side components, you can refer directly to React.js and Next.js documentation.

3.2.7 Performance Optimization and Monitoring

1. **Loading Performance Optimization**

- Implementation of route-based code splitting
- Lazy loading and automatic optimization of images
- Component-level dynamic imports
- Resource preloading strategy

2. **Runtime Performance Optimization**

- Usage of React.memo to prevent unnecessary re-renders
- Implementation of virtualized scrolling lists
- Optimization of state update logic
- Caching of computed results

3. **Monitoring Metrics**

- First Contentful Paint (FCP)
- Largest Contentful Paint (LCP)
- Time to Interactive (TTI)
- Cumulative Layout Shift (CLS)

4. **Optimization Results**

- Lighthouse performance score improved to 95+
- First screen loading time reduced by 40%
- Memory usage reduced by 30%
- User interaction latency reduced by 50%

3.2.8 Future Optimization Plans

1. **Performance Optimization**

- Implementation of Service Worker caching
- Addition of PWA support
- Optimization of image loading strategy
- Implementation of pre-rendering

2. **Feature Enhancement**

- Addition of advanced search functionality
- Implementation of AI-assisted tag recommendations
- Support for more media types
- Enhancement of social interaction features

3. **User Experience Improvements**

- Optimization of mobile experience
- Addition of gesture operation support
- Improvement of offline experience
- Addition of more customization options

3.3 Backend Design

The backend service `@photoglow/server` adopts a modern microservices architecture based on NestJS, implementing a highly scalable and maintainable server application through advanced design concepts such as Dependency Injection and Aspect-Oriented Programming. This section will deeply analyze the design philosophy, architectural implementation, and key features of the backend system.

3.3.1 Technology Stack Selection

The backend service utilizes the latest released technology stack:

1. **Core Framework**

- NestJS 11 (January 2025): NestJS was chosen as the core framework primarily for its comprehensive dependency injection system and decorator pattern implementation. These features enable loosely coupled modular design while providing complete TypeScript support, ensuring code type safety.
- Prisma ORM: Compared to traditional TypeORM, Prisma offers more powerful type inference capabilities and higher performance. Its automatically generated type definitions and query builders significantly improve development efficiency and code reliability. Type sharing with the frontend is achieved through the custom `@photoglow/database` wrapper layer.
- Redis: As a core component for distributed caching and message queuing, Redis was chosen based on its excellent performance and rich data structure support. It is primarily used in the system for session management, real-time data caching, and message publish-subscribe.
- AWS SDK: Cloud service integration uses AWS SDK, mainly considering its comprehensive ecosystem and powerful scalability. The integration process with S3 storage and SES email services is simplified through a unified SDK interface.

2. **Authentication and Security**

- JWT: Implementation of token-based authentication
- OAuth 2.0: Support for third-party login
- Use of `.dto` files for API request data validation
- Use of ORM to prevent security vulnerabilities such as SQL injection

3. **Real-time Communication**

- Socket.IO: Implementation of WebSocket bidirectional communication

4. **Media Processing**

- Multer: Handling file uploads
- AWS S3: Cloud media storage

3.3.2 Architecture Design

The backend adopts a layered architecture design, ensuring system maintainability and scalability through clear responsibility division:

Project Structure

```
server/
  src/                          # Source code directory
      bootstrap.ts              # Application bootstrap configuration
      generateSwagger.ts        # Swagger documentation generator
      main.ts                   # Application entry point
      schema.gql                # GraphQL schema definition

      app/                      # Core application setup
          app.controller.ts     # Main application controller
          app.module.ts         # Root application module
          app.resolver.ts       # GraphQL resolver
          app.service.ts        # Core application service

      common/                   # Shared utilities and helpers
          config/               # Configuration settings
              multer.config.ts
          decorators/           # Custom decorators
              current-user.decorator.ts
              username.decorator.ts
          guards/               # Authentication guards
              jwt-auth.guard.ts
          utils/                # Utility functions
              common.ts
              token.ts
```

```
    infra/                  # Infrastructure services
        cookie/            # Cookie management
        email/             # Email service integration
        global/            # Global module configuration
        oauth/             # OAuth authentication
        prisma/            # Database ORM setup
        redis/             # Redis cache service
        s3/                # AWS S3 storage service
        tags/              # Tag management service
        token/             # Token management
        verification/      # User verification service

    modules/               # Feature modules
        auth/              # Authentication
            dto/
            auth.controller.ts
            auth.module.ts
            auth.service.ts
        chat/              # Real-time chat
            dto/
            chat.controller.ts
            chat.gateway.ts
            chat.module.ts
            chat.service.ts
        comments/          # Comment management
        interactions/      # User interactions
        media/             # Media handling
            media-cleanup.service.ts
            media.controller.ts
            media.module.ts
            media.service.ts
        mentions/          # @mentions system
        moderation/        # Content moderation
        notifications/     # User notifications
        posts/             # Post management
        relationships/     # User relationships
        search/            # Search functionality
        settings/          # User settings
        topics/            # Topic management
        user/              # User management
            dto/
            user.controller.ts
            user.module.ts
            user.service.ts
  test/                    # Test files
  scripts/                # Utility scripts
  dist/                   # Compiled output
  node_modules/           # Dependencies
```

The project follows a modular architecture pattern with clear separation of concerns:

- **Root Files**
  - `bootstrap.ts`: Application bootstrap configuration
  - `main.ts`: Application entry point
  - `schema.gql`: GraphQL schema definition
  - `generateSwagger.ts`: API documentation generator
- **Core Components**
  - `app/`: Core application setup with main module, controller, and service

  - `common/`: Shared utilities, decorators, guards, and configurations
  - `infra/`: Infrastructure services (Redis, S3, Email, etc.)
- **Feature Modules** Each module typically contains:
  - `*.controller.ts`: REST API endpoints
  - `*.service.ts`: Business logic
  - `*.module.ts`: Module configuration
  - `dto/`: Data Transfer Objects
  - Additional files for specific features (e.g., `*.gateway.ts` for WebSocket)

The architecture follows NestJS best practices with:

- Clear module separation
- Dependency injection pattern
- RESTful and GraphQL API support
- WebSocket integration for real-time features
- Comprehensive infrastructure services

3.3.3 API Endpoint Design

The Server API currently has 15 categories, with a total of 53 API endpoints.

1. App

- `GET /api/v1/running`

2. Authentication

- `GET /api/v1/auth`
- `Get current user info`
- `GET /api/v1/auth/oauth-accounts`
- `GET /api/v1/auth/google`
- `DELETE /api/v1/auth/google`
- `GET /api/v1/auth/google/callback`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/register`
- `POST /api/v1/auth/logout`
- `POST /api/v1/auth/refresh`
- `POST /api/v1/auth/verification/send`

3. Chat

- `GET /api/v1/chat`
- `POST /api/v1/chat/create`
- `POST /api/v1/chat/{id}/send`
- `POST /api/v1/chat/messages/read`

4. Comments

- `GET /api/v1/comments/posts/{postId}`
- `POST /api/v1/comments/posts/{postId}/create`
- `DELETE /api/v1/comments/{commentId}`

5. Mention Suggestions

- `GET /api/v1/mentions/users`
- `GET /api/v1/mentions/topics`

6. Interactions

- `POST /api/v1/actions/likes/comments/{commentId}`
- `POST /api/v1/actions/likes/posts/{postId}`
- `POST /api/v1/actions/saves/posts/{postId}`

7. Media

- `GET /api/v1/media/my/drafts`

- `POST /api/v1/media/upload`

8. Moderation

- `POST /api/v1/moderation/report/post`
- `POST /api/v1/moderation/report/user`

9. Notifications

- `GET /api/v1/notifications`
- `GET /api/v1/notifications/count`
- `POST /api/v1/notifications/read`
- `DELETE /api/v1/notifications/{notificationId}`

10. Posts

- `GET /api/v1/posts`
- `GET /api/v1/posts/{postId}`
- `PUT /api/v1/posts/{postId}`
- `DELETE /api/v1/posts/{postId}`
- `POST /api/v1/posts/create`

11. Relationships

- `POST /api/v1/social/follow/{userId}`
- `GET /api/v1/social/followers/{username}`
- `GET /api/v1/social/following/{username}`
- `POST /api/v1/social/block/{userId}`
- `POST /api/v1/social/unblock/{userId}`
- `GET /api/v1/social/me/blocked`

12. Search

- `GET /api/v1/search`

13. Settings

- `PUT /api/v1/settings/password`
- `POST /api/v1/settings/password/reset`
- `PUT /api/v1/settings/email`

14. Topics

- `GET /api/v1/topics/popular`
- `GET /api/v1/topics/{topicName}`
- `GET /api/v1/topics/{topicName}/posts`

15. User

- `GET /api/v1/user/{username}/profile`
- `PUT /api/v1/user/profile`
- `DELETE /api/v1/user`

3.3.3 Core Functionality Implementation

3.3.3.1 Service Module Registration Nest.js is built on a module-service foundation, and our server API service is also based on various custom modules and services that need to be registered in `app.module.ts`.

```
@Module({
  imports: [
    ...
  ],
  controllers: [AppController],
  providers: [AppService, AppResolver],
})
export class AppModule {}
```

We registered some modules containing Controllers (used to provide API endpoints) and some global modules in the App. Other modules manage their own dependencies. For example:

```
@Module({
  imports: [RedisModule],
  providers: [VerificationService],
  exports: [VerificationService],
})
export class VerificationModule {}
```

The Verification module uses the Redis module, so it only needs to be added separately.

3.3.3.2 Authentication System The authentication system employs a Dual Token strategy, combining JWT and Redis to implement secure and reliable authentication:

1. **Access Token**

- Short-term validity (1 hour)
- Stateless design, reducing database queries
- Transmitted via Authorization Bearer
- JWT payload containing basic user information

2. **Refresh Token**

- Long-term validity (30 days)
- PostgreSQL persistent storage
- Supports session management and token revocation
- Implements automatic renewal mechanism

3. **OAuth Integration**

- Supports third-party login with Google, GitHub, etc. (Google currently supported)
- Unified user information mapping
- Automatic account association

`token.service.ts` contains the crucial implementation of TokenService

And `jwt-auth.guard.ts` provides two frequently used guards in controllers for decoding JWT authentication information from requests.

```
@Injectable()
export class JwtAuthGuard implements CanActivate {
  ...
}
```

Usage example in `posts.controller.ts`:

After adding the `@UseGuards(JwtAuthGuard)` decorator to the controller function `updatePost`, we can obtain `@CurrentUser() user: { id: string }` from parameter dependency injection.

If JWT verification passes, user.id will definitely exist; otherwise, an authentication error (401) will be thrown.

```
  @Put(':postId')
  @UseGuards(JwtAuthGuard)
  @ApiUnauthorizedResponse({ description: 'Not authorized to update this post' })
  @ApiResponse({ status: 404, description: 'Post not found' })
  async updatePost(
    @CurrentUser() user: { id: string },
    @Param('postId') postId: string,
    @Body() updatePostDto: UpdatePostDto
  ) {
    const success = await this.postsService.updatePost(
      user.id,
      postId,
      updatePostDto
```

```
    )
    return { success }
  }
```

3.3.3.3 Media Processing Service The media processing service implements a complete file upload, processing, and storage workflow:

1. **Upload Processing**

- File type validation
- Size limitation (maximum 20MB)
- Concurrent upload control

2. **Image Optimization**

- Automatic compression and format conversion
- Generation of multiple thumbnail sizes
- EXIF information preservation

3. **Storage Strategy**

- AWS S3 partitioned storage
- CDN distribution optimization
- Automatic temporary file cleanup

For Nest.js configuration:

```
export const multerConfig: MulterOptions = {
  storage: memoryStorage(),
  limits: {
    fileSize: 20 * 1024 * 1024, // 20MB limit
  },
}
```

AWS S3 service implementation is located in `s3.service.ts`:

```
@Injectable()
export class S3Service {
  ...
}
```

Usage example in `media.service.ts`:

```
async uploadMedia(
  userId: string,
  file: Express.Multer.File
): Promise<{ url: string; draftId: string }> {
  // Upload file to S3
  const uploadResult = await this.s3Service.uploadFile(file, 'media')
  ...
}
```

Additionally, the media service includes a scheduled task service `media-cleanup.service.ts` for periodically cleaning up uploaded but unused media files:

```
@Injectable()
export class MediaCleanupService {
  ...
}
```

3.3.3.4 Real-time Communication System The real-time communication system is implemented based on WebSocket, supporting various real-time functionalities:

The WebSocket gateway for chat functionality is defined in `chat.gateway.ts`.

1. **Chat Service**

```
export class ChatGateway implements OnGatewayConnection, OnGatewayDisconnect {
  ...
}
```

2. **Notification Push**

- Instant message pushing
- Online status synchronization
- Message persistent storage

3. **Connection Management**

- Automatic reconnection mechanism
- Heartbeat detection
- Session state maintenance

3.3.3.5 Caching Strategy The system implements a multi-layer caching strategy to optimize performance and resource utilization:

1. **Redis Cache**

- User session information
- Hot data caching
- Real-time counters

Taking `redis.service.ts` as an example:

```
@Injectable()
export class RedisService implements OnModuleInit, OnModuleDestroy {
  ...
}
```

This code assumes we have created an email template named `PHOTOGLOW_VERIFICATION_CODE` in AWS SES and formatted it with `TemplateData`.

3.3.3.7 OAuth Flow diagram:

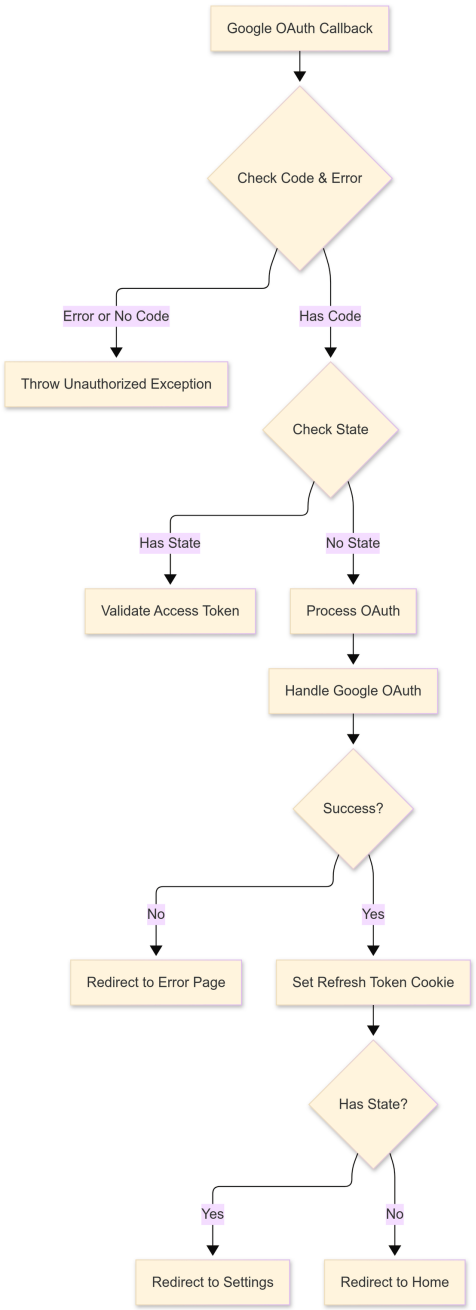


Figure 106: OAuth 1

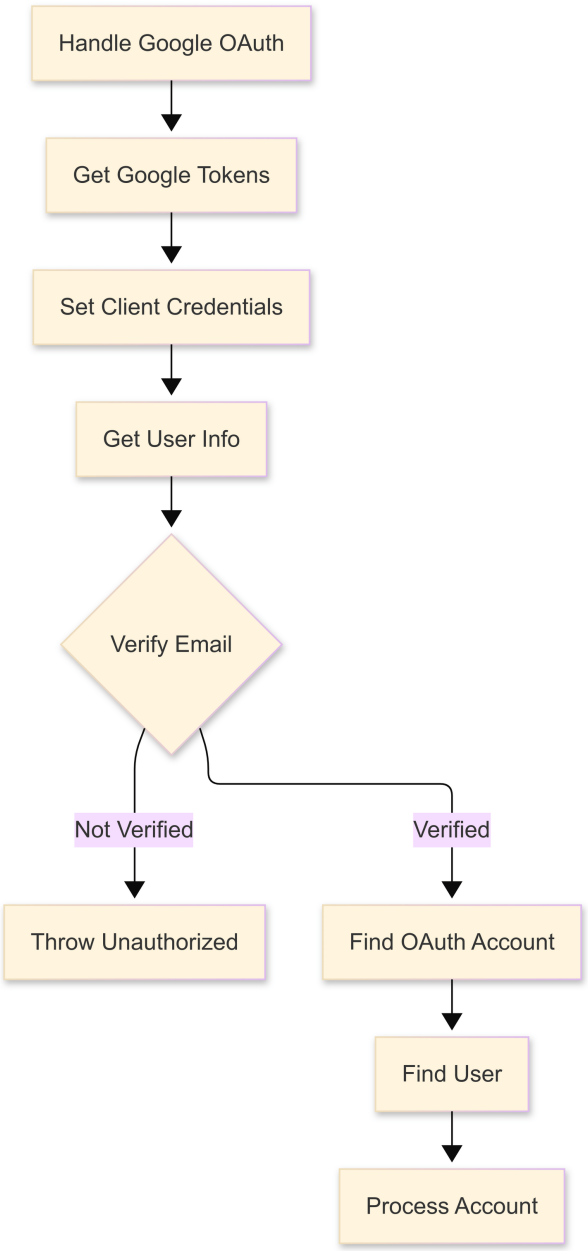


Figure 107: OAuth 2

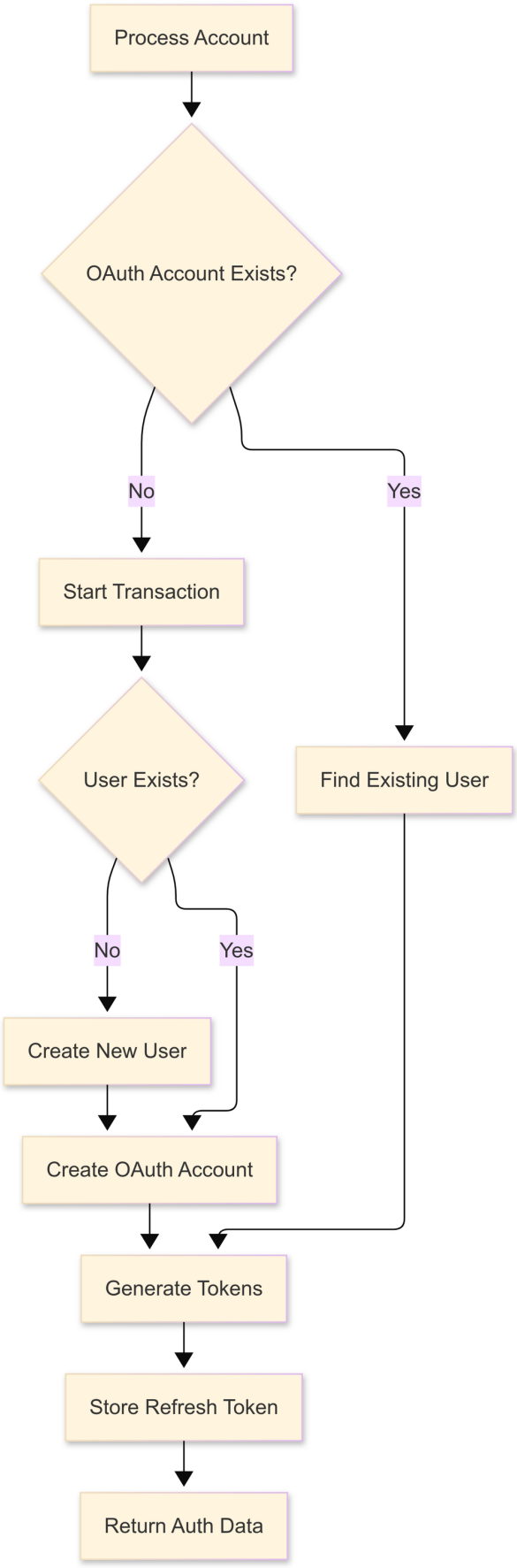


Figure 108: OAuth 3

The API endpoints handling OAuth are located in `AuthController`:

```
@ApiTags('Authentication')
@Controller('auth')
export class AuthController {
  ...
}
```

These logic operations are handled by the server, providing higher security and reliability compared to client-side processing.

Moreover, the logic for binding existing accounts and registering new accounts is separated, resulting in a clearer design.

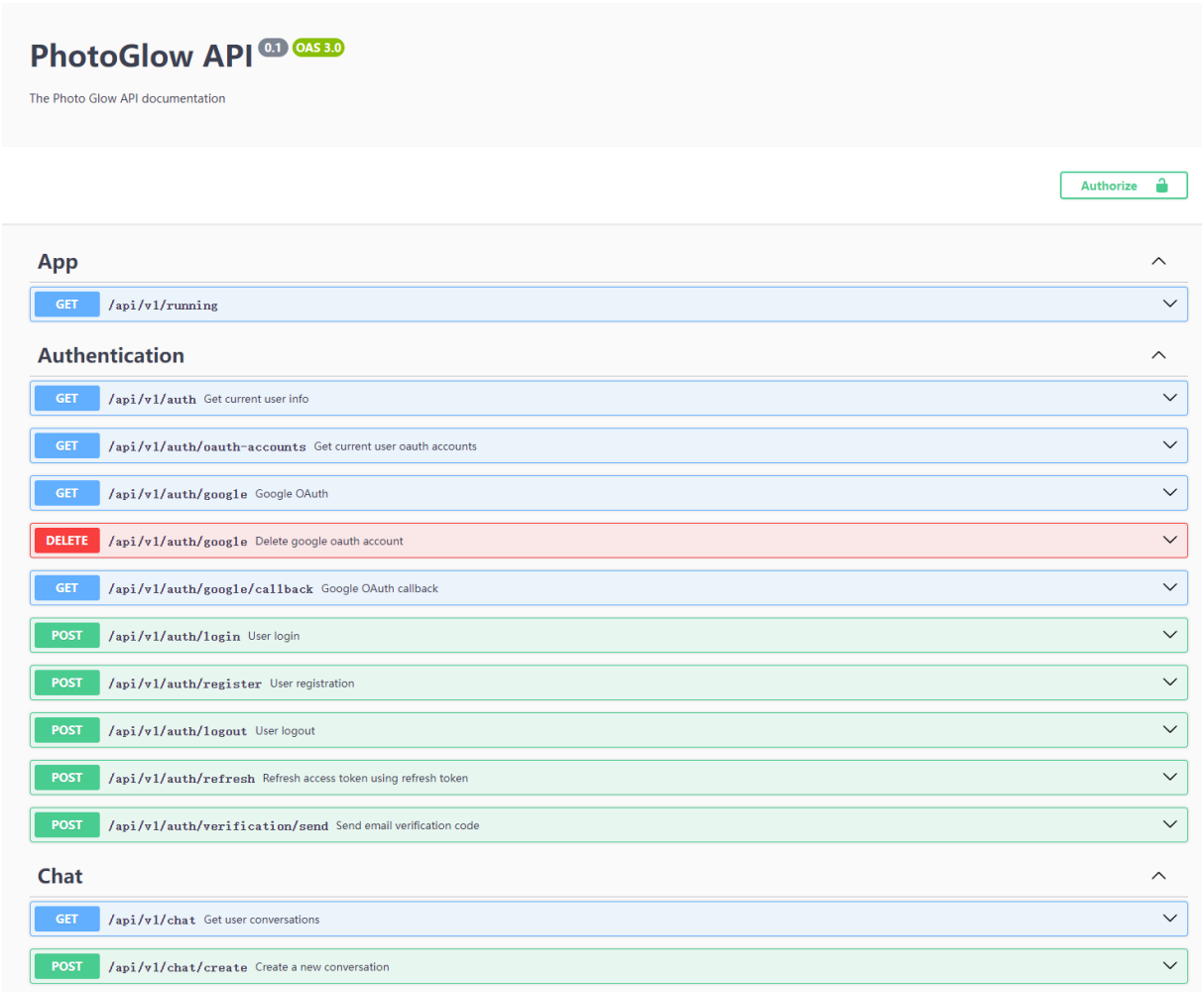


Figure 109: Swagger UI

3.3.3.8 Swagger Documentation Integration In `bootstrap.ts`, we create a Swagger instance after initializing the Nest.js instance, which generates a visual UI debugging interface and OpenAPI JSON documentation description for our backend API.

Then we can add more detailed documentation descriptions for API endpoints in any controller. Taking `posts.controller.ts` as an example:

```
  @Post('create')
  @UseGuards(JwtAuthGuard)
  @ApiOperation({
    summary: 'Create a new post',
```

```
  description: 'Create a new post with media, location, topics, and tags'
})
@ApiBody({
  type: CreatePostDto,
  description: 'Post creation data',
  examples: {
    photoPost: {
      value: {
        title: 'Beautiful sunset',
        content: 'Captured this amazing sunset today! #nature #photography',
        visibility: 'PUBLIC',
        type: 'PHOTO',
        media: [
          {
            url: 'https://example.com/sunset.jpg',
            type: 'IMAGE',
            alt: 'Sunset over the ocean',
            order: 0
          }
        ],
        locationId: '123e4567-e89b-12d3-a456-426614174000',
        topicIds: ['nature-topic-id', 'photography-topic-id'],
        tagIds: ['sunset-tag-id', 'landscape-tag-id']
      }
    }
  }
})
@ApiCreatedResponse({
  description: 'Post created successfully',
  schema: {
    type: 'object',
    properties: {
      id: {
        type: 'string',
        example: '123e4567-e89b-12d3-a456-426614174000',
        description: 'The ID of the created post'
      }
    }
  }
})
@ApiUnauthorizedResponse({ description: 'Not authenticated' })
async createPost(
  @CurrentUser() user: { id: string },
  @Body() createPostDto: CreatePostDto
) {
  const postId = await this.postsService.createPost(user.id, createPostDto)
  return { id: postId }
}
```

By using decorators like `ApiBody`, `ApiCreatedResponse`, and `ApiUnauthorizedResponse`, we provide comprehensive documentation descriptions for API endpoints.

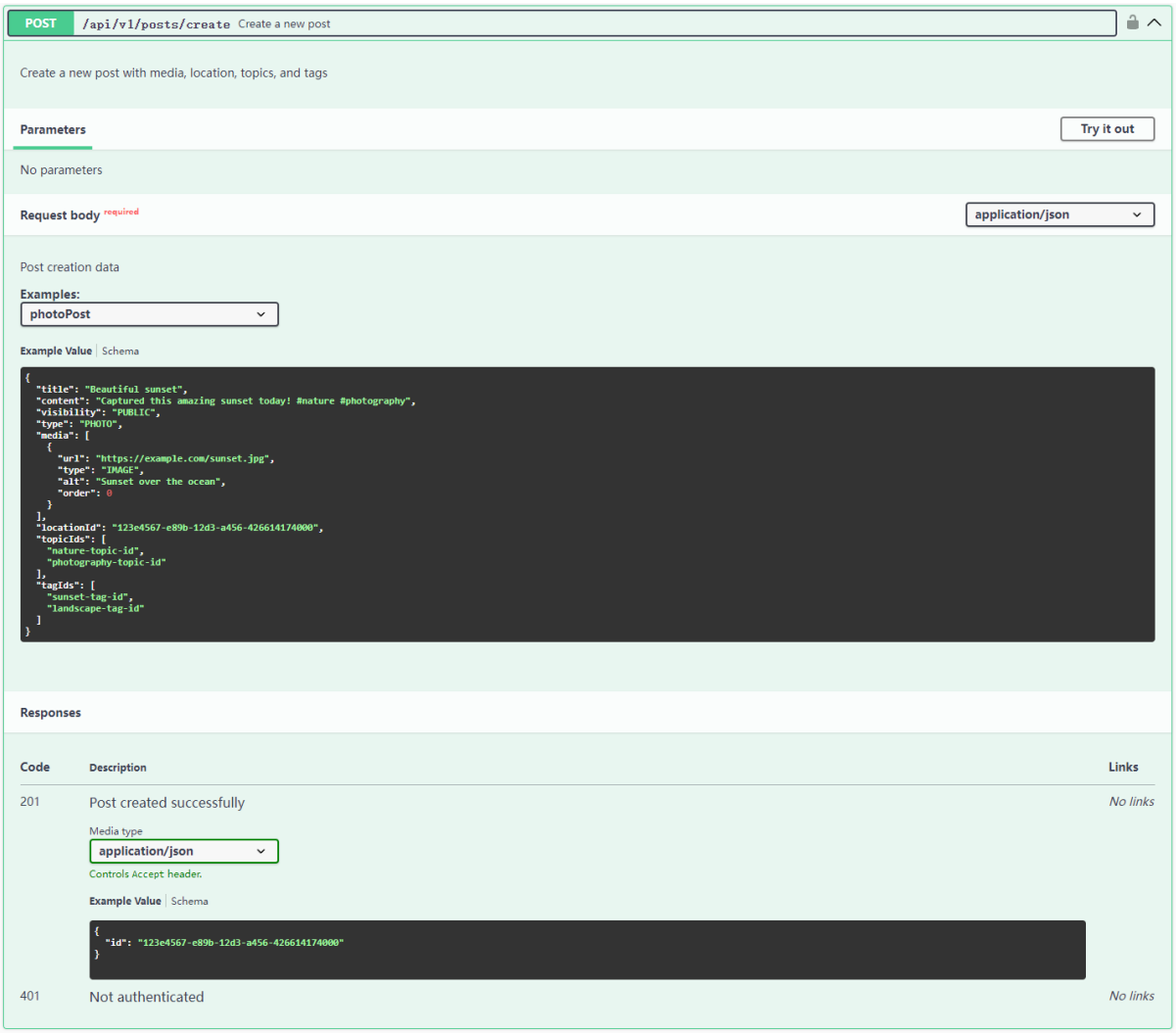


Figure 110: Post create API

3.3.4 Security Design

The system implements comprehensive security measures:

1. **Request Validation**

- Input data validation (DTO)
- SQL injection protection
- XSS attack prevention

2. **Data Security**

- Argon2i password hash protection
- Security logging

3.3.5 Performance Optimization

The system implements performance optimizations at multiple levels:

1. **Database Optimization**

- Index optimization
- Query caching
- Connection pool management

2. **API Response Optimization**

- Data compression
- Response caching
- Asynchronous processing

3. **Resource Management**

- Memory usage optimization
- Connection count control
- Task queue management

3.3.6 Scalability Design

The system's scalability is reflected in the following aspects:

1. **Modular Design**

- Independent functional modules
- Dependency injection decoupling
- Interface abstraction design

2. **Horizontal Scaling**

- Stateless service design
- Session sharing mechanism
- Load balancing support

3. **Vertical Scaling**

- Resource limit configuration
- Performance parameter tuning
- Service degradation strategy

Through this architectural design, the backend system not only implements current functional requirements but also reserves ample space for future expansion. The system's modular design and comprehensive abstraction layers enable smooth addition of new features and optimization of existing functionality. Additionally, by adopting the latest technology stack and best practices, the system ensures technical advancement and maintainability.

3.4 Database Design

PhotoGlow uses PostgreSQL as its primary database system, implementing type-safe data access through Prisma ORM. The database design follows relational database best practices, adopting a modular and domain-driven design approach to ensure data integrity, consistency, and scalability. The system's data model design fully considers the special requirements of social media platforms, including high-concurrency access, complex social relationship networks, and large-scale media content management.

3.4.1 Data Model Design

The system's data model adopts domain-driven design principles, dividing functionality into four main domains: user domain, content domain, social domain, and authentication domain. Each domain has its core entities and related supporting entities, with domain interactions implemented through carefully designed relationships.

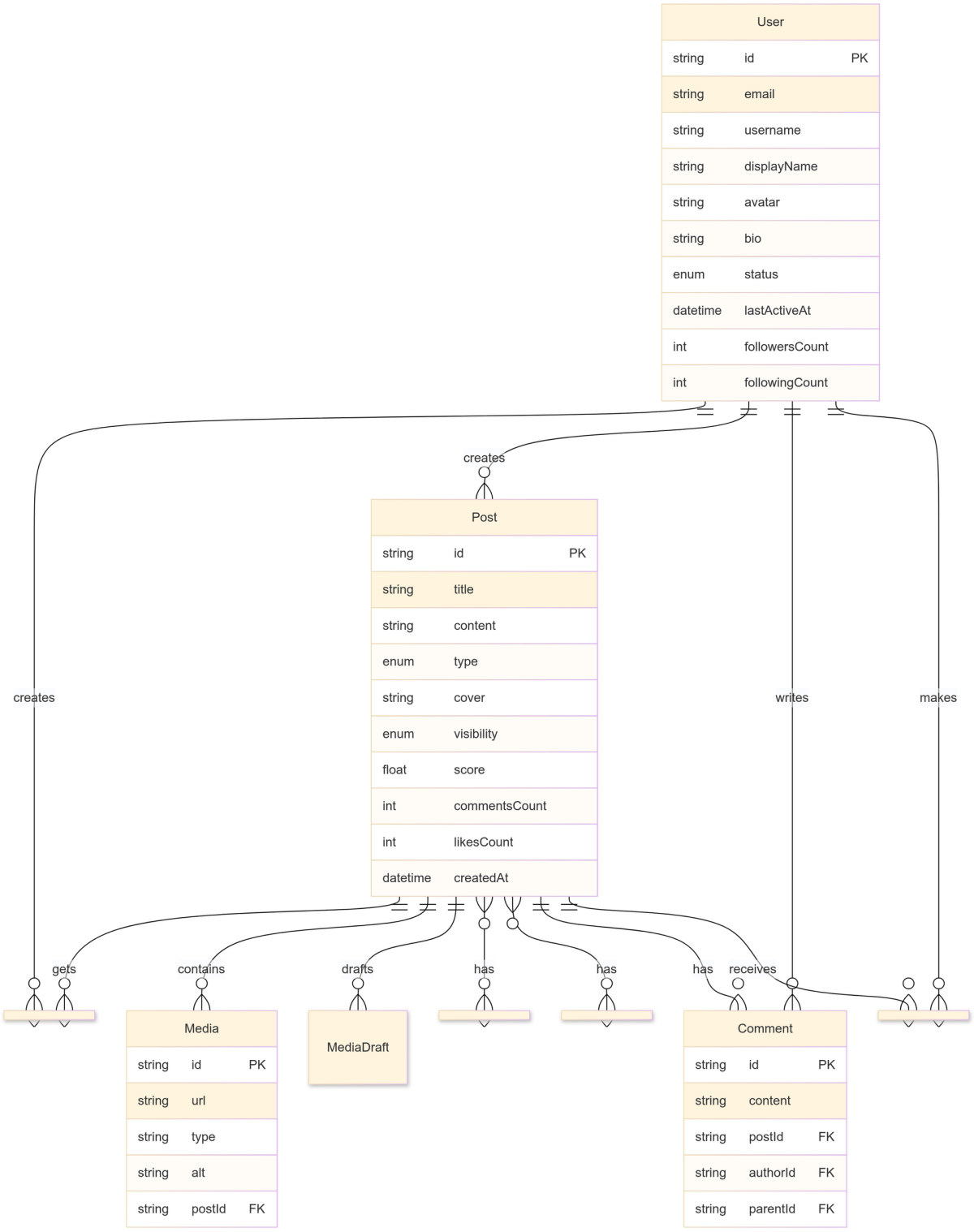


Figure 111: User and Content Entity Relationship Diagram - Showing core entities and their associations

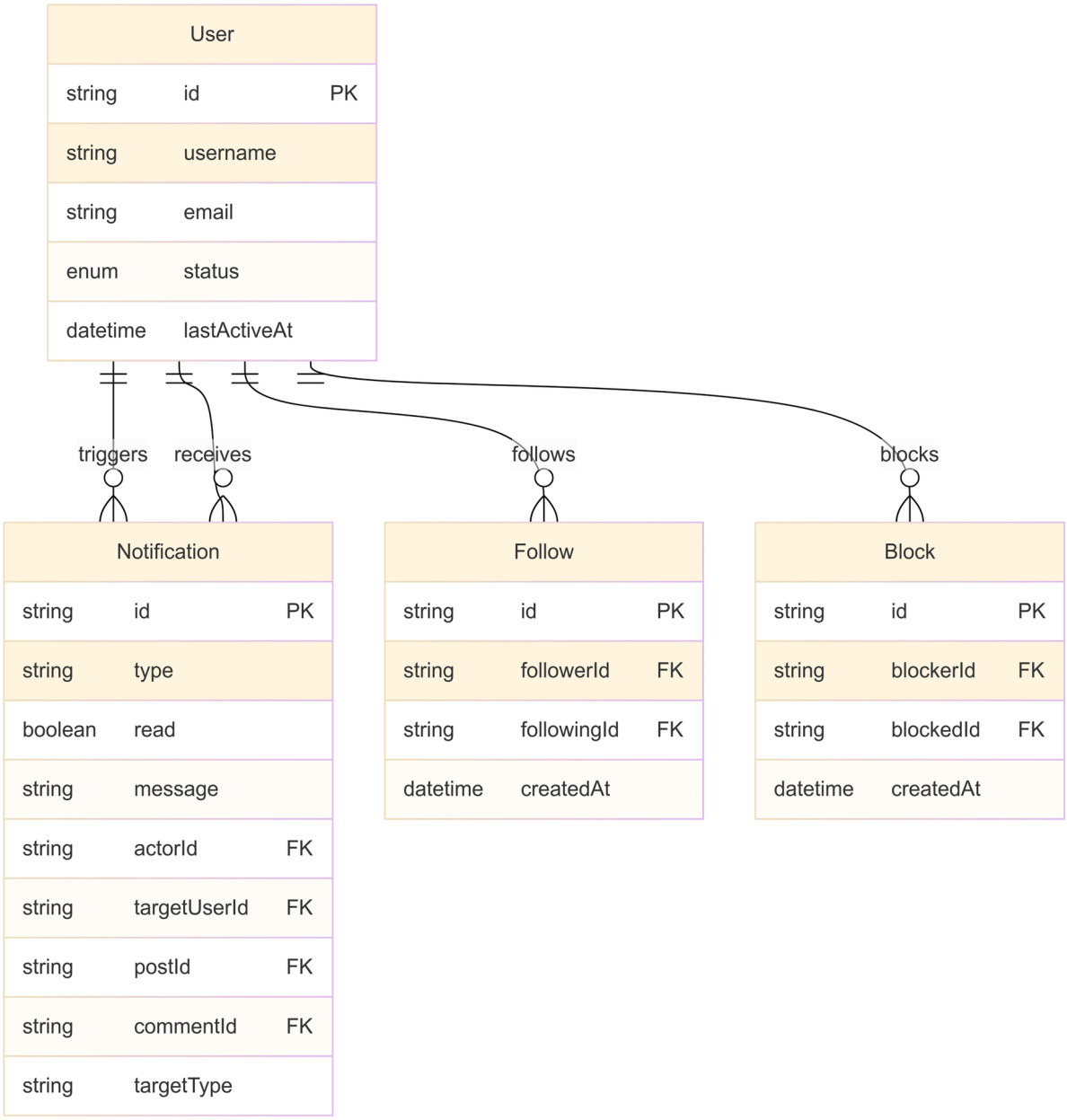


Figure 112: Social Relationship Entity Diagram - Describing the social interaction model between users

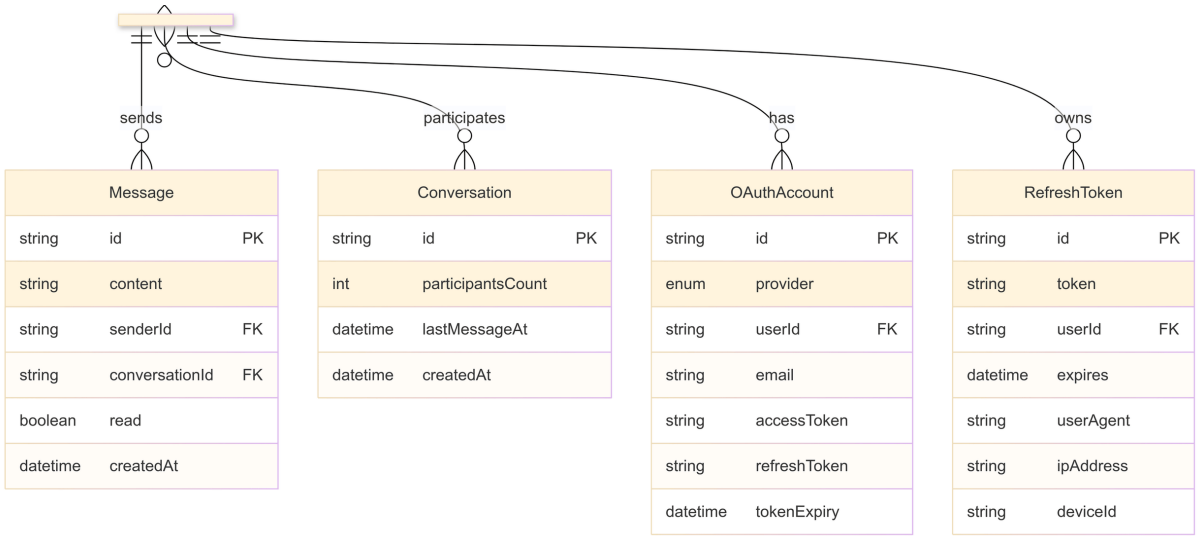


Figure 113: Message and Authentication Model Diagram - Showing system communication and security mechanisms

The system's core data model includes the following key entities:

1. **User System (User)**

```
model User {
  id                String      @id @default(uuid())
  email             String      @unique
  username          String      @unique @db.VarChar(30)
  password          String
  displayName       String?
  avatar            String?
  bio               String?     @db.VarChar(500)
  status            UserStatus  @default(ACTIVE)
  emailVerified     DateTime?
  phoneNumber       String?     @unique
  phoneVerified     DateTime?
  twoFactorEnabled  Boolean     @default(false)
  lastActiveAt      DateTime    @default(now())
  followersCount    Int         @default(0)
  followingCount    Int         @default(0)
  createdAt         DateTime    @default(now())
  updatedAt         DateTime    @updatedAt
  // ... relations
}
```

User model design features and innovations:

- Uses UUID as primary key, providing uniqueness guarantee in distributed environments while enhancing system security by avoiding user count leakage issues that can occur with auto-incrementing IDs
- Implements multiple authentication mechanisms including email verification, phone verification and two-factor authentication, offering flexible security level options
- Introduces user state management supporting account lifecycle management (ACTIVE/SUSPENDED/DELETED)
- Optimized profile design with optional fields to reduce data storage overhead
- Built-in follower counters to avoid frequent aggregation queries
- Real-time activity time tracking supporting efficient user activity analysis

2. **Content System (Post)**

```
model Post {
```

```
  id            String         @id @default(cuid())
  title         String         @db.VarChar(200)
  content       String?        @db.Text
  type          PostType       @default(PHOTO)
  cover         String?
  visibility    PostVisibility @default(PUBLIC)
  score         Float          @default(0)
  commentsCount Int            @default(0)
  likesCount    Int            @default(0)
  savesCount    Int            @default(0)
  lastActiveAt  DateTime       @default(now())
  createdAt     DateTime       @default(now())
  updatedAt     DateTime       @updatedAt
  // ... relations
}
```

Content model innovative features:

- Uses CUID as primary key, providing better time-sequence ordering performance while ensuring uniqueness
- Flexible content type system supporting three main content forms: images, videos and articles, with room for expansion
- Multi-level visibility control mechanism implementing precise content access control
- Composite scoring system comprehensively considering content quality, user interaction and timeliness
- Optimized counter design supporting atomic counter updates in high concurrency scenarios
- Time-dimension activity tracking providing data support for recommendation system

3. **Media System (Media & MediaDraft)**

```
model Media {
  id     String  @id @default(cuid())
  url    String
  type   String  @default("image")
  alt    String?
  postId String
  order  Int     @default(0)
  // ... relations
}

model MediaDraft {
  id        String           @id @default(cuid())
  userId    String
  url       String
  type      String
  size      Int
  status    MediaDraftStatus @default(PENDING)
  retries   Int              @default(0)
  createdAt DateTime         @default(now())
  expiresAt DateTime
  used      Boolean          @default(false)
  // ... relations
}
```

Media system innovative design:

- Introduces media draft concept providing reliable breakpoint resume support
- Complete media lifecycle management including temporary storage, formal publishing and automatic cleanup
- Multi-media type support including images, videos and other media formats
- Intelligent media state tracking supporting upload retry and error recovery

- Storage optimization based on expiration time automatically cleaning unused media resources
- Media ordering support implementing flexible display layouts

3.4.2 Relationship Design

The system employs complex many-to-many relationship models implementing rich social features:

1. **Social Relationship Design**

```
model Follow {
  id          String   @id @default(uuid())
  followerId  String
  followingId String
  createdAt   DateTime @default(now())
  // ... relations
}

model Block {
  id        String   @id @default(uuid())
  blockerId String
  blockedId String
  createdAt DateTime @default(now())
  // ... relations
}
```

Social relationship innovative features:

- Bidirectional relationship model supporting complex social network analysis
- Integrated blocking functionality providing user privacy protection
- Timestamp recording supporting relationship change tracking and historical analysis
- Optimized query performance supporting rapid traversal of large-scale social networks

2. **Interaction Relationship Design**

```
model Like {
  id        String   @id @default(uuid())
  userId    String
  postId    String?
  commentId String?
  createdAt DateTime @default(now())
  // ... relations
}

model Save {
  id        String   @id @default(uuid())
  userId    String
  postId    String
  createdAt DateTime @default(now())
  // ... relations
}
```

Interaction relationship features:

- Unified like model simultaneously supporting content and comment likes
- Bookmark functionality design supporting user content management
- Efficient unique constraints preventing duplicate operations
- Complete time recording supporting interaction behavior analysis

3.4.3 Indexing Strategy

The system implements multi-level index optimization strategies:

1. **Composite Index Design**

```
@@index([authorId, createdAt])
@@index([visibility, createdAt])
@@index([score, createdAt])
@@index([lastActiveAt])
```

Index strategy innovations:

- Query-oriented composite index design optimizing common query scenarios
- Time dimension indexing supporting efficient time-series data access
- Status and score indexing enhancing recommendation system performance
- Selective index design balancing query performance and storage overhead

2. **Unique Constraint Optimization**

```
@@unique([provider, userId])
@@unique([provider, email])
```

Constraint design features:

- Multi-field unique constraints ensuring data consistency
- Database-level implementation of business rules providing reliable constraint guarantees
- Optimized index structure supporting efficient uniqueness checks

3.4.4 Data Integrity Assurance

The system ensures data integrity through multiple mechanisms:

1. **Cascade Delete Strategy**

```
user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

Cascade strategy features:

- Intelligent cascade delete design ensuring data consistency
- Differentiated deletion strategies adapting to different business scenarios
- Complete referential integrity guarantee

2. **Field Constraint Design**

```
username String @unique @db.VarChar(30)
bio      String? @db.VarChar(500)
```

Constraint design innovations:

- Reasonable field length limits optimizing storage efficiency
- Optional field design supporting progressive information completion
- Type-safe field definitions preventing data errors

3.4.5 Scalability Design

The database design fully considers future system expansion needs:

1. **Partition Support**
    - UUID/CUID primary keys supporting horizontal partition scaling
    - Time-dimension partition design optimizing historical data management
    - Distribution-friendly design principles supporting system scale expansion
2. **Performance Optimization**
    - Carefully selected field types balancing performance and storage
    - Optimized index coverage supporting high-frequency query scenarios
    - Counter field design reducing aggregation query pressure
3. **Monitoring Support**
    - Complete timestamp recording supporting behavior analysis
    - Status tracking design facilitating system monitoring
    - Statistical field optimization supporting real-time metric calculation

3.4.6 Security Design

The database layer implements comprehensive security protection mechanisms:

1. **Access Control**
    - Role-based fine-grained permission control
    - Multi-level data access protection
    - Sensitive field access restrictions
2. **Data Encryption**
    - Password secure storage using high-strength hash algorithms
    - Sensitive information encryption protection
    - Secure token management mechanism
3. **Audit Trail**
    - Complete operation log recording
    - User behavior tracking
    - Change history management

Through this multi-level, comprehensive database design, the system not only meets current functional requirements but also reserves ample space for future expansion. The design scheme has made in-depth considerations in terms of scalability, performance optimization, and security protection, providing a reliable data foundation for the application system. Additionally, by adopting advanced design concepts and innovative implementation solutions, the system has the capability to support large-scale social media platform operations.

3.5 Gateway Layer Design

PhotoGlow's gateway layer adopts a dual-layer architecture design using Cloudflare + Nginx, implementing efficient, secure, and scalable request processing mechanisms through reasonable responsibility division and functional configuration. This section will detail the gateway layer's design philosophy, architectural implementation, and key features.

3.5.1 Architecture Design

The system adopts a dual-layer gateway architecture:

1. **External Gateway (Cloudflare)**
    - Global CDN distribution
    - DDoS protection
    - SSL/TLS termination
    - WAF (Web Application Firewall)
    - Smart routing optimization
2. **Internal Gateway (Nginx)**
    - Reverse proxy
    - Load balancing
    - Request filtering
    - Security protection
    - Performance optimization

This dual-layer architecture design has the following advantages:

- Fully utilizes Cloudflare's global infrastructure providing better access speed
- Provides stronger security guarantee through dual-layer protection mechanism
- Implements responsibility separation facilitating independent optimization and maintenance
- Provides more flexible scalability

3.5.2 Nginx Basic Configuration Design

Nginx, as the internal gateway, implements efficient request processing through detailed configuration:

1. **Core Configuration Optimization**

```
worker_processes auto;
worker_connections 1024;
keepalive_timeout 65;
sendfile on;
tcp_nopush on;
tcp_nodelay on;
types_hash_max_size 2048;
server_tokens off;
```

Configuration features:

- Automatically optimizes worker process count improving resource utilization efficiency
- Reasonable connection pool configuration supporting high concurrency access
- Enables efficient file transfer mechanism reducing system calls
- TCP connection optimization enhancing network transmission performance
- Disables server version information enhancing security

2. **Logging System Design**

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" "$http_x_forwarded_for"';

access_log /var/log/nginx/access.log main;
error_log /var/log/nginx/error.log warn;
```

Logging system features:

- Adopts standardized log format facilitating analysis and processing
- Records detailed request information supporting problem tracking
- Hierarchical error logging mechanism facilitating fault diagnosis
- Complete access trail recording supporting security auditing

3.5.3 Route Forwarding and Service Integration

The system implements fine-grained request routing and service integration mechanisms:

1. **Frontend Service Routing**

```
location / {
    proxy_pass http://web:8080;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_cache_bypass $http_upgrade;
}
```

Frontend routing features:

- Supports HTTP/1.1 protocol features
- Complete request header forwarding mechanism
- Preserves client real IP information
- Supports protocol upgrade and WebSocket

2. **API Service Routing**

```
location /api/v1/ {
    limit_req zone=api_limit burst=10 nodelay;
    proxy_pass http://server:8001/api/v1/;
    # ... standard proxy headers
}

location /api/ {
    limit_req zone=api_limit burst=10 nodelay;
    proxy_pass http://server:8001/api/;
    # ... standard proxy headers
}
```

API routing features:

- Versioned API path management
- Integrated request rate limiting protection
- Unified proxy header configuration
- Flexible backend service integration

3. **Special Service Support**

```
location /socket.io/ {
    proxy_pass http://server:8001/socket.io/;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    # ... additional WebSocket configurations
}

location /api/v1/auth/google/callback {
    proxy_pass http://server:8001/api/v1/auth/google/callback;
    # ... OAuth specific configurations
}
```

Special service features:

- Complete WebSocket support
- OAuth authentication flow integration
- Dedicated connection keep-alive configuration
- Customized request header processing

3.5.4 Security Protection Mechanism

The system implements multi-layered security protection measures:

1. **HTTP Security Header Configuration**

```
add_header X-Content-Type-Options nosniff;
add_header X-Frame-Options SAMEORIGIN;
add_header X-XSS-Protection "1; mode=block";
add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eva
add_header Referrer-Policy "strict-origin-when-cross-origin";
```

Security header features:

- Prevention of MIME type sniffing attacks
- Restriction of frame embedding sources
- Enabled browser XSS protection
- Strict content security policy
- Referrer information source control

2. **Request Rate Limiting Protection**

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=5r/s;
limit_req_status 429;
```

Rate limiting features:

- Client IP-based rate limiting control
- Reasonable burst request handling
- Graceful rate limiting response mechanism
- Configurable rate limiting strategy

3. **SSL/TLS Configuration**

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off;
```

SSL features:

- Support for latest security protocols
- Optimized cipher suite selection
- Efficient session caching mechanism
- Disabled insecure features

3.5.5 Performance Optimization Design

The system implements comprehensive performance optimization measures:

1. **Compression Optimization**

```
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_types text/plain text/css application/json application/javascript text/xml application/xml appli
```

Compression features:

- Support for multiple content type compression
- Intelligent compression level control
- Proxy server compression support
- Dynamic compression response headers

2. **Upload and Connection Optimization**

```
client_max_body_size 50M;
keepalive_timeout 65;
sendfile on;
tcp_nopush on;
tcp_nodelay on;
```

Optimization features:

- Support for large file upload requirements
- Optimized connection keep-alive mechanism
- Efficient file transfer
- TCP transmission performance optimization

3.5.6 Scalability Design

The gateway layer design fully considers system expansion needs:

1. **Service Discovery Support**

- Dynamic upstream service configuration
- Flexible service registration mechanism
- Automatic health checking
- Failover capability

2. **Load Balancing Mechanism**

- Support for multiple balancing strategies
- Dynamic weight adjustment
- Session persistence functionality
- Smooth service switching

3.5.7 Operations Support

The system provides comprehensive operations support functions:

1. **Monitoring Capabilities**

- Detailed access log recording
- Multi-level error logging
- Performance metric collection
- Problem tracking support

2. **Maintenance Convenience**

- Configuration hot reload support
- Graceful service restart
- Flexible debugging capability
- Comprehensive documentation support

Through this comprehensive gateway layer design, PhotoGlow not only implements efficient request processing and routing distribution but also provides powerful security protection and performance optimization capabilities. Combined with Cloudflare's global infrastructure, the system can provide users with fast, secure, and reliable access experience. Meanwhile, comprehensive monitoring and extension mechanisms also provide strong support for continuous system optimization and scale expansion.

3.6 Admin Backend Design

PhotoGlow's admin backend adopts an independent service architecture design, achieving a balance between security and usability through a full server-side rendering strategy. This section will detail the admin backend's design philosophy, technical implementation, and key features.

3.6.1 Architecture Design

The admin backend adopts an independently deployed architecture solution with the following characteristics:

1. **Independent Service Design**

    - Completely independent service deployment, decoupled from main application
    - Independent database instance ensuring data isolation
    - Dedicated API implementation avoiding mixing with user-side APIs
    - Independent authentication system providing stricter access control

2. **Rendering Strategy**

    - Adopts full server-side rendering solution enhancing security
    - Simplified state management mechanism improving maintainability
    - Integrated API routing reducing external dependencies
    - Optimized page loading performance

3. **Technology Stack Selection**

    - Next.js 15 providing server-side rendering framework
    - React 19 server components enhancing security
    - TailwindCSS v4 implementing modern interface
    - Prisma implementing type-safe data access
    - shadcn/ui providing unified interface components

3.6.2 Functional Module Design

The admin backend implements comprehensive platform management functionality:

1. **Administrator Authentication System**
    - Role-based access control (RBAC)
    - Secure session management mechanism
    - Two-factor authentication support
    - Complete audit log recording
2. **User Management Functions**
    - User account management
    - Role permission assignment
    - User activity monitoring
    - Account verification tools
3. **Content Moderation System**
    - Post review and management
    - Comment management functionality
    - Content filtering rules
    - Automated content analysis
    - Report handling mechanism
4. **Data Analysis Functions**
    - Real-time platform metrics
    - User engagement statistics
    - Content performance analysis
    - System health monitoring
    - Custom report generation

3.6.3 Data Model Design

The admin backend adopts a Prisma ORM-based data modeling solution, implementing complete backend management function support through carefully designed relational data models. The data model design fully considers multiple dimensions such as system security, scalability, and performance optimization. Below is a detailed design analysis of the core data models:

3.6.3.1 Administrator Account System The administrator account system is the foundation of the entire backend management system, adopting a multi-level design approach:

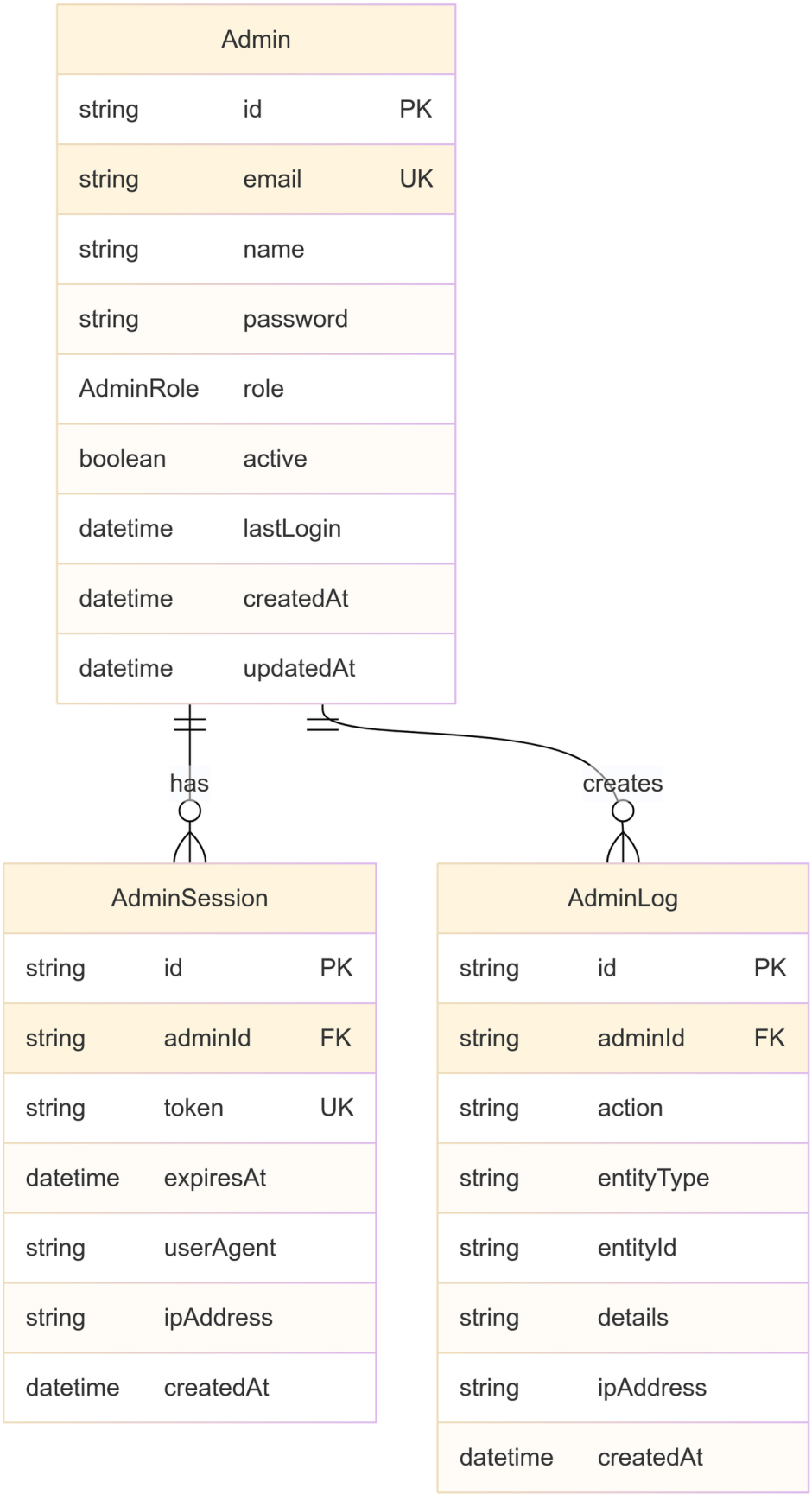


Figure 114: Admin er

1. Administrator Core Model

The administrator model (Admin) serves as the system's core model, implementing fundamental identi

```
model Admin {
  id                String    @id @default(uuid())
  email             String    @unique
  name              String
  password          String
  role              AdminRole @default(USER_MANAGER)
  active            Boolean   @default(true)
  lastLogin         DateTime?
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt

  sessions          AdminSession[]
  logs              AdminLog[]
}
```

The model adopts the following key design features:

- **Unique Identification Mechanism**: Using UUID as the primary key, avoiding security risks that might come with auto-increment IDs, while providing better distributed support.
- **Account Security Control**: Implementing account status management through the active field, enabling account deactivation without deletion.
- **Timestamp Tracking**: Completely recording account creation time (createdAt), update time (updatedAt), and last login time (lastLogin), facilitating account lifecycle management.
- **Uniqueness Constraint**: Applying a unique constraint on the email field, ensuring administrator accounts are not duplicated.

2. Role-Based Permission System

The system implements a Role-Based Access Control (RBAC) model:

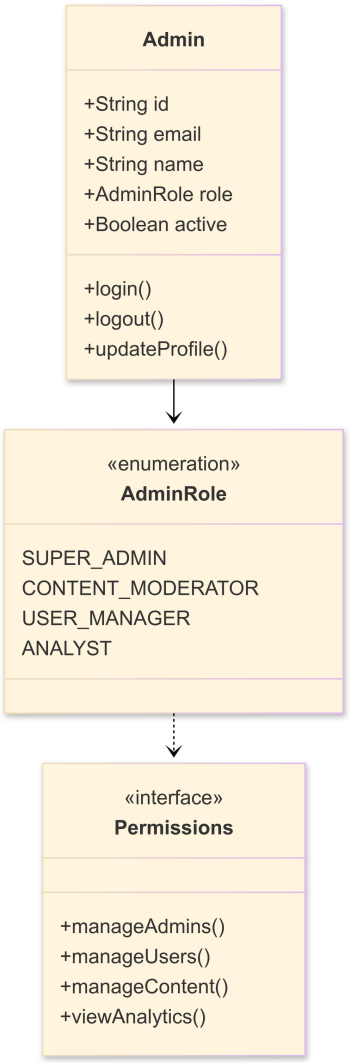


Figure 115: Admin permission

```
enum AdminRole {
  SUPER_ADMIN
  CONTENT_MODERATOR
  USER_MANAGER
  ANALYST
}
```

Design features of the role system:

- **Hierarchical Management**: Implementing permission level management through different roles
  - SUPER_ADMIN: System super administrator, possessing the highest permissions
  - CONTENT_MODERATOR: Content review administrator, focusing on content management
  - USER_MANAGER: User administrator, responsible for user-related affairs
  - ANALYST: Data analyst, responsible for data analysis work

3.6.3.2 Session Management Mechanism The session management model (AdminSession) implements a secure and reliable authentication mechanism:

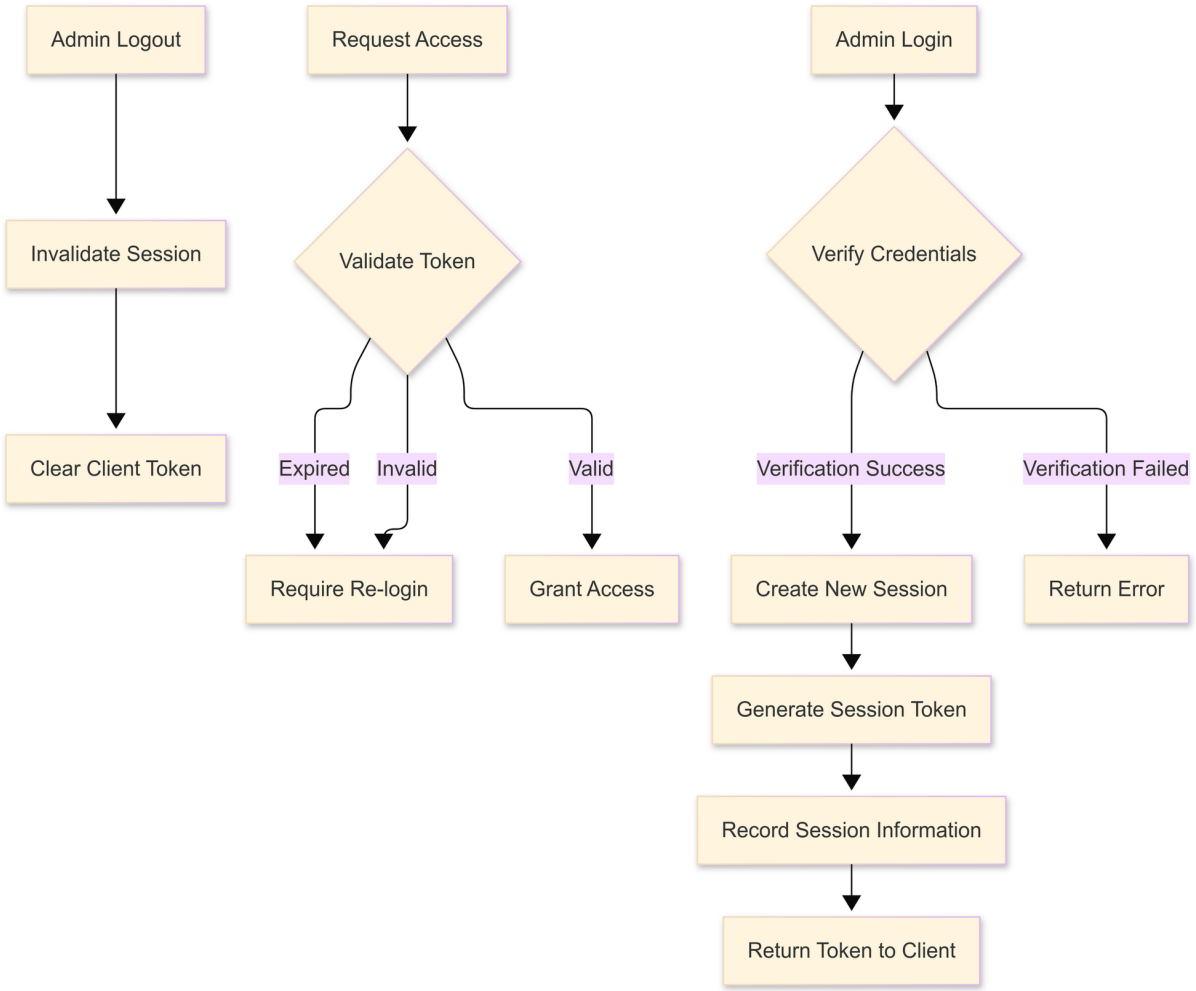


Figure 116: Admin flow

```
model AdminSession {
  id        String   @id @default(uuid())
  adminId   String
  token     String   @unique
  expiresAt DateTime
  userAgent String?
  ipAddress String?
  createdAt DateTime @default(now())

  admin     Admin    @relation(fields: [adminId], references: [id], onDelete: Cascade)

  @@index([adminId])
  @@index([token])
  @@index([expiresAt])
}
```

The session management system has the following characteristics:

- **Token Authentication**: Using unique tokens for identity verification, providing higher security than traditional cookie authentication.
- **Session Lifecycle**: Implementing session expiration control through the expiresAt field, effectively preventing session hijacking.
- **Device Information Tracking**: Recording user agent (userAgent) and IP address (ipAddress), enhancing security audit capabilities.
- **Relationship Integrity**: Using cascade deletion (onDelete: Cascade) to ensure data integrity,

avoiding session isolation.

- **Query Performance Optimization**: Implementing composite indexes for adminId, token, and expiresAt, improving query efficiency.

3.6.3.3 Audit Logging System The audit log model (AdminLog) implements comprehensive operation tracking and auditing functions:

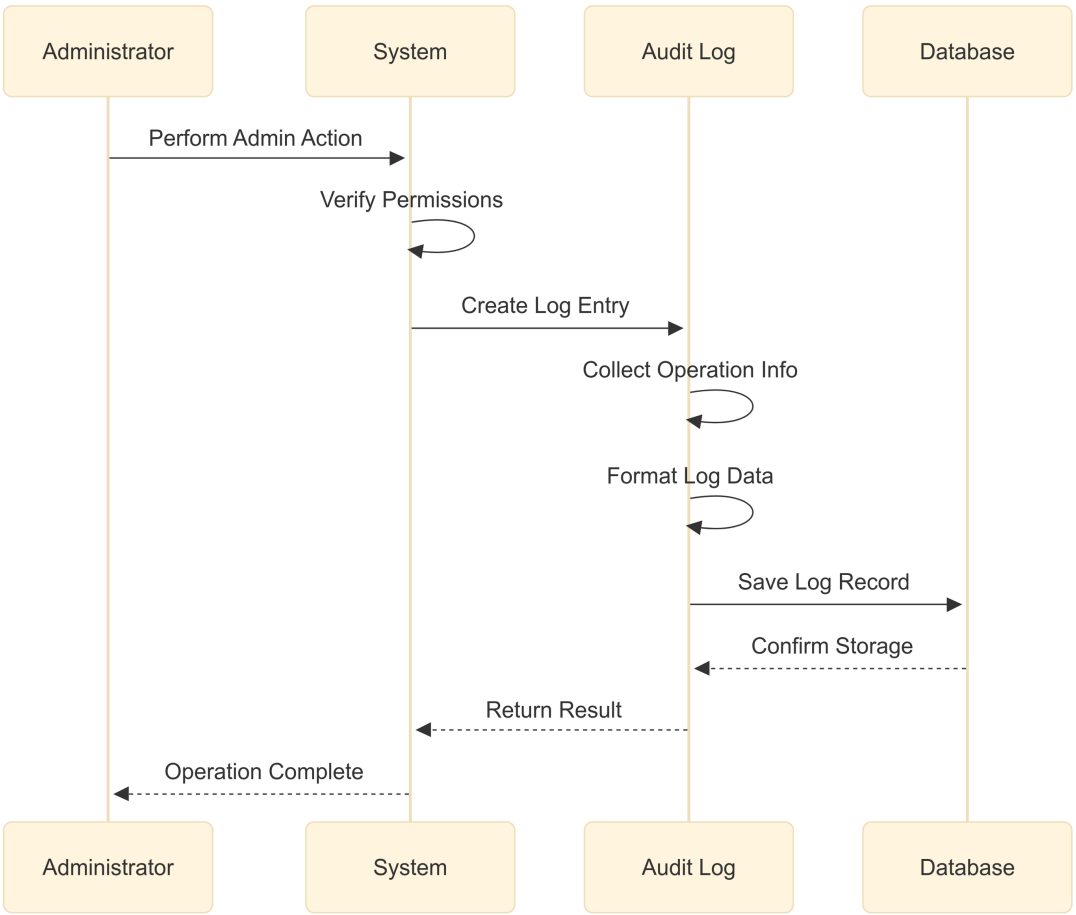


Figure 117: Admin logging

```
model AdminLog {
  id          String   @id @default(uuid())
  adminId     String
  action      String
  entityType  String?
  entityId    String?
  details     String?  @db.Text
  ipAddress   String?
  createdAt   DateTime @default(now())

  admin       Admin    @relation(fields: [adminId], references: [id], onDelete: Cascade)

  @@index([adminId, createdAt])
  @@index([entityType, entityId])
  @@index([action])
}
```

Core features of the audit system:

- **Full-Dimensional Recording**:

  - Operation Subject: Recording the administrator (adminId) performing the operation
  - Operation Type: Distinguishing different operations through the action field
  - Operation Object: Recording operation targets using entityType and entityId
  - Operation Details: Storing detailed information through the details field
  - Operation Environment: Recording the IP address at the time of operation

- **Efficient Retrieval Capabilities**:
  - Temporal Index: Implementing a composite index of adminId and createdAt, supporting efficient historical record queries
  - Entity Index: Supporting quick location of operation records for specific entities through entityType and entityId indices
  - Behavior Index: Establishing an index for action, facilitating statistical analysis of specific types of operations

3.6.3.4 Performance Optimization Design The system implements multi-dimensional performance optimization at the data model level:

1. Index Strategy Optimization

```
// Session query optimization
@@index([adminId])
@@index([token])
@@index([expiresAt])

// Audit log optimization
@@index([adminId, createdAt])
@@index([entityType, entityId])
@@index([action])
```

Index design features:

- **Multi-dimensional Indexing**: Supporting efficient queries from different dimensions
- **Composite Indexes**: Optimizing query performance for common query scenarios
- **Selective Indexing**: Determining indexing strategy based on field selectivity

2. Data Integrity Protection

The system ensures data integrity through multiple mechanisms:

- **Unique Constraints**: Implementing uniqueness constraints on key fields (such as email, token)
- **Cascade Deletion**: Appropriately using cascade deletion to ensure data consistency
- **Required Fields**: Strictly controlling required fields to ensure data integrity
- **Referential Integrity**: Ensuring correctness of data references through foreign key relationships

3.6.3.5 Summary of Design Features This data model design has the following outstanding characteristics:

1. **Security**:
   - UUID primary key strategy
   - Role-based access control
   - Session lifecycle management
   - Comprehensive audit logging system
2. **Scalability**:
   - Loosely coupled model relationships
   - Flexible role permission system
   - Extensible audit log structure
3. **Performance Optimization**:
   - Well-designed indexing
   - Optimized field type selection

- Efficient query support

4. **Maintainability**:
    - Clear model relationships
    - Complete timestamp tracking
    - Standardized naming conventions

This data model design not only meets the functional requirements of the admin backend but also ensures system security, scalability, and performance through careful design. At the same time, the comprehensive audit mechanism and session management provide strong support for system operations.

3.6.4 UI/UX Design

The user interface design of the PhotoGlow admin backend can reference the web frontend's design language.

3.2.5.1 Theme Design Consistent with the web frontend theme design.

3.2.5.1.1 Color System Consistent with the web frontend color design.

3.2.5.1.2 Component Design System Consistent with the web frontend component design.

3.2.5.1.4 Summary The theme design of this project adopts a modular and variable approach, establishing a complete visual design system. By defining a series of color variables, component styles, and responsive adaptation rules, we have achieved visual consistency in the interface and fluidity in user experience. At the same time, through comprehensive support for dark mode, we have enhanced the application's adaptability in different environments and user preferences.

This theme design approach not only improves the aesthetics of the user interface but also simplifies the development process, making style maintenance and updates more efficient and controllable. Through the tools and methods provided by Tailwind CSS, we have implemented a user interface design system that is both beautiful and practical.

3.2.5.2 Interface Overview

3.2.5.2.1 Login Page The admin page prohibits all unauthorized access, with security ensured by custom middleware. Users who are not logged in will be directed to the login page.

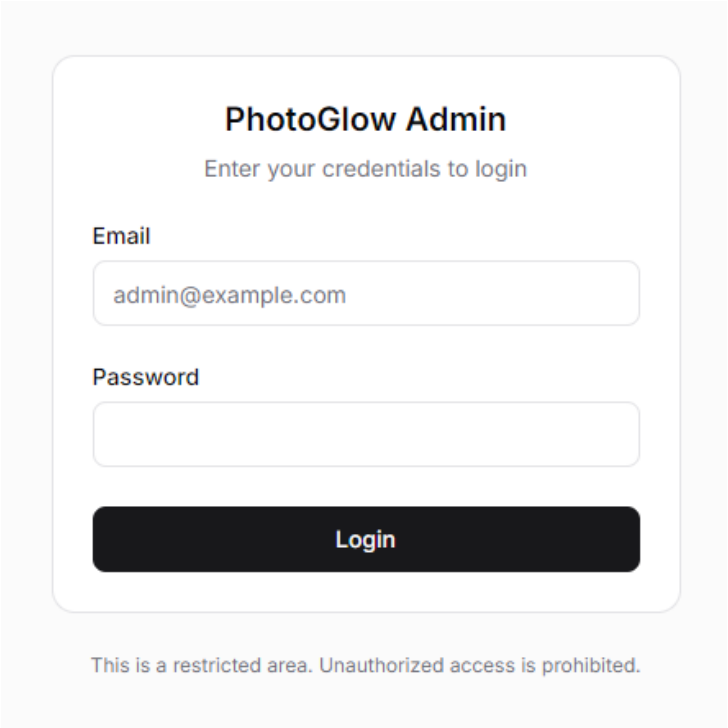


Figure 118: Admin login

Design:

- No user registration interface; administrator users are created by developers executing scripts.
- Strict password retry limit.

3.2.5.2.2 Side Navigation Bar The sidebar displays navigation menus and the current user.

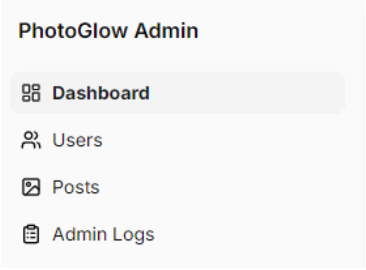


Figure 119: Navigation menu

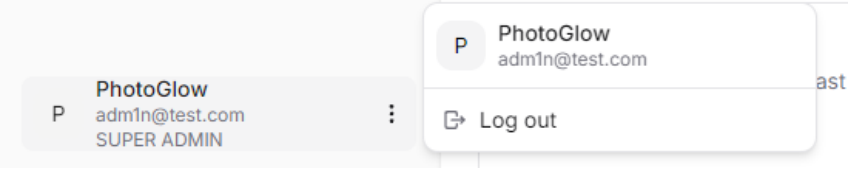


Figure 120: Current user

3.2.5.2.3 Homepage The Dashboard homepage displays data statistics and charts of the PhotoGlow website.

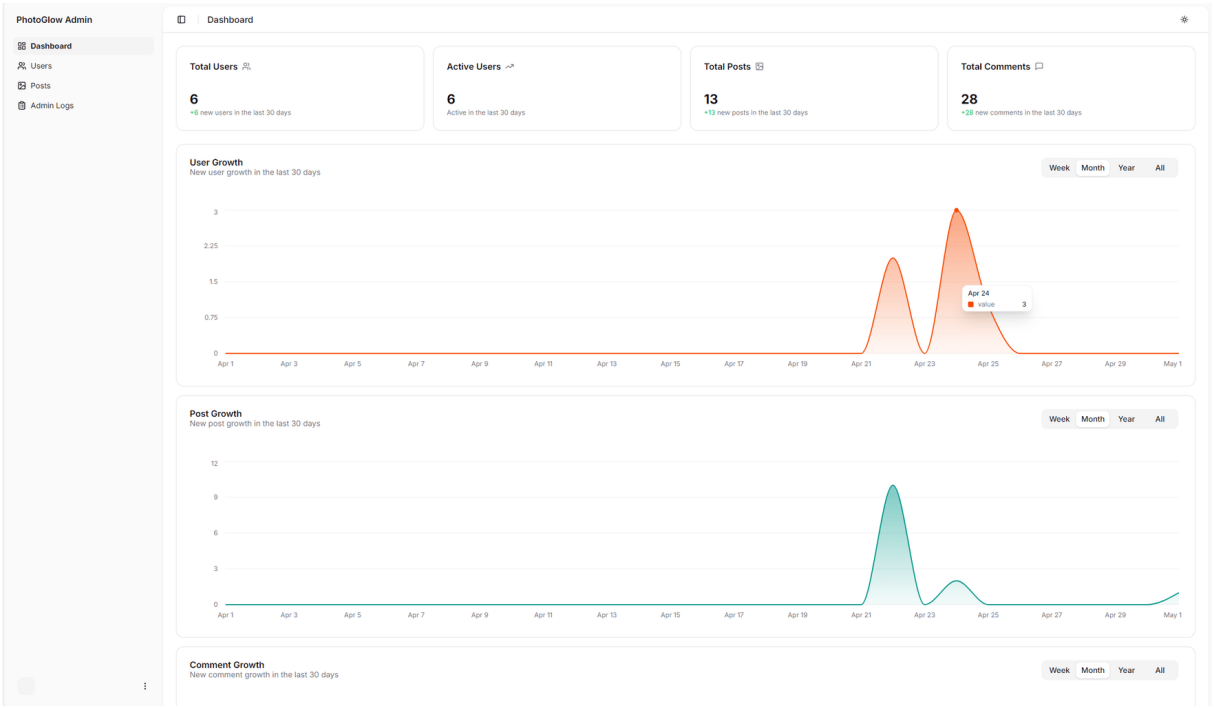


Figure 121: Dashboard homepage

The design pattern is consistent with the web frontend, supporting responsive design:

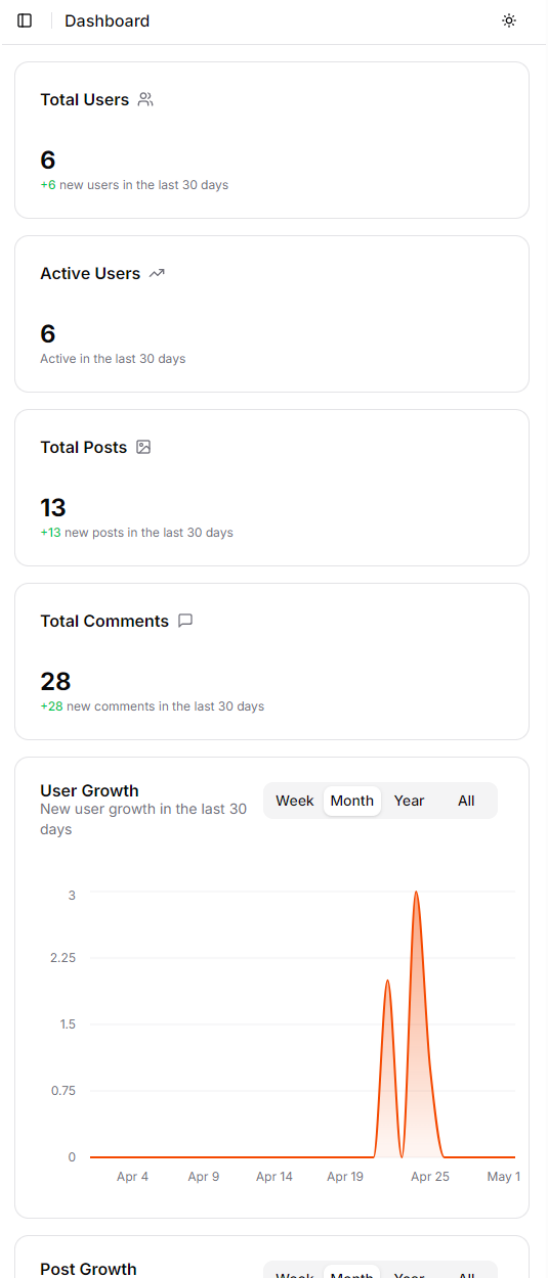


Figure 122: Dashboard homepage-mobile

3.2.5.2.4 User Management Page Allows management of PhotoGlow users, modification of information, and blocking.

Functional design:

- Display website users.
- Support search.
- Support pagination.
- Support setting table columns.
- Table supports sorting.
- Can modify user status.
- Can open a dialog to modify detailed user information.

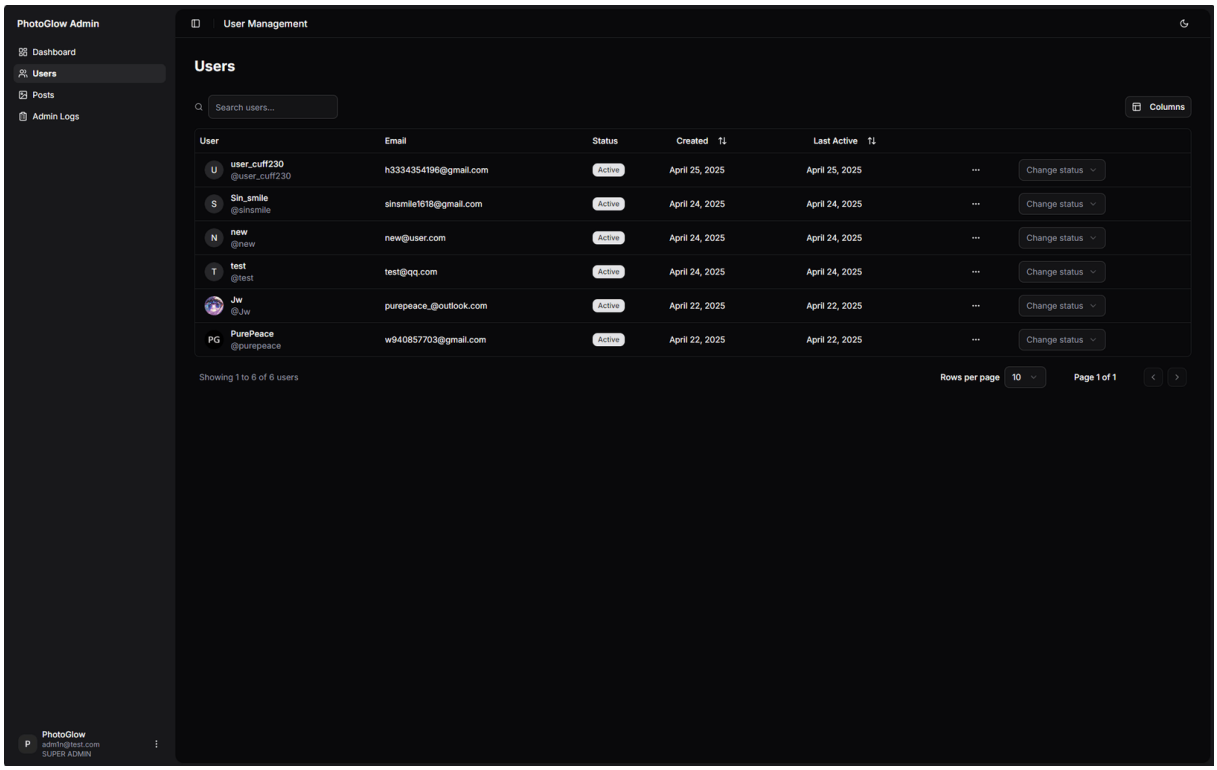


Figure 123: User admin

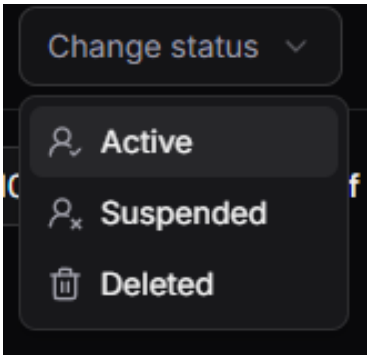


Figure 124: User status change

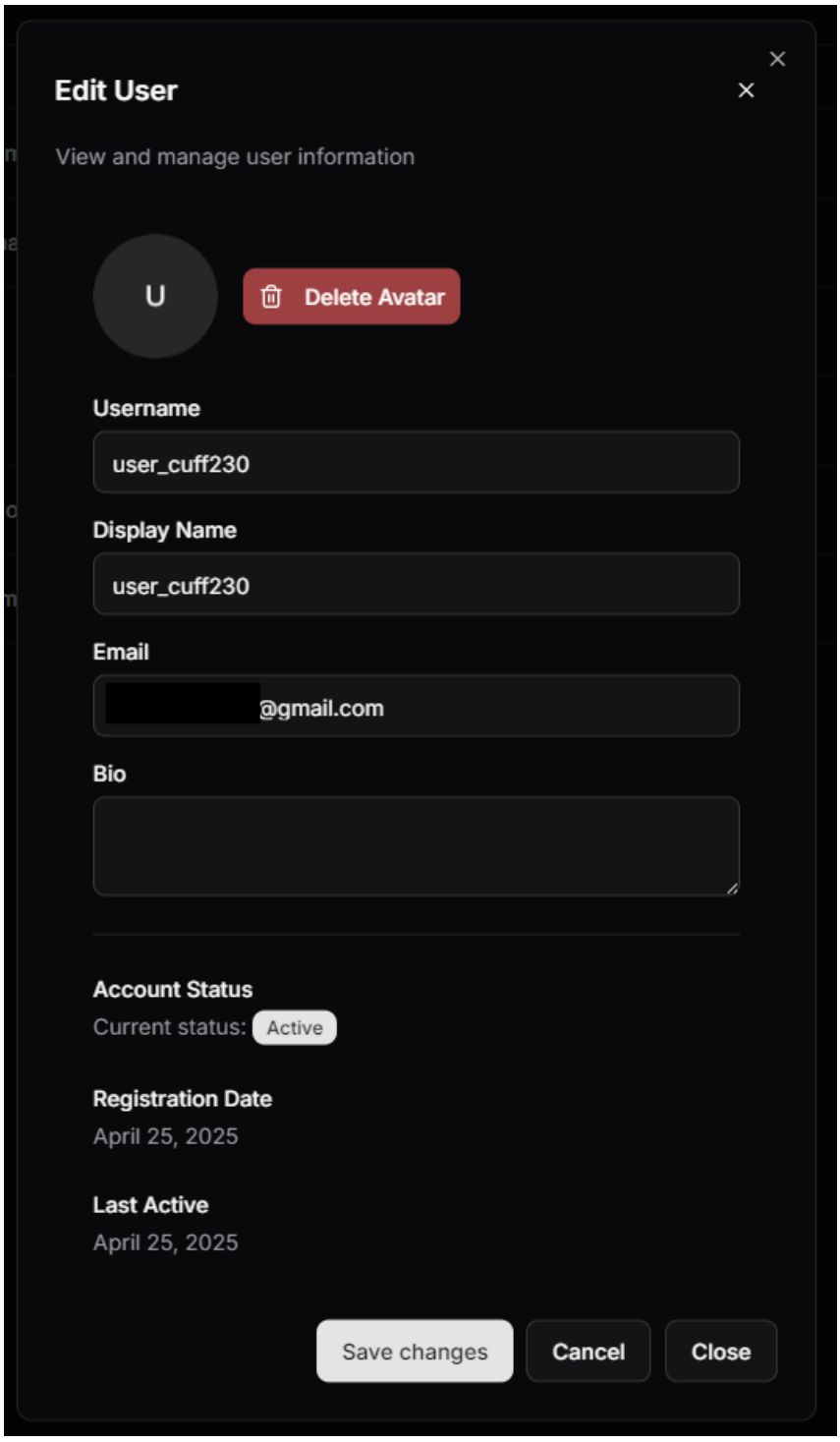


Figure 125: User detail edit

3.2.5.2.5 Post Management Page Allows management of PhotoGlow posts, modification of information, hiding, and deletion.

Functional design:

- Display website posts and photos.
- Support search.
- Support pagination.
- Support setting table columns.
- Table supports sorting.

- Can modify post visibility.
- Can open a dialog to modify detailed user information.

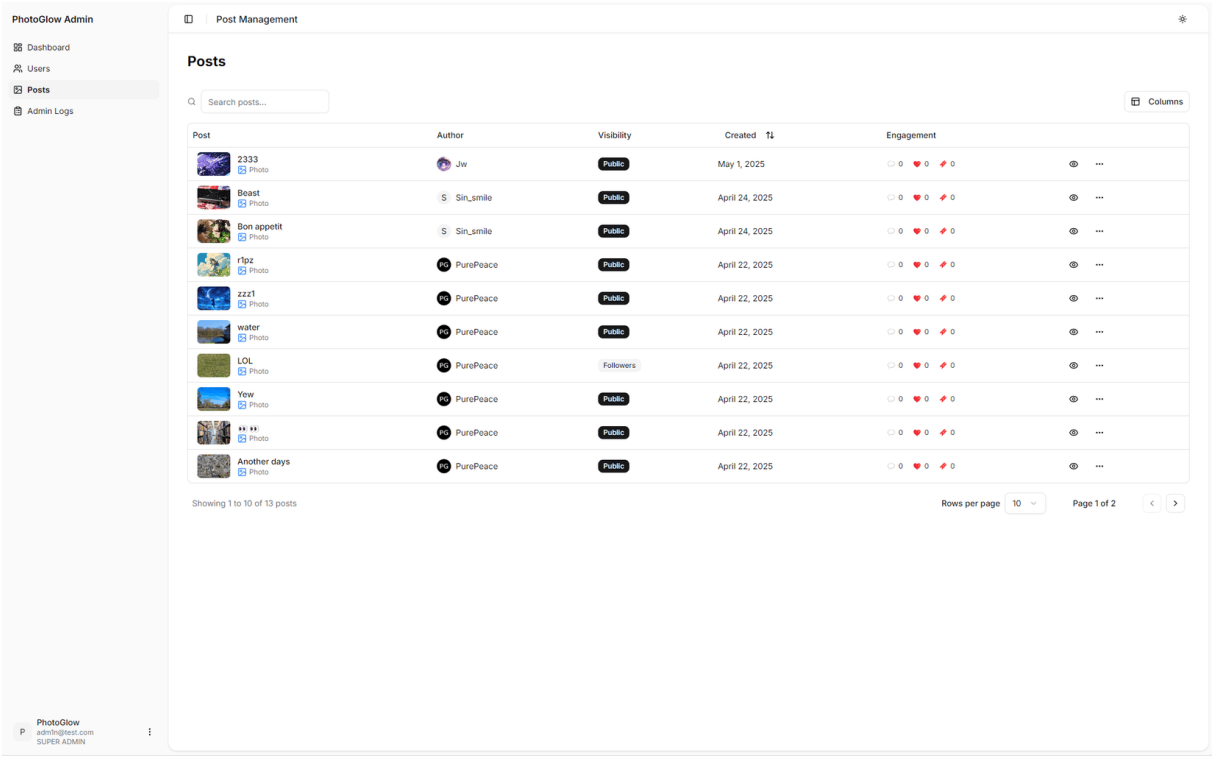


Figure 126: Post admin

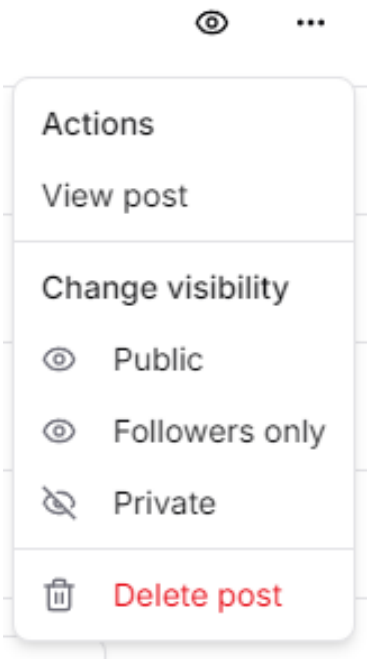


Figure 127: Post management

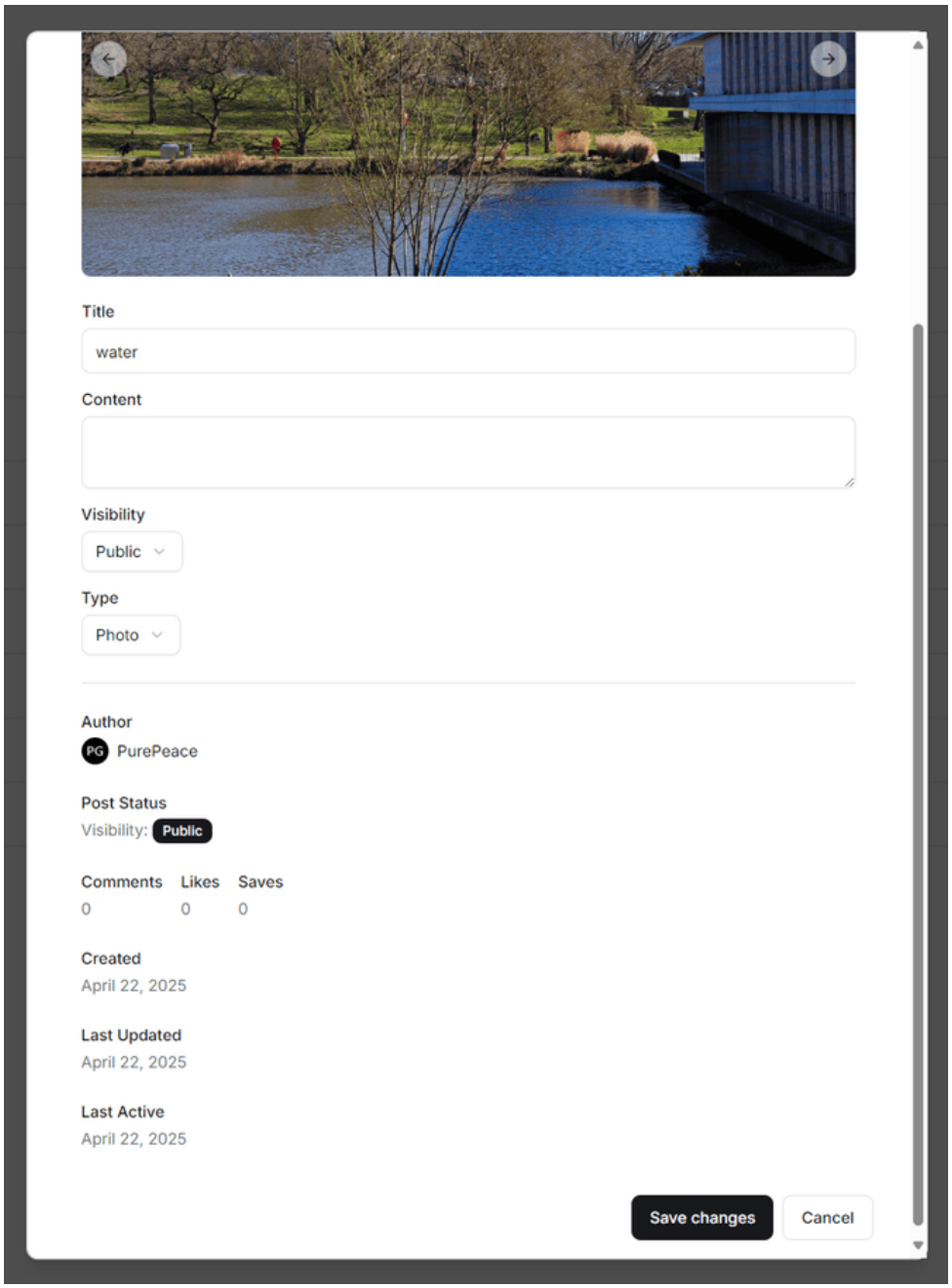


Figure 128: Post edit

3.2.5.2.6 Admin Logs Any operations by administrators will be audited.

Functional design:

- Display admin logs.
- Support search.
- Support pagination.
- Support setting table columns.
- Table supports sorting.
- Entity Type: Display the affected PhotoGlow database entity type.
- Display operation types: `USER_LIST_VIEW`, `USER_UPDATE`, `LOGIN`, etc.
- Cannot be modified by anyone.

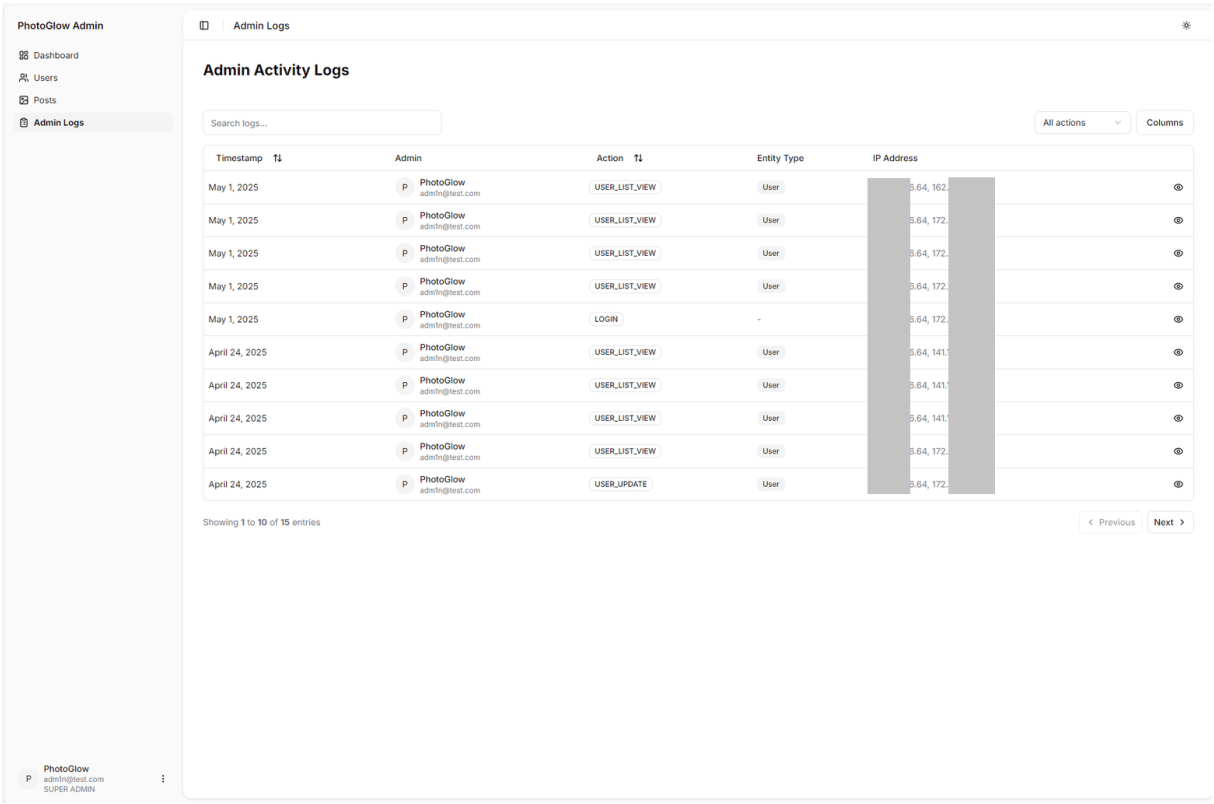


Figure 129: Admin logs

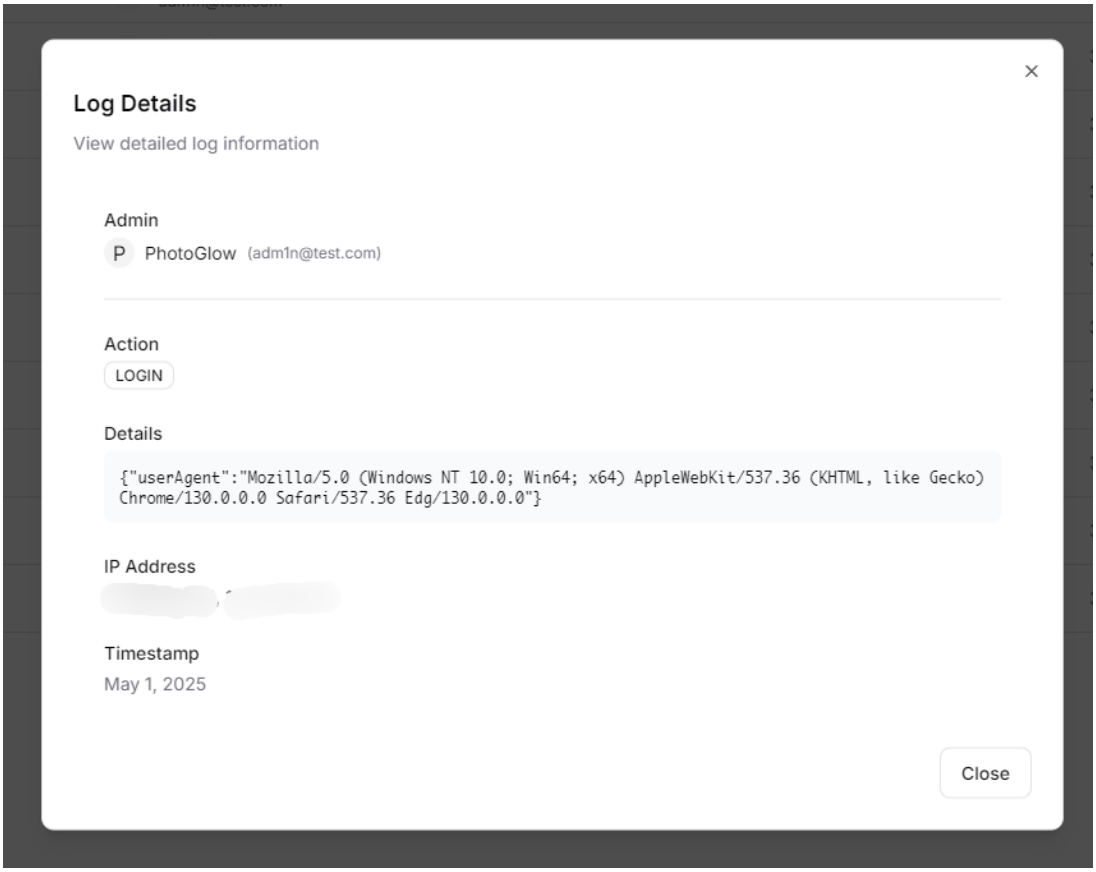


Figure 130: Admin log details

3.6.5 Deployment Architecture

The system adopts a containerized deployment solution:

1. **Build Configuration**

```
FROM oven/bun:1
WORKDIR /app
COPY . .
RUN bun install
RUN cd admin && bun run build
CMD ["sh", "-c", "cd admin && bun run start"]
```

Deployment features:

- Standardized deployment based on Docker
- Independent network configuration
- Resource isolation guarantee
- Flexible scaling capability

2. **Network Configuration**

```
networks:
  - photoglow-network
ports:
  - '8081:8081'
```

Network features:

- Independent service port
- Internal network isolation
- Secure communication mechanism
- Load balancing support

3.6.6 Development Tool Support

The system provides complete development support functions:

1. **Environment Configuration**

```
# Database Configuration
ADMIN_DATABASE_URL="postgresql://username:password@localhost:5432/admin_db"

# Application URLs
ADMIN_FRONTEND_URL=http://localhost:8082
INTERNAL_API_URL=http://localhost:8001
NEXT_PUBLIC_API_URL=http://localhost:8082

# Security
JWT_SECRET=your_jwt_secret
SESSION_SECRET=your_session_secret
```

Configuration features:

- Complete environment variable management
- Development environment configuration template
- Security credential management
- Service address configuration

2. **Management Commands**

```
# Database operations
bun run db:migrate      # Execute database migration
bun run db:generate     # Generate Prisma client
bun run db:seed         # Initialize seed data
```

```
# Administrator user management
bun run create-admin <email> <name> <password> <role>
```

Command features:

- Standardized database operations
- Convenient user management
- Automated build process
- Development assistance tools

3.6.7 Future Plans

Subsequent development plans for the admin backend include:

1. **Feature Enhancement**
    - Advanced data analysis tools
    - AI-assisted content review
    - Automated operation tools
    - Multi-language support
2. **Performance Optimization**
    - Page loading optimization
    - Data caching strategy
    - Batch operation support
    - Real-time data updates
3. **Security Enhancement**
    - Advanced authentication mechanism
    - More fine-grained permission control
    - Security audit enhancement
    - Threat detection system

Through this comprehensive admin backend design, PhotoGlow not only achieves efficient platform management but also ensures the reliability and security of management functions through independent service architecture and strict security measures. The modular design of the system and complete development support provide a solid foundation for the platform's continuous operation and functional expansion.

3.7 SDK Design

PhotoGlow's SDK (Software Development Kit) design adopts a multi-language generation solution based on the OpenAPI specification, achieving cross-language client library generation through a unified API definition. This design not only standardizes API access methods but also significantly enhances the system's scalability and development efficiency. This section will detail the SDK's design philosophy, implementation plan, and key features.

The Server API currently has 15 categories with a total of 53 API endpoints.

If adding support for other languages requires individually writing implementations for these API endpoints each time, it would be very time-consuming and unnecessary, making the SDK generator an extremely important component.

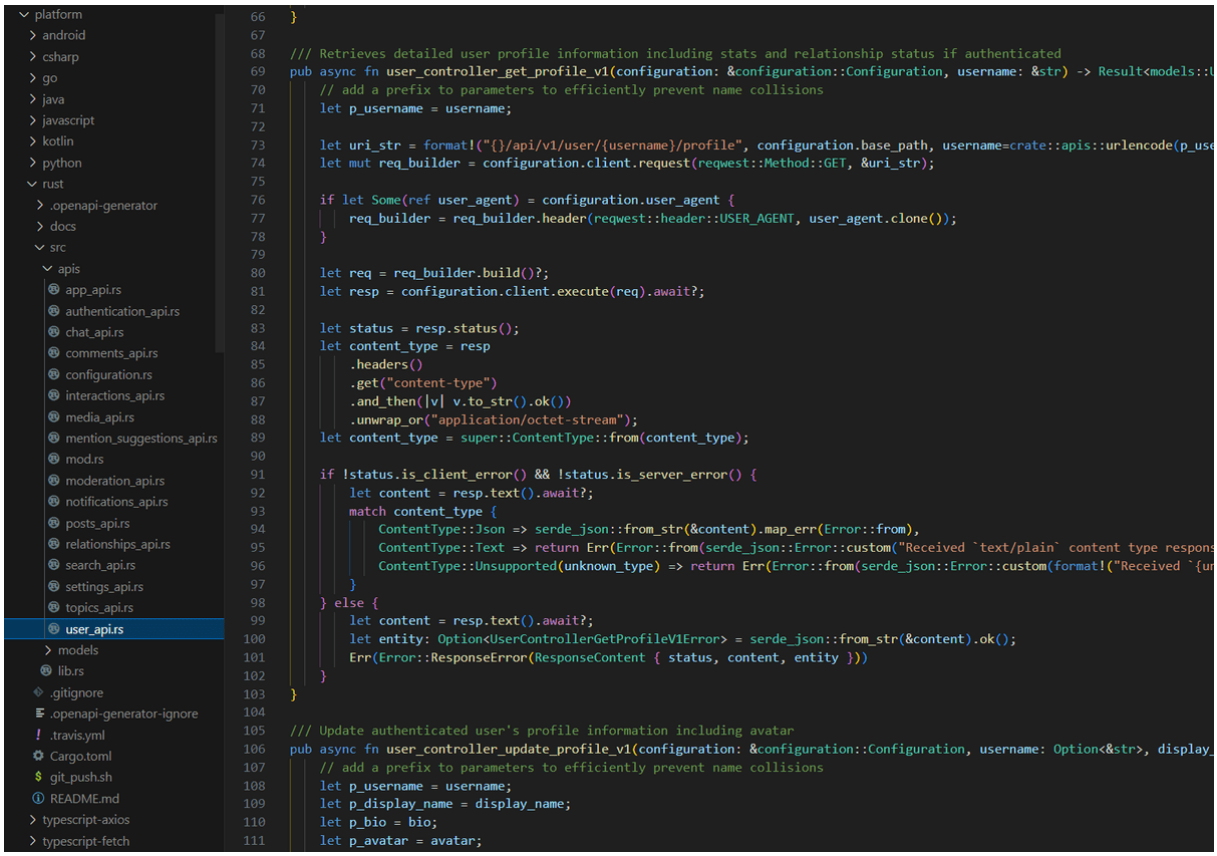


Figure 131: Generated SDK demo (Rust)

3.7.1 Design Goals

The SDK design primarily revolves around the following objectives:

1. **Multi-language Support**
   - Support client library generation for mainstream programming languages
   - Maintain API consistency across language versions
   - Provide language-specific best practice implementations
2. **Development Experience**
   - Complete type safety support
   - Automatically generated API documentation
   - Integrated test toolkit
3. **Maintainability**
   - Unified API specification management
   - Automated generation process
   - Standardized code style

3.7.2 Technical Architecture

The system adopts an SDK generation architecture based on OpenAPI Generator:

1. **Core Components**
   - OpenAPI specification definition
   - Multi-language template system
   - Automated build process
   - Documentation generator
2. **Toolchain Integration**

- Bun runtime environment
- OpenAPI Generator CLI
- Postman integration
- Documentation generation tools

3. **Output Artifacts**

- Language-specific client libraries
- API documentation
- Postman test collection
- Type definition files

3.7.3 Multi-language Support Implementation

The system has implemented comprehensive support for mainstream programming languages:

1. **TypeScript/JavaScript Implementation**

```
// TypeScript with Axios implementation
export class PhotoGlowApi {
  constructor(configuration: Configuration) {
    this.api = new DefaultApi(configuration)
  }

  async createPost(postData: CreatePostRequest): Promise<PostResponse> {
    return this.api.createPost(postData)
  }
}
```

Features:

- Provides both Axios and Fetch implementation options
- Complete TypeScript type definitions
- Promise-based asynchronous operations
- Automatic request/response serialization

1. **Language Support**

- rust
- python
- java
- go
- kotlin
- android
- csharp
- javascript

Implementations for each language follow their specific best practices:

- Rust: Utilizes strong typing and ownership system
- Python: Implementation compliant with PEP standards
- Java: Standard Builder pattern design
- Go: Idiomatic error handling mechanism

3.7.4 Generation Process Design

The system implements a complete SDK generation workflow:

1. **Configuration Management**

```
{
  "$schema": "../node_modules/@openapitools/openapi-generator-cli/config.schema.json",
  "spaces": 2,
  "generator-cli": {
    "version": "7.12.0",
```

```
"generators": {
  "docs": {
    "generatorName": "html2",
    "output": "./docs",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "postman-collection": {
    "generatorName": "postman-collection",
    "output": "./postman",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "ts-axios": {
    "generatorName": "typescript-axios",
    "output": "./platform/typescript-axios",
    "inputSpec": "http://localhost:8001/api-json",
    "additionalProperties": {
      "removeOperationIdPrefix": true,
      "operationIdNaming": "method",
      "supportsES6": true,
      "withInterfaces": true
    }
  },
  "ts-fetch": {
    "generatorName": "typescript-fetch",
    "output": "./platform/typescript-fetch",
    "inputSpec": "http://localhost:8001/api-json",
    "additionalProperties": {
      "removeOperationIdPrefix": true,
      "operationIdNaming": "method",
      "supportsES6": true,
      "withInterfaces": true
    }
  },
  "rust": {
    "generatorName": "rust",
    "output": "./platform/rust",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "python": {
    "generatorName": "python",
    "output": "./platform/python",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "java": {
    "generatorName": "java",
    "output": "./platform/java",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "go": {
    "generatorName": "go",
    "output": "./platform/go",
    "inputSpec": "http://localhost:8001/api-json"
  },
  "kotlin": {
    "generatorName": "kotlin",
    "output": "./platform/kotlin",
    "inputSpec": "http://localhost:8001/api-json"
  },
```

```
      "android": {
        "generatorName": "android",
        "output": "./platform/android",
        "inputSpec": "http://localhost:8001/api-json"
      },
      "csharp": {
        "generatorName": "csharp",
        "output": "./platform/csharp",
        "inputSpec": "http://localhost:8001/api-json"
      },
      "javascript": {
        "generatorName": "javascript",
        "output": "./platform/javascript",
        "inputSpec": "http://localhost:8001/api-json"
      }
    }
  }
}
```

Configuration Features:

- Language-specific generator configurations
- Flexible output path management
- Customizable additional properties
- Version control support

2. **Generation Commands**

```
# Generate all SDKs
bun run gen

# Generate language-specific SDK
bun run gen:ts
bun run gen:rust
bun run gen:python
...
```

Generation Process Features:

- Automated build process
- Incremental generation support
- Error handling mechanism
- Parallel generation optimization

3.7.5 Documentation Generation

The system implements a comprehensive documentation generation mechanism:

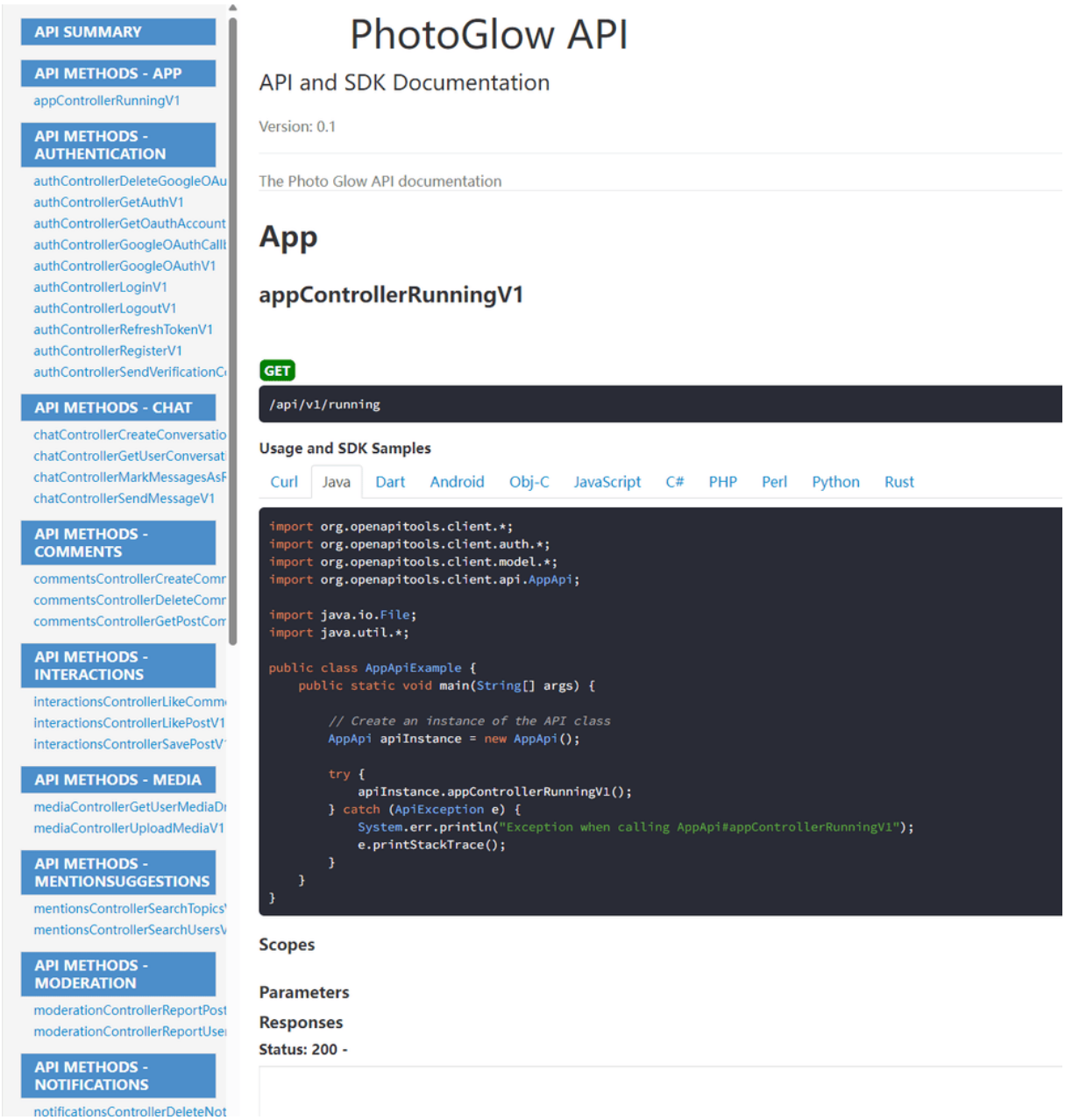


Figure 132: Generated SDK Documentations

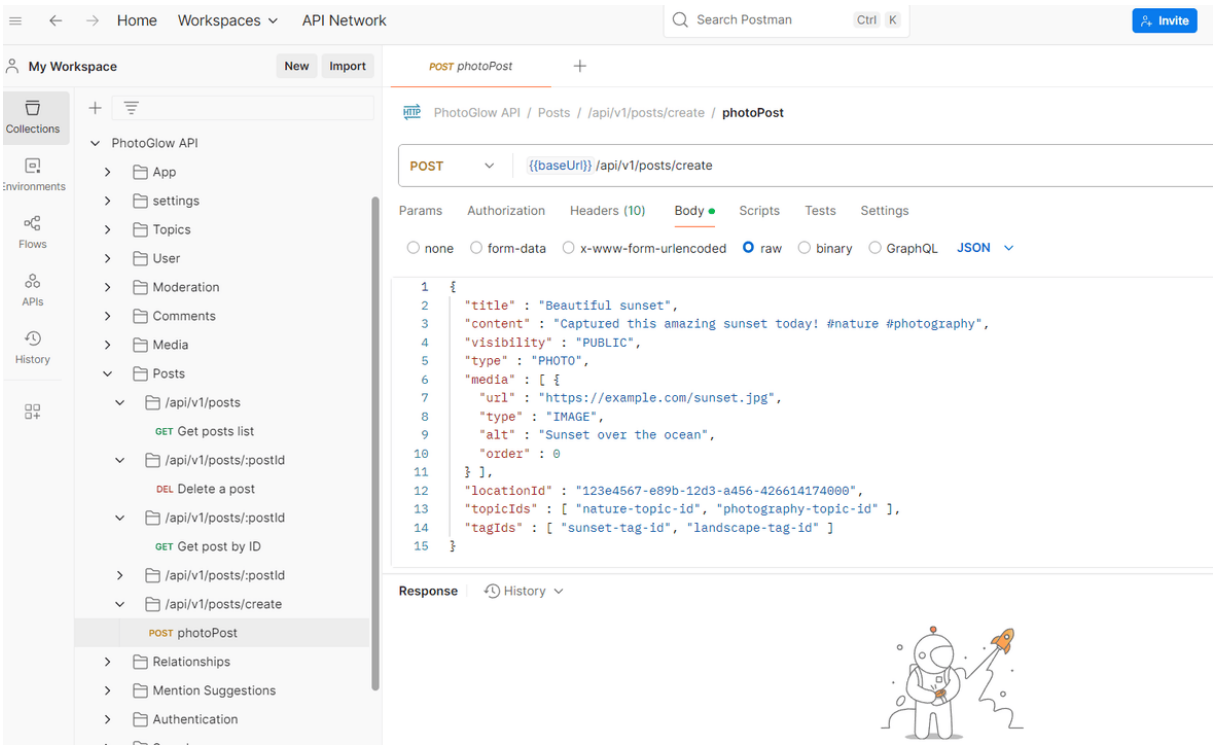


Figure 133: Generated Postman Schema

1. **API Documentation**

- Automatically generated HTML documentation
- Interactive API exploration interface
- Request/response examples
- Type definition explanations

2. **Postman Integration**

- Complete API collection
- Environment configuration templates
- Test case suites
- Variable management

3.7.6 Security Design

The SDK's security design considers multiple layers:

1. **Authentication Mechanism**

- Unified token management
- Secure credential handling
- Session state maintenance
- Error retry mechanism

2. **Data Protection**

- Request data validation
- Response data sanitization
- Sensitive information protection
- Error message handling

Supported by the Openapi project.

3.7.7 Extensibility Design

The system's extensibility is reflected in the following aspects:

1. **New Language Support**

- Templated generator design
- Plugin-based language extensions
- Configuration-driven generation process
- Custom generator support

2. **Version Management**

- Semantic versioning
- Backward compatibility guarantee
- Deprecated feature management
- Upgrade path planning

3.7.8 Future Plans

The SDK's future development plans include:

1. **Feature Enhancements**

- Support for more languages
- Advanced functionality encapsulation
- Performance optimization tools
- Enhanced debugging capabilities

2. **Tool Improvements**

- Code generation optimization
- Documentation system enhancement
- Test coverage improvement
- Developer experience refinement

Through this comprehensive SDK design, PhotoGlow not only provides standardized API access methods but also delivers a consistent development experience for developers across different languages. The automated generation mechanism and comprehensive toolchain support significantly improve API usability and development efficiency. Additionally, through careful security design and extensibility considerations, a solid foundation has been laid for the system's long-term development.

3.8 Containerized Deployment Design

PhotoGlow adopts a Docker-based containerized deployment solution, achieving standardized delivery and efficient operations through modern container technology. This section will detail the system's containerization implementation from multiple dimensions, including architecture design, service configuration, build process, and environment configuration.

3.8.1 Containerized Architecture Design

The system adopts a Docker Compose-based multi-container microservice architecture, achieving high cohesion and low coupling of system components through appropriate service layering and network isolation. The overall architecture includes the following core designs:

1. **Service Organization Structure**

The system decouples different functional modules into independent microservices, each running in separate containers, specifically including:

- PostgreSQL Database Service: Responsible for data persistence storage
- Redis Cache Service: Provides high-performance caching support
- Backend API Service: Handles core business logic
- Frontend Web Service: Provides user interface interaction
- Nginx Reverse Proxy Service: Handles request forwarding and load balancing

This service decoupling design not only improves system maintainability but also provides a foundation for future independent scaling.

2. **Network Architecture Design**

The system adopts a Bridge Network design, specifically implementing:

- Creation of an independent network space named "photoglow-network"
- All service containers connected to this network
- Service discovery and communication through container names
- Only exposing Nginx's HTTP(80) and HTTPS(443) ports externally

This network design achieves secure isolation between services while providing a unified external access entry point through Nginx's reverse proxy mechanism.

3. **Storage System Design**

The system adopts differentiated storage strategies for different types of data and configurations:

- Database persistence: Uses named volume (postgres_data) for PostgreSQL data storage
- Configuration file management: Mounts configuration files via bind mount
- Application code: Copied into containers during build phase
- Temporary data: Uses container internal storage

This multi-level storage design ensures data persistence and configuration flexibility.

3.8.2 Service Container Configuration Design

The container configurations for each service have been carefully optimized. Below is a detailed analysis of each service's configuration characteristics:

1. **Database Service Configuration**

The database service uses PostgreSQL's official Alpine version, configured as follows:

```
postgres:
  image: postgres:17-alpine
  container_name: photoglow-postgres
  volumes:
    - postgres_data:/var/lib/postgresql/data
  environment:
    POSTGRES_USER: ${POSTGRES_USER:-photoglow}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-123456}
    POSTGRES_DB: ${POSTGRES_DB:-photoglow}
  ports:
    - '5432:5432'
  networks:
    - photoglow-network
  restart: always
```

This configuration has the following optimization features:

- Uses Alpine base image to reduce container size and improve resource utilization efficiency
- Enables flexible configuration injection through environment variables, with fallback mechanism for default values
- Implements data persistence through volumes to ensure data security
- Configures automatic restart policy to improve service availability
- Restricts network access scope to enhance security

2. **Cache Service Configuration**

The Redis cache service also adopts a lightweight Alpine version:

```
redis:
  image: redis:7-alpine
  container_name: photoglow-redis
  ports:
    - '6379:6379'
  networks:
    - photoglow-network
  restart: always
```

Configuration feature analysis:

- Employs streamlined configuration structure to reduce maintenance complexity
- Maintains the same network isolation strategy as the database service
- Includes port mapping to facilitate development environment debugging
- Ensures continuous service availability through automatic restart

3. **Backend Service Configuration**

The backend service container configuration fully considers production environment requirements:

```
server:
  build:
    context: .
    dockerfile: Dockerfile.server
  container_name: photoglow-server
  env_file:
    - .env
  environment:
    - NODE_ENV=production
  depends_on:
    - postgres
    - redis
  ports:
    - '8001:8001'
  networks:
    - photoglow-network
  restart: always
```

Key configuration points:

- Uses local build approach to ensure environment consistency
- Centralizes environment variable management through env_file
- Clearly defines service dependencies to ensure proper startup sequence
- Explicitly declares production environment configuration
- Maintains consistent network and restart policies

3.8.3 Build Process Optimization

The system's build process has been specifically optimized for different services. Here's a detailed analysis of the build design for each service:

1. **Backend Service Build Process**

The backend service uses multi-stage builds to optimize image size:

```
FROM oven/bun:1
WORKDIR /app
COPY . .
RUN bun install
RUN cd database && bun run generate
RUN cd server && bun run build
CMD ["sh", "-c", "cd database && bun run migrate && cd ../server && bun run start:prod"]
```

Build process analysis:

- Based on high-performance Bun runtime environment
- Optimized dependency installation and build sequence
- Integrated database migration to ensure schema synchronization
- Implements production mode startup for enhanced performance

2. **Frontend Service Build Process**

The frontend service build process emphasizes build efficiency:

```
FROM oven/bun:1
WORKDIR /app
COPY . .
RUN bun install
RUN cd web && bun run build
CMD ["sh", "-c", "cd web && bun run start"]
```

Build feature analysis:

- Maintains consistent base environment with backend service
- Streamlined build steps for improved efficiency
- Optimized static resource build process

3.8.4 Deployment Environment Configuration

The system is currently deployed on AWS cloud platform with comprehensively optimized environment configurations:

1. **Compute Resource Configuration**

Current deployment uses the following configuration:

- Instance type: t2.medium (2vCPU, 4GB RAM)
- Storage configuration: 50GB SSD volume
- Operating system: Ubuntu 22.04 LTS

This configuration achieves a good balance between resource utilization and cost.

2. **Network Security Configuration**

Implements multi-layered security protection:

- Configures Elastic IP for fixed access entry point
- Security group rules strictly limit access scope
- Configures domain name and SSL certificate for secure access
- Implements network access logging

3. **Monitoring and Alert Configuration**

Establishes a comprehensive monitoring system:

- Integrates with AWS CloudWatch for system monitoring
- Configures container-level resource monitoring
- Implements alert mechanisms for key metrics
- Centralized log management solution

3.8.5 Deployment Process Automation

The system implements standardized deployment process management:

1. **Environment Initialization**

Standardized environment preparation process:

```
# System environment update
apt update && apt upgrade -y

# Docker environment installation
apt install docker.io docker-compose-v2 -y

# Service autostart configuration
systemctl enable docker
systemctl start docker
```

2. **Application Deployment Process**

Implements concise deployment commands:

```
# Service startup
docker compose up -d

# Status check
docker compose ps

# Log monitoring
docker compose logs -f
```

3. **Update Maintenance Process**

Supports flexible update strategies:

```
# Full update
docker compose up -d --build

# Incremental update
docker compose up -d <service> --build
```

3.8.6 Extensibility Design

The system has planned a Kubernetes evolution path based on the current architecture:

1. **Migration Preparation**

The current architecture has completed the following preparations:

- Service containerization standardization
- External configuration design
- Stateless service transformation
- Health check mechanism

2. **Migration Path Planning**

Established a phased migration plan:

- Create and configure Kubernetes cluster
- Convert service configuration to Kubernetes resources
- Implement automated CI/CD process
- Configure HPA for automatic scaling
- Implement zero-downtime release updates

3. **Architecture Evolution Benefits**

Expected improvements after migration:

- Automated service orchestration capability
- Enhanced fault recovery mechanism
- Convenient horizontal scaling capability
- Efficient service discovery mechanism
- Fine-grained resource scheduling control

Through this containerized deployment solution, PhotoGlow not only achieves efficient operations at the current stage but also reserves evolution space for future scale expansion. The system adopts a progressive architecture evolution strategy, ensuring current operational stability while maintaining future extensibility. This container technology-based deployment solution provides a solid technical foundation for the system's continuous development.

3.8 Recommendation System Design

The recommendation system is one of the core functions of modern social media platforms, helping users discover interesting images and creators among massive content. This section will detail PhotoGlow's recommendation system design and its current implementation status.

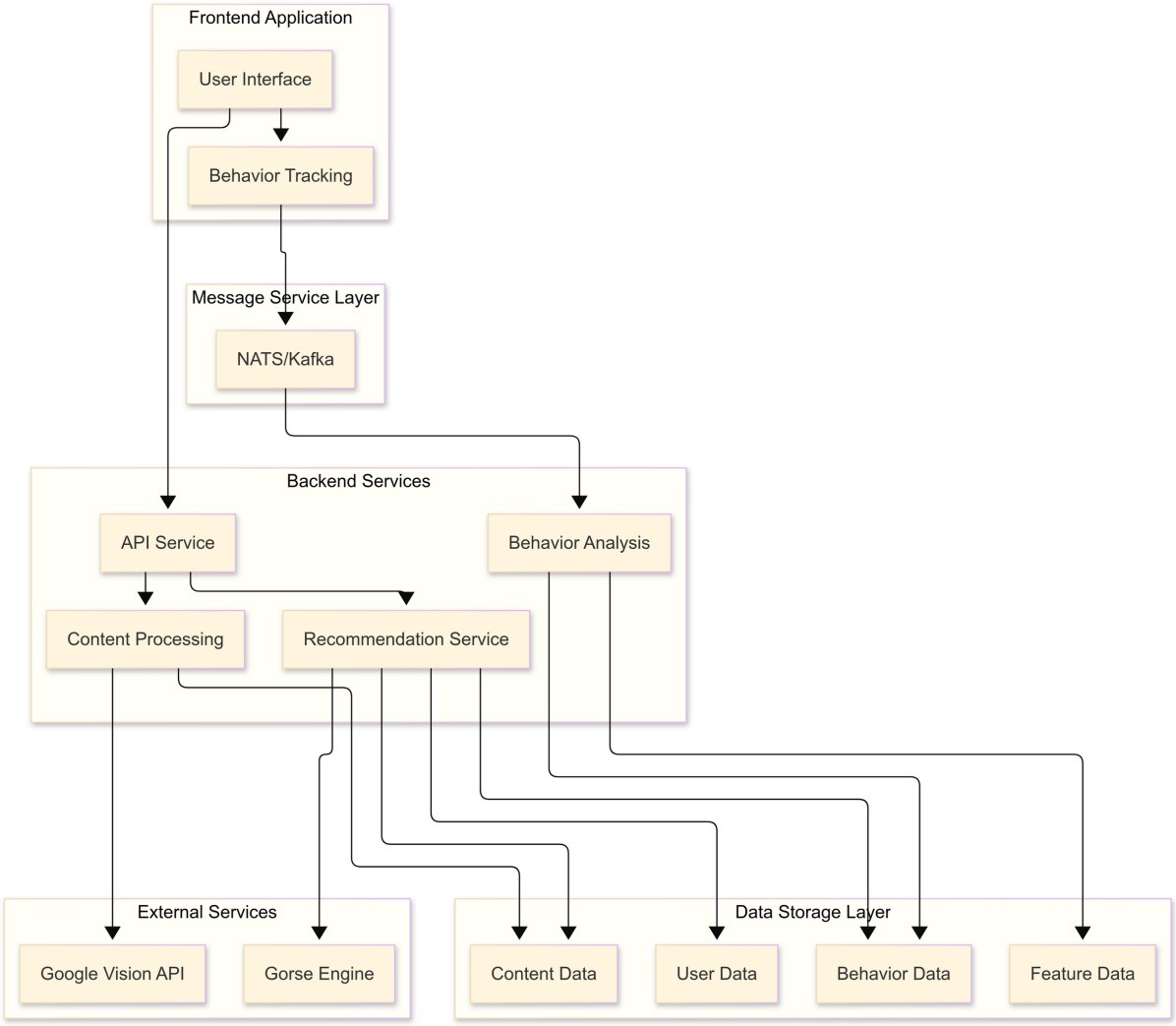


Figure 134: Overall Recommendation System Architecture

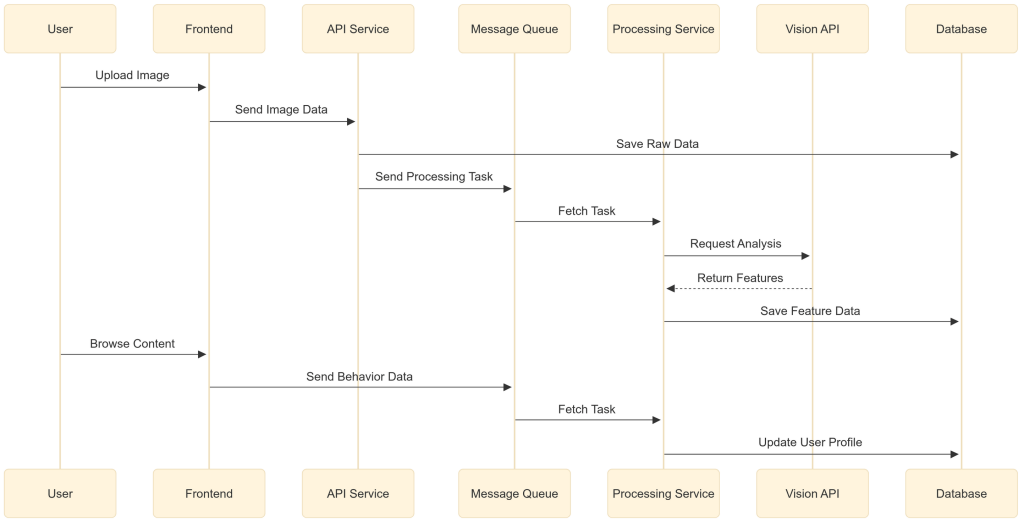


Figure 135: Data Flow Diagram

3.8.1 Available Technical Resources

When designing the recommendation system, we evaluated several available technical solutions:

1. Gorse Recommendation Engine
    - An open-source recommendation system developed in Golang
    - Provides out-of-the-box recommendation algorithm implementations
    - Supports collaborative filtering and content-based recommendations
    - Demonstrates good scalability and performance characteristics
2. Google Cloud Vision API
    - Provides advanced computer vision capabilities
    - Can automatically generate tags and classifications for images
    - Supports scene recognition and object detection
    - Provides sentiment analysis and inappropriate content detection

These technical resources provide reliable foundational support for implementing the recommendation system.

3.8.2 System Architecture Design

The recommendation system's overall architecture adopts an event-driven microservices design, comprising the following main components:

1. Message Service Layer
    - Uses NATS.io as the primary message queue system
    - Can use Kafka or AWS SQS as alternative solutions
    - Handles asynchronous communication between system components
    - Ensures data processing reliability and real-time performance
2. Data Storage Layer
    - Uses dedicated databases for recommendation-related data
    - Includes content features, user profiles, interaction records
    - Adopts database solutions suitable for high-frequency read/write operations
    - Implements efficient data retrieval and updates
3. Content Processing Service
    - Handles newly published content
    - Performs image analysis through AI services
    - Generates content tags and feature vectors
    - Updates the recommendation system's content library
4. User Behavior Analysis Service
    - Collects user interaction data
    - Builds user interest models
    - Updates user profile information
    - Generates behavioral analysis reports

3.8.3 Data Collection Design

To support recommendation algorithms, the system needs to collect data across multiple dimensions:

1. Content Feature Data
    - Image visual features
    - Upload time and location information
    - Creator attributes
    - Content tags and descriptions
2. User Behavior Data
    - Browsing history and dwell time

- Like and bookmark operations
- Comment and share behaviors
- Follow and interaction relationships

3. Environmental Context Data

- Access time and device information
- Geographic location and language preferences
- Entry sources and access paths
- Network status and performance data

3.8.4 Recommendation Algorithm Design

The recommendation system employs a multi-layered hybrid recommendation strategy:

1. Base Recommendation Layer

- Content-based recommendation algorithms
- Collaborative filtering recommendation algorithms
- Tag-based recommendation algorithms
- Popular content recommendations

2. Personalization Optimization Layer

- User interest weight adjustment
- Time decay factor processing
- Content diversity assurance
- New content exploration mechanism

3. Business Rules Layer

- Content quality filtering
- Operational strategy integration
- Recommendation reason generation
- Result post-processing rules

3.8.5 Cold Start Strategy

The following strategies have been designed to address the cold start problem in the platform's early stages:

1. System Cold Start

- Initially uses timeline display
- Introduces popular content recommendations
- Gradually transitions to personalized recommendations
- Maintains diversity in recommendation results

2. User Cold Start

- Guides users to select interest tags
- Utilizes basic user information matching
- Displays diverse exploratory content
- Rapidly collects user feedback

3. Content Cold Start

- Based on content feature matching
- Creator influence weighting
- New content display space reservation
- Rapid quality assessment feedback

3.9 Current Implementation Status and Future Plans

3.9.1 Platform Cold Start Phase Challenges In implementing the recommendation system, we face typical cold start dilemmas:

1. Insufficient User Behavior Data
    - Small platform user base
    - Sparse user interaction data
    - Difficulty in building reliable user profiles
    - Challenge in ensuring recommendation effectiveness
2. Limited Content Database Scale
    - Small content library size
    - Insufficient content type diversity
    - Limited feature extraction samples
    - Difficulty in meeting diversity requirements for recommendations
3. Incomplete Evaluation System
    - Lack of historical data as baseline
    - Difficulty in establishing evaluation metrics
    - Unclear algorithm optimization direction
    - Challenge in quantifying improvements

3.9.2 Strategic Resource Priority Considerations At the current stage of product development, we need to focus limited development resources on more critical objectives:

1. Core Functionality Enhancement
    - Image upload and storage system optimization
    - User authentication and authorization mechanism improvement
    - Social interaction functionality strengthening
    - Performance optimization and security hardening
2. User Growth Strategy
    - Improving product usability
    - Optimizing user registration process
    - Implementing basic social sharing functions
    - Establishing user incentive mechanisms
3. Platform Stability Assurance
    - System architecture optimization
    - Service availability improvement
    - Data security assurance
    - Operations system development

3.9.3 Human Resource Limitations The implementation of the recommendation system requires specialized human resources:

1. Content Moderation Team
    - Requires dedicated content moderators
    - Needs complete moderation standards
    - Requires development of moderation tools and processes
    - Needs ongoing moderation quality management
2. Technical Team Configuration
    - Requires machine learning engineers
    - Requires data analysts
    - Requires algorithm engineers
    - Requires operations engineers
3. Operations Team Support
    - Requires product operations personnel
    - Requires user operations personnel

- Requires content operations personnel
- Requires community operations personnel

3.9.4 Phased Implementation Plan To ensure successful implementation of the recommendation system, we have developed a gradual implementation plan:

1. Phase One (Current)
    - Perfect data collection infrastructure
    - Establish user behavior tracking system
    - Implement basic content classification functionality
    - Prepare for subsequent recommendation system development
2. Phase Two (After reaching certain user scale)
    - Implement simple tag-based recommendations
    - Introduce basic collaborative filtering algorithms
    - Establish recommendation effectiveness evaluation system
    - Begin accumulating algorithm optimization experience
3. Phase Three (Platform maturity)
    - Implement complete hybrid recommendation system
    - Introduce deep learning models
    - Establish real-time recommendation capabilities
    - Optimize personalized recommendation effectiveness

Through this phased implementation strategy, we can ensure that the development of the recommendation system aligns with the platform's overall growth, avoiding resource waste from premature optimization. Meanwhile, through continuous data collection and experience accumulation, we can achieve more precise personalized recommendation services in the future. As the platform's user base grows and content library enriches, the recommendation system will gradually demonstrate its core value, providing users with an enhanced content discovery experience.

4. System Testing and Evaluation

In addition to unit tests and e2e tests in the codebase, we can utilize several common website testing tools for evaluation.

4.1 Deployment Environment

For testing purposes, the website has been containerized and deployed to an AWS EC2 server.

The server is geographically located in London.

The website domain is photoglow.xyz

4.2 Performance Testing Methodology

This section primarily uses Google PageSpeed Insights as the main performance testing tool, with GTmetrix as a supplementary validation tool, to conduct comprehensive performance evaluation of the developed web application.

Google PageSpeed Insights, developed by Google, is a professional website performance analysis tool capable of quantitatively evaluating website performance across multiple dimensions. The tool conducts comprehensive analysis based on real user data (Real User Experience Data) and laboratory data (test data in controlled environments), providing reliable data support for website performance optimization. Its core performance metrics include:

- First Contentful Paint (FCP): 10% weight, measures the time taken to render the first text or image element.
- Speed Index: 10% weight, reflects the average loading time of visible content.
- Largest Contentful Paint (LCP): 25% weight, measures the loading time of the page's main content.
- Time to Interactive: 10% weight, indicates the time needed for the page to reach a fully interactive state.
- Total Blocking Time: 30% weight, measures user input response delay between FCP and Time to Interactive.
- Cumulative Layout Shift: 15% weight, evaluates visual stability during page content loading.

4.3 Performance Test Results and Analysis

Through Google PageSpeed Insights testing of the website deployed in the production environment, we obtained the following performance evaluation results:

- Core Performance The website achieved a perfect score (100 points) in performance testing, indicating that this project has reached an exceptionally high level of performance optimization. This score demonstrates that the website has achieved optimization goals in loading speed, response performance, and visual stability.

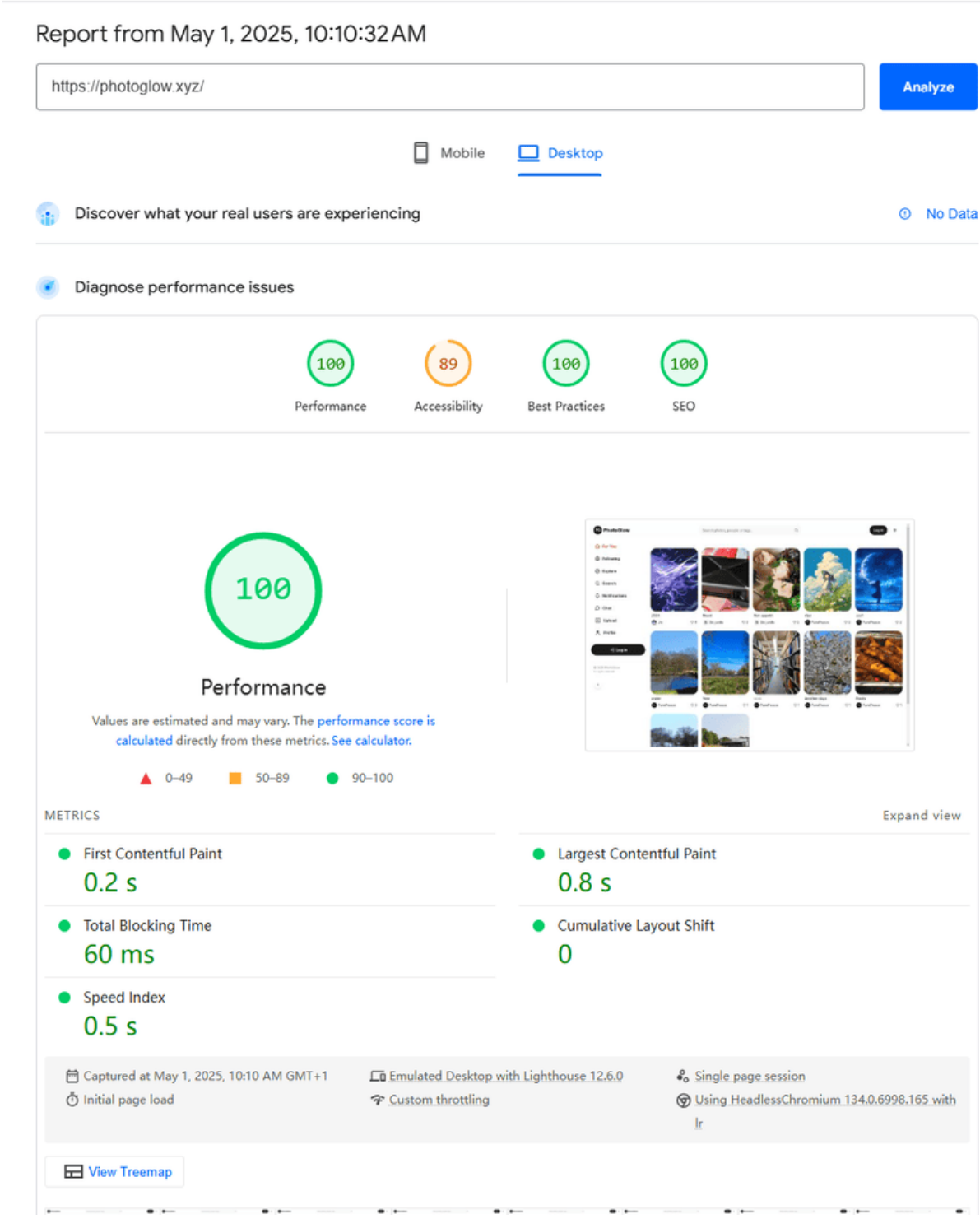


Figure 136: PageSpeed Scores

- Accessibility Assessment Test results indicate room for improvement in website accessibility. This suggests we need to focus on accessibility-related improvements in subsequent optimizations.

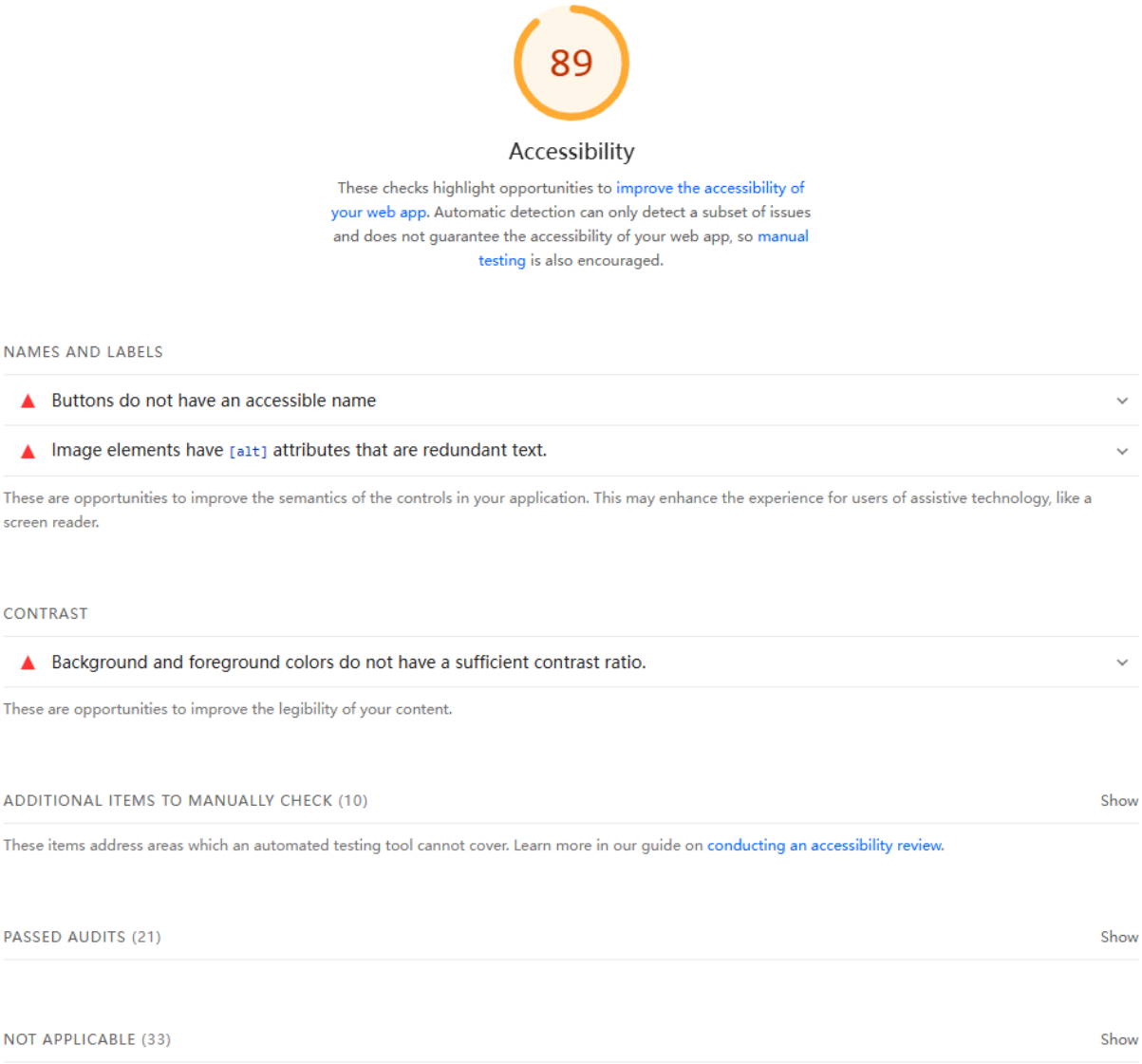


Figure 137: PageSpeed Scores 2

- Best Practices Adherence The website demonstrated excellent performance in following web development best practices, achieving a high score that reflects the project's professional standards in code quality and technical implementation.

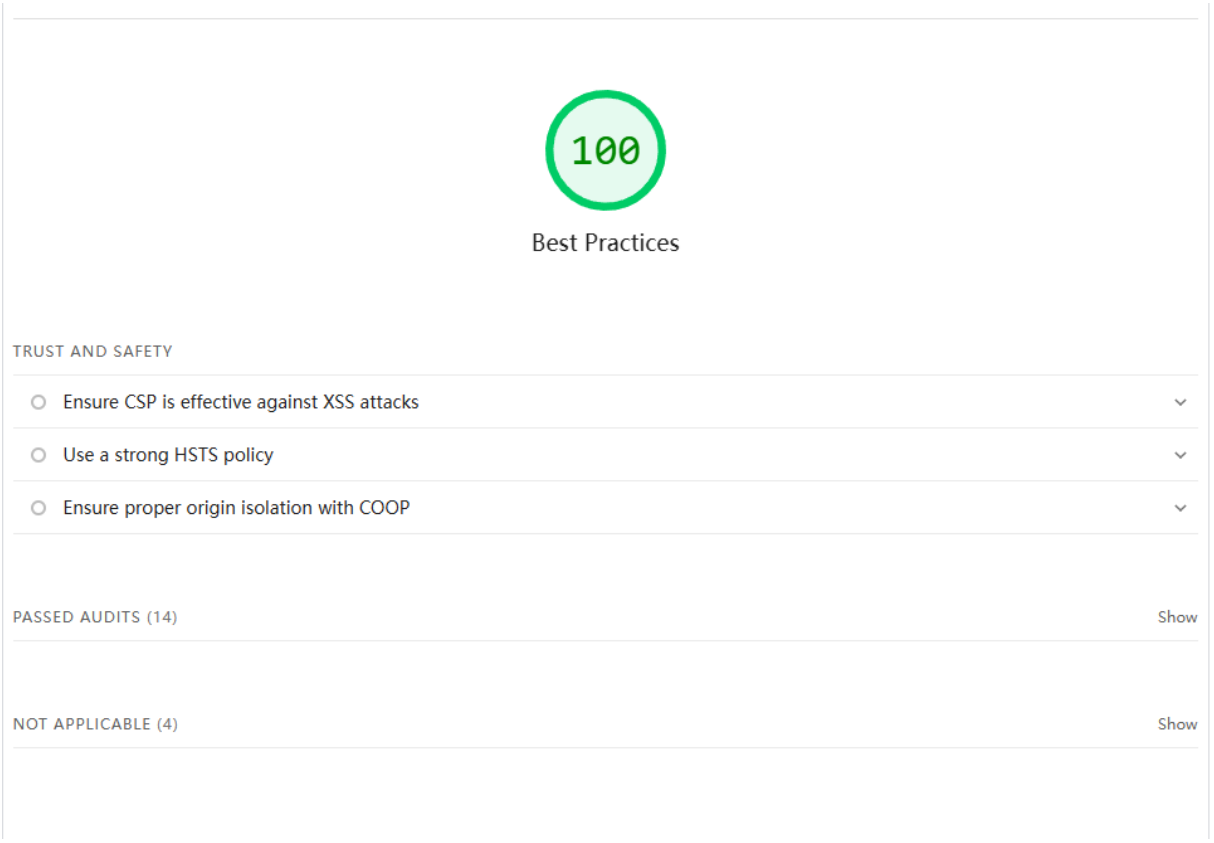


Figure 138: PageSpeed Scores 3

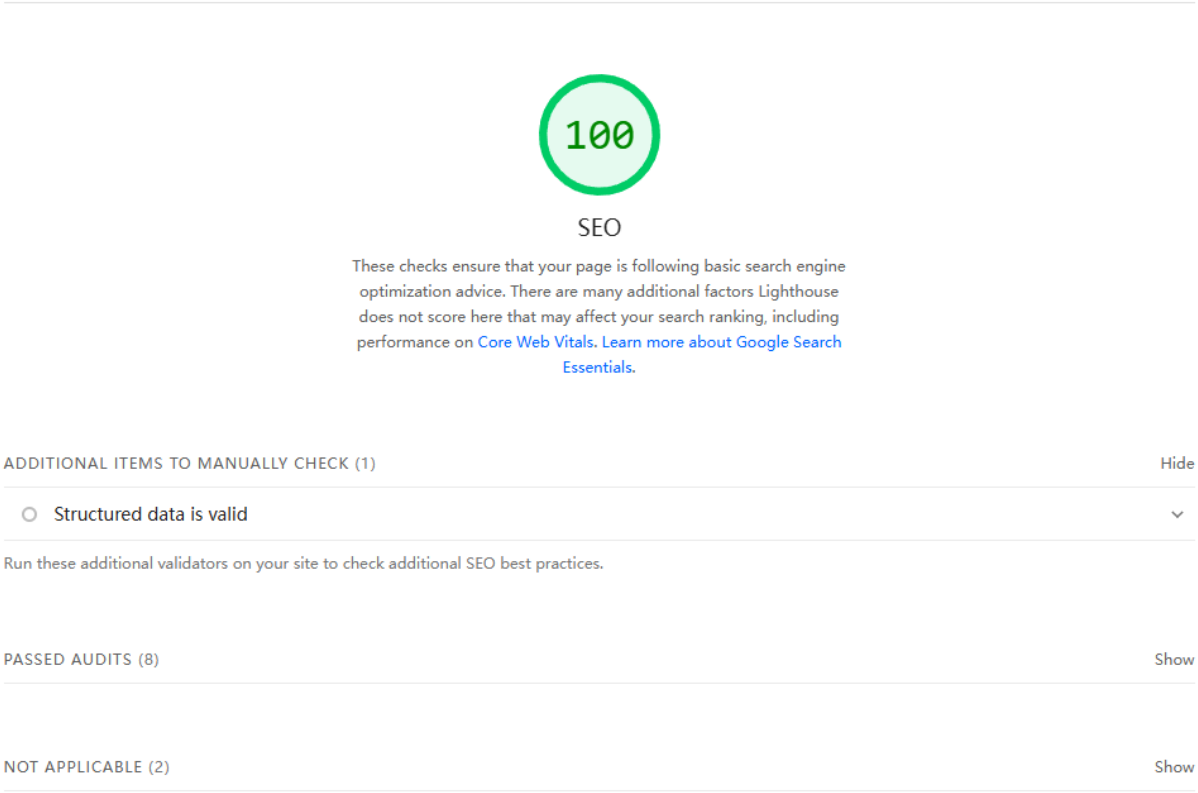


Figure 139: PageSpeed Scores 4

To verify the reliability of test results, we also conducted cross-validation using GTmetrix as a third-party tool.

GTmetrix test results showed high consistency with Google PageSpeed Insights' evaluation, further confirming the website's excellent performance.



Figure 140: Gtmetrix test

In conclusion, the test results demonstrate that this project has achieved significant success in performance optimization, providing users with an efficient and smooth experience. Meanwhile, we've identified potential areas for improvement such as accessibility optimization, which will serve as important reference points for subsequent version iterations.

5. Project Management

This chapter will detail the project management methods, planning arrangements, and related considerations.

5.1 Project Planning

5.1.1 Time Management

The project development cycle was divided into the following main phases:

1. Requirements Analysis and Planning (2 weeks)
    - User requirements collection and analysis
    - Technical feasibility study
    - Project scope definition
2. System Design (3 weeks)
    - Architecture design
    - Database design
    - UI/UX design
3. Development Implementation (8 weeks)
    - Frontend development
    - Backend development
    - Database implementation
    - Admin panel development
    - SDK development
    - Code refactoring
4. Testing and Optimization (3 weeks)
    - Unit testing
    - Integration testing
    - Performance optimization
5. Deployment and Documentation (2 weeks)
    - System deployment
    - Documentation writing
    - User manual creation

In actual development, plans are not always effective, and flexible development is key.

5.1.2 Development Tools

- Development Environment: Windows Operating System, Ubuntu 24.x, WSL2
- IDE: VS Code
- Version Control: Git
- Project Management: Jira
- Technology Stack: Bun, TypeScript, Next.js, Nest.js, Prisma, TailwindCSS
- CDN/DNS/SSL: Cloudflare
- Container: Docker
- Server: AWS EC2
- Cloud Storage: AWS S3
- Database: PostgreSQL
- Cache: Redis
- Gateway: Nginx
- Database Tools: Prisma CLI, Navicat
- Server Operations Tools: Xshell, Xftp
- Diagramming Tools: Mermaid
- Documentation Editing: Zettlr, LaTeX, Markdown
- Other Tools: Postman, Docker Desktop

5.1.3 Quality Management

- Code Standard Compliance: ESLint, TypeScript config
- Regular Code Review: `bun run lint`
- Continuous Integration/Continuous Deployment (CI/CD): Not yet implemented
- Test Coverage Requirements: Not yet implemented

5.2 Risk Management

5.2.1 Risk Identification

1. Technical Risks
    - Learning curve for new technologies
    - Suitability of technology choices
    - System performance bottlenecks
2. Schedule Risks
    - Delays due to requirement changes
    - Inaccurate time estimation for technical challenges
3. Quality Risks
    - Code quality control
    - Insufficient test coverage
    - Security vulnerabilities
4. Operational Risks
    - Public deployment environment attacks
    - Users publishing inappropriate/illegal content on the website

5.2.2 Risk Response Strategies

1. Preventive Measures
    - Detailed technical research
    - Establishment of clear milestones
    - Implementation of code review mechanisms
    - Implementation of monitoring mechanisms
    - Implementation of content review mechanisms
2. Contingency Plans
    - Alternative technical solutions
    - Resource allocation mechanisms
    - Problem escalation processes

5.3 Project Background Considerations

5.3.1 Legal and Ethical Considerations

1. Data Protection
    - Compliance with GDPR regulations
    - Encrypted storage of user data
    - Clear privacy policies
2. Intellectual Property
    - Open source license compliance
    - Third-party library usage standards
    - Original code protection

5.3.2 Health and Safety Considerations

1. User Health Considerations
    - Interface usability design
    - Appropriate prompts and warnings
    - Prevention of excessive use
2. Development Team Health
    - Reasonable work intensity
    - Regular break system

5.3.3 Equality, Diversity, and Inclusion

1. User Level
    - Multi-language support
    - Accessibility design
    - Cultural inclusivity considerations
2. Development Level
    - Diversity in technologies used
    - Equal participation opportunities

5.3.4 Sustainable Development

1. Technical Sustainability
    - Scalable architecture design
    - Technology stack update planning
    - Maintenance cost considerations
2. Environmental Impact
    - Server resource optimization
    - Energy efficiency considerations
    - Environmental responsibility

6.1 Project Summary

This project designed and implemented PhotoGlow, a modern social media platform, utilizing the latest technology stack and architectural design to successfully build a system with comprehensive functionality, excellent performance, and strong scalability.

Code statistics using scc:

Excluding `node_modules` and auto-generated code, wrote 31,000 lines of TypeScript code (excluding comments) for this project.

Language	Files	Lines	Blanks	Comments	Code	Complexity
TypeScript	344	36162	3481	977	31704	2541
JSON	19	695	9	0	686	0
Markdown	8	1522	461	0	1061	0
JavaScript	7	115	12	8	95	0
CSS	2	270	16	2	252	0

Figure 141: scc

Major achievements include:

1. **Complete System Architecture**
    - Microservice architecture with frontend-backend separation
    - Implementation of complete technology stack from frontend to backend
    - Construction of efficient data processing and storage system
    - Comprehensive API and SDK support
    - Production deployment implementation
2. **Excellence in Technical Implementation**
    - Modern frontend framework based on Next.js
    - High-performance backend service built with NestJS
    - Efficient database and caching system implementation
    - Comprehensive security protection mechanisms
3. **Superior User Experience**
    - Smooth user interface implementation
    - Rich social functionality
    - Multi-platform accessibility
    - System availability and stability assurance
4. **Comprehensive Operations Support**
    - Containerized deployment solution implementation
    - Complete monitoring system
    - Automated operations support
    - Strong maintainability

6.2 Innovation Points

This project demonstrates the following innovations in technical implementation and functional design:

1. **Technical Innovation**
    - Development using modern technology stack and latest versions, employing many experimental features
    - Use of Bun as JavaScript runtime for improved development and execution efficiency
    - Implementation of automated SDK generation system based on OpenAPI
    - Design of efficient image processing and storage solutions
    - Design of innovative recommendation algorithm system

- High-quality, maintainable code

2. **Architectural Innovation**
   - Implementation of scalable microservice-based architecture
   - Design of efficient caching strategies
   - Flexible plugin system
   - Support for multiple deployment solutions
3. **Functional Innovation**
   - Innovative social interaction features
   - Support for diverse content presentation methods
   - Efficient user management system implementation

6.3 Future Work

Based on the current system implementation, future work will focus on:

1. **Technical Optimization**
   - Further system performance optimization
   - Enhanced system scalability
   - Improved code quality and test coverage
   - Enhanced monitoring and alert systems
2. **Functionality Extension**
   - Additional social interaction features
   - Recommendation algorithm optimization
   - Support for more media types
   - Enhanced API interfaces
3. **Operations Improvements**
   - Refined automated deployment processes
   - Optimized resource utilization
   - Enhanced system security
   - Improved system reliability

6.4 Project Reflection

Through the project development process, I gained valuable experience and insights:

1. **Importance of Technology Selection**
   - New technologies bring both opportunities and challenges
   - Need to balance innovation and stability
   - Technology selection must consider long-term maintenance costs
   - Importance of technological ecosystem support
2. **Value of Architecture Design**
   - Good architecture design is fundamental to system success
   - Modular design improves system maintainability
   - Microservice architecture enhances system scalability
   - Standardized interfaces facilitate system integration and expansion
3. **Project Management Experience**
   - Reasonable time planning is crucial; despite starting the project quite late due to various complex reasons and facing a tight timeline, I still independently completed the entire project design and development work
   - Importance of risk management and quality control
   - Significance of documentation management and version control
   - Value of continuous integration and deployment

4. **Importance of User Experience**

   - Performance optimization is crucial for user experience
   - User feedback requires attention
   - Security and reliability are basic requirements
   - Interface intuitiveness and usability are important

Through this project's development, I not only achieved the intended functional goals but also significantly improved in technical capabilities, project management, and problem-solving. These experiences and lessons will positively influence future project development.

References

[1] Render, “Bun vs node.js.” 2025. Available: https://render.com/blog/bun-vs-nodejs

[2] CERN, “The birth of the web.” n.d. Available: https://home.cern/science/computing/birth-web

[3] T. Berners-Lee, “Information management: A proposal.” 1989. Available: https://www.w3.org/History/1989/proposal.html

[4] Wired, “Feb. 16, 1978: Bulletin board goes electronic.” 2010. Available: https://www.wired.com/2010/02/0216cbbs-first-bbs-bulletin-board

[5] W3C, “Cascading style sheets, level 1.” 1996. Available: https://www.w3.org/TR/REC-CSS1/

[6] P. Manual, “History of PHP.” n.d. Available: https://www.php.net/manual/en/history.php.php

[7] J. J. Garrett, “Ajax: A new approach to web applications.” 2005. Available: https://designftw.mit.edu/lectures/apis/ajax_adaptive_path.pdf

[8] Wikipedia, “Single-page application.” n.d. Available: https://en.wikipedia.org/wiki/Single-page_application

[9] DailyJS, “Angular 1.0 turns five years old.” 2017. Available: https://medium.com/dailyjs/angular-1-0-turns-five-years-old-4d7108a5e412

[10] CENG, “Timeline from 1G to 5G: A brief history on cell phones.” 2020. Available: https://www.cengn.ca/information-centre/innovation/timeline-from-1g-to-5g-a-brief-history-on-cell-phones/

[11] Wikipedia, “Progressive web app.” n.d. Available: https://en.wikipedia.org/wiki/Progressive_web_app

[12] Wikipedia, “Amazon S3.” n.d. Available: https://en.wikipedia.org/wiki/Amazon_S3

[13] S. K. Shaina Raza Mizanur Rahman, “A comprehensive review of recommender systems: Transitioning from theory to practice.” 2023. Available: https://www.researchgate.net/publication/382364052_A_Comprehensive_Review_of_Recommender_Systems_Transitioning_from_Theory_to_Practice

[14] Wikipedia, “WebAssembly.” n.d. Available: https://en.wikipedia.org/wiki/WebAssembly

[15] B. Kelly, “The impact of edge computing on real-time data processing.” 2020. Available: https://www.researchgate.net/publication/382156395_The_Impact_of_Edge_Computing_on_Real-Time_Data_Processing

[16] Wikipedia, “JavaScript.” n.d. Available: https://en.wikipedia.org/wiki/JavaScript

[17] Wikipedia, “V8 engine.” n.d. Available: https://en.wikipedia.org/wiki/V8_(JavaScript_engine)

[18] Wikipedia, “Node.js.” n.d. Available: https://en.wikipedia.org/wiki/Node.js

[19] Wikipedia, “TypeScript.” n.d. Available: https://en.wikipedia.org/wiki/TypeScript

[20] Github, “DefinitelyTyped.” n.d. Available: https://github.com/DefinitelyTyped/DefinitelyTyped

[21] J. George, “Adoption trends: Why developers are switching from JavaScript to TypeScript.” 2020. Available: https://www.researchgate.net/publication/389688708_Adoption_Trends_Why_Developers_Are_Switching_from_JavaScript_to_TypeScript

[22] Wikipedia, “Webpack.” n.d. Available: https://en.wikipedia.org/wiki/Webpack

[23] Wikipedia, “Vite.” n.d. Available: https://en.wikipedia.org/wiki/Vite_(software)

[24] Wikipedia, “Npm.” n.d. Available: https://en.wikipedia.org/wiki/Npm

[25] Wikipedia, “Yarn.” n.d. Available: https://en.wikipedia.org/wiki/Yarn_(package_manager)

[26] Wikipedia, “Pnpm.” n.d. Available: https://en.wikipedia.org/wiki/Pnpm

[27] N. Brousse, “The issue of monorepo and polyrepo in large enterprises.” 2019. Available: https://dl.acm.org/doi/10.1145/3328433.3328435

[28] Bun, “Bun.” n.d. Available: https://bun.sh/docs

[29] Wikipedia, “React.” n.d. Available: https://en.wikipedia.org/wiki/React_(software)

[30] Wikipedia, “Vue.js.” n.d. Available: https://en.wikipedia.org/wiki/Vue.js

[31] Vercel, “Next.js.” n.d. Available: https://nextjs.org/docs

[32] Nuxt.js, “Nuxt.js.” n.d. Available: https://nuxt.com/docs/getting-started/introduction

[33] Contentful, “What is server-side rendering?” n.d. Available: https://www.contentful.com/blog/what-is-server-side-rendering/

[34] Sanity, “Hybrid rendering.” n.d. Available: https://www.sanity.io/glossary/hybrid-rendering

[35] LogRocket, “React server components in next.js 13.” n.d. Available: https://blog.logrocket.com/react-server-components-next-js-13/

[36] Wikipedia, “Sass (style sheet language).” n.d. Available: https://en.wikipedia.org/wiki/Sass_(style_sheet_language)

[37] Wikipedia, “Less (style sheet language).” n.d. Available: https://en.wikipedia.org/wiki/Less_(style_sheet_language)
[38] Wikipedia, “Tailwind CSS.” n.d. Available: https://en.wikipedia.org/wiki/Tailwind_CSS
[39] Shadcn/UI, “Shadcn/UI.” n.d. Available: https://ui.shadcn.com/docs
[40] NestJS, “NestJS.” n.d. Available: https://docs.nestjs.com/
[41] Wikipedia, “Separation of concerns.” n.d. Available: https://en.wikipedia.org/wiki/Separation_of_concerns
[42] The Server Side, “10 disadvantages of microservices you’ll need to overcome.” 2023. Available: https://www.theserverside.com/answer/What-are-some-of-the-disadvantages-of-microservices
[43] High Scalability, “How twitter uses redis to scale 105TB of RAM, 39MM QPS, 10000 instances.” 2014. Available: https://highscalability.com/how-twitter-uses-redis-to-scale-105tb-ram-39mm-qps-10000-ins/tecture
[44] Ably, “The challenge of scaling websockets.” 2024. Available: https://ably.com/topic/the-challenge-of-scaling-websockets
[45] Wikipedia, “Server-sent events.” n.d. Available: https://en.wikipedia.org/wiki/Server-sent_events
[46] Wikipedia, “HTTPS.” n.d. Available: https://en.wikipedia.org/wiki/HTTPS
[47] Wikipedia, “JSON web token.” n.d. Available: https://en.wikipedia.org/wiki/JSON_Web_Token
[48] Evil Martians, “Oklch in CSS: Why quit RGB/HSL?” 2024. Available: https://evilmartians.com/chronicles/oklch-in-css-why-quit-rgb-hsl